Extensions

Reference manual pages

2026-10-25

Contents

bwidget **6**
ArrowButton 6
BWidget 9
Button 11
ButtonBox 14
ComboBox 17
Dialog 21
DragSite 24
DropSite 26
DynamicHelp 29
Entry 31
Label 35
LabelEntry 39
LabelFrame 41
ListBox 43
MainFrame 49
MessageDlg 53
NoteBook 56
PagesManager 59
PanedWindow 61
PanelFrame 63
PasswdDlg 65
ProgressBar 67
ProgressDlg 69
ScrollView 71
ScrollableFrame 72
ScrolledWindow 74
SelectColor 76
SelectFont 78
Separator 79
SpinBox 80
StatusBar 83
TitleFrame 85
Tree 86
Widget 95
contents 100
index 102
navtree 102
options 103
expect **111**
expect 111
libexpect 137
itcl **146**
body 146

class 148
code 156
configbody 157
delete 159
ensemble 160
find 163
is 164
itcl 165
itclcomponent 167
itcldelegate 167
itclextendedclass 170
itcloption 178
itclvars 180
itclwidget 182
local 189
scope 191
itcl-c **192**
Class 192
List 193
Object 194
Preserve 194
RegisterC 195
Stack 197
itk **198**
Archetype 198
Toplevel 204
Widget 206
itk 208
itkvars 209
usual 210
mclistbox **211**
mclistbox 211
mpexpr **217**
mpexpr 217
tclhttpd **224**
httpd 224
tcltls **227**
tls 227
tcludp **241**
udp 241
tclvfs **245**
vfs-filesystems 246
vfs-fsapi 247
vfs 252
vfslib 256
tclws **257**

Calling_a_Web_Service 257
Creating_a_Tcl_Web_Service 270
Creating_a_Web_Service_Type 276
Defining_Types 279
Dictionary_Representation_of_XML_Arrays 282
Embedded_Web_Service 283
Rest_flavor_service_response 285
Tcl_Web_Service_Example 286
Tcl_Web_Service_Math_Example 289
Using_Options 292
index 295
tdbc 296
tdbc 296
tdbc_connection 297
tdbc_mapSqlState 302
tdbc_resultset 303
tdbc_statement 304
tdbc_tokenize 307
tdbc-c 308
Tdbc_Init 308
tdbcmysql 309
tdbc_mysql 309
tdbcodbc 311
tdbc_odbc 311
tdbcpostgres 314
tdbc_postgres 314
tdbcsqlite3 316
tdbc_sqlite3 316
tdom 317
dom 317
domDoc 327
domNode 335
expat 343
expatapi 353
pullparser 357
schema 359
tdomcmd 379
tnc 380
xpathFunc 382
thread 384
thread 384
tpool 392
tsv 396
ttrace 401
License 404
bwidget license terms 404

expect license terms 405
itcl, itcl-c license terms 405
itk license terms 406
mclistbox license terms 407
mpexpr license terms 407
tclhttpd license terms 408
tcltls license terms 408
tcludp license terms 409
tclvfs license terms 409
tclws license terms 410
tdbc, tdbc-c, tdbcmysql, tdbcodbc license terms 410
tdbcpostgres license terms 411
tdbcsqlite3 license terms 412
tdom license terms 412
thread license terms 419

bwidget

ArrowButton

NAME

ArrowButton - Button widget with an arrow shape.

CREATION

ArrowButton *pathName* ?*option value...*?

STANDARD OPTIONS

-activebackground	
-activeforeground	
-background or -bg	-borderwidth or -bd
-disabledforeground	
-foreground or -fg	
-highlightbackground	
-highlightcolor	
-highlightthickness	
-relief	
-repeatdelay	
-repeatinterval	
-takefocus	
-troughcolor	

WIDGET-SPECIFIC OPTIONS

-armcommand
-arrowbd
-arrowrelief
-clean
-command
-dir
-disarmcommand

-height
-helptext
-helptype
-helpvar
-ipadx
-ipady
-state
-type
-width

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName invoke

DESCRIPTION

ArrowButton can be of two types following **type** option: for **button** type, it is standard button with an arrow drawn on it; for **arrow** type, it is an arrow like scrollbar's arrow.

WIDGET-SPECIFIC OPTIONS

-armcommand
Specifies a Tcl command to associate with the ArrowButton when mouse button 1 is pressed over the ArrowButton. When **repeatdelay** or **repeatinterval** option is positive integer, this command is repeatedly called if mouse pointer is over the button and until mouse button 1 is released.

-arrowbd
When ArrowButton **type** is *arrow*, specifies the border width of the arrow. Must be 1 or 2.

-arrowrelief
When ArrowButton **type** is *arrow*, specifies the relief of the arrow. Must be **raised** or **sunken**.

-clean
Specifies a level of quality, between 0 and 2, for the arrow. If 0, the arrow is drawn with its maximum width and height. If 1, the base of arrow is arranged to be odd to have same edges. If 2, the base of arrow is arranged to be odd and the orthogonal to be (base+1)/2 to have 'straight' diagonal for edges.

-command
Specifies a Tcl command to associate with the ArrowButton. This command is typically

invoked when mouse button 1 is released over the ArrowButton window.

-dir
Specifies the direction of the arrow: **top**, **bottom**, **left** or **right**.

-disarmcommand
Specifies a Tcl command to associate with the ArrowButton when mouse button 1 is released. This command is called even if pointer is not over the ArrowButton, and always before the command specified by **command** option. It is typically used in conjuntion with **armcommand**, **repeatdelay** and **repeatinterval**.

-height
Specifies a desired height for the ArrowButton. The value is in screen units.

-helptext
Text for dynamic help. If empty, no help is available for this widget. See also DynamicHelp.

-helptype
Type of dynamic help. Use *balloon* or *variable*. See also DynamicHelp.

-helpvar
Variable to use when **helptype** option is *variable*. See also DynamicHelp.

-ipadx
Specifies a minimun pad between the ArrowButton border and the right and left side of the arrow. The value is in screen units.

-ipady
Specifies a minimun pad between the ArrowButton border and the top and bottom side of the arrow. The value is in screen units.

-state
Specifies one of three states for the ArrowButton: **normal**, **active**, or **disabled**.

If ArrowButton type is *button*:
In normal state the ArrowButton is displayed using the **foreground** and **background** options. The active state is typically used when the pointer is over the ArrowButton. In active state the ArrowButton is displayed using the **activeforeground** and **activebackground** options. In disabled state the **disabledforeground** and **background** options determine how the ArrowButton is displayed.

If ArrowButton type is *arrow*:
Only colors of arrow change. The background of ArrowButton is always displayed using **troughcolor** option. In normal state the ArrowButton is displayed using the **background** option. The active state is typically used when the pointer is over the ArrowButton. In active state the ArrowButton is displayed using the **activebackground** option. In disabled state the ArrowButton is displayed with a dark stipple.

Disabled state means that the ArrowButton should be insensitive: the default bindings will refuse to activate the widget and will ignore mouse button presses.

-type
Determines the type of the ArrowButton: **button** for standard button look, or **arrow** scrollbar's arrow look.

-width
Specifies a desired width for the ArrowButton. The value is in screen units.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* invoke**
If ArrowButton **state** is not disabled, this invoke the commands of the button. ArrowButton is redisplayed with active color and sunken relief, and **armcommand** is called. Then ArrowButton is redisplayed with normal color and its defined relief, and **disarmcommand** then **command** are called.

invoke is called when ArrowButton has input focus and user press the space bar.

Keywords: ArrowButton

BWidget

Under construction ...

NAME
BWidget - Description text

COMMAND
BWidget::XLFDfont *cmd* ?*arg...*?

BWidget::assert *exp* ?*msg*?

BWidget::badOptionString *type value list*

BWidget::bindMouseWheel *widget*

BWidget::classes *class*

BWidget::clonename *menu*

BWidget::focus *option path*

BWidget::get3dcolor *path bgcolor*

BWidget::getname *name*

BWidget::grab *option path*

BWidget::inuse *class*

BWidget::library *class* ?*class ...*?

BWidget::lreorder *list neworder*

BWidget::parsetext *text*

BWidget::place *path w h* ?*arg...*?

BWidget::write *filename* ?*mode*?

BWidget::wrongNumArgsString *string*

DESCRIPTION

Description text

COMMAND

BWidget::XLFDfont *cmd* ?*arg...*?
Description text

BWidget::assert *exp* ?*msg*?
Description text

BWidget::badOptionString *type value list*
Return a proper error string for a *value* of *type* that doesn't match *list*.

BWidget::bindMouseWheel *widget*
Bind the given *widget* with the standard mouse wheel bindings.

BWidget::classes *class*
Returns a list of classes needed by the given *class*.

BWidget::clonename *menu*
Description text

BWidget::focus *option path*
Description text

BWidget::get3dcolor *path bgcolor*
Description text

BWidget::getname *name*
Description text

BWidget::grab *option path*
Description text

BWidget::inuse *class*
Returns true or false if the given *class* is being used by the current running program.

BWidget::library *class ?class ...?*
Returns a string of code that contains all the libraries needed to use the widgets given by *class*. Each class's code and the code of its dependent classes is appended to the string and returned. This is mostly useful for saving BWidgets to another project.

BWidget::lreorder *list neworder*
Description text

BWidget::parsetext *text*
Description text

BWidget::place *path w h ?arg...?*
Used to position and resize the widget specified by *path*. *w* and *h* are used to specify the requested width and height of the *path* widget for use by **wm geometry** (set to 0 to use current values). The placement of the widget relative to other widgets or the screen is controlled by additional arguments:

at *x y*
Place the widget specified by the *path* argument at screen position x,y. See **wm geometry** for information about window placement values.

center*?widget?*
Place the *path* widget centered against *widget* or against the root widget if *widget* is not given.

right*?widget?*
Place the *path* widget either left or right of the reference widget (or the root widget if *widget* is not specified). If the reference widget's position is such that the newly placed window might be obscured then the opposite side will be tried.

below*?widget?*
As for left/right above, this option causes the widget to be placed either above or below the reference widget with the opposite placement being attempted if the newly placed widget will not be visible.

BWidget::write *filename ?mode?*
Writes the currently used set of BWidget class code to the given *filename*. All the code necessary to run the BWidgets currently in use is written to the file. This is mostly useful for saving BWidget code to another project as a single file instead of the entire BWidget package.

BWidget::wrongNumArgsString *string*
Returns a standard error string for the wrong number of arguments. *string* is appended to the standard string.

Keywords: BWidget

Button

NAME

Button - Button widget with enhanced options

CREATION

Button *pathName* ?*option value...*?

STANDARD OPTIONS

-activebackground	-activeforeground
-anchor	-background or -bg

WIDGET-SPECIFIC OPTIONS

-armcommand	-command
-default	-disarmcommand
-height	-helptext
-helptype	-helpvar
-name	-relief
-state	-underline
-width	

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName invoke

DESCRIPTION

Button widget extends the Tk button with new options. DynamicHelp options, a new relief style, callback for **arm**/**disarm**, and **repeatdelay**/**repeatinterval** options has been added.

WIDGET-SPECIFIC OPTIONS

-armcommand

Specifies a Tcl command to associate with the Button when mouse button 1 is pressed over the Button. When **repeatdelay** or **repeatinterval** option is positive integer, this command is repeatedly called if mouse pointer is over the Button and until mouse button 1 is released.

-command

Specifies a Tcl command to associate with the Button. This command is typically invoked when mouse button 1 is released over the Button window.

-default

Specifies one of three states for the default ring: **normal**, **active**, or **disabled**. In active state, the button is drawn with the platform specific appearance for a default button. In normal state, the button is drawn with the platform specific appearance for a non-default button, leaving enough space to draw the default button appearance. The normal and active states will result in buttons of the same size. In disabled state, the button is drawn with the non-default button appearance without leaving space for the default appearance. The disabled state may result in a smaller button than the active state.

-disarmcommand
Specifies a Tcl command to associate with the Button when mouse button 1 is released. This command is called even if pointer is not over the Button, and always before the command specified by **command** option. It is typically used in conjuntion with **armcommand**, **repeatdelay** and **repeatinterval**.

-height
Specifies a desired height for the Button. If an image or bitmap is being displayed in the Button then the value is in screen units; for text it is in lines of text. If this option isn't specified, the Button's desired height is computed from the size of the image or bitmap or text being displayed in it.

-helptext
Text for dynamic help. If empty, no help is available for this widget. See also DynamicHelp.

-helptype
Type of dynamic help. Use *balloon* or *variable*. See also DynamicHelp.

-helpvar
Variable to use when **helptype** option is *variable*. See also DynamicHelp.

-name
Specifies a standard name for the button. If the option ***nameName** is found in the resources database, then **text** and **underline** options are extracted from its value.

-relief
Specifies the 3-D effect desired for the widget. Acceptable values are standard values for button relief (**raised**, **sunken**, **flat**, **ridge**, **solid**, and **groove**) and **link**, which specifies that button relief is **flat** when pointer is outside the button and **raised** when pointer is inside.

-state
Specifies one of three states for the Button: **normal**, **active**, or **disabled**. In normal state the Button is displayed using the **foreground** and **background** options. The active state is typically used when the pointer is over the Button. In active state the Button is displayed using the **activeforeground** and **activebackground** options. Disabled state means that the Button should be insensitive: the default bindings will refuse to activate the widget and will ignore mouse button presses. In this state the **disabledforeground** and **background** options determine how the Button is displayed.

-underline
Specifies the integer index of a character to underline in the label of the button. 0 corresponds to the first character of the text displayed, 1 to the next character, and so on. The binding **<Alt-char>** is automatically set on the toplevel of the Button to call Button:: **setfocus**.

-width
If an image or bitmap is being displayed in the Button then the value is in screen units; for text it is in characters. If this option isn't specified, the Button's desired width is computed from the size of the image or bitmap or text being displayed in it.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* invoke**
If Button **state** is not disabled, this invoke the commands of the Button. Button is redisplayed with active color and sunken relief, and **armcommand** is called. Then Button is redisplayed with normal color and its defined relief, and **disarmcommand** then **command** are called.

invoke is called when Button has input focus and user press the space bar.

Keywords: Button

ButtonBox

NAME
ButtonBox - Set of buttons with horizontal or vertical layout

CREATION
ButtonBox *pathName* ?*option value...*?

WIDGET-SPECIFIC OPTIONS

-background or -bg	-default
-homogeneous	
-orient	
-padx	
-pady	
-spacing	
-state	

WIDGET COMMAND

pathName add ?*option value...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName delete *index*

pathName index *index*

pathName insert *index* ?*option value...*?

pathName invoke *index*

pathName itemcget *index option*

pathName itemconfigure *index* ?*option*? ?*value option value* ...?

pathName setfocus *index*

DESCRIPTION

ButtonBox layouts Button horizontally or vertically. Some commands take an *index* as argument indicating on which Button it work. This index may be specified in any of the following forms:

number
Specifies the Button numerically, where 0 corresponds to the first added Button, 1 to the next, and so on.

end or last
Indicates the last item added.

default
Indicates the default Button.

name
Indicates the button whose **-name** option is *name*.

text
Indicates the button whose **-text** option is *text*.

WIDGET-SPECIFIC OPTIONS

-background
Specifies a default background color for all added buttons and for the frame.

-default
Specifies the default button of the button box. The value is an integer referencing the n-th added button, starting from 0. If this value is -1 (the default), all button wil be drawn with their -default option set to disabled, and this value can not be changed. If this value is not -1, the associated button is drawn with -default option set to active and the others are drawn

with -default option set to normal. The value can be changed by configure.

-homogeneous (read-only)
Specifies wether or not buttons must have the same width for horizontal layout.

-orient (read-only)
Specifies the orientation of the button box. If this option is **horizontal** (the default), buttons are added from top to bottom. If this option is **vertical**, buttons are added from left to right.

-padx
Specifies a default value for the -padx option of all added buttons.

-pady
Specifies a default value for the -pady option of all added buttons.

-spacing
Specifies the default spacing between buttons. This value can be changed before each call to **add**.

-state
Specifies a state for all the buttons in the button box. Can be any state supported by buttons.

WIDGET COMMAND

***pathName* add ?*option value...*?**
Add a button to the button box.

See Button for description of options.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

pathName* delete *index
Delete the button at *index* from the button box.

pathName* index *index
Return the numerical index corresponding to the item.

***pathName* insert *index* ?*option value...*?**
Insert a new button into the button box before the given index.

pathName* invoke *index
Invoke the Button given by *index*.

pathName* itemcget *index option

Returns the current value of a configuration option for the item. *Option* may have any of the values accepted by the item creation command.

***pathName* itemconfigure *index* ?*option*? ?*value option value ...*?**
This command is similar to the **configure** command, except that it applies to the options for an individual item, whereas **configure** applies to the options for the widget as a whole. **Options** may have any of the values accepted by the item creation widget command. If options are specified, options are modified as indicated in the command and the command returns an empty string. If no options are specified, returns a list describing the current options for the item. Read-only options are not be modified.

pathName* setfocus *index
Set the focus to the Button given by *index*.

Keywords: ButtonBox

ComboBox

NAME
ComboBox - ComboBox widget

CREATION
ComboBox *pathName* ?*option value...*?

OPTIONS from ArrowButton

-background or -bg	-disabledforeground
-foreground or -fg	-state

OPTIONS from Entry

-command	-disabledforeground
-dragenabled	-dragendcmd
-dragevent	-draginitcmd
-dragtype	-dropcmd
-dropenabled	-dropovercmd
-droptypes	-editable
-entrybg (see -background)	-entryfg (see -foreground)
-exportselection	-font
-helptext	-helptype
-helpvar	-highlightbackground
-highlightcolor	-highlightthickness

-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-justify
-selectbackground	-selectborderwidth
-selectforeground	-show
-state	-takefocus
-text	-textvariable
-width	-xscrollcommand

WIDGET-SPECIFIC OPTIONS

-autocomplete	-autopost
-bwlistbox	-expand
-height	-hottrack
-images	-listboxwidth
-modifycmd	-postcommand
-values	

WIDGET COMMAND

pathName bind ?*arg...*?

pathName cget *option*

pathName clearvalue

pathName configure ?*option*? ?*value option value* ...?

pathName get

pathName getlistbox

pathName getvalue

pathName icursor *index*

pathName post

pathName setvalue *index*

pathName unpost

DESCRIPTION

ComboBox widget enables the user to select a value among a list given by the **values** option. The list of possible values can be popped by pressing the ArrowButton or by clicking in the entry when **editable** value of the ComboBox is **false**.
If **editable** value of the ComboBox is **true** and the entry has the focus, the user can press the top and bottom arrow keys to modify its value. If the current value exactly match a value in the list, then the previous (for top arrow key) or then next (for bottom arrow key) value in the list is displayed. If the current value match the beginning of a value in the list, then this value is displayed. If the current value doesnt match anything, then the first value is displayed.

WIDGET-SPECIFIC OPTIONS

-autocomplete
Specifies whether or not the combobox should attempt to auto-complete the value in the entry field as the user types. If true, the combobox will fill in a value that it finds in its value list as the user types into the entry.

-autopost
Specifies whether or not the combobox should post the drop down as the user types. If true, the combobox will post and scroll to the entry that most closely matches the user entry.

-bwlistbox
Specifies that the combobox should use a BWidget listbox in its drop down instead of the standard Tk option. This option is enabled by default if the **-images** option is not empty.

-expand
Specifies the value expansion behavior for the window. It must be **none** (default) or **tab**. If **tab** is specified, then a Tab binding is added to attempt to expand the current value based on the other values in **-values**.

-height
Specifies the desired height for the window, in lines. If zero or less, then the desired height for the window is made just large enough to hold all the elements in the listbox.

-modifycmd
Specifies a Tcl command called when the user modify the value of the ComboBox by selecting it in the listbox or pressing arrow key.

-postcommand
Specifies a Tcl command called before the listbox of the ComboBox is mapped.

-values
Specifies the values to display in the listbox of the ComboBox.

WIDGET COMMAND

***pathName* bind ?*arg...*?**
Set bindings on the entry widget.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* clearvalue**
Clears the current text of the ComboBox.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* get**
Returns the current contents of the entry.

***pathName* getlistbox**
Returns the path to the listbox in the drop down.

***pathName* getvalue**
Returns the index of the current text of the ComboBox in the list of values, or -1 if it doesn't match any value.

pathName* icursor *index
Arrange for the insertion cursor to be displayed just before the character given by *index*. Returns an empty string.

***pathName* post**
Post the drop down.

pathName* setvalue *index
Set the text of the ComboBox to the value indicated by *index* in the list of values. *index* may be specified in any of the following forms:

last
Specifies the last element of the list of values.

first
Specifies the first element of the list of values.

next
Specifies the element following the current (ie returned by **getvalue**) in the list of values.

previous
Specifies the element preceding the current (ie returned by **getvalue**) in the list of values.

@*number*
Specifies the integer index in the list of values.

***pathName* unpost**
Unpost the drop down.

BINDINGS

When Entry of the ComboBox has the input focus, it has the following bindings, in addition to the default Entry bindings:

- Page up set the value of the ComboBox to the first value.
- Page down set the value of the ComboBox to the last value.
- Arrow up set the value of the ComboBox to the previous value.
- Arrow down set the value of the ComboBox to the next value.
- If **-autopost** is enabled, Escape unposts the listbox.

If the listbox is not mapped and ComboBox is not editable or disabled, mouse button 1 on the Entry cause the listbox to popup, as if the user press the ArrowButton.

Keywords: ComboBox

Dialog

NAME

Dialog - Dialog abstraction with custom buttons

CREATION

Dialog *pathName* ?*option value...*?

OPTIONS from ButtonBox

-background or -bg	-homogeneous
-padx	-pady
-spacing	

WIDGET-SPECIFIC OPTIONS

-anchor
-bitmap
-cancel
-default
-geometry
-image
-modal
-parent
-place
-separator
-side

-title
-transient

WIDGET COMMAND

pathName add ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName draw ?*focus*?

pathName enddialog *result*

pathName getframe

pathName invoke *index*

pathName itemcget *index option*

pathName itemconfigure *index* ?*option*? ?*value option value* ...?

pathName setfocus *index*

pathName withdraw

DESCRIPTION

Dialog widget enables the user to create a dialog box. Some commands take an *index* as argument indicating on which Button it work. This index is the same specified for equivalent ButtonBox command:

number
Specifies the Button numerically, where 0 corresponds to the first added Button, 1 to the next, and so on.

end or last
Indicates the last item added.

default
Indicates the default Button.

WIDGET-SPECIFIC OPTIONS

-anchor (read-only)
Specifies the anchor point of the ButtonBox. Must be one of **w**, **e**, **n**, **s** or **c**. If **side** option is set to *top* or *bottom*, **anchor** values *n*, *s* and *c* have the same effect. If **side** option is set to

left or *right*, **anchor** values *w*, *e* and *c* have the same effect.

-bitmap (read-only)
Specifies a bitmap to display at the left of the user frame. **image** option override **bitmap**.

-cancel
Specifies the number of the cancel button of the Dialog. When user presses Esc or the windows close button, this button is invoked. If set to **-1**, Esc does not invoke anything. The window close button *destroys* the dialog in this case and returns **-1**.

-default
Specifies the number of the default button of the Dialog. When user press Return in the Dialog, this button is invoked.

-geometry
Set the **geometry** used when method **draw** is called. See **wm geometry** for a parameter description.

-image (read-only)
Specifies an image to display at the left of the user frame. **image** option override **bitmap**.

-modal
This option must be **none**, **local** or **global**. The value of this option specifies the grab mode of the dialog and how works Dialog::**draw**.

-parent
Parent of the Dialog. Dialog is placed relative to its parent. If empty, it is placed relative to the root window. Also see **place** option.

-place
Specifies where to draw the Dialog toplevel relative to the dialog's parent. Must be one of **none**, **center**, **left**, **right**, **above**, **below**. Default value of **place** is *center*.

-separator (read-only)
Specifies wether or not to draw a separator between the user frame and the ButtonBox.

-side (read-only)
Specifies where to draw the ButtonBox relative to the user frame. Must be one of **top**, **left**, **bottom** or **right**.

-title
Title of the Dialog toplevel.

-transient (read-only)
Specifies if the Dialog Toplevel should be a transient window or not. Default value of **transient** is *true*.

WIDGET COMMAND

***pathName* add ?*arg...*?**
Add a button to the button box of the dialog box. Default -command option is *Dialog::enddialog $path index* where *index* is number of button added.

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* draw ?*focus*?**
This command draw the Dialog, and set grab to it following **modal** option. If **modal** option is set to *none*, the command returns immediatly an empty string. In all other case, the command returns when Dialog::**enddialog** is called or when Dialog is destroyed. The return value is the result argument of Dialog::**enddialog** or -1 if it is destroyed.

By default, the focus is set to the default button referenced by **default** option, or to the toplevel of Dialog if no default button has been set. If *focus* is present, it must be a pathname, or an index to a button. Initial focus is set on this pathname or corresponding button.

pathName* enddialog *result
This command is typically called within a command of a button to make Dialog::**draw** return.

***pathName* getframe**
Returns the pathname of the user window.

pathName* invoke *index
Invoke the Button given by *index*.

pathName* itemcget *index option
Returns the current value of a configuration option for the item. *Option* may have any of the values accepted by the item creation command.

***pathName* itemconfigure *index* ?*option*? ?*value option value ...*?**
This command is similar to the **configure** command, except that it applies to the options for an individual item, whereas **configure** applies to the options for the widget as a whole. **Options** may have any of the values accepted by the item creation widget command. If options are specified, options are modified as indicated in the command and the command returns an empty string. If no options are specified, returns a list describing the current options for the item. Read-only options are not be modified.

pathName* setfocus *index
Set the focus to the Button given by *index*.

***pathName* withdraw**
Call this command to hide the dialog box.

Keywords: Dialog

DragSite

NAME

DragSite - Commands set for Drag facilities

COMMAND

DragSite::include *class type event*

DragSite::register *path* ?*option value...*?

DragSite::setdrag *path subpath initcmd endcmd* ?*force*?

DESCRIPTION

Commands of this namespace enable user to define a BWidget or a Tk widget as a drag site.

COMMAND

DragSite::include *class type event*

This command provides a simple way to include options relatives to a drag site into BWidget resources definition. It includes the options needed for **register**: *-dragevent*, initialized to *event*, *-draginitcmd* and *-dragendcmd*, initialized to empty string, and two new options:

-dragenabled	Specifies wether or not drag is active (initialized to 0)

DragSite::register *path* ?*option value...*?

This command is used to declare *path* as a drag site. Options are:

-dragendcmd

Command called when drag terminates (ie when user release drag icon). This command is called with the following arguments:

- the pathname of the drag source (the widget itself), - the pathname of the drop target, - the operation, - the type of the dragged data, - the dragged data, - result of the drop (result of the call to **-dropcmd** of the target),

If the drop does not occurs, the target and the operation are empty string and the result is 0.

-dragevent

Specifies the number of the mouse button associated to the drag. Must be **1**, **2** or **3**.

-draginitcmd

Command called when drag initiates. When the event of option **dragevent** occurs on *path*, this command is called with the following arguments:

- pathname of the drag source (*path*), - root x-coordinate of pointer, - root y-coordinate of pointer, - a toplevel created to represent dragged data. When returning, if it has no children, a bitmap is automatically displayed.

If the command returns an empty string, then the drag will be suppressed. Otherwise the command must return a list containing three elements:

- the type of the data,
- the list of acceptable basic operations (**copy**, **move** and **link**)

- and the data.

Note that even if **copy** does not appear in the list of basic operation, it is considered as an acceptable operation, since **copy** semantic does not modify the drag source.

DragSite::setdrag *path subpath initcmd endcmd ?force?*
This command provides a simple way to call **register** during a BWidget creation or configuration.

- *path* is the pathname of the BWidget,
- *subpath* is the pathname of the tk widget where drag event occurs,
- *initcmd* BWidget command for *drag-init* event,
- *endcmd* BWidget command for *drag-end* event,
- *force* specifies wether or not to call **register** whenever no option value has changed (0 by default - for BWidget configuration, use 1 for BWidget creation).

setdrag verifies the modification flag of options **dragenabled** and **dragevent** and calls **register** if needed according to the options values and *initcmd* and *endcmd* arguments. **draginitcmd** and **dragendcmd** are not taken from options of widget because they are considered as user command, called by BWidget implementation of *drag-init* and *drag-end* events.

Keywords: DragSite

DropSite

NAME

DropSite - Commands set for Drop facilities

COMMAND

DropSite::include *class types*

DropSite::register *path* ?*option value...*?

DropSite::setcursor *cursor*

DropSite::setdrop *path subpath dropover drop* ?*force*?

DropSite::setoperation *op*

DESCRIPTION

Commands of this namespace enable user to define a BWidget or a Tk widget as a drop site. A drop site is composed of the type of object that can be dropped and associated operation, a command called when drop occurs, and a command when an object is dragged over the widget. A drop site must have at least one type of acceptable object and a drop command.

COMMAND

DropSite::include *class types*
This command provides a simple way to include options relatives to a drop site into BWidget resources definition. It includes the options needed for **register**, *-dropovercmd* and *-dropcmd*

, initialized to empty string, and *-droptypes*, initialized to *types*, and one new option:

-dropenabled	Specifies wether or not drop is active (initialized to 0)

DropSite::register *path ?option value...?*

This command is used to declare *path* as a drop site. Options are:

-dropcmd

This command is called when user release the drag icon over a valid drop target widget. Arguments passed to the command are:

- pathname of the drop target (the widget itself), - pathname of the drag source, - root x-coordinate of the pointer, - root y-coordinate of the pointer, - operation, - type of the dragged data, - dragged data.

Its return values is passed as a result to the **-dragendcmd** command of the drag source widget.

-dropovercmd

This command can be used to provide a dynamic drag while *drag-over* events. While a drag occurs, the events <Enter>, <Motion> and <Leave> are caught and handled internally. Arguments passed to the command are:

- pathname of the drop target (the widget itself), - pathname of the drag source, - event over the drop target: *enter*, *motion* or *leave*, - root x-coordinate of the pointer, - root y-coordinate of the pointer, - operation, - type of the dragged data, - dragged data.

Command must the new status of the drag:

- 0 if widget refuse this drag. Command will not be recalled on motion/leave event.
- 1 if widget accept this drag. Command will not be recalled on motion/leave event.
- 2 if widget refuse this drag. Command will be recalled on each motion event to reevaluate.
- 3 if widget accept this drag. Command will be recalled on each motion event to reevaluate.

Here is a list of events and associated actions on a DropSite widget. This example assumes that dragged data type is valid for the drop target. **status** is the status of the drag on a DropSite. Its value is:

Event	Old status	Action	New status
<Enter>	-	if DropSite has dropovercmd, call it with enter	result of dropovercmd
else	1		
<Motion>	0 or 1		unchanged
2 or 3	call dropovercmd with motion	result of dropovercmd	
<Leave>	0 or 1		-
2 or 3	call dropovercmd with leave	-	

<Drop>	0	call dragendcmd of drag source	-
1	call dropcmd and call dragendcmd of drag source		
2	call dropovercmd with leave and call dragendcmd of drag source		
3	call dropcmd and call dragendcmd of drag source		

-droptypes
Specifies a list {*type oplist* ?*type oplist*? ...} of acceptable types and associated operations for the drop target. For each type, *oplist* is a list {*descops mod* ?*descops mod*? ...} describing operations and modifier keys for these operations. *descops* describe an operation. It can be a predefined operations (**copy**, **move** or **link**) or a new user defined operation, of the form { *subop* **baseop** ?*bitmap*?}. *subop* is the name given to the sub operation, *baseop* is the name of the base operation (**copy**, **move** or **link**) and *bitmap* is a bitmap to display for the operation. If *bitmap* is empty, the default bitmap of the base operation is used for the sub operation. *subop* can be a base operation, in order to change the bitmap of a base operation. In this case, *baseop* must be empty or equal to *subop*. *mod* is the modifer key for the operation. It can be:

- **none** to specify that no modifier key is pressed. This modifier can only be used with a sub operation named **default** (and vice versa), which has the behaviour of not display any bitmap operation. For all type, if the modifier **none** is not given, it is automatically associated to the **default** sub operation of a **copy** base operation. - **program** to specifies a sub operation accessible only by **DropSite::setoperation**. - A list combining **shift**, **control** and **alt**, which means their corresponding key.

DropSite::setcursor *cursor*
This command can be used within the script **dragovercmd**. It is usefull to provide visual effect about the state of the drag.

DropSite::setdrop *path subpath dropover drop* ?*force*?
This command provides a simple way to call **register** during a BWidget creation or configuration.

- *path* is the pathname of the BWidget,
- *subpath* is the pathname of the tk widget where drag event occurs,
- *dropover* is a command for *drag-over* event,
- *drop* is a command for *drop* event,
- *force* specifies wether or not to call **register** whenever no option value has changed (0 by default - for BWidget configuration, use 1 for BWidget creation).

setdrop verifies the modification flag of options **dropenabled** and **droptypes** and calls **register** if needed according to the options values and *dropover* and *drop* arguments.

dropovercmd and **dropcmd** are not taken from options of widget because they are considered as user command, called by BWidget implementation of *drag-over* and *drop* events.

DropSite::setoperation *op*
Description text

Keywords: DropSite

DynamicHelp

NAME

DynamicHelp - Provide help to Tk widget or BWidget

COMMAND

DynamicHelp::add *widget* ?*option value ...*?

DynamicHelp::configure ?*option*? ?*value option value* ...?

DynamicHelp::delete *widget*

DynamicHelp::include *class type*

DynamicHelp::register *path type* ?*arg...*?

DynamicHelp::sethelp *path subpath* ?*force*?

DESCRIPTION

Description text

COMMAND

DynamicHelp::add ?*option value ...*?
This command adds dynamic help to the given *widget*.

-command
If specified, refers to a command to execute to get the help text to display. The command must return a string to display. If the command returns an empty string, no help is displayed.

-index
If specified, refers to a menu index to bind the help to instead of a widget. If **-type** is not menu, this option is ignored.

-item
If specified, refers to an item or tag on a canvas widget or to a tag in a text widget to bind the help to instead of to a widget.

-type
Specifies the type of help. Can be: **balloon**, **variable** or **menu**. Default is **balloon**.

-text
The text to be displayed as help.

-variable

Specifies a variable name to which the help string will be written. Some other widget (e.g., a status bar) is responsible for displaying this variable.

Creating dynamic help for a menu is a two-step process. The menu itself must first be added and then each menu entry must be added separately. Here is a brief example.

```
# create menu
menu .m -type menubar
# associate menubar to toplevel BEFORE DynamicHelp::register
# to make it works with menu clone name
. configure -menu .m
.m add cascade -label "File" -menu .m.file
menu .m.file
.m.file add command -label "Open..."
.m.file add command -label "Quit"
# create label for help, using variable varinfo
label .l -textvariable varinfo
# associate all entries of menu .m.file to variable varinfo
DynamicHelp::add .m.file -type menu -variable varinfo
# then declare entries of .m.file
DynamicHelp::add .m.file -type menu -index 0 -text "Detach menu"
DynamicHelp::add .m.file -type menu -index 1 -text "Open a file"
DynamicHelp::add .m.file -type menu -index 2 -text "Exit demo"
```

DynamicHelp::configure ?*option*? ?*value option value* ...?

This command configure the ballon help.

-borderwidth

Width of the black border around the balloon.

-delay

Define the delay in millisecond of mouse inactivity before displaying the balloon.

-state

Specifies one of two states for help balloons: **normal** and **disabled**.

If **state** is **disabled**, help balloons will not be displayed for any registered widget.

-topbackground

The background color of the toplevel window created for a balloon.

Other standard options are:

-background or -bg	-font
-foreground or -fg	-justify
-padx	-pady

DynamicHelp::delete *widget*

Delete all dynamic help for the given *widget*.

DynamicHelp::include *class type*
Description text

DynamicHelp::register *path type ?arg...?*
Its use is deprecated. Use DynamicHelp::add instead.

Register a help text to the widget *path*. *type* determines the type of the help or the type of the widget. Depending on *type*, other options must be provided.

type	options
balloon	?tagOrItem? text
variable	?tagOrItem? varName text
menu	varName
menuentry	index text

If one of the option is missing or is empty, help is removed for this widget.

If *tagOrItem* is specified, then *path* is a canvas or a text. In case of a canvas, *tagOrItem* is the name of a tag or item on the canvas to which the help will be bound. In case of a text, *tagOrItem* is the name of a tag on the text to which the help will be bound.

For type other than *balloon*, *varName* is typically a variable linked to a label.
For menu, balloon type help is not available. To declare a help for menu, you first declare the menu, and then entries of this menu.
For example:

create menu menu .m -type menubar # associate menubar to toplevel BEFORE DynamicHelp::register # to make it works with menu clone name. configure -menu .m .m add cascade -label "File" -menu .m.file menu .m.file .m.file add command -label "Open..." .m.file add command -label "Quit" # create label for help, using variable varinfo label .l -textvariable varinfo # associate all entries of menu .m.file to variable varinfo DynamicHelp::register .m.file menu varinfo # then declare entries of .m.file DynamicHelp::register .m.file menuentry 0 "Detach menu" DynamicHelp::register .m.file menuentry 1 "Open a file" DynamicHelp::register .m.file menuentry 2 "Exit demo"

Notice that if popup menu is owned by a menubar, you must associate first the menubar to its toplevel. In this case, when you create a menu popup, its clone window is also created, and DynamicHelp::register detects the exitence of the clone window and maps events to it.

DynamicHelp::sethelp *path subpath ?force?*
Description text

Keywords: DynamicHelp

Entry

NAME

Entry - Entry widget with **state** option, dynamic help and drag and drop facilities

CREATION

Entry *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-disabledbackground	-disabledforeground
-exportselection	-font
-foreground or -fg	-highlightbackground
-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-justify
-relief	-selectbackground
-selectborderwidth	-selectforeground
-takefocus	-text
-textvariable	-xscrollcommand

WIDGET-SPECIFIC OPTIONS

-command	-dragenabled
-dragendcmd	-dragevent
-draginitcmd	-dragtype
-dropcmd	-dropenabled
-dropovercmd	-droptypes
-editable	-helptext
-helptype	-helpvar
-show	-state
-width	

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName invoke

DESCRIPTION

The **Entry** widget extends the default Tk entry. Options have been added to provide visual effect depending on the state of the Entry, DynamicHelp options, and Drag and Drop. Entry behaves much like a Label, with **text** option to set its contents.
Tk entry command can also be used on Entry widget.

WIDGET-SPECIFIC OPTIONS

-command
Specifies a command when user press <Return> in the Entry.

-dragenabled
A boolean specifying if drag is enabled.

-dragendcmd
Specifies a command to be called when drag ended. **dragendcmd** must be a command conforming to the description of the option **dragendcmd** of **DragSite::register**. If **dragendcmd** is empty, the internal *dragend* command updates the entry following the operation (**move** or **copy**) and the dragged data (whole or selected part of the entry).

-dragevent
Specifies the number of the mouse button associated to the drag. Must be **1**, **2** or **3**.

-draginitcmd
Specifies a command to be called when **dragevent** occurs on widget. **draginitcmd** must be a command conforming to the description of the option **draginitcmd** of **DragSite::register**. if **draginitcmd** is empty, the command refuse the drag if entry is empty or if portion of text is selected and event doesn't occur above the selection. In all other cases, the command returns:

- as the data type, the value of option **dragtype** or *TEXT* if empty,
- as the operations, *{copy move}* if **state** is normal and **editable** is true, or *{copy}* only in other cases,
- as the data, the whole content or the selected portion of the entry.

-dragtype
Specifies an alternate type of dragged object.

-dropcmd
Entry has a command wrapper for *drop* events. This command stops auto scrolling and extract current position. If **dropcmd** is not empty, it is called with the following arguments:

- the pathname of the Entry,
- the pathname of the drag source,
- the numeric index in the entry designated by the cursor,
- the current operation,
- the data type,
- the data.

and must return a value conforming to **dropcmd** option described in **DropSite::register**. If **dropcmd** is empty, the wrapper updates the entry following the type of data:

COLOR or FGCOLOR	reconfigure the foreground of the Entry
BGCOLOR	reconfigure the background of the Entry
TEXT, or any other tag	reconfigure the Entry to display the associated string.

and returns 1.

-dropenabled
A boolean specifying if drop is enabled.

-dropovercmd
Entry has a command wrapper for *drag-over* events. This command enables auto scrolling and position extraction during the *drag-over*. If **dropovercmd** is empty, the wrapper accepts the drop if **editable** option is true and **state** option is normal. If **dropovercmd** is not empty, it is called with the following arguments:

- the pathname of the Entry,
- the pathname of the drag source,
- the event,
- the numeric index in the entry designated by the cursor,
- the current operation,
- the data type,
- the data.

and must return a value conforming to **dropovercmd** option described in **DropSite::register**.

-droptypes
Specifies a list of accepted dropped object/operation. See option **droptypes** of **DropSite::register**. for more infromation. Default accepts *FGCOLOR*, *COLOR*, *BGCOLOR* and *TEXT*, all with **copy** and **move** operations.

-editable
Specifies whether the Entry is editable by the user. Equivalent to the **state** option of the Tk entry widget.

-helptext
Text for dynamic help. If empty, no help is available for this widget. See also DynamicHelp.

-helptype
Type of dynamic help. Use *balloon* or *variable*. See also DynamicHelp.

-helpvar
Variable to use when **helptype** option is *variable*. See also DynamicHelp.

-show
If this option is specified, then the true contents of the entry are not displayed in the window. Instead, each character in the entry's value will be displayed as the first character in the value of this option, such as ``*''. This is useful, for example, if the entry is to be used to enter a password. If characters in the entry are selected and copied elsewhere, the information copied will be what is displayed, not the true contents of the entry.

-state

Specifies one of two states for the Entry: **normal** or **disabled**. In normal state the text of the Entry is displayed using the **foreground** option. In disabled state the text of the Entry is displayed using the **disabledforeground** option. If the entry is disabled then the value may not be changed by user input and no insertion cursor will be displayed, even if the input focus is in the widget. Disabled state is the same as not editable with visual effect.

-width
Specifies an integer value indicating the desired width of the entry window, in average-size characters of the widget's font. If the value is less than or equal to zero, the widget picks a size just large enough to hold its current text.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* invoke**
Calls the command specified by the option **-command**.

Keywords: Entry

Label

NAME
Label - Label widget with **state** option, dynamic help and drag and drop facilities

CREATION
Label *pathName* ?*option value...*?

STANDARD OPTIONS

-anchor
-background or -bg
-bitmap
-borderwidth or -bd
-cursor
-disabledforeground

-font
-foreground or -fg
-highlightbackground
-highlightcolor
-highlightthickness
-image
-justify
-padx
-pady
-relief
-takefocus
-text
-textvariable
-wraplength

WIDGET-SPECIFIC OPTIONS

-dragenabled
-dragendcmd
-dragevent
-draginitcmd
-dragtype
-dropcmd
-dropenabled
-dropovercmd
-droptypes
-focus
-height
-helptext
-helptype

-helpvar
-name
-state
-underline
-width

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName setfocus

DESCRIPTION

The **Label** widget extends the default Tk label. Options have been added to provide visual effect depending on the state of the Label, DynamicHelp options, and Drag and Drop.

WIDGET-SPECIFIC OPTIONS

-dragenabled
A boolean specifying if drag is enabled.

-dragendcmd
Specifies a command to be called when drag ended. **dragendcmd** must be a command conforming to the description of the option **dragendcmd** of **DragSite::register**.

-dragevent
Specifies the number of the mouse button associated to the drag. Must be **1**, **2** or **3**.

-draginitcmd
Specifies a command to be called when **dragevent** occurs on widget. **draginitcmd** must be a command conforming to the description of the option **draginitcmd** of **DragSite::register**. If **draginitcmd** is empty, the internal **draginitcmd** command is used instead and returns:

IMAGE {copy} <image name>	if an image is displayed.

Note that if **dragtype** option is not empty, its value is used instead of those above.

-dragtype
Specifies an alternate type of dragged object.

-dropcmd
Specifies a command to be called when drop occurs on the widget. **dropcmd** must be a command conforming to the description of the option **dropcmd** of **DropSite::register**. If **dropcmd** is empty, the command updates the label following the type of the data:

COLOR or FGCOLOR	reconfigure the foreground of the Label.
BGCOLOR	reconfigure the background of the Label.
IMAGE	reconfigure the Label to display the associated image.
BITMAP	reconfigure the Label to display the associated bitmap. image option is set to empty.
TEXT, or any other tag	reconfigure the Label to display the associated string. image and bitmap options are set to empty.

and returns 1.

-dropenabled
A boolean specifying if drop is enabled.

-dropovercmd
Specifies a command to be called when drag icon is over the widget. **dropovercmd** must be a command conforming to the description of the option **dropovercmd** of **DropSite::register**. If **dropovercmd** is empty, Label always accepts the drop if data type is *FGCOLOR*, *COLOR*, *BGCOLOR*, and accepts all other data type only if **state** is normal.

-droptypes
Specifies a list of accepted dropped object/operation. See option **droptypes** of **DropSite::register**. for more infromation. Default accepts *FGCOLOR*, *COLOR*, *BGCOLOR*, *TEXT*, *BITMAP* and *IMAGE*, all with **copy** and **move** operations.

-focus
Specifies a pathname to set the focus on for Label::**setfocus** command.

-height
Specifies a desired height for the label. If an image or bitmap is being displayed in the label then the value is in screen units, for text it is in lines of text. If this option isn't specified, the label's desired height is computed from the size of the image or bitmap or text being displayed in it.

-helptext
Text for dynamic help. If empty, no help is available for this widget. See also DynamicHelp.

-helptype
Type of dynamic help. Use *balloon* or *variable*. See also DynamicHelp.

-helpvar
Variable to use when **helptype** option is *variable*. See also DynamicHelp.

-name
Specifies a standard name for the label. If the option ***nameName** is found in the resource database, then **text** and **underline** options are extracted from its value.

-state
Specifies one of two states for the Label: **normal** or **disabled**. In normal state the text of the Label is displayed using the **foreground** option. In disabled state the text of the Label is

displayed using the **disabledforeground** option.

-underline
Specifies the integer index of a character to underline in the label. 0 corresponds to the first character of the text displayed, 1 to the next character, and so on. The binding **<Alt-char>** is automatically set on the toplevel of the Label to call Label::**setfocus**.

-width
Specifies a desired width for the label. If an image or bitmap is being displayed in the label then the value is in screen units, for text it is in characters. If this option isn't specified, the label's desired width is computed from the size of the image or bitmap or text being displayed in it.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* setfocus**
Set the focus on the pathname given by **-focus** option if **-state** is *normal*.

Keywords: Label

LabelEntry

NAME
LabelEntry - LabelFrame containing an Entry widget.

CREATION
LabelEntry *pathName* ?*option value...*?

OPTIONS from Entry

-borderwidth or -bd	-command
-disabledforeground	-dragenabled
-dragendcmd	-dragevent
-draginitcmd	-dragtype
-dropcmd	-dropenabled

-dropovercmd	-droptypes
-editable	-entrybg (see -background)
-entryfg (see -foreground)	-exportselection
-font	-helptext
-helptype	-helpvar
-highlightbackground	-highlightcolor
-highlightthickness	-insertbackground
-insertborderwidth	-insertofftime
-insertontime	-insertwidth
-justify	-relief
-selectbackground	-selectborderwidth
-selectforeground	-show
-state	-takefocus
-text	-textvariable
-width	-xscrollcommand

OPTIONS from LabelFrame

-background or -bg	-disabledforeground
-foreground or -fg	-helptext
-helptype	-helpvar
-label (see -text)	-labelanchor (see -anchor)
-labelfont (see -font)	-labelheight (see -height)
-labeljustify (see -justify)	-labelwidth (see -width)
-name	-padx
-pady	-side
-state	-underline
-wraplength	

WIDGET COMMAND

pathName bind ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

LabelEntry is a widget composed of LabelFrame widget containing an Entry widget. Tk entry command can also be used on LabelEntry widget.

WIDGET COMMAND

***pathName* bind ?*arg...*?**
Set bindings on the entry widget.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

Keywords: LabelEntry

LabelFrame

NAME
LabelFrame - Frame with a Label

CREATION
LabelFrame *pathName* ?*option value...*?

STANDARD OPTIONS

-borderwidth or -bd	-relief

OPTIONS from Label

-anchor	-background or -bg
-bitmap	-disabledforeground
-focus	-font
-foreground or -fg	-height

-helptext	-helptype
-helpvar	-image
-justify	-name
-padx	-pady
-state	-text
-textvariable	-underline
-width	-wraplength

WIDGET-SPECIFIC OPTIONS

-side

WIDGET COMMAND

LabelFrame::align ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

DESCRIPTION

LabelFrame enables user to create a frame with a Label positionned at any side. LabelFrame is used by ComboBox and SpinBox.

WIDGET-SPECIFIC OPTIONS

-side (read-only)

Specifies where to position the Label relative to the user frame: **top**, **bottom**, **left** or **right**.

WIDGET COMMAND

LabelFrame::align ?*arg...*?

This command align label of all widget given by *args* of class LabelFrame (or "derived") by setting their width to the max one +1

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value* ...?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more

option-value pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getframe**
Return the frame where the user can create any other widget.

Keywords: LabelFrame

ListBox

NAME
ListBox - ListBox widget

CREATION
ListBox *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-cursor	-highlightbackground
-highlightcolor	-highlightthickness
-relief	-selectbackground
-selectforeground	-takefocus
-xscrollcommand	-yscrollcommand

WIDGET-SPECIFIC OPTIONS

-autofocus	-deltax	-deltay
-dragenabled	-dragendcmd	
-dragevent	-draginitcmd	
-dragtype	-dropcmd	
-dropenabled	-dropovercmd	
-dropovermode	-droptypes	
-height	-multicolumn	
-padx	-redraw	
-selectfill	-selectmode	-width

WIDGET COMMAND
pathName bindImage *event script*

pathName bindText *event script*

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName delete ?*arg...*?

pathName edit *item text* ?*verifycmd*? ?*clickres*? ?*select*?

pathName exists *item*

pathName index *item*

pathName insert *index item* ?*option value...*?

pathName item *first* ?*last*?

pathName itemcget *item option*

pathName itemconfigure *item* ?*option*? ?*value option value* ...?

pathName items ?*first*? ?*last*?

pathName move *item index*

pathName reorder *neworder*

pathName see *item*

pathName selection *cmd* ?*arg...*?

pathName xview ?*arg...*?

pathName yview ?*arg...*?

DESCRIPTION

ListBox widget uses canvas to display a list of items. Each item is composed of a label with its own font and foreground attributes, and an optional image or window. Each item is drawn in a single line, whose height is defined by the **deltay** option, so they must have at most this height. A item is uniquely identified by a string given at creation (by the **insert** command). The ListBox can have one or more columns, depending on **multicolumn** option. The user do not handle columns; the number of columns is determined following the height of the ListBox in order to see each item vertically.

WIDGET-SPECIFIC OPTIONS

-autofocus
If this option is true, the listbox will take focus any time the user clicks in it. Without focus, the listbox's mouse wheel bindings will not work properly. The default is true.

-deltax
Specifies horizontal pad between each columns.

-deltay
Specifies vertical size of the items.

-dragenabled
A boolean specifying if drag is enabled.

-dragendcmd
Specifies a command to be called when drag ended. **dragendcmd** must be a command conforming to the description of the option **dragendcmd** of **DragSite::register**.

-dragevent
Specifies the number of the mouse button associated to the drag. Must be **1**, **2** or **3**.

-draginitcmd
ListBox has a command wrapper for *drag-init* events. This command refused the drag if no item is designated. In other cases: If **draginitcmd** is empty, it returns:

- the value of option **dragtype** or *LISTBOX_ITEM* if empty as the data type,
- *{move copy link}* as the operations,
- the item identifier as the data.

If **draginitcmd** is not empty, it is called with the following arguments:

- the pathname of the listbox,
- the identifier of the dragged item,
- the toplevel created to represent dragged data.

and must return a value conforming to **draginitcmd** option described in **DragSite::register**.

-dragtype
Specifies an alternate type of dragged object.

-dropcmd
ListBox has a command wrapper for *drop* events. This command stops auto scrolling and extract item and position. If **dropcmd** is not empty, it is called with the following arguments:

- the pathname of the listbox,
- the pathname of the drag source,
- a list describing where the drop occurs. It can be:
- the current operation,
- the data type,
- the data.
  - *{***widget***}*,
  - *{***item** *item}* or
  - *{***position** *index}*.

The default drop command allows for drag-and-drop within the listbox but not to or from other

widgets.

-dropenabled
A boolean specifying if drop is enabled.

-dropovercmd
LsitBox has a command wrapper for *drag-over* events. This command enables auto scrolling and position extraction during the *drag-over*. If **dropovercmd** is not empty, the command is called with the following aguments:

- the pathname of the listbox,
- the pathname of the drag source,
- a list describing where the drop can occur, whose elements are:
- the current operation,
- the data type,
- the data.
  - the string *widget* if **dropovertype** option contains *w*, else empty string.
  - the targeted item if drag icon points an item and **dropovertype** option contains *i*, else empty string.
  - an index within two items where drag icon points to if **dropovertype** option contains *p*, else empty string.
  - optionally, the preferred method if drop can occur both inside an item and between two items. The value is *position* or *item*.

The command must return a list with two elements:

- the drop status, conforming to those described in **dropovercmd** option of **DropSite::register** ,
- the choosen method: *widget*, *item* or *position*.

-dropovermode
Specifies the type of *drop-over* interaction. Must be a combination of **w**, which specifies that drop can occurs everywhere on widget, **p**, which specifies that drop can occurs between two items, and **i**, which specifies that drop occurs inside items.

-droptypes
Specifies a list of accepted dropped object/operation. See option **droptypes** of **DropSite::register**. for more infromation. Default is *LISTBOX_ITEM* with operations **copy** and **move**.

-height
Specifies the desired height for the listbox in units of **deltay** pixels.

-multicolumn
Specifies wether or not ListBox layouts items in order to see each one vertically.

-padx
Specifies distance between image or window and text of the items.

-redraw
Specifies wether or not the listbox should be redrawn when entering idle. Set it to false if you call **update** while modifying the listbox.

-selectfill

If true, the listbox will draw a selection rectangle that fills the listbox from left-to-right instead of just drawing a box around the selected item. This more closely mimics the standard Tk listbox.

-selectmode
Specifies the desired selection-mode for the listbox. Must be one of **none**, **single** or **multiple** . **selectmode** *single* allows to select 1 item by its text or image. **selectmode** *multiple* allows to select multiple items by their text or image. For more info on selectmodes *single* or *multiple*, see the Tk **listbox** command. Default value for **selectmode** is *none*.

-width
Specifies the desired width for the listbox in units of 8 pixels.

WIDGET COMMAND

pathName* bindImage *event script
This command associates a command to execute whenever the event sequence given by *event* occurs on the image of a item. The item idenfier on which the event occurs is appended to the command.

Any occurrence of **%W** in *script* is substituted with the path of the listbox.

pathName* bindText *event script
This command associates a command to execute whenever the event sequence given by *event* occurs on the label of a item. The item idenfier on which the event occurs is appended to the command.

Any occurrence of **%W** in *script* is substituted with the path of the listbox.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* delete ?*arg...*?**
Deletes all items in *arg*. *arg* can be a list of items or a list of list of items. To delete all items, do *$pathName delete [$pathName items]*.

***pathName* edit *item text* ?*verifycmd*? ?*clickres*? ?*select*?**
Provides a way for the user to edit in place the label of an item. The command takes the initial text as argument and does not modify the label of the edited node, but returns an empty string if edition is canceled, or the typed text if edition is accepted. When editing, the user can cancel by pressing Escape, or accept by pressing Return. *clickres* specifies what to do if the user click outside the editable area. If *clickres* is 0 (the default), the edition is canceled. If

clickres is 1, the edition is accepted. In all other case, the edition continues. If edition is accepted and *modifycmd* is not empty, then it is called with the new text as argument and must return 1 to accept the new text, 0 to refuse it and continue edition. *select* specifies wether or not the initial text should be selected. Default is 1.

pathName* exists *item
Returns 1 if *item* exists in the listbox, else 0.

pathName* index *item
Returns the position of *item* in the list.

***pathName* insert *index item* ?*option value...*?**
Inserts a new item identified by *item* in the list at position *index*.

Any instance of *#auto* within the item name will be replaced by the number of the item in the order of insertion.

-data
User data associated to the item.

-fill
Specifies the foreground color of the label of the item.

-font
Specifies a font for the label of the item.

-image
Specifies an image to display at the left of the label of the item. **window** option override **image**.

-indent
Specifies the amount of extra space in pixels at the left of the item.

-text
Specifies the label of the item.

-window
Specifies a pathname to display at the left of the label of the item. **window** option override **image**.

***pathName* item *first* ?*last*?**
Its use is deprecated. Use items instead. If *last* is omitted, returns the item at index *first* in the list, or an empty string if *first* refers to a non-existent element. If *last* is specified, the command returns a list whose elements are all of the items between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

pathName* itemcget *item option
Returns the current value of a configuration option for the item. *Option* may have any of the values accepted by the item creation command.

***pathName* itemconfigure *item* ?*option*? ?*value option value* ...?**
This command is similar to the **configure** command, except that it applies to the options for an individual item, whereas **configure** applies to the options for the widget as a whole. **Options** may have any of the values accepted by the item creation widget command. If options are specified, options are modified as indicated in the command and the command

returns an empty string. If no options are specified, returns a list describing the current options for the item. Read-only options are not be modified.

***pathName* items ?*first*? ?*last*?**
If *first* and *last* are omitted, returns the list of all items. If *first* is specified and *last* omitted, returns the item at index *first*, or an empty string if *first* refers to a non-existent element. If *first* and *last* are specified, the command returns a list whose elements are all of the items between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

pathName* move *item index
Moves *item* at position *index* in the list.

pathName* reorder *neworder
Modifies the order of items in the listbox given by *neworder*. Items that do not appear in *neworder* are no moved.

pathName* see *item
Arrange the scrolling area to make *item* visible.

***pathName* selection *cmd* ?*arg...*?**
Modifies the list of selected items following *cmd*:

clear
remove all items of the selection.

set
set the selection to all items in *arg*

add
add all items of *arg* in the selection

remove
remove all items of *arg* of the selection

get
return the current selected items

***pathName* xview ?*arg...*?**
Standard command to enable horizontal scrolling of *pathName*.

***pathName* yview ?*arg...*?**
Standard command to enable vertical scrolling of *pathName*.

BINDINGS

A **<<ListboxSelect>>** virtual event is generated any time the selection in the listbox changes.

The listbox has all the standard mouse wheel bindings when it has focus.

Keywords: ListBox

MainFrame

NAME
MainFrame - Manage toplevel with menu, toolbar and statusbar

CREATION
MainFrame *pathName* ?*option value...*?

OPTIONS from ProgressBar

-background or -bg	-progressfg (see -foreground)
-progressmax (see -maximum)	-progresstype (see -type)
-progressvar (see -variable)	

WIDGET-SPECIFIC OPTIONS

-height
-menu
-separator
-textvariable
-width

WIDGET COMMAND

pathName addindicator ?*arg...*?

pathName addtoolbar

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

pathName getindicator *index*

pathName getmenu *menuid*

pathName gettoolbar *index*

pathName setmenustate *tag state*

pathName showstatusbar *name*

pathName showtoolbar *index bool*

DESCRIPTION

MainFrame manage toplevel to have:

- simple menu creation, with automatic accelerator bindings and DynamicHelp association,
- one or more toolbars that user can hide,

- a status bar, displaying a user message or a menu description, and optionally a ProgressBar.

WIDGET-SPECIFIC OPTIONS

-height
Specifies the desired height for the user frame in any of the forms acceptable to Tk_GetPixels. If this option is less than or equal to zero (the default) then the window will not request any size at all.

-menu (read-only)
This option describes the menu. This is a list whose each five elements describe one cascade menu. It has the following form: {*menuname tags menuId tearoff menuentries*...} where *menuentries* is a list where each element describe one menu entry, which can be:

- for a separator: {**separator**}
- for a command: {**command** *menuname* ?*tags*? ?*description*? ?*accelerator*? ?*option value*? ...}
- for a check button: {**checkbutton** *menuname* ?*tags*? ?*description*? ?*accelerator*? ?*option value* ? ...}
- for a radio button: {**radiobutton** *menuname* ?*tags*? ?*description*? ?*accelerator* ?*option value*? ...}
- for a cascade menu: {**cascade** *menuname tags menuId tearoff menuentries*}

where:

- *menuname* is the name of the menu. If it contains a &, the following character is automatically converted to the corresponding **-underline** option of **menu add** command.
- *tags* is the tags list for the entry, used for enabling or disabling menu entries with **MainFrame::setmenustate**.
- *menuId* is an id for the menu, from which you can get menu pathname with **MainFrame::getmenu**.
- *tearoff* specifies if menu has tearoff entry.
- *description* specifies a string for DynamicHelp.
- *accelerator* specifies a key sequence. It is a list of two elements, where the first is one of **Ctrl**, **Alt** or **CtrlAlt**, and the second as letter or a digit. An accelerator string is build and corresponding binding set on the toplevel to invoke the menu entry.
- *option value* specifies additionnal options for the entry (see **menu add** command).

Each value enclosed by ? are optional and defaulted to empty string, but must be provided if one or more following options is not empty.
Example:

```
set descmenu {
    "&File" {} {} 0 {
        {command "&New"     {} "Create a new document"     {Ctrl n}
    -command Menu::new}
        {command "&Open..." {} "Open an existing document" {Ctrl o}
    -command Menu::open}
        {command "&Save"    open "Save the document" {Ctrl s} -command
    Menu::save}
        {cascade  "&Export"  {} export 0 {
```

```
            {command "Format &1" open "Export document to format 1" {}
    -command {Menu::export 1}}
            {command "Format &2" open "Export document to format 2" {}
    -command {Menu::export 2}}
        }}
        {separator}
        {cascade "&Recent files" {} recent 0 {}}
        {separator}
        {command "E&xit" {} "Exit the application" {} -command
    Menu::exit}
    }
    "&Options" {} {} 0 {
        {checkbutton "Toolbar" {} "Show/hide toolbar" {}
            -variable Menu::_drawtoolbar
            -command  {$Menu::_mainframe showtoolbar toolbar
    $Menu::_drawtoolbar}
        }
    }
}
```

-separator (read-only)
Specifies if separator should be drawn at the top and/or at the bottom of the user window. Must be one of the values **none**, **top**, **bottom** or **both**. It depends on the relief of subwidgets of user window.

-textvariable
Specifies the textvariable option for the label of the status bar. DynamicHelp description of menu entries are mapped to this variable at the creation of the MainFrame. If this variable is changed by MainFrame::configure, menu description will not be available. You change the text of the label by modifying the value of the variable.

-width
Specifies the desired width for the user frame in any of the forms acceptable to Tk_GetPixels. If this option is less than or equal to zero (the default) then the window will not request any size at all.

WIDGET COMMAND

***pathName* addindicator ?*arg...*?**
Add an indicator box at the right of the status bar. Each indicator are added from left to right. An indicator is a Tk label widget configured with option-value pair given by ?*arg...*?. **-relief** and **-borderwidth** options are respetively defaulted to *sunken* and 1. Returns the pathname of the created label.

***pathName* addtoolbar**
Add a toolbar to the MainFrame. Returns the pathname of the new window where to place toolbar items.

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getframe**
Returns the pathname of the user window.

pathName* getindicator *index
Returns the pathname of the *index*th added indicator.

pathName* getmenu *menuid
Returns the pathname of the menu whose id is *menuid*.

pathName* getmenustate *tag state
Returns the state of the given menu *tag*.

pathName* gettoolbar *index
Returns the pathname of the *index*th added toolbar.

pathName* setmenustate *tag state
Set the **-state** option value of all the menu entries that have the tag *tag* to *state*. A menu entry is disabled, if one of its associated tags have state **disabled**.

pathName* showstatusbar *name
name is one of **none**, **status** or **progression**. Use **none** to hide the status bar, **status** to display the label only, or **progression** to display the label and the ProgressBar.

pathName* showtoolbar *index bool
Hide if *bool* is 0, or show if *bool* is 1 the *index*th added toolbar. To prevent your toplevel from resizing while hiding/showing toolbar, do [wm geometry $top [wm geometry $top]] when it is managed.

Keywords: MainFrame

MessageDlg

NAME

MessageDlg - Message dialog box

CREATION

MessageDlg *pathName* ?*option value...*?

STANDARD OPTIONS

-anchor	-font

-foreground or -fg	-padx
-pady	

OPTIONS from Dialog

-background or -bg	-cancel
-default	-parent

WIDGET-SPECIFIC OPTIONS

-aspect	-buttons
-buttonwidth	-icon
-justify	-message
-title	-type
-width	

DESCRIPTION

MessageDlg provides a simple way to display a message dialog. MessageDlg::**create** creates the message dialog, displays it and return the index of the pressed button, or -1 if it is destroyed. When returning, the dialog no longer exists.

WIDGET-SPECIFIC OPTIONS

-aspect
Specifies a non-negative integer value indicating desired aspect ratio for the text. The aspect ratio is specified as 100*width/height. 100 means the text should be as wide as it is tall, 200 means the text should be twice as wide as it is tall, 50 means the text should be twice as tall as it is wide, and so on. Used to choose line length for text if **width** option isn't specified. Defaults to 150.

-buttons
Specifies a list of buttons to display when **type** option is *user*. If a button has a symbolic name, its associated text will be displayed.

-buttonwidth
Specifies the standard width of the buttons in the dialog.

-icon
Specifies an icon to display. Must be one of the following: **error**, **info**, **question** or **warning**.

-justify
Specifies how to justify lines of text. Must be one of **left**, **center**, or **right**. Defaults to **left**.

This option works together with the **anchor**, **aspect**, **padx**, **pady**, and **width** options to provide a variety of arrangements of the text within the window. The **aspect** and **width** options determine the amount of screen space needed to display the text. The **anchor**, **padx**, and **pady** options determine where this rectangular area is displayed within the widget's window, and the **justify** option determines how each line is displayed within that rectangular region. For example, suppose **anchor** is **e** and **justify** is **left**, and that the message window is much larger than needed for the text. The the text will displayed so that the left edges of all the lines line up and the right edge of the longest line is **padx** from the right side of the window; the entire text block will be centered in the vertical span of the window.

-message
Specifies the message to display in this message box.

-title
Specifies a string to display as the title of the message box. If the value is empty (the default), a default title will be set corresponding to the **icon** option. The default associated title is in english, and can be modified to set it in another language by specifying the resource:

```
    *MessageDlg.nameTitle:   value
```

or the equivalent tcl command:

```
    option add *MessageDlg.nameTitle value
```

where *name* is the name of an icon as defined in the **icon** option.
For example, for french language, you can specify for a warning dialog:

```
    option add *MessageDlg.warningTitle  "Attention"
```

-type
Specifies a set of buttons to be displayed. The following values are possible:

abortretryignore
Displays three buttons whose symbolic names are **abort**, **retry** and **ignore**.

ok

Displays one button whose symbolic name is **ok**.

okcancel

Displays two buttons whose symbolic names are **ok** and **cancel**.

retrycancel

Displays two buttons whose symbolic names are **retry** and **cancel**.

yesno

Displays two buttons whose symbolic names are **yes** and **no**.

yesnocancel

Displays three buttons whose symbolic names are **yes**, **no** and **cancel**.

user

Displays buttons of **-buttons** option.

-width
Specifies the length of lines in the window. If this option has a value greater than zero then the **aspect** option is ignored and the **width** option determines the line length. If this option has a value less than or equal to zero, then the **aspect** option determines the line length.

Keywords: MessageDlg

NoteBook

NAME

NoteBook - Notebook manager widget

CREATION

NoteBook *pathName* ?*option value...*?

STANDARD OPTIONS

-font

OPTIONS from ArrowButton

-activebackground	-activeforeground
-background or -bg	-borderwidth or -bd
-disabledforeground	-foreground or -fg
-repeatdelay	-repeatinterval

WIDGET-SPECIFIC OPTIONS

-arcradius	-height
-homogeneous	-side
-tabbevelsize	-tabpady
-width	

WIDGET COMMAND

pathName bindtabs *event script*

pathName cget *option*

pathName compute_size

pathName configure ?*option*? ?*value option value ...*?

pathName delete *page* ?*destroyframe*?

pathName getframe *page*

pathName index *page*

pathName insert *index page* ?*option value...*?

pathName itemcget *page option*

pathName itemconfigure *page* ?*option*? ?*value option value* ...?

pathName move *page index*

pathName page *first* ?*last*?

pathName pages ?*first*? ?*last*?

pathName raise ?*page*?

pathName see *page*

DESCRIPTION

NoteBook widget manage a set of pages and displays one of them.

WIDGET-SPECIFIC OPTIONS

-arcradius
Specifies the amount of rounding effect at the corners of a tab. This value can be adjusted from 0 to 8 pixels with the default being 2 pixels. Usually, small values are preferable.

-height
Specifies the desired height for the pages. If this option is equal to zero (the default) then the window will not request any size at all. In this case, user may want to call NoteBook:: **compute_size** to make NoteBook larger enough to contains the largest page.

-homogeneous
Specifies wether or not the label of the pages must have the same width.

-side
Specifies the side where to place the label of the pages. Must be one of **top** or **bottom**.

-tabbevelsize
Specifies the amount of bevel the tabs should have. This value can be adjusted from 0 to 8 pixels with the default being 0 pixels. A zero pixel bevel is essentially rectangular while non-zero bevel size will look trapezoidal.

-tabpady
Specifies the padding between the text in the tab and the top and bottom of the tab. Padding may be a list of two values to specify padding for top and bottom separately. Padding defaults to {0 6}.

-width
Specifies the desired width for the pages. If this option is equal to zero (the default) then the

window will not request any size at all. In this case, user may want to call NoteBook:: **compute_size** to make NoteBook larger enough to contains the largest page.

WIDGET COMMAND

pathName* bindtabs *event script
This command associates a command to execute whenever the event sequence given by *event* occurs on a tab. The page identifier on which the event occurs is appended to the command.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* compute_size**
This command can be called to make the NoteBook large enough to contain the largest page. Note that if all pages use -createcmd, they will have no requested size.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* delete *page* ?*destroyframe*?**
Deletes the page *page*. If *destroyframe* is 1 (the default), the frame associated to *page* is destroyed. If *destroyframe* is 0, the frame is not destroyed and is reused by further call to **insert** with the same *page*.

pathName* getframe *page
Returns the pathname of the page *page*.

pathName* index *page
Return the numerical index corresponding to the item.

***pathName* insert *index page* ?*option value...*?**
Insert a new page idendified by *page* at position *index* in the pages list. *index* must be numeric or **end**. The pathname of the new page is returned.

-createcmd
Specifies a command to be called the first time the page is raised.

-image
Specifies an image to display for the page at the left of the label

-leavecmd
Specifies a command to be called when a page is about to be leaved. The command must return 0 if the page can not be leaved, or 1 if it can.

-raisecmd

Specifies a command to be called each time the page is raised.

-state
Specifies the state of the page. Must be **normal** or **disabled**.

-text
Specifies a label to display for the page.

pathName* itemcget *page option
Returns the current value of a configuration option for the item. *Option* may have any of the values accepted by the item creation command.

***pathName* itemconfigure *page* ?*option*? ?*value option value ...*?**
This command is similar to the **configure** command, except that it applies to the options for an individual item, whereas **configure** applies to the options for the widget as a whole. **Options** may have any of the values accepted by the item creation widget command. If options are specified, options are modified as indicated in the command and the command returns an empty string. If no options are specified, returns a list describing the current options for the item. Read-only options are not be modified.

pathName* move *page index
Moves *page* tab to index *index*.

***pathName* page *first* ?*last*?**
Its use is deprecated. Use pages instead. If *last* is omitted, returns the page at index *first* , or an empty string if *first* refers to a non-existent element. If *last* is specified, the command returns a list whose elements are all of the pages between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

***pathName* pages ?*first*? ?*last*?**
If *first* and *last* are omitted, returns the list of all pages. If *first* is specified and *last* omitted, returns the page at index *first*, or an empty string if *first* refers to a non-existent element. If *first* and *last* are specified, the command returns a list whose elements are all of the pages between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

***pathName* raise ?*page*?**
Raise the page *page*, or return the raised page if *page* is omitted.

pathName* see *page
Scrolls labels to make the label of the page *page* visible.

Keywords: NoteBook

PagesManager

NAME
PagesManager - Pages manager widget

CREATION
PagesManager *pathName* ?*option value...*?

STANDARD OPTIONS
-background

WIDGET-SPECIFIC OPTIONS

-height

WIDGET COMMAND

pathName add *page*

pathName cget *option*

pathName compute_size

pathName configure ?*option*? ?*value option value* ...?

pathName delete *page*

pathName getframe *page*

pathName page *first* ?*last*?

pathName pages ?*first*? ?*last*?

pathName raise ?*page*?

DESCRIPTION

PagesManager widget manages a set of pages and displays one of them. PagesManager does not provide any user access method, as NoteBook does, so it can be done through a listbox, a menu, radiobutton, or whatever. The widget shows no pages during creation; there must be an explicit call to **raise** to display one.

WIDGET-SPECIFIC OPTIONS

-height

Specifies the desired height for the pages. If this option is equal to zero (the default) then the window will not request any size at all. In this case, user may want to call PagesManager:: **compute_size** to make PagesManager larger enough to contains the largest page.

-width

Specifies the desired width for the pages. If this option is equal to zero (the default) then the window will not request any size at all. In this case, user may want to call PagesManager:: **compute_size** to make PagesManager larger enough to contains the largest page.

WIDGET COMMAND

pathName* add *page

Add a new page identified by *page*, which is an arbitrary identifier. The pathname of the new page is returned and widgets for the page should be created with this as the parent or ancestor. However, PagesManager manages its own geometry. **pack**, **grid** or an equivalent should not be used with the pathname returned by **add**.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* compute_size**
This command can be called to make the PagesManager large enough to contain the largest page.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

pathName* delete *page
Deletes the page *page*.

pathName* getframe *page
Returns the pathname of the page *page*.

***pathName* page *first* ?*last*?**
Its use is deprecated. Use pages instead. If *last* is omitted, returns the page at index *first* , or an empty string if *first* refers to a non-existent element. If *last* is specified, the command returns a list whose elements are all of the pages between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

***pathName* pages ?*first*? ?*last*?**
If *first* and *last* are omitted, returns the list of all pages. If *first* is specified and *last* omitted, returns the page at index *first*, or an empty string if *first* refers to a non-existent element. If *first* and *last* are specified, the command returns a list whose elements are all of the pages between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

***pathName* raise ?*page*?**
Raise the page *page*, or return the raised page if *page* is omitted.

Keywords: PagesManager

PanedWindow

NAME
PanedWindow - Tiled layout manager widget

CREATION
PanedWindow *pathName* ?*option value...*?

STANDARD OPTIONS
-background or -bg

WIDGET-SPECIFIC OPTIONS

-activator	-pad
-side	-weights
-width	

WIDGET COMMAND

pathName add ?*option value...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe *index*

DESCRIPTION

PanedWindow is a widget that lays out children in a vertically or horizontally tiled format. The user can adjust the size of the panes, with a pane control sash created between children.

WIDGET-SPECIFIC OPTIONS

-activator (read-only)
By default, the control sash is either a button or a line depending upon the underlying operating system. This sash may be explicitly set using this option. If set to **line** then the sash consists of just a dividing line. Otherwise if set to **button** then it constists of a knob that can be dragged.

-pad (read-only)
Specifies additional space between the button of the sash and children.

-side (read-only)
Specifies the side of the sash, which implies the layout: **top** or **bottom** (horizontal layout), **left** or **right** (vertical layout).

-weights (read-only)
Specifies how the weights specified when adding panes should be used. Must be **extra** or **available**. When using *extra*, only extra space is devided among the diffferent panes relative to their weight. When using *available*, all space is devided among the diffferent panes relative to their weight. Default value for **weights** is *extra*.

-width (read-only)
Specifies the width of the button of the sash. This option is ignored if the activator is set to **line**.

WIDGET COMMAND

***pathName* add ?*option value...*?**
This command add a new pane. The new pane is placed below the previous pane for vertical

layout or at right for horizontal layout. This command returns a frame where user can place its widget. Valid options are:

-minsize
Specifies the minimum size requested for the pane. See the **grid** command for more information.

-weight
Specifies the relative weight for apportioning any extra spaces among panes. See the **grid** command for more information.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

pathName* getframe *index
Returns the pathname of the *index*th added pane.

Keywords: PanedWindow

PanelFrame

NAME
PanelFrame - Frame with a boxed title area

CREATION
PanelFrame *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-text	
-textvariable	
-font	-relief

WIDGET-SPECIFIC OPTIONS

-height	
-width	

-panelbackground	-panelforeground
-ipad	

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

PanelFrame creates a frame area with a boxed title area. The boxed title area contains a label ands allows for other items to be placed in it. This can serve like a mini-toolbar.

WIDGET-SPECIFIC OPTIONS

-height
Specifies the desired height for the widget.

-ipad
The spacing to place around individual panel area items.

-panelbackground
The color for the panel area background. Defaults to the selection highlight background color.

-panelforeground
The color for the title text. Defaults to the selection highlight foreground color.

-width
Specifies the desired width for the widget.

WIDGET COMMAND

***pathName* add widget ?*option value option value ...*?**
Add a widget to the panel. Widgets are **pack**ed in. Possible options are:

-side
Side to place item on (defaults to **right**).

-fill
Whether to fill space (defaults to **none**).

-expand
Whether to expand space (defaults to **0**).

-pad
Override of the widget's **-ipad** option for this item.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then

the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* delete widget ?*widget* ...?**
Delete a widget and associated state from the panel.

***pathName* getframe**
Get the frame widget for the status bar in which status bar items should be created.

***pathName* items**
List of items in the status bar.

Keywords: PanelFrame

PasswdDlg

NAME

PasswdDlg - Login/Password dialog box

CREATION

PasswdDlg *pathName* ?*option value...*?

OPTIONS from Dialog

-anchor	-background or -bg
-homogeneous	-modal
-padx	-pady
-parent	-spacing
-title	

OPTIONS from LabelEntry

-background or -bg	-borderwidth or -bd
-disabledforeground	-entrybg
-entryfg	-exportselection
-font	-foreground or -fg
-helptype	-highlightbackground
-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime

-insertwidth	-labelanchor
-labelfont	-labelheight
-labeljustify	-labelwidth
-loginhelptext (see -helptext)	-loginhelpvar (see -helpvar)
-loginlabel (see -label)	-logintext (see -text)
-logintextvariable (see -textvariable)	-loginunderline (see -underline)
-passwdeditable (see -editable)	-passwdhelptext (see -helptext)
-passwdhelpvar (see -helpvar)	-passwdlabel (see -label)
-passwdstate (see -state)	-passwdtext (see -text)
-passwdtextvariable (see -textvariable)	-passwdunderline (see -underline)
-relief	-selectbackground
-selectborderwidth	-selectforeground
-wraplength	

WIDGET-SPECIFIC OPTIONS

-command
-type

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

PasswdDlg provides a simple way to display a login/password dialog. PasswdDlg::**create** creates the dialog, displays it, and return the value of login and password in a list, or an empty list if it is destroyed or user press cancel. When returning, the dialog no longer exists.
Additionnal resources can be set to modify other text:

```
    *loginName     Label for login LabelEntry
    *passwordName  Label for password LabelEntry
```

WIDGET-SPECIFIC OPTIONS

-command

Specifies a command to call when user press ok button.

-type
Specifies a set of buttons to be displayed. The following values are possible:

ok
Displays one button whose symbolic name is **ok**.

okcancel

Displays two buttons whose symbolic names are **ok** and **cancel**.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

Stephane Lavirotte

(Stephane.Lavirotte@sophia.inria.fr)

Keywords: PasswdDlg

ProgressBar

NAME
ProgressBar - Progress indicator widget

CREATION
ProgressBar *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-foreground or -fg	-orient
-relief	
-troughcolor	

WIDGET-SPECIFIC OPTIONS

-height

-maximum
-type
-variable
-width

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

ProgressBar widget indicates the user the progress of a lengthly operation. It is used by MainFrame and ProgressDlg.

WIDGET-SPECIFIC OPTIONS

-height
Specifies the desired height for the progress indicator.

-maximum
Specifies the maximum value of the variable. This value must be greater than zero.

-type
Specifies the type of the ProgressBar. Must be one of **normal**, **incremental**, **infinite** or **nonincremental_infinite**.

If **type** is *normal*, the progress indicator is drawn proportional to the variable value and **maximum** option each time the variable is set.

If **type** is *incremental*, the value of the progress indicator is maintained internally, and incremented each time the variable is set by its value. The progress indicator is drawn proportional to the internal value and **maximum** option.

If **type** is *infinite*, the value of the progress indicator is maintained internally, and incremented each time the variable is set by its value. The progress indicator moves from left to right if internal value (modulo **maximum**) is less than **maximum**/2, and from right to left if internal value is greater than **maximum**/2.

If **type** is *nonincremental_infinite*, the value of the progress indicator taken from the variable value, The progress indicator moves from left to right if variable value (modulo **maximum**) is less than **maximum**/2, and from right to left if internal value is greater than **maximum**/2.

See **-variable** option for special case of its value,

Default value for **type** is *normal*.

-variable
Specifies the variable attached to the progress indicator. Progress indicator is updated when the value of the variable changes. If the value of the variable is negative, the progress

indicator is not displayed (it is drawn flat with **background** color - usefull for ProgressDlg to make it invisible). If its value 0, progress indicator is reinitialized.

-width
Specifies the desired width for the progress indicator.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

Keywords: ProgressBar

ProgressDlg

NAME
ProgressDlg - Progress indicator dialog box

CREATION
ProgressDlg *pathName* ?*option value...*?

STANDARD OPTIONS

-font
-textvariable

OPTIONS from Dialog

-background or -bg	-parent
-separator	-title

OPTIONS from ProgressBar

-background or -bg	-borderwidth or -bd
-foreground or -fg	-maximum

-relief	-troughcolor
-type	-variable

WIDGET-SPECIFIC OPTIONS

-command
-height
-stop
-width

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

ProgressDlg provides a simple way to display a progress indicator dialog. ProgressDlg::**create** creates the dialog, displays it, set a local grab to it and immediatly return. The dialog is updated by modifying the value of the variable of options **-textvariable** and **-variable**. You have to destroy the dialog after use.

WIDGET-SPECIFIC OPTIONS

-command
Specifies a command to call when user press stop button. Note that it is the program's responsibility to periodically call **update** so that button press events can be generated.

-height
Specifies a desired height for the label in lines of text.

-stop
Specifies the text of the button typically used to stop process. If empty, no button will be drawn. This can be a symbolic name.

-width
Specifies a desired width for the label in characters.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value* ...?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list

describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

Keywords: ProgressDlg

ScrollView

NAME

ScrollView - Display the visible area of a scrolled window

CREATION

ScrollView *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-cursor	

WIDGET-SPECIFIC OPTIONS

-fill
-height

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

ScrollView displays the visible area of a scrolled window within its scroll region.

WIDGET-SPECIFIC OPTIONS

-fill
Specifies the fill color of the rectangle.

-foreground
Specifies the color of the border of the rectangle.

-height
Specifies the desired height for the ScrollView.

-width
Specifies the desired width for the ScrollView.

-window
Specifies the window to view. This widget must have **-xscrollcommand** and **-yscrollcommand** options, and respond to **xview** and **yview** command. In order to make ScrollView working with other scrollbar, **-xscrollcommand** and **-yscrollcommand** options of the widget must be set before the widget is passed to the **-window** option of the ScrollView (for example, if the widget is handled by a ScrolledWindow, call **setwidget** before setting **-window** option).

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

BINDINGS

Keywords: ScrollView

ScrollableFrame

NAME
ScrollableFrame - Scrollable frame containing widget

CREATION
ScrollableFrame *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-xscrollcommand
-yscrollcommand	

WIDGET-SPECIFIC OPTIONS

-areaheight
-areawidth
-constrainedheight

-constrainedwidth
-height
-width
-xscrollincrement
-yscrollincrement

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

pathName see *widget* ?*vert*? ?*horz*?

pathName xview ?*arg...*?

pathName yview ?*arg...*?

DESCRIPTION

ScrollableFrame widget containing widget.

WIDGET-SPECIFIC OPTIONS

-areaheight
Specifies the height for the scrollable area. If zero, then the height of the scrollable area is made just large enough to hold all its children.

-areawidth
Specifies the width for the scrollable area. If zero, then the width of the scrollable area window is made just large enough to hold all its children.

-constrainedheight
Specifies whether or not the scrollable area should have the same height of the scrolled window. If true, vertical scrollbar is not needed.

-constrainedwidth
Specifies whether or not the scrollable area should have the same width of the scrolled window. If true, horizontal scrollbar is not needed.

-height
Specifies the desired height for the window in pixels.

-width
Specifies the desired width for the window in pixels.

-xscrollincrement
See **xscrollincrement** option of **canvas** widget.

-yscrollincrement
See **yscrollincrement** option of **canvas** widget.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getframe**
Return the pathname of the scrolled frame where widget should be created.

***pathName* see *widget* ?*vert*? ?*horz*?**
Arrange scrollable area to make *widget* visible in the window. *vert* and *horz* specify which part of *widget* must be preferably visible, in case where *widget* is too tall or too large to be entirely visible. *vert* must be **top** (the default) or **bottom**, and *horz* must be **left** (the default) or **right**. If *vert* or *horz* is not a valid value, area is not scrolled in this direction.

***pathName* xview ?*arg...*?**
Standard command to enable horizontal scrolling of *pathName*.

***pathName* yview ?*arg...*?**
Standard command to enable vertical scrolling of *pathName*.

Keywords: ScrollableFrame

ScrolledWindow

NAME
ScrolledWindow - Generic scrolled widget

CREATION
ScrolledWindow *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-relief	

WIDGET-SPECIFIC OPTIONS

-auto
-ipad
-managed
-scrollbar
-sides
-size

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

pathName setwidget *widget*

DESCRIPTION

ScrolledWindow enables user to create easily a widget with its scrollbar. Scrollbars are created by ScrolledWindow and scroll commands are automatically associated to a scrollable widget with **ScrolledWindow::setwidget**.

WIDGET-SPECIFIC OPTIONS

-auto
Specifies the desired auto managed scrollbar: none means scrollbar are always drawn horizontal means horizontal scrollbar is drawn as needed vertical means vertical scrollbar is drawn as needed both means horizontal and vertical scrollbars are drawn as needed (default value)

-ipad (read-only)
Padding in pixels between client widget and scrollbars. Default value: **1**.

-managed (read-only)
If true, scrollbar are managed during creation, so their size are included in the requested size of the ScrolledWindow. If false, they are not. Default value: **true**.

-scrollbar
Specifies the desired scrollbar: **none**, **horizontal**, **vertical** or **both** (default value).

-sides (read-only)
Side of the scrollbars. Possible values are: **ne**, **en**, **nw**, **wn**, **se** (default value), **es**, **sw**, **ws**.

-size (read-only)
Size of the scrollbars in pixels. Use 0 for standard size (default value).

WIDGET COMMAND

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getframe**

Return the pathname of the frame where the scrolled widget should be created. This command is no longer needed. You can directly create the scrolled widget as the child of *pathName*.

pathName* setwidget *widget

Associate *widget* to the the scrollbars. *widget* becomes managed by the ScrolledWindow. The user should not attempt to manage *widget* until it is no longer managed by the ScrolledWindow. *widget* must be a scrollable widget, i.e. have the options **xscrollcommand**/**yscrollcommand** and the command **xview**/**yview**, such as canvas or text.

Keywords: ScrolledWindow

SelectColor

NAME

SelectColor - Color selection widget

CREATION

SelectColor *pathName* ?*option value...*?

WIDGET-SPECIFIC OPTIONS

-color	-parent
-placement	-title
-type	

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

SelectColor::dialog *pathName?option value ...?*

SelectColor::menu *pathName placement?option value ...?*

SelectColor::setcolor *index color*

DESCRIPTION

SelectColor provides a simple way to select color. It can be displayed as a dialog box or as a menubutton.

WIDGET-SPECIFIC OPTIONS

-color
Specifies the color value of the widget.

-parent
Parent of the Dialog. Dialog is centered in its parent. If empty, it is centered in root window.

-placement
Where to place the *popup* color dialog when displaying it. Must be any of: **at**, **center**, **left**, **right**, **above**, or **below**. If *-parent* is specified, placement will be in relation to the parent widget.

-title
Title of the Dialog toplevel.

-type (read-only)
Specifies the type of the SelectColor widget. Must be **dialog** or **popup**. If **type** option is *dialog*, SelectColor::**create** directly creates the dialog, displays it and return an empty string if cancel button is pressed or if dialog is destroyed, and the selected color if ok button is pressed. In all cases, dialog is destroyed. If **type** option is *popup*, SelectColor::**create** creates a small, popup dialog with a small set of predefined colors and a button to activate a full color dialog.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

SelectColor::dialog *pathName?option value ...?*
Creates a dialog for the user to select a custom color.

SelectColor::menu *pathName placement?option value ...?*
Creates a small, popup dialog for the user to select from a predefined list of colors with an

additional button to display a full color dialog.

placement can be any of **at**, **center**, **left**, **right**, **above**, or **below**. If *-parent* is specified, placement will be in relation to the parent widget.

SelectColor::setcolor *index color*
Set the value of user predefined color at index *index* to *color*. *index* must be between 0 and 10.

Keywords: SelectColor

SelectFont

NAME
SelectFont - Font selection widget

CREATION
SelectFont *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-font

WIDGET-SPECIFIC OPTIONS
-command

WIDGET COMMAND
pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

SelectFont::loadfont

DESCRIPTION

SelectFont provides a simple way to choose font. It can be displayed as a dialog box or as a toolbar.
Textual items in Dialog box uses **-name** options so they can be translated to any language. Symbolic name used are **ok**, **cancel**, **font**, **size**, **style**, **bold**, **italic**, **underline** and **overstrike**.

WIDGET-SPECIFIC OPTIONS

-command
Specifies a command to call when user select a new font when SelectFont **type** option is *toolbar*.

-initialcolor
If specified, add an additional button that lets the user pick a color. This option is ignored if **type** is *toolbar*.

-nosizes
If true, don't show the listbox containing valid font sizes. This option is ignored if **type** is

toolbar.

-parent
Parent of the Dialog. Dialog is centered in its parent. If empty, it is centered in root window.

-sampletext
Specifies the text displayed in the preview area.

-title
Title of the Dialog toplevel.

-type
Specifies the type of the SelectFont widget. Must be **dialog** or **toolbar**. If **type** option is *dialog*, SelectFont::**create** directly creates the dialog, displays it and return an empty string if cancel button is pressed or if dialog is destroyed, and the selected font (and color if **initialcolor**) if ok button is pressed. In all cases, dialog is destroyed. If **type** option is *toolbar*, SelectFont::**create** returns the pathname of the widget created.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

SelectFont::loadfont
Load the font available in the system.

Keywords: SelectFont

Separator

NAME
Separator - 3D separator widget

CREATION
Separator *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-orient

WIDGET-SPECIFIC OPTIONS
-relief

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

Separator is a widget that display an horizontal or vertical 3-D line.

WIDGET-SPECIFIC OPTIONS

-relief

Specifies the relief of the Separator. Must be **groove** (the default) or **ridge**.

WIDGET COMMAND

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value* ...?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

Keywords: Separator

SpinBox

NAME

SpinBox - SpinBox widget

CREATION

SpinBox *pathName* ?*option value...*?

OPTIONS from ArrowButton

-background or -bg	-disabledforeground
-foreground or -fg	-repeatdelay
-repeatinterval	-state

OPTIONS from Entry

-command	-disabledforeground
-dragenabled	-dragendcmd
-dragevent	-draginitcmd
-dragtype	-dropcmd
-dropenabled	-dropovercmd
-droptypes	-editable
-entrybg (see -background)	-entryfg (see -foreground)
-exportselection	-font
-helptext	-helptype
-helpvar	-highlightbackground
-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-justify
-selectbackground	-selectborderwidth
-selectforeground	-show
-state	-takefocus
-text	-textvariable
-width	-xscrollcommand

WIDGET-SPECIFIC OPTIONS

-modifycmd
-range
-values

WIDGET COMMAND

pathName bind ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getvalue

pathName setvalue *index*

DESCRIPTION

SpinBox widget enables the user to select a value among a list given by the **values** option or a set of values defined by a mininum, a maximum and an increment. Notice that **range** option defines a list of values, so **getvalue** and **setvalue** work with both values and range.

WIDGET-SPECIFIC OPTIONS

-modifycmd
Specifies a Tcl command called when the user modify the value of the SpinBox.

-range
Specifies a list of three intergers (or real) describing the minimum, maximum and increment of the SpinBox.

-values
Specifies the values accepted by the SpinBox. This option takes precedence over **range** option.

WIDGET COMMAND

***pathName* bind ?*arg...*?**
Set bindings on the entry widget.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value* ...?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getvalue**
Returns the index of the current text of the SpinBox in the list of values, or -1 if it doesn't match any value.

pathName* setvalue *index
Set the text of the SpinBox to the value indicated by *index* in the list of values. *index* may be specified in any of the following forms:

last
Specifies the last element of the list of values.

first
Specifies the first element of the list of values.

next
Specifies the element following the current (ie returned by **getvalue**) in the list of values.

previous
Specifies the element preceding the current (ie returned by **getvalue**) in the list of values.

@*number*
Specifies the integer index in the list of values.

BINDINGS

When Entry of the SpinBox has the input focus, it has the following bindings, in addition to the default Entry bindings:

- Page up set the value of the SpinBox to the last value.
- Page down set the value of the SpinBox to the first value.
- Arrow up set the value of the SpinBox to the next value.
- Arrow down set the value of the SpinBox to the previous value.

Keywords: SpinBox

StatusBar

NAME

StatusBar - status bar widget

CREATION

StatusBar *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-foreground or -fg	-orient
-relief	
-troughcolor	

WIDGET-SPECIFIC OPTIONS

-height
-showresize
-ipad
-pad
-width

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

StatusBar widget is a simple container widget with a corner resize control, meant to be placed at the bottom of a toplevel dialog.

WIDGET-SPECIFIC OPTIONS

-height
Specifies the desired height for the widget.

-showresize
Specifies whether to show the corner resize control.

-pad
The spacing to place around the status bar.

-ipad
The spacing to place around individual status bar items.

-width
Specifies the desired width for the widget.

WIDGET COMMAND

***pathName* add widget ?*option value option value* ...?**
Add a widget to the status bar. Possible options are:

-weight
Weighting of this item for resizing (passed to **grid**).

-separator
Whether to use a separator for this item.

-sticky
Passed on to grid.

-pad
Override of the widget's **-ipad** option for this item.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value* ...?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have

any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* delete widget ?*widget* ...?**
Delete a widget and associated state from the status bar.

***pathName* getframe**
Get the frame widget for the status bar in which status bar items should be created.

***pathName* items**
List of items in the status bar.

Keywords: StatusBar

TitleFrame

NAME

TitleFrame - Frame with a title

CREATION

TitleFrame *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-font	
-relief	

WIDGET-SPECIFIC OPTIONS

-baseline

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

DESCRIPTION

TitleFrame enables user to create a frame with a title like XmFrame Motif widget.

WIDGET-SPECIFIC OPTIONS

-baseline
Specifies the vertical alignment of the title: **top**, **center** or **bottom**.

-ipad
Specifies a pad between the border of the frame and the user frame. The value is in screen units.

-side
Specifies the horizontal alignment of the title: **left**, **center** or **right**.

WIDGET COMMAND

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getframe**

Return the frame where the user can create any other widget.

Keywords: TitleFrame

Tree

NAME

Tree - Tree widget

CREATION

Tree *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-cursor	-highlightbackground
-highlightcolor	-highlightthickness
-relief	-selectbackground
-selectforeground	-takefocus
-xscrollcommand	-yscrollcommand

WIDGET-SPECIFIC OPTIONS

-closecmd	-crossfill
-crossclosebitmap	-crosscloseimage
-crossopenbitmap	-crossopenimage
-deltax	-deltay

-dragenabled	-dragendcmd
-dragevent	-draginitcmd
-dragtype	-dropcmd
-dropenabled	-dropovercmd
-dropovermode	-droptypes
-height	-linesfill
-linestipple	-opencmd
-padx	-redraw
-selectcommand	-selectfill
-showlines	-width

WIDGET COMMAND

pathName bindArea *event script*

pathName bindImage *event script*

pathName bindText *event script*

pathName cget *option*

pathName closetree *node*

pathName configure ?*option*? ?*value option value* ...?

pathName delete ?*arg...*?

pathName edit *node text* ?*verifycmd*? ?*clickres*? ?*select*?

pathName exists *node*

pathName find *findinfo* ?*confine*?

pathName index *node*

pathName insert *index parent node* ?*option value...*?

pathName itemcget *node option*

pathName itemconfigure *node* ?*option*? ?*value option value* ...?

pathName line *node*

pathName move *parent node index*

pathName nodes *node* ?*first*? ?*last*?

pathName opentree *node*

pathName parent *node*

pathName reorder *node neworder*

pathName see *node*

pathName selection *cmd* ?*arg...*?

pathName toggle *node*

pathName visible *node*

pathName xview ?*arg...*?

pathName yview ?*arg...*?

DESCRIPTION

Tree widget uses canvas to display a hierarchical list of items (called nodes). Each node is composed of a label with its own font and foreground attributes, and an optional image or window. Each node can have a list of subnodes, which can be collapsed or expanded. Each node is drawn in a single line, whose height is defined by the **deltay** option, so they must have at most this height. A node is uniquely identified by a string given at creation (by the **insert** command). The node named *root* is the root of the tree and is not drawn. The tree structure is directly maintained by the widget.

WIDGET-SPECIFIC OPTIONS

-closecmd
Specifies a command to be called when user close a node. The closed node is appended to the command.

-crossfill
Specifies a foreground color for the cross bitmap.

-crossclosebitmap
Specifies a bitmap to be displayed in place of the standard cross when a node is closed.

-crosscloseimage
Specifies an image to be displayed in place of the standard cross when a node is closed. Overrides the -crossclosebitmap option.

-crossopenbitmap
Specifies a bitmap to be displayed in place of the standard cross when a node is open.

-crossopenimage
Specifies an image to be displayed in place of the standard cross when a node is open. Overrides the -crossopenbitmap option.

-deltax
Specifies horizontal indentation between a node and its children.

-deltay
Specifies vertical size of the nodes.

-dragenabled
A boolean specifying if drag is enabled.

-dragendcmd
Specifies a command to be called when drag ended. **dragendcmd** must be a command conforming to the description of the option **dragendcmd** of **DragSite::register**.

-dragevent
Specifies the number of the mouse button associated to the drag. Must be **1**, **2** or **3**.

-draginitcmd
Tree has a command wrapper for *drag-init* events. This command refused the drag if no node is designated. In other cases: If **draginitcmd** is empty, it returns:

- the value of option **dragtype** or *TREE_NODE* if empty as the data type,
- *{copy move link}* as the operations,
- the node identifier as the data.

If **draginitcmd** is not empty, it is called with the following arguments:

- the pathname of the tree,
- the identifier of the dragged node,
- the toplevel created to represent dragged data.

and must return a value conforming to **draginitcmd** option described in **DragSite::register**.

-dragtype
Specifies an alternate type of dragged object.

-dropcmd
Tree has a command wrapper for *drop* events. This command stops auto scrolling and extract node and position. If **dropcmd** is not empty, it is called with the following arguments:

- the pathname of the tree,
- the pathname of the drag source,
- a list describing where the drop occurs. It can be:
- the current operation,
- the data type,
- the data.
  - *{***widget***}*,
  - *{***node** *node}* or
  - *{***position** *node index}*.

-dropenabled
A boolean specifying if drop is enabled.

-dropovercmd
Tree has a command wrapper for *drag-over* events. This command enables auto scrolling and position extraction during the *drag-over*. If **dropovercmd** is not empty, the command is called with the following aguments:

- the pathname of the tree,
- the pathname of the drag source,
- a list describing where the drop can occur, whose elements are:
- the current operation,
- the data type,
- the data.
  - the string *widget* if **dropovertype** option contains *w*, else empty string.
  - the targeted node if drag icon points a node and **dropovertype** option contains *n*, else empty string.
  - a list containing a node and the position within the children of the node where drag icon points to if **dropovertype** option contains *p*, else empty string.
  - optionally, the preferred method if drop can occur both inside a node and between two nodes. The value is *position* or *node*.

The command must return a list with two elements:

- the drop status, conforming to those described in **dropovercmd** option of **DropSite::register** ,
- the choosen method: *widget*, *node* or *position*.

-dropovermode
Specifies the type of *drop-over* interaction. Must be a combination of **w**, which specifies that drop can occurs everywhere on widget, **p**, which specifies that drop can occurs between two nodes, and **n**, which specifies that drop occurs inside nodes.

-droptypes
Specifies a list of accepted dropped object/operation. See option **droptypes** of **DropSite::register**. for more infromation. Default is *TREE_NODE* with operations **copy** and **move**.

-height
Specifies the desired height for the tree in units of **deltay** pixels.

-linesfill
Specifies a foreground color for the lines between nodes.

-linestipple
Specifies a stipple bitmap for the lines between nodes.

-opencmd
Specifies a command to be called when the user opens a node. The name of the opened node is appended to the command.

-padx
Specifies distance between image or window and text of the nodes.

-redraw
Specifies wether or not the tree should be redrawn when entering idle. Set it to false if you call **update** while modifying the tree.

-selectcommand
Specifies a command to be called when the selection is changed. The path of the tree widget and the selected nodes are appended to the command.

-selectfill
If true, the selection box will be drawn across the entire tree from left-to-right instead of just around the item text.

-showlines
Specifies whether or not lines should be drawn between nodes.

-width
Specifies the desired width for the tree in units of 8 pixels.

NODE NAMES

Certain special characters in node names are automatically substituted by the tree during operation. These characters are **& | ^!**. They are all substituted with a **_** character. This is only to avoid errors because the characters are special to the tree widget.

WIDGET COMMAND

pathName* bindArea *event script
This command associates a command to execute whenever the event sequence given by *event* occurs anywhere within the Tree area.

pathName* bindImage *event script
This command associates a command to execute whenever the event sequence given by *event* occurs on the image of a node. The node idenfier on which the event occurs is appended to the command and may be used to manipulate the tree (e.g. don't use **%W**).

If **-selectfill** is given, an eventual binding of the background box by **bindText** is overwritten.

pathName* bindText *event script
This command associates a command to execute whenever the event sequence given by *event* occurs on the label of a node. The node idenfier on which the event occurs is appended to the command and may be used to manipulate the tree (e.g. don't use **%W**).

If **-selectfill** is given, an eventual binding of the background box by **bindImage** is overwritten.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* closetree *node* ?*recurse*?**
This command close all the subtree given by *node*. Recurse through the tree starting at *node* and set **open** option to 0 depending on *recurse*. Default value of *recurse* is *true*.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then

the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* delete ?*arg...*?**
Deletes all nodes (and children of them) in *arg*. *arg* can be a list of nodes or a list of list of nodes. To delete all the tree, do *$pathName delete [$pathName nodes root]*.

***pathName* edit *node text* ?*verifycmd*? ?*clickres*? ?*select*?**
Provides a way for the user to edit in place the label of a node. This is possible only if *node* is visible (all its parents are open). The command takes the initial text as argument and does not modify the label of the edited node, but returns an empty string if edition is canceled, or the typed text if edition is accepted. When editing, the user can cancel by pressing Escape, or accept by pressing Return. *clickres* specifies what to do if the user click outside the editable area. If *clickres* is 0 (the default), the edition is canceled. If *clickres* is 1, the edition is accepted. In all other case, the edition continues. If edition is accepted and *modifycmd* is not empty, then it is called with the new text as argument and must return 1 to accept the new text, 0 to refuse it and continue edition. *select* specifies wether or not the initial text should be selected. Default is 1.

pathName* exists *node
Returns whether or not *node* exists in the tree.

***pathName* find *findinfo* ?*confine*?**
Returns the node given by the position *findinfo*. *findinfo* can take the form of a pixel position *@x,y* or of the line number of a currently visible Tree node. The first line of the Tree has the value of zero.

If *confine* is non-empty, then confine *findinfo* to only match pixel positions for the area consumed by Tree labels, not just anywhere on their lines. (*confine* has no effect if *findinfo* is a line number.)

pathName* index *node
Returns the position of *node* in its parent.

***pathName* insert *index parent node* ?*option value...*?**
Inserts a new node identified by *node* in the children list of *parent* at position *index*.

Any instance of *#auto* within the node name will be replaced by the number of the item in the order of insertion. The non-printable characters \1 to \5 are reserved for internal use and should not be present in node names.

-anchor
Specifies the anchor of the image or window of the node. Defaults to w.

-data
User data associated to the node.

-deltax
Specifies the horizontal indentation of the node. If the value is -1, the node will be drawn with the deltax for the entire tree.

-drawcross
Specifies how the cross used to expand or collapse the children of a node should be drawn. Must be one of **auto**, **always** or **never**. If **auto**, the cross is drawn only if the node has children. If **always**, the cross is always drawn. If **never**, the cross is never drawn. To maintain compatibility with older versions of this widget, **allways** is a deprecated synonym to **always**.

-fill
Specifies the foreground color of the label of the node.

-font
Specifies a font for the label of the node.

-helpcmd
If specified, refers to a command to execute to get the help text to display. The command must return a string to display. If the command returns an empty string, no help is displayed. See also DynamicHelp.

-helptext
Text for dynamic help. See also DynamicHelp.

-helptype
Type of dynamic help. Use *balloon* or *variable*. See also DynamicHelp.

-helpvar
Variable to use when **helptype** option is *variable*. See also DynamicHelp.

-image
Specifies an image to display at the left of the label of the node. **window** option override **image**.

-open
Specifies wether or not the children of the node should be drawn.

-padx
Specifies the distance between image or window and the text of the node. If the value is -1, the node will be drawn with the padx for the entire tree.

-selectable
Specifies if the node can be selected or not.

-text
Specifies the label of the node.

-window
Specifies a pathname to display at the left of the label of the node. **window** option override **image**.

pathName* itemcget *node option
Returns the current value of a configuration option for the item. *Option* may have any of the values accepted by the item creation command.

pathName* itemconfigure *node ?option? ?value option value ...?
This command is similar to the **configure** command, except that it applies to the options for an individual item, whereas **configure** applies to the options for the widget as a whole. **Options** may have any of the values accepted by the item creation widget command. If

options are specified, options are modified as indicated in the command and the command returns an empty string. If no options are specified, returns a list describing the current options for the item. Read-only options are not be modified.

pathName* line *node
Returns the line number where *node* was drawn. If the node is not visible then return -1. The first line of the tree has the value of 0.

pathName* move *parent node index
Moves *node* to the children list of *parent* at position *index*. *parent* can not be a descendant of *node*.

***pathName* nodes *node* ?*first*? ?*last*?**
Returns parts of the children of *node*, following *first* and *last*. If *first* and *last* are omitted, returns the list of all children. If *first* is specified and *last* omitted, returns the child at index *first*, or an empty string if *first* refers to a non-existent element. If *first* and *last* are specified, the command returns a list whose elements are all of the children between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

***pathName* opentree *node* ?*recurse*?**
This command open all the subtree given by *node*. Recurse through the tree starting at *node* and set **open** option to 1 depending on value of *recurse*. Default value of *recurse* is *true*.

pathName* parent *node
Returns the parent of *node*.

pathName* reorder *node neworder
Modifies the order of children of *node* given by *neworder*. Children of *node* that do not appear in *neworder* are no moved.

pathName* see *node
Arrange the scrolling area to make *node* visible.

***pathName* selection *cmd* ?*arg...*?**
Modifies the list of selected nodes following *cmd*:

add
Adds all nodes in *arg* to the selection.

clear
Removes all nodes from the selection.

get
Returns a list containing the indices of current selected nodes.

includes
Tests if the specified node is selected. Returns true if the answer is yes, and false else.

range
Sets the selection to all nodes between the two specified ones.

remove
Removes all nodes in *arg* from the selection.

set
Sets the selection to all nodes in *arg*.

toggle

Toggles the selection status of all nodes in *arg*.

The subcommands **add**, **range**, and **set** silently ignore nodes which are declared unselectable. See the node option -selectable to influence this.

pathName* toggle *node
Toggle the open/close status of the given *node*.

pathName* visible *node
Returns whether or not *node* is visible (all its parents are open).

***pathName* xview ?*arg...*?**
Standard command to enable horizontal scrolling of *pathName*.

***pathName* yview ?*arg...*?**
Standard command to enable vertical scrolling of *pathName*.

BINDINGS

A **<<TreeSelect>>** virtual event is generated any time the selection in the tree changes. This is the default behavior of an item in the tree, but it can be overridden with the bindText or bindImage command. If the button 1 binding is overridden, this event may not be generated.

The tree has all the standard mouse wheel bindings when it has focus.

Keywords: Tree

Widget

Under construction ...

NAME
Widget - The Widget base class

COMMAND
Widget::addmap *class subclass subpath options*

Widget::bwinclude *class subclass subpath* ?*arg...*?

Widget::cget *path option*

Widget::configure *path options*

Widget::create *class path* ?*rename*?

Widget::declare *class optlist*

Widget::define *class filename* ?*class ...*?

Widget::destroy *path*

Widget::focusNext *w*

Widget::focusOK *w*

Widget::focusPrev *w*

Widget::generate-doc *dir widgetlist*

Widget::generate-widget-doc *class iscmd file*

Widget::getoption *path option*

Widget::getVariable *path varName?myVarName?*

Widget::hasChanged *path option pvalue*

Widget::init *class path options*

Widget::setoption *path option value*

Widget::subcget *path subwidget*

Widget::syncoptions *class subclass subpath options*

Widget::tkinclude *class tkwidget subpath* ?*arg...*?

DESCRIPTION

The **Widget** namespace handle data associated to all BWidget and provide commands to easily define BWidget.
For commands can be used to define a BWidget: **tkinclude**, **bwinclude**, **declare**, **addmap** and **syncoptions**. Here is the definition of ComboBox widget:

```
namespace eval ComboBox {
    # We're using ArrowButton, Entry and LabelFrame
    ArrowButton::use
    Entry::use
    LabelFrame::use

    # Include resources of LabelFrame
    Widget::bwinclude ComboBox LabelFrame .labf \
        rename     {-text -label} \
        remove     {-focus} \
        prefix     {label -justify -width -anchor -height -font} \
        initialize {-relief sunken -borderwidth 2}

    # Include resources of Entry
    Widget::bwinclude ComboBox Entry .e \
        remove {-relief -bd -borderwidth -bg -fg} \
        rename {-foreground -entryfg -background -entrybg}
```

```
    # Declare new resources
    Widget::declare ComboBox {
        {-height      TkResource 0  0 listbox}
        {-values      String     "" 0}
        {-modifycmd   String     "" 0}
        {-postcommand String     "" 0}
    }

    # Map resources to subwidget
    Widget::addmap ComboBox "" :cmd {-background {}}
    Widget::addmap ComboBox ArrowButton .a \
        {-foreground {} -background {} -disabledforeground {} -state
    {}}

    # Synchronize subwidget options
    Widget::syncoptions ComboBox Entry .e {-text {}}
    Widget::syncoptions ComboBox LabelFrame .labf {-label -text
    -underline {}}

    proc use {} {}
}
```

COMMAND

Widget::addmap *class subclass subpath options*

This command map some resources to subwidget. Mapped resources automatically configure subwidget when widget is configured.

- *class* is the class of the new BWidget
- *subclass* is the class the subwidget (BWidget class, e.g Entry, or empty for Tk widget)
- *subpath* is the path of the subwidget
- *options* is the list *{option realres ...}* of options to map to subwidget

Widget::bwinclude *class subclass subpath ?arg...?*

This command includes into a new BWidget the resources of another BWidget. Arguments are:

- *class* class of the new widget
- *subclass* class name of the BWidget to be included
- *subpath* path of the widget to configure when BWidget is configured
- *options* is:
  - *include {option option ...}* list of options to include (all if not defined)
  - *remove {option option ...}* list of options to remove
  - *rename {option name option name ...}* list of options to rename
  - *prefix {prefix option option ...}* pefix all *option* by *prefix*
  - *initialize {option value option value ...}* default value of options
  - *readonly {option value option value ...}* new readonly flag

Widget::cget *path option*

Returns the value of *option* of BWidget *path*. **cget** tests the option existence and takes care of synchronization with subwidget. Typically called by the BWidget **cget** command.

Widget::configure *path options*

Description text

Widget::create *class path ?rename?*

The standard method for creating a BWidget. The real widget path is renamed to $path:cmd, and a new proc is created to replace the path which points to the BWidget's commands.

If *rename* is false, the path will not be renamed, but the proc will still be created. This is useful when inheriting another BWidget who will already have renamed the widget.

The command returns the widget path. This command is usually the last command executed in the ::create command for the widget.

Widget::declare *class optlist*

This command declare new resources for a BWidget.

- *class* is class of the new widget
- *options* is the list describing new options. Each new option is a list **{option type value ro ?args?}** where:
  - *option* is the name of the option
  - *type* is the type of the option
  - *value* is the default value of the option
  - *ro* is the readonly flag of the option
  - *args* depends on type

type can be:

TkResource

value of *option* denotes a resource of a Tk widget. *args* must be *class* or *{class realoption}*. *class* is the creation command of the Tk widget, e.g. **entry**. The second form must be used if *option* has not the same name in Tk widget, but *realoption*. If *value* is empty, it is initialized to the default value of the Tk widget.

BwResource

value of *option* denotes a resource of a BWidget. *args* must be *class* or *{class realoption}*. *class* is the name of the namespace of the BWidget, e.g. **LabelFrame**. The second form must be used if *option* has not the same name in BWidget, but *realoption*. If *value* is empty, it is initialized to the default value of the BWidget.

Int

value of *option* is an integer. *args* can be *{?min? ?max?}* to force it to be in a range. The test is *[expr $option > $min] && [expr $option < $max]* so if args is *{0 10}*, value must be beetween 0 and 10 exclude, if *args* is *{=0 =10}*, value must be beetween 0 and 10 include.

Boolean

value of *option* is a boolean. True values can be **1**, **true** or **yes**. False values can be **0**, **false** or **no**. **Widget::cget** always return 0 or 1.

Enum

value of *option* is a element of a enumeration. *args* must be the list of enumeration, e.g. *{top left bottom right}*.

Flag

value of *option* is a combination of a set of chars. *args* must be a string defining the set.

String
value of *option* is any uncontrolled string.

Synonym
option is a synonym of option *args*. *value* has no effect here.

Widget::define *class filename ?class ...?*
This command is used to define a new BWidget class. It is usually the first command executed in a new widget definition.

- *class* is the name of the new widget class.
- *filename* is the name of the file (without extension) in the BWidget distribution that defines this class.

Each class defined after the filename is a class that this widget depends on. The ::use command will be called for each of these classes after the new widget has been defined.

This command does several things to setup the new class. First, it creates an alias in the global namespace for the name of the class that points to the class's ::create subcommand. Second, it defines a ::use subcommand for the class which other classes can use to load this class on the fly. Lastly, it creates a default binding to the <Destroy> event for the class that calls Widget::destroy on the path. This is the default setup for almost all widgets in the BWidget package.

Widget::destroy *path*
Description text

Widget::focusNext *w*
Description text

Widget::focusOK *w*
Description text

Widget::focusPrev *w*
Description text

Widget::generate-doc *dir widgetlist*
Description text

Widget::generate-widget-doc *class iscmd file*
Description text

Widget::getoption *path option*
Returns the value of *option* of BWidget *path*. This command does not test option existence, does not handle synonym and does not take care of synchronization with subwidget.

Widget::getVariable *path varName?myVarName?*
Make the variable *varName* relational to *path* accessible in the current procedure. The variable will be created in the widget namespace for *path* and can be used for storing widget-specific information. When *path* is destroyed, any variable accessed in this manner will be destroyed with it.

If *myVarName* is specified, the variable will be accessible in the current procedure as that name.

Widget::hasChanged *path option pvalue*

Description text

Widget::init *class path options*
Description text

Widget::setoption *path option value*
Set the value of *option* of BWidget *path* without option test, subwidget mapping, synonym handling and does not set the modification flag.

Widget::subcget *path subwidget*
Returns the list of all option/value of BWidget *path* that are mapped to *subwidget*.

Widget::syncoptions *class subclass subpath options*
This command synchronize options value of a subwidget. Used when an option of a subwidget is modified out of the BWidget **configure** command.

- *class* is the class of the new BWidget
- *subclass* is the class the subwidget (BWidget class, e.g Entry, or empty for Tk widget)
- *subpath* is the path of the subwidget
- *options* is the list *{option realres ...}* of options to synchronize with subwidget

Widget::tkinclude *class tkwidget subpath ?arg...?*
This command includes into a new BWidget the resources of a Tk widget. Arguments are:

- *class* class of the new widget
- *tkwidger* command name of the Tk widget to be included
- *subpath* path of the widget to configure when BWidget is configured
- *options* is:
  - *include {option option ...}* list of options to include (all if not defined)
  - *remove {option option ...}* list of options to remove
  - *rename {option name option name ...}* list of options to rename
  - *prefix {prefix option option ...}* pefix all *option* by *prefix*
  - *initialize {option value option value ...}* default value of options
  - *readonly {option value option value ...}* new readonly flag

Keywords: Widget

contents

Simple Widgets

Label	Label widget with state option, dynamic help and drag and drop facilities
Entry	Entry widget with state option, dynamic help and drag and drop facilities
Button	Button widget with enhanced options
ArrowButton	Button widget with an arrow shape.
ProgressBar	Progress indicator widget

ScrollView	Display the visible area of a scrolled window
Separator	3D separator widget

Manager Widgets

MainFrame	Manage toplevel with menu, toolbar and statusbar
LabelFrame	Frame with a Label
TitleFrame	Frame with a title (consider Tk 8.4+ labelframe instead)
PanelFrame	Frame with a boxed title area
ScrolledWindow	Generic scrolled widget
ScrollableFrame	Scrollable frame containing widget
PanedWindow	Tiled layout manager widget (consider Tk 8.4+ panedwindow instead)
ButtonBox	Set of buttons with horizontal or vertical layout
PagesManager	Pages manager widget
NoteBook	Notebook manager widget
Dialog	Dialog abstraction with custom buttons
StatusBar	Status bar widget with resize control

Composite Widgets

LabelEntry	LabelFrame containing an Entry widget.
ComboBox	ComboBox widget
SpinBox	SpinBox widget (consider Tk 8.4+ spinbox instead)
Tree	Tree widget
ListBox	ListBox widget
MessageDlg	Message dialog box
ProgressDlg	Progress indicator dialog box
PasswdDlg	Login/Password dialog box
SelectFont	Font selection widget
SelectColor	Color selection widget

Commands Classes

Widget	The Widget base class
DynamicHelp	Provide help to Tk widget or BWidget
DragSite	Commands set for Drag facilities
DropSite	Commands set for Drop facilities
BWidget	Description text

Keywords: contents

index

Keywords: index

navtree

Brief description

Simple Widgets

Label

Entry

Button

ArrowButton

ProgressBar

ScrollView

Separator

Manager Widgets

MainFrame

LabelFrame

TitleFrame

PanelFrame

ScrolledWindow

ScrollableFrame

PanedWindow

ButtonBox

PagesManager

NoteBook

Dialog

StatusBar

Composite Widgets

LabelEntry
ComboBox
SpinBox
Tree
ListBox
MessageDlg
ProgressDlg
PasswdDlg
SelectFont
SelectColor
Commands Classes
Widget
DynamicHelp
DragSite
DropSite
BWidget
Keywords: navtree

options

NAME

options - Standard options supported by widgets

DESCRIPTION

-activebackground, activeBackground, Foreground

-activeborderwidth, activeBorderWidth, BorderWidth

-activeforeground, activeForeground, Background

-anchor, anchor, Anchor

-background or -bg, background, Background

-bitmap, bitmap, Bitmap

-borderwidth or -bd, borderWidth, BorderWidth

-cursor, cursor, Cursor

-disabledbackground, disabledBackground, DisabledBackground

-disabledforeground, disabledForeground, DisabledForeground

-exportselection, exportSelection, ExportSelection

-font, font, Font

-foreground or -fg, foreground, Foreground

-highlightbackground, highlightBackground, HighlightBackground

-highlightcolor, highlightColor, HighlightColor

-highlightthickness, highlightThickness, HighlightThickness

-image, image, Image

-insertbackground, insertBackground, Foreground

-insertborderwidth, insertBorderWidth, BorderWidth

-insertofftime, insertOffTime, OffTime

-insertontime, insertOnTime, OnTime

-insertwidth, insertWidth, InsertWidth

-jump, jump, Jump

-justify, justify, Justify

-orient, orient, Orient

-padx, padX, Pad

-pady, padY, Pad

-relief, relief, Relief

-repeatdelay, repeatDelay, RepeatDelay

-repeatinterval, repeatInterval, RepeatInterval

-selectbackground, selectBackground, Foreground

-selectborderwidth, selectBorderWidth, BorderWidth

-selectforeground, selectForeground, Background

-setgrid, setGrid, SetGrid

-takefocus, takeFocus, TakeFocus

-text, text, Text

-textvariable, textVariable, Variable

-troughcolor, troughColor, Background

-underline, underline, Underline

-wraplength, wrapLength, WrapLength

-xscrollcommand, xScrollCommand, ScrollCommand

-yscrollcommand, yScrollCommand, ScrollCommand

KEYWORDS

NAME

options - Standard options supported by widgets

DESCRIPTION

This manual entry describes the common configuration options supported by widgets in the Tk toolkit. Every widget does not necessarily support every option (see the manual entries for individual widgets for a list of the standard options supported by that widget), but if a widget does support an option with one of the names listed below, then the option has exactly the effect described below.

In the descriptions below, ``Command-Line Name'' refers to the switch used in class commands and **configure** widget commands to set this value. For example, if an option's command-line switch is **-foreground** and there exists a widget **.a.b.c**, then the command

```
.a.b.c  configure  -foreground black
```

may be used to specify the value

black

for the option in the the widget

.a.b.c

. Command-line switches may be abbreviated, as long as the abbreviation is unambiguous. ``Database Name'' refers to the option's name in the option database (e.g. in .Xdefaults files). ``Database Class'' refers to the option's class value in the option database.

Database Class: Foreground

Specifies background color to use when drawing active elements. An element (a widget or portion of a widget) is active if the mouse cursor is positioned over the element and pressing a mouse button will cause some action to occur. If strict Motif compliance has been requested by setting the **tk_strictMotif** variable, this option will normally be ignored; the normal background color will be used instead. For some elements on Windows and Macintosh systems, the active color will only be used while mouse button 1 is pressed over the element.

Command-Line Name: -activeborderwidth Database Name: activeBorderWidth Database Class: BorderWidth

Specifies a non-negative value indicating the width of the 3-D border drawn around active elements. See above for definition of active elements. The value may have any of the forms acceptable to **Tk_GetPixels**. This option is typically only available in widgets displaying more than one element at a time (e.g. menus but not buttons).

Command-Line Name: -activeforeground Database Name: activeForeground Database Class: Background

Specifies foreground color to use when drawing active elements. See above for definition of active elements.

Command-Line Name: -anchor Database Name: anchor Database Class: Anchor

Specifies how the information in a widget (e.g. text or a bitmap) is to be displayed in the widget. Must be one of the values **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, or **center**. For example, **nw** means display the information such that its top-left corner is at the top-left corner of the widget.

Command-Line Name: -background or -bg Database Name: background Database Class: Background

Specifies the normal background color to use when displaying the widget.

Command-Line Name: -bitmap Database Name: bitmap Database Class: Bitmap

Specifies a bitmap to display in the widget, in any of the forms acceptable to **Tk_GetBitmap**. The exact way in which the bitmap is displayed may be affected by other options such as **anchor** or **justify**. Typically, if this option is specified then it overrides other options that specify a textual value to display in the widget; the **bitmap** option may be reset to an empty string to re-enable a text display. In widgets that support both **bitmap** and **image** options, **image** will usually override **bitmap**.

Command-Line Name: -borderwidth or -bd Database Name: borderWidth Database Class: BorderWidth

Specifies a non-negative value indicating the width of the 3-D border to draw around the outside of the widget (if such a border is being drawn; the **relief** option typically determines this). The value may also be used when drawing 3-D effects in the interior of the widget. The value may have any of the forms acceptable to **Tk_GetPixels**.

Command-Line Name: -cursor Database Name: cursor Database Class: Cursor

Specifies the mouse cursor to be used for the widget. The value may have any of the forms acceptable to **Tk_GetCursor**.

Command-Line Name: **-disabledforeground**

Command-Line Name: -disabledbackground Database Name: disabledBackground Database Class: DisabledBackground Specifies background color to use when drawing a disabled element.

If the option is specified as an empty string (which is typically the case on monochrome displays), disabled elements are drawn with the normal background color but they are dimmed by drawing them with a stippled fill pattern. Database Name: disabledForeground Database Class: DisabledForeground

Specifies foreground color to use when drawing a disabled element. If the option is specified as an empty string (which is typically the case on monochrome displays), disabled elements are drawn with the normal foreground color but they are dimmed by drawing them with a stippled fill pattern.

Command-Line Name: -exportselection Database Name: exportSelection Database Class: ExportSelection

Specifies whether or not a selection in the widget should also be the X selection. The value may have any of the forms accepted by **Tcl_GetBoolean**, such as **true**, **false**, **0**, **1**, **yes**, or **no**. If the selection is exported, then selecting in the widget deselects the current X selection, selecting outside the widget deselects any widget selection, and the widget will respond to selection retrieval requests when it has a selection. The default is usually for widgets to export selections.

Command-Line Name: -font Database Name: font Database Class: Font

Specifies the font to use when drawing text inside the widget.

Command-Line Name: -foreground or -fg Database Name: foreground Database Class: Foreground

Specifies the normal foreground color to use when displaying the widget.

Command-Line Name: -highlightbackground Database Name: highlightBackground Database Class: HighlightBackground

Specifies the color to display in the traversal highlight region when the widget does not have the input focus.

Command-Line Name: -highlightcolor Database Name: highlightColor Database Class: HighlightColor

Specifies the color to use for the traversal highlight rectangle that is drawn around the widget when it has the input focus.

Command-Line Name: -highlightthickness Database Name: highlightThickness Database Class: HighlightThickness

Specifies a non-negative value indicating the width of the highlight rectangle to draw around the outside of the widget when it has the input focus. The value may have any of the forms acceptable to **Tk_GetPixels**. If the value is zero, no focus highlight is drawn around the widget.

Command-Line Name: -image Database Name: image Database Class: Image

Specifies an image to display in the widget, which must have been created with the **image create** command. Typically, if the **image** option is specified then it overrides other options that specify a bitmap or textual value to display in the widget; the **image** option may be reset to an empty string to re-enable a bitmap or text display.

Command-Line Name: -insertbackground Database Name: insertBackground Database Class: Foreground

Specifies the color to use as background in the area covered by the insertion cursor. This color will normally override either the normal background for the widget (or the selection background

if the insertion cursor happens to fall in the selection).

Command-Line Name: -insertborderwidth Database Name: insertBorderWidth Database Class: BorderWidth

Specifies a non-negative value indicating the width of the 3-D border to draw around the insertion cursor. The value may have any of the forms acceptable to **Tk_GetPixels**.

Command-Line Name: -insertofftime Database Name: insertOffTime Database Class: OffTime

Specifies a non-negative integer value indicating the number of milliseconds the insertion cursor should remain ``off'' in each blink cycle. If this option is zero then the cursor doesn't blink: it is on all the time.

Command-Line Name: -insertontime Database Name: insertOnTime Database Class: OnTime

Specifies a non-negative integer value indicating the number of milliseconds the insertion cursor should remain ``on'' in each blink cycle.

Command-Line Name: -insertwidth Database Name: insertWidth Database Class: InsertWidth

Specifies a value indicating the total width of the insertion cursor. The value may have any of the forms acceptable to **Tk_GetPixels**. If a border has been specified for the insertion cursor (using the **insertBorderWidth** option), the border will be drawn inside the width specified by the **insertWidth** option.

Command-Line Name: -jump Database Name: jump Database Class: Jump

For widgets with a slider that can be dragged to adjust a value, such as scrollbars, this option determines when notifications are made about changes in the value. The option's value must be a boolean of the form accepted by **Tcl_GetBoolean**. If the value is false, updates are made continuously as the slider is dragged. If the value is true, updates are delayed until the mouse button is released to end the drag; at that point a single notification is made (the value ``jumps'' rather than changing smoothly).

Command-Line Name: -justify Database Name: justify Database Class: Justify

When there are multiple lines of text displayed in a widget, this option determines how the lines line up with each other. Must be one of **left**, **center**, or **right**. **Left** means that the lines' left edges all line up, **center** means that the lines' centers are aligned, and **right** means that the lines' right edges line up.

Command-Line Name: -orient Database Name: orient Database Class: Orient

For widgets that can lay themselves out with either a horizontal or vertical orientation, such as scrollbars, this option specifies which orientation should be used. Must be either **horizontal** or **vertical** or an abbreviation of one of these.

Command-Line Name: -padx Database Name: padX Database Class: Pad

Specifies a non-negative value indicating how much extra space to request for the widget in the X-direction. The value may have any of the forms acceptable to **Tk_GetPixels**. When computing how large a window it needs, the widget will add this amount to the width it would normally need (as determined by the width of the things displayed in the widget); if the geometry manager can satisfy this request, the widget will end up with extra internal space to the left and/or right of what it displays inside. Most widgets only use this option for padding text: if they are displaying a bitmap or image, then they usually ignore padding options.

Command-Line Name: -pady Database Name: padY Database Class: Pad

Specifies a non-negative value indicating how much extra space to request for the widget in the Y-direction. The value may have any of the forms acceptable to **Tk_GetPixels**. When computing how large a window it needs, the widget will add this amount to the height it would normally need (as determined by the height of the things displayed in the widget); if the geometry manager can satisfy this request, the widget will end up with extra internal space above and/or below what it displays inside. Most widgets only use this option for padding text: if they are displaying a bitmap or image, then they usually ignore padding options.

Command-Line Name: -relief Database Name: relief Database Class: Relief

Specifies the 3-D effect desired for the widget. Acceptable values are **raised**, **sunken**, **flat**, **ridge**, **solid**, and **groove**. The value indicates how the interior of the widget should appear relative to its exterior; for example, **raised** means the interior of the widget should appear to protrude from the screen, relative to the exterior of the widget.

Command-Line Name: -repeatdelay Database Name: repeatDelay Database Class: RepeatDelay

Specifies the number of milliseconds a button or key must be held down before it begins to auto-repeat. Used, for example, on the up- and down-arrows in scrollbars.

Command-Line Name: -repeatinterval Database Name: repeatInterval Database Class: RepeatInterval

Used in conjunction with **repeatDelay**: once auto-repeat begins, this option determines the number of milliseconds between auto-repeats.

Command-Line Name: -selectbackground Database Name: selectBackground Database Class: Foreground

Specifies the background color to use when displaying selected items.

Command-Line Name: -selectborderwidth Database Name: selectBorderWidth Database Class: BorderWidth

Specifies a non-negative value indicating the width of the 3-D border to draw around selected items. The value may have any of the forms acceptable to **Tk_GetPixels**.

Command-Line Name: -selectforeground Database Name: selectForeground Database Class: Background

Specifies the foreground color to use when displaying selected items.

Command-Line Name: -setgrid Database Name: setGrid Database Class: SetGrid

Specifies a boolean value that determines whether this widget controls the resizing grid for its top-level window. This option is typically used in text widgets, where the information in the widget has a natural size (the size of a character) and it makes sense for the window's dimensions to be integral numbers of these units. These natural window sizes form a grid. If the **setGrid** option is set to true then the widget will communicate with the window manager so that when the user interactively resizes the top-level window that contains the widget, the dimensions of the window will be displayed to the user in grid units and the window size will be constrained to integral numbers of grid units. See the section GRIDDED GEOMETRY MANAGEMENT in the **wm** manual entry for more details.

Command-Line Name: -takefocus Database Name: takeFocus Database Class: TakeFocus

Determines whether the window accepts the focus during keyboard traversal (e.g., Tab and Shift-Tab). Before setting the focus to a window, the traversal scripts consult the value of the

takeFocus option. A value of **0** means that the window should be skipped entirely during keyboard traversal. **1** means that the window should receive the input focus as long as it is viewable (it and all of its ancestors are mapped). An empty value for the option means that the traversal scripts make the decision about whether or not to focus on the window: the current algorithm is to skip the window if it is disabled, if it has no key bindings, or if it is not viewable. If the value has any other form, then the traversal scripts take the value, append the name of the window to it (with a separator space), and evaluate the resulting string as a Tcl script. The script must return **0**, **1**, or an empty string: a **0** or **1** value specifies whether the window will receive the input focus, and an empty string results in the default decision described above. Note: this interpretation of the option is defined entirely by the Tcl scripts that implement traversal: the widget implementations ignore the option entirely, so you can change its meaning if you redefine the keyboard traversal scripts.

Command-Line Name: -text Database Name: text Database Class: Text

Specifies a string to be displayed inside the widget. The way in which the string is displayed depends on the particular widget and may be determined by other options, such as **anchor** or **justify**.

Command-Line Name: -textvariable Database Name: textVariable Database Class: Variable

Specifies the name of a variable. The value of the variable is a text string to be displayed inside the widget; if the variable value changes then the widget will automatically update itself to reflect the new value. The way in which the string is displayed in the widget depends on the particular widget and may be determined by other options, such as **anchor** or **justify**.

Command-Line Name: -troughcolor Database Name: troughColor Database Class: Background

Specifies the color to use for the rectangular trough areas in widgets such as scrollbars and scales.

Command-Line Name: -underline Database Name: underline Database Class: Underline

Specifies the integer index of a character to underline in the widget. This option is used by the default bindings to implement keyboard traversal for menu buttons and menu entries. 0 corresponds to the first character of the text displayed in the widget, 1 to the next character, and so on.

Command-Line Name: -wraplength Database Name: wrapLength Database Class: WrapLength

For widgets that can perform word-wrapping, this option specifies the maximum line length. Lines that would exceed this length are wrapped onto the next line, so that no line is longer than the specified length. The value may be specified in any of the standard forms for screen distances. If this value is less than or equal to 0 then no wrapping is done: lines will break only at newline characters in the text.

Command-Line Name: -xscrollcommand Database Name: xScrollCommand Database Class: ScrollCommand

Specifies the prefix for a command used to communicate with horizontal scrollbars. When the view in the widget's window changes (or whenever anything else occurs that could change the display in a scrollbar, such as a change in the total size of the widget's contents), the widget will generate a Tcl command by concatenating the scroll command and two numbers. Each of the numbers is a fraction between 0 and 1, which indicates a position in the document. 0 indicates the beginning of the document, 1 indicates the end, .333 indicates a position one third the way

through the document, and so on. The first fraction indicates the first information in the document that is visible in the window, and the second fraction indicates the information just after the last portion that is visible. The command is then passed to the Tcl interpreter for execution. Typically the **xScrollCommand** option consists of the path name of a scrollbar widget followed by ``set'', e.g. ``.x.scrollbar set'': this will cause the scrollbar to be updated whenever the view in the window changes. If this option is not specified, then no command will be executed.

Command-Line Name: -yscrollcommand Database Name: yScrollCommand Database Class: ScrollCommand Specifies the prefix for a command used to communicate with vertical scrollbars. This option is treated in the same way as the xScrollCommand option, except that it is used for vertical scrollbars and is provided by widgets that support vertical scrolling. See the description of xScrollCommand for details on how this option is used.

KEYWORDS
class

,

name

,

standard option

,

switch

```
Copyright © 1990-1994 The Regents of the University of California.
Copyright © 1994-1996 Sun Microsystems, Inc.
Copyright © 1995-1997 Roger E. Critchlow Jr.
```

Keywords: options

expect

expect

Name
expect - programmed dialogue with interactive programs, Version 5

Synopsis
expect [**-dDinN**] [**-c** *cmds*] [[**-[f|b]**] *cmdfile*] [*args*]

Introduction
Expect is a program that "talks" to other interactive programs according to a script. Following the script, **Expect** knows what can be expected from a program and what the correct response should be. An interpreted language provides branching and high-level control structures to direct the dialogue. In addition, the user can take control and interact directly when desired, afterward returning control to the script.

Expectk is a mixture of **Expect** and **Tk**. It behaves just like **Expect** and **Tk**'s **wish**. **Expect** can

also be used directly in C or C++ (that is, without Tcl). See libexpect(3).

The name "Expect" comes from the idea of *send/expect* sequences popularized by uucp, kermit and other modem control programs. However unlike uucp, **Expect** is generalized so that it can be run as a user-level command with any program and task in mind. **Expect** can actually talk to several programs at the same time.

For example, here are some things **Expect** can do:

- Cause your computer to dial you back, so that you can login without paying for the call.
- Start a game (e.g., rogue) and if the optimal configuration doesn't appear, restart it (again and again) until it does, then hand over control to you.
- Run fsck, and in response to its questions, answer "yes", "no" or give control back to you, based on predetermined criteria.
- Connect to another network or BBS (e.g., MCI Mail, CompuServe) and automatically retrieve your mail so that it appears as if it was originally sent to your local system.
- Carry environment variables, current directory, or any kind of information across rlogin, telnet, tip, su, chgrp, etc.

There are a variety of reasons why the shell cannot perform these tasks. (Try, you'll see.) All are possible with **Expect**.

In general, **Expect** is useful for running any program which requires interaction between the program and the user. All that is necessary is that the interaction can be characterized programmatically. **Expect** can also give the user back control (without halting the program being controlled) if desired. Similarly, the user can return control to the script at any time.

Usage

Expect reads *cmdfile* for a list of commands to execute. **Expect** may also be invoked implicitly on systems which support the #! notation by marking the script executable, and making the first line in your script:

#!/usr/local/bin/expect -f

Of course, the path must accurately describe where **Expect** lives. /usr/local/bin is just an example.

The **-c** flag prefaces a command to be executed before any in the script. The command should be quoted to prevent being broken up by the shell. This option may be used multiple times. Multiple commands may be executed with a single **-c** by separating them with semicolons. Commands are executed in the order they appear. (When using Expectk, this option is specified as **-command**.)

The **-d** flag enables some diagnostic output, which primarily reports internal activity of commands such as **expect** and **interact**. This flag has the same effect as "exp_internal 1" at the beginning of an Expect script, plus the version of **Expect** is printed. (The **strace** command is useful for tracing statements, and the **trace** command is useful for tracing variable

assignments.) (When using Expectk, this option is specified as **-diag**.)

The **-D** flag enables an interactive debugger. An integer value should follow. The debugger will take control before the next Tcl procedure if the value is non-zero or if a ^C is pressed (or a breakpoint is hit, or other appropriate debugger command appears in the script). See the README file or SEE ALSO (below) for more information on the debugger. (When using Expectk, this option is specified as **-Debug**.)

The **-f** flag prefaces a file from which to read commands from. The flag itself is optional as it is only useful when using the #! notation (see above), so that other arguments may be supplied on the command line. (When using Expectk, this option is specified as **-file**.)

By default, the command file is read into memory and executed in its entirety. It is occasionally desirable to read files one line at a time. For example, stdin is read this way. In order to force arbitrary files to be handled this way, use the **-b** flag. (When using Expectk, this option is specified as **-buffer**.) Note that stdio-buffering may still take place however this shouldn't cause problems when reading from a fifo or stdin.

If the string "-" is supplied as a filename, standard input is read instead. (Use "./-" to read from a file actually named "-".)

The **-i** flag causes **Expect** to interactively prompt for commands instead of reading them from a file. Prompting is terminated via the **exit** command or upon EOF. See **interpreter** (below) for more information. **-i** is assumed if neither a command file nor **-c** is used. (When using Expectk, this option is specified as **-interactive**.)

-- may be used to delimit the end of the options. This is useful if you want to pass an option-like argument to your script without it being interpreted by **Expect**. This can usefully be placed in the #! line to prevent any flag-like interpretation by Expect. For example, the following will leave the original arguments (including the script name) in the variable *argv*.

#!/usr/local/bin/expect --

Note that the usual getopt(3) and execve(2) conventions must be observed when adding arguments to the #! line.

The file $exp_library/expect.rc is sourced automatically if present, unless the **-N** flag is used. (When using Expectk, this option is specified as **-NORC**.) Immediately after this, the file ~/.expect.rc is sourced automatically, unless the **-n** flag is used. If the environment variable DOTDIR is defined, it is treated as a directory and .expect.rc is read from there. (When using Expectk, this option is specified as **-norc**.) This sourcing occurs only after executing any **-c** flags.

-v causes Expect to print its version number and exit. (The corresponding flag in Expectk, which uses long flag names, is -version.)

Optional *args* are constructed into a list and stored in the variable named *argv*. *argc* is initialized to the length of argv.

argv0 is defined to be the name of the script (or binary if no script is used). For example, the following prints out the name of the script and the first three arguments:

```
    send_user "$argv0 [lrange $argv 0 2]\n"
```

Commands

Expect uses *Tcl* (Tool Command Language). Tcl provides control flow (e.g., if, for, break),

expression evaluation and several other features such as recursion, procedure definition, etc. Commands used here but not defined (e.g., **set**, **if**, **exec**) are Tcl commands (see tcl(3)). **Expect** supports additional commands, described below. Unless otherwise specified, commands return the empty string.

Commands are listed alphabetically so that they can be quickly located. However, new users may find it easier to start by reading the descriptions of **spawn**, **send**, **expect**, and **interact**, in that order.

Note that the best introduction to the language (both Expect and Tcl) is provided in the book "Exploring Expect" (see SEE ALSO below). Examples are included in this man page but they are very limited since this man page is meant primarily as reference material.

Note that in the text of this man page, "Expect" with an uppercase "E" refers to the **Expect** program while "expect" with a lower-case "e" refers to the **expect** command within the **Expect** program.)

close *[-slave] [-onexec 0|1] [-i spawn_id]*

closes the connection to the current process. Most interactive programs will detect EOF on their stdin and exit; thus **close** usually suffices to kill the process as well. The **-i** flag declares the process to close corresponding to the named spawn_id. Both **expect** and **interact** will detect when the current process exits and implicitly do a **close**. But if you kill the process by, say, "exec kill $pid", you will need to explicitly call **close**. The **-onexec** flag determines whether the spawn id will be closed in any new spawned processes or if the process is overlayed. To leave a spawn id open, use the value 0. A non-zero integer value will force the spawn closed (the default) in any new processes. The **-slave** flag closes the slave associated with the spawn id. (See "spawn -pty".) When the connection is closed, the slave is automatically closed as well if still open. No matter whether the connection is closed implicitly or explicitly, you should call **wait** to clear up the corresponding kernel process slot. **close** does not call **wait** since there is no guarantee that closing a process connection will cause it to exit. See **wait** below for more info.

debug *[[-now] 0|1]*

controls a Tcl debugger allowing you to step through statements, set breakpoints, etc. With no arguments, a 1 is returned if the debugger is not running, otherwise a 0 is returned. With a 1 argument, the debugger is started. With a 0 argument, the debugger is stopped. If a 1 argument is preceded by the **-now** flag, the debugger is started immediately (i.e., in the middle of the **debug** command itself). Otherwise, the debugger is started with the next Tcl statement. The **debug** command does not change any traps. Compare this to starting Expect with the **-D** flag (see above). See the README file or SEE ALSO (below) for more information on the debugger.

disconnect

disconnects a forked process from the terminal. It continues running in the background. The process is given its own process group (if possible). Standard I/O is redirected to /dev/null.

The following fragment uses **disconnect** to continue running the script in the background.

```
if {[fork]!=0} exit
disconnect
. . .
```

The following script reads a password, and then runs a program every hour that demands a password each time it is run. The script supplies the password so that you only have to type it once. (See the **stty** command which demonstrates how to turn off password echoing.)

```
send_user "password?\ "
expect_user -re "(.*)\n"
for {} 1 {} {
    if {[fork]!=0} {sleep 3600;continue}
    disconnect
    spawn priv_prog
    expect Password:
    send "$expect_out(1,string)\r"
    . . .
    exit
}
```

An advantage to using **disconnect** over the shell asynchronous process feature (&) is that **Expect** can save the terminal parameters prior to disconnection, and then later apply them to new ptys. With &, **Expect** does not have a chance to read the terminal's parameters since the terminal is already disconnected by the time **Expect** receives control.

exit ***[-opts] [status]***

causes **Expect** to exit or otherwise prepare to do so. The **-onexit** flag causes the next argument to be used as an exit handler. Without an argument, the current exit handler is returned. The **-noexit** flag causes **Expect** to prepare to exit but stop short of actually returning control to the operating system. The user-defined exit handler is run as well as Expect's own internal handlers. No further Expect commands should be executed. This is useful if you are running Expect with other Tcl extensions. The current interpreter (and main window if in the Tk environment) remain so that other Tcl extensions can clean up. If Expect's **exit** is called again (however this might occur), the handlers are not rerun. Upon exiting, all connections to spawned processes are closed. Closure will be detected as an EOF by spawned processes. **exit** takes no other actions beyond what the normal _exit(2) procedure does. Thus, spawned processes that do not check for EOF may continue to run. (A variety of conditions are important to determining, for example, what signals a spawned process will be sent, but these are system-dependent, typically documented under exit(3).) Spawned processes that continue to run will be inherited by init. *status* (or 0 if not specified) is returned as the exit status of **Expect**. **exit** is implicitly executed if the end of the script is reached.

exp_continue [-continue_timer]

The command **exp_continue** allows **expect** itself to continue executing rather than returning as it normally would. By default **exp_continue** resets the timeout timer. The *-continue_timer* flag prevents timer from being restarted. (See **expect** for more information.)

exp_internal ***[-f file] value***

causes further commands to send diagnostic information internal to **Expect** to stderr if *value* is non-zero. This output is disabled if *value* is 0. The diagnostic information includes every character received, and every attempt made to match the current output against the patterns.

If the optional *file* is supplied, all normal and debugging output is written to that file

(regardless of the value of *value*). Any previous diagnostic output file is closed. The **-info** flag causes exp_internal to return a description of the most recent non-info arguments given.

exp_open ***[args] [-i spawn_id]***
returns a Tcl file identifier that corresponds to the original spawn id. The file identifier can then be used as if it were opened by Tcl's **open** command. (The spawn id should no longer be used. A **wait** should not be executed. The **-leaveopen** flag leaves the spawn id open for access through Expect commands. A **wait** must be executed on the spawn id.

exp_pid ***[-i spawn_id]***
returns the process id corresponding to the currently spawned process. If the **-i** flag is used, the pid returned corresponds to that of the given spawn id.

exp_send
is an alias for **send**.

exp_send_error
is an alias for **send_error**.

exp_send_log
is an alias for **send_log**.

exp_send_tty
is an alias for **send_tty**.

exp_send_user
is an alias for **send_user**.

exp_version ***[[-exit] version]***
is useful for assuring that the script is compatible with the current version of Expect.

With no arguments, the current version of **Expect** is returned. This version may then be encoded in your script. If you actually know that you are not using features of recent versions, you can specify an earlier version.

Versions consist of three numbers separated by dots. First is the major number. Scripts written for versions of **Expect** with a different major number will almost certainly not work. **exp_version** returns an error if the major numbers do not match.

Second is the minor number. Scripts written for a version with a greater minor number than the current version may depend upon some new feature and might not run. **exp_version** returns an error if the major numbers match, but the script minor number is greater than that of the running **Expect**.

Third is a number that plays no part in the version comparison. However, it is incremented when the **Expect** software distribution is changed in any way, such as by additional documentation or optimization. It is reset to 0 upon each new minor version.

With the **-exit** flag, **Expect** prints an error and exits if the version is out of date.

expect ***[[-opts] pat1 body1] ... [-opts] patn [bodyn]***
waits until one of the patterns matches the output of a spawned process, a specified time period has passed, or an end-of-file is seen. If the final body is empty, it may be omitted.

Patterns from the most recent **expect_before** command are implicitly used before any other

patterns. Patterns from the most recent **expect_after** command are implicitly used after any other patterns.

If the arguments to the entire **expect** statement require more than one line, all the arguments may be "braced" into one so as to avoid terminating each line with a backslash. In this one case, the usual Tcl substitutions will occur despite the braces.

If a pattern is the keyword **eof**, the corresponding body is executed upon end-of-file. If a pattern is the keyword **timeout**, the corresponding body is executed upon timeout. If no timeout keyword is used, an implicit null action is executed upon timeout. The default timeout period is 10 seconds but may be set, for example to 30, by the command "set timeout 30". An infinite timeout may be designated by the value -1. If a pattern is the keyword **default**, the corresponding body is executed upon either timeout or end-of-file.

If a pattern matches, then the corresponding body is executed. **expect** returns the result of the body (or the empty string if no pattern matched). In the event that multiple patterns match, the one appearing first is used to select a body.

Each time new output arrives, it is compared to each pattern in the order they are listed. Thus, you may test for absence of a match by making the last pattern something guaranteed to appear, such as a prompt. In situations where there is no prompt, you must use **timeout** (just like you would if you were interacting manually).

Patterns are specified in three ways. By default, patterns are specified as with Tcl's **string match** command. (Such patterns are also similar to C-shell regular expressions usually referred to as "glob" patterns). The **-gl** flag may may be used to protect patterns that might otherwise match **expect** flags from doing so. Any pattern beginning with a "-" should be protected this way. (All strings starting with "-" are reserved for future options.)

For example, the following fragment looks for a successful login. (Note that **abort** is presumed to be a procedure defined elsewhere in the script.)

```
expect {
    busy              {puts busy\n ; exp_continue}
    failed            abort
    "invalid password" abort
    timeout           abort
    connected
}
```

Quotes are necessary on the fourth pattern since it contains a space, which would otherwise separate the pattern from the action. Patterns with the same action (such as the 3rd and 4th) require listing the actions again. This can be avoid by using regexp-style patterns (see below). More information on forming glob-style patterns can be found in the Tcl manual.

Regexp-style patterns follow the syntax defined by Tcl's **regexp** (short for "regular expression") command. regexp patterns are introduced with the flag **-re**. The previous example can be rewritten using a regexp as:

```
    expect {
        busy       {puts busy\n ; exp_continue}
        -re "failed|invalid password" abort
        timeout    abort
        connected
    }
```

Both types of patterns are "unanchored". This means that patterns do not have to match the entire string, but can begin and end the match anywhere in the string (as long as everything else matches). Use ^ to match the beginning of a string, and $ to match the end. Note that if you do not wait for the end of a string, your responses can easily end up in the middle of the string as they are echoed from the spawned process. While still producing correct results, the output can look unnatural. Thus, use of $ is encouraged if you can exactly describe the characters at the end of a string.

Note that in many editors, the ^ and $ match the beginning and end of lines respectively. However, because expect is not line oriented, these characters match the beginning and end of the data (as opposed to lines) currently in the expect matching buffer. (Also, see the note below on "system indigestion.")

The **-ex** flag causes the pattern to be matched as an "exact" string. No interpretation of *, ^, etc is made (although the usual Tcl conventions must still be observed). Exact patterns are always unanchored.

The **-nocase** flag causes uppercase characters of the output to compare as if they were lowercase characters. The pattern is not affected.

While reading output, more than 2000 bytes can force earlier bytes to be "forgotten". This may be changed with the function **match_max**. (Note that excessively large values can slow down the pattern matcher.) If *patlist* is **full_buffer**, the corresponding body is executed if *match_max* bytes have been received and no other patterns have matched. Whether or not the **full_buffer** keyword is used, the forgotten characters are written to expect_out(buffer). If *patlist* is the keyword **null**, and nulls are allowed (via the **remove_nulls** command), the corresponding body is executed if a single ASCII 0 is matched. It is not possible to match 0 bytes via glob or regexp patterns. Upon matching a pattern (or eof or full_buffer), any matching and previously unmatched output is saved in the variable *expect_out(buffer)*. Up to 9 regexp substring matches are saved in the variables *expect_out(1,string)* through *expect_out(9,string)*. If the **-indices** flag is used before a pattern, the starting and ending indices (in a form suitable for **lrange**) of the 10 strings are stored in the variables *expect_out(X,start)* and *expect_out(X,end)* where X is a digit, corresponds to the substring position in the buffer. 0 refers to strings which matched the entire pattern and is generated for glob patterns as well as regexp patterns. For example, if a process has produced output of "abcdefgh\n", the result of:

```
    expect "cd"
```

is as if the following statements had executed:

```
    set expect_out(0,string) cd
    set expect_out(buffer) abcd
```

and "efgh\n" is left in the output buffer. If a process produced the output "abbbcabkkkka\n", the result of:

```
    expect -indices -re "b(b*).*(k+)"
```

is as if the following statements had executed:

```
    set expect_out(0,start) 1
    set expect_out(0,end) 10
    set expect_out(0,string) bbbcabkkkk
    set expect_out(1,start) 2
    set expect_out(1,end) 3
    set expect_out(1,string) bb
    set expect_out(2,start) 10
    set expect_out(2,end) 10
    set expect_out(2,string) k
    set expect_out(buffer) abbbcabkkkk
```

and "a\n" is left in the output buffer. The pattern "*" (and -re ".*") will flush the output buffer without reading any more output from the process.

Normally, the matched output is discarded from Expect's internal buffers. This may be prevented by prefixing a pattern with the **-notransfer** flag. This flag is especially useful in experimenting (and can be abbreviated to "-not" for convenience while experimenting). The spawn id associated with the matching output (or eof or full_buffer) is stored in *expect_out(spawn_id)*. The **-timeout** flag causes the current expect command to use the following value as a timeout instead of using the value of the timeout variable. By default, patterns are matched against output from the current process, however the **-i** flag declares the output from the named spawn_id list be matched against any following patterns (up to the next **-i**). The spawn_id list should either be a whitespace separated list of spawn_ids or a variable referring to such a list of spawn_ids. For example, the following example waits for "connected" from the current process, or "busy", "failed" or "invalid password" from the spawn_id named by $proc2.

```
   expect {
       -i $proc2 busy {puts busy\n ; exp_continue}
       -re "failed|invalid password" abort
       timeout abort
       connected
   }
```

The value of the global variable *any_spawn_id* may be used to match patterns to any spawn_ids that are named with all other **-i** flags in the current **expect** command. The spawn_id from a **-i** flag with no associated pattern (i.e., followed immediately by another **-i**) is made available to any other patterns in the same **expect** command associated with *any_spawn_id.*

The **-i** flag may also name a global variable in which case the variable is read for a list of spawn ids. The variable is reread whenever it changes. This provides a way of changing the I/O source while the command is in execution. Spawn ids provided this way are called "indirect" spawn ids.

Actions such as **break** and **continue** cause control structures (i.e., **for**, **proc**) to behave in the

usual way. The command **exp_continue** allows **expect** itself to continue executing rather than returning as it normally would.

This is useful for avoiding explicit loops or repeated expect statements. The following example is part of a fragment to automate rlogin. The **exp_continue** avoids having to write a second **expect** statement (to look for the prompt again) if the rlogin prompts for a password.

```
expect {
    Password: {
        stty -echo
        send_user "password (for $user) on $host: "
        expect_user -re "(.*)\n"
        send_user "\n"
        send "$expect_out(1,string)\r"
        stty echo
        exp_continue
    } incorrect {
        send_user "invalid password or account\n"
        exit
    } timeout {
        send_user "connection to $host timed out\n"
        exit
    } eof {
        send_user \
            "connection to host failed: $expect_out(buffer)"
        exit
    } -re $prompt
}
```

For example, the following fragment might help a user guide an interaction that is already totally automated. In this case, the terminal is put into raw mode. If the user presses "+", a variable is incremented. If "p" is pressed, several returns are sent to the process, perhaps to poke it in some way, and "i" lets the user interact with the process, effectively stealing away control from the script. In each case, the **exp_continue** allows the current **expect** to continue pattern matching after executing the current action.

```
stty raw -echo
expect_after {
    -i $user_spawn_id
    "p" {send "\r\r\r"; exp_continue}
    "+" {incr foo; exp_continue}
    "i" {interact; exp_continue}
    "quit" exit
}
```

By default, **exp_continue** resets the timeout timer. The timer is not restarted, if **exp_continue** is called with the **-continue_timer** flag.

expect_after ***[expect_args]***

works identically to the **expect_before** except that if patterns from both **expect** and **expect_after** can match, the **expect** pattern is used. See the **expect_before** command for more information.

expect_background ***[expect_args]***
takes the same arguments as **expect**, however it returns immediately. Patterns are tested whenever new input arrives. The pattern **timeout** and **default** are meaningless to **expect_background** and are silently discarded. Otherwise, the **expect_background** command uses **expect_before** and **expect_after** patterns just like **expect** does. When **expect_background** actions are being evaluated, background processing for the same spawn id is blocked. Background processing is unblocked when the action completes. While background processing is blocked, it is possible to do a (foreground) **expect** on the same spawn id. It is not possible to execute an **expect** while an **expect_background** is unblocked. **expect_background** for a particular spawn id is deleted by declaring a new expect_background with the same spawn id. Declaring **expect_background** with no pattern removes the given spawn id from the ability to match patterns in the background.

expect_before ***[expect_args]***
takes the same arguments as **expect**, however it returns immediately. Pattern-action pairs from the most recent **expect_before** with the same spawn id are implicitly added to any following **expect** commands. If a pattern matches, it is treated as if it had been specified in the **expect** command itself, and the associated body is executed in the context of the **expect** command. If patterns from both **expect_before** and **expect** can match, the **expect_before** pattern is used. If no pattern is specified, the spawn id is not checked for any patterns. Unless overridden by a **-i** flag, **expect_before** patterns match against the spawn id defined at the time that the **expect_before** command was executed (not when its pattern is matched). The -info flag causes **expect_before** to return the current specifications of what patterns it will match. By default, it reports on the current spawn id. An optional spawn id specification may be given for information on that spawn id. For example

```
expect_before -info -i $proc
```

At most one spawn id specification may be given. The flag -indirect suppresses direct spawn ids that come only from indirect specifications.

Instead of a spawn id specification, the flag "-all" will cause "-info" to report on all spawn ids.

The output of the -info flag can be reused as the argument to expect_before.

expect_tty ***[expect_args]***
is like **expect** but it reads characters from /dev/tty (i.e. keystrokes from the user). By default, reading is performed in cooked mode. Thus, lines must end with a return in order for **expect** to see them. This may be changed via **stty** (see the **stty** command below).

expect_user ***[expect_args]***
is like **expect** but it reads characters from stdin (i.e. keystrokes from the user). By default, reading is performed in cooked mode. Thus, lines must end with a return in order for **expect** to see them. This may be changed via **stty** (see the **stty** command below).

fork
creates a new process. The new process is an exact copy of the current **Expect** process. On success, **fork** returns 0 to the new (child) process and returns the process ID of the child process to the parent process. On failure (invariably due to lack of resources, e.g., swap

space, memory), **fork** returns -1 to the parent process, and no child process is created.

Forked processes exit via the **exit** command, just like the original process. Forked processes are allowed to write to the log files. If you do not disable debugging or logging in most of the processes, the result can be confusing.

Some pty implementations may be confused by multiple readers and writers, even momentarily. Thus, it is safest to **fork** before spawning processes.

interact ***[string1 body1] ... [stringn [bodyn]]***

gives control of the current process to the user, so that keystrokes are sent to the current process, and the stdout and stderr of the current process are returned.

String-body pairs may be specified as arguments, in which case the body is executed when the corresponding string is entered. (By default, the string is not sent to the current process.) The **interpreter** command is assumed, if the final body is missing.

If the arguments to the entire **interact** statement require more than one line, all the arguments may be "braced" into one so as to avoid terminating each line with a backslash. In this one case, the usual Tcl substitutions will occur despite the braces.

For example, the following command runs interact with the following string-body pairs defined: When ^Z is pressed, **Expect** is suspended. (The **-reset** flag restores the terminal modes.) When ^A is pressed, the user sees "you typed a control-A" and the process is sent a ^A. When $ is pressed, the user sees the date. When ^C is pressed, **Expect** exits. If "foo" is entered, the user sees "bar". When ~~ is pressed, the **Expect** interpreter runs interactively.

```
set CTRLZ \032
interact {
    -reset $CTRLZ {exec kill -STOP [pid]}
    \001   {send_user "you typed a control-A\n";
            send "\001"
           }
    $      {send_user "The date is [clock format [clock
seconds]]."}
    \003   exit
    foo    {send_user "bar"}
    ~~
}
```

In string-body pairs, strings are matched in the order they are listed as arguments. Strings that partially match are not sent to the current process in anticipation of the remainder coming. If characters are then entered such that there can no longer possibly be a match, only the part of the string will be sent to the process that cannot possibly begin another match. Thus, strings that are substrings of partial matches can match later, if the original strings that was attempting to be match ultimately fails.

By default, string matching is exact with no wild cards. (In contrast, the **expect** command uses glob-style patterns by default.) The **-ex** flag may be used to protect patterns that might

otherwise match **interact** flags from doing so. Any pattern beginning with a "-" should be protected this way. (All strings starting with "-" are reserved for future options.) The **-re** flag forces the string to be interpreted as a regexp-style pattern. In this case, matching substrings are stored in the variable *interact_out* similarly to the way **expect** stores its output in the variable **expect_out**. The **-indices** flag is similarly supported. The pattern **eof** introduces an action that is executed upon end-of-file. A separate **eof** pattern may also follow the **-output** flag in which case it is matched if an eof is detected while writing output. The default **eof** action is "return", so that **interact** simply returns upon any EOF. The pattern **timeout** introduces a timeout (in seconds) and action that is executed after no characters have been read for a given time. The **timeout** pattern applies to the most recently specified process. There is no default timeout. The special variable "timeout" (used by the **expect** command) has no affect on this timeout. For example, the following statement could be used to autologout users who have not typed anything for an hour but who still get frequent system messages:

```
interact -input $user_spawn_id timeout 3600 return -output \
    $spawn_id
```

If the pattern is the keyword **null**, and nulls are allowed (via the **remove_nulls** command), the corresponding body is executed if a single ASCII 0 is matched. It is not possible to match 0 bytes via glob or regexp patterns.

Prefacing a pattern with the flag **-iwrite** causes the variable *interact_out(spawn_id)* to be set to the spawn_id which matched the pattern (or eof).

Actions such as **break** and **continue** cause control structures (i.e., **for**, **proc**) to behave in the usual way. However **return** causes interact to return to its caller, while **inter_return** causes **interact** to cause a return in its caller. For example, if "proc foo" called **interact** which then executed the action **inter_return**, **proc foo** would return. (This means that if **interact** calls **interpreter** interactively typing **return** will cause the interact to continue, while **inter_return** will cause the interact to return to its caller.)

During **interact**, raw mode is used so that all characters may be passed to the current process. If the current process does not catch job control signals, it will stop if sent a stop signal (by default ^Z). To restart it, send a continue signal (such as by "kill -CONT <pid>"). If you really want to send a SIGSTOP to such a process (by ^Z), consider spawning csh first and then running your program. On the other hand, if you want to send a SIGSTOP to **Expect** itself, first call interpreter (perhaps by using an escape character), and then press ^Z.

String-body pairs can be used as a shorthand for avoiding having to enter the interpreter and execute commands interactively. The previous terminal mode is used while the body of a string-body pair is being executed.

For speed, actions execute in raw mode by default. The **-reset** flag resets the terminal to the mode it had before **interact** was executed (invariably, cooked mode). Note that characters entered when the mode is being switched may be lost (an unfortunate feature of the terminal driver on some systems). The only reason to use **-reset** is if your action depends on running in cooked mode.

The **-echo** flag sends characters that match the following pattern back to the process that generated them as each character is read. This may be useful when the user needs to see feedback from partially typed patterns.

If a pattern is being echoed but eventually fails to match, the characters are sent to the spawned process. If the spawned process then echoes them, the user will see the characters twice. **-echo** is probably only appropriate in situations where the user is unlikely to not complete the pattern. For example, the following excerpt is from rftp, the recursive-ftp script, where the user is prompted to enter ~g, ~p, or ~l, to get, put, or list the current directory recursively. These are so far away from the normal ftp commands, that the user is unlikely to type ~ followed by anything else, except mistakenly, in which case, they'll probably just ignore the result anyway.

```
interact {
    -echo ~g {getcurdirectory 1}
    -echo ~l {getcurdirectory 0}
    -echo ~p {putcurdirectory}
}
```

The **-nobuffer** flag sends characters that match the following pattern on to the output process as characters are read.

This is useful when you wish to let a program echo back the pattern. For example, the following might be used to monitor where a person is dialing (a Hayes-style modem). Each time "atd" is seen the script logs the rest of the line.

```
proc lognumber {} {
    interact -nobuffer -re "(.*)\r" return
    puts $log "[clock format [clock seconds]]: dialed
$interact_out(1,string)"
}

interact -nobuffer "atd" lognumber
```

During **interact**, previous use of **log_user** is ignored. In particular, **interact** will force its output to be logged (sent to the standard output) since it is presumed the user doesn't wish to interact blindly.

The **-o** flag causes any following key-body pairs to be applied to the output of the current process. This can be useful, for example, when dealing with hosts that send unwanted characters during a telnet session.

By default, **interact** expects the user to be writing stdin and reading stdout of the **Expect** process itself. The **-u** flag (for "user") makes **interact** look for the user as the process named by its argument (which must be a spawned id).

This allows two unrelated processes to be joined together without using an explicit loop. To aid in debugging, Expect diagnostics always go to stderr (or stdout for certain logging and debugging information). For the same reason, the **interpreter** command will read

interactively from stdin.

For example, the following fragment creates a login process. Then it dials the user (not shown), and finally connects the two together. Of course, any process may be substituted for login. A shell, for example, would allow the user to work without supplying an account and password.

```
spawn login
set login $spawn_id
spawn tip modem
# dial back out to user
# connect user to login
interact -u $login
```

To send output to multiple processes, list each spawn id list prefaced by a **-output** flag. Input for a group of output spawn ids may be determined by a spawn id list prefaced by a **-input** flag. (Both **-input** and **-output** may take lists in the same form as the **-i** flag in the **expect** command, except that any_spawn_id is not meaningful in **interact**.) All following flags and strings (or patterns) apply to this input until another -input flag appears. If no **-input** appears, **-output** implies "-input $user_spawn_id -output". (Similarly, with patterns that do not have **-input**.) If one **-input** is specified, it overrides $user_spawn_id. If a second **-input** is specified, it overrides $spawn_id. Additional **-input** flags may be specified.

The two implied input processes default to having their outputs specified as $spawn_id and $user_spawn_id (in reverse). If a **-input** flag appears with no **-output** flag, characters from that process are discarded.

The **-i** flag introduces a replacement for the current spawn_id when no other **-input** or **-output** flags are used. A -i flag implies a -o flag.

It is possible to change the processes that are being interacted with by using indirect spawn ids. (Indirect spawn ids are described in the section on the expect command.) Indirect spawn ids may be specified with the -i, -u, -input, or -output flags.

interpreter [args]

causes the user to be interactively prompted for **Expect** and Tcl commands. The result of each command is printed.

Actions such as **break** and **continue** cause control structures (i.e., **for**, **proc**) to behave in the usual way. However **return** causes interpreter to return to its caller, while **inter_return** causes **interpreter** to cause a return in its caller. For example, if "proc foo" called **interpreter** which then executed the action **inter_return**, **proc foo** would return. Any other command causes **interpreter** to continue prompting for new commands.

By default, the prompt contains two integers. The first integer describes the depth of the evaluation stack (i.e., how many times Tcl_Eval has been called). The second integer is the Tcl history identifier. The prompt can be set by defining a procedure called "prompt1" whose return value becomes the next prompt. If a statement has open quotes, parens, braces, or brackets, a secondary prompt (by default "+> ") is issued upon newline. The secondary prompt may be set by defining a procedure called "prompt2".

During **interpreter**, cooked mode is used, even if the its caller was using raw mode.

If stdin is closed, **interpreter** will return unless the **-eof** flag is used, in which case the subsequent argument is invoked.

log_file ***[args] [[-a] file]***
If a filename is provided, **log_file** will record a transcript of the session (beginning at that point) in the file. **log_file** will stop recording if no argument is given. Any previous log file is closed. Instead of a filename, a Tcl file identifier may be provided by using the **-open** or **-leaveopen** flags. This is similar to the **spawn** command. (See **spawn** for more info.) The **-a** flag forces output to be logged that was suppressed by the **log_user** command. By default, the **log_file** command *appends* to old files rather than truncating them, for the convenience of being able to turn logging off and on multiple times in one session. To truncate files, use the **-noappend** flag. The **-info** flag causes log_file to return a description of the most recent non-info arguments given.

log_user ***-info|0|1***
By default, the send/expect dialogue is logged to stdout (and a logfile if open). The logging to stdout is disabled by the command "log_user 0" and reenabled by "log_user 1". Logging to the logfile is unchanged. The **-info** flag causes log_user to return a description of the most recent non-info arguments given.

match_max ***[-d] [-i spawn_id] [size]***
defines the size of the buffer (in bytes) used internally by **expect**. With no *size* argument, the current size is returned.

With the **-d** flag, the default size is set. (The initial default is 2000.) With the **-i** flag, the size is set for the named spawn id, otherwise it is set for the current process.

overlay ***[-# spawn_id] [-# spawn_id] [...] program [args]***
executes *program args* in place of the current **Expect** program, which terminates. A bare hyphen argument forces a hyphen in front of the command name as if it was a login shell. All spawn_ids are closed except for those named as arguments. These are mapped onto the named file identifiers.

Spawn_ids are mapped to file identifiers for the new program to inherit. For example, the following line runs chess and allows it to be controlled by the current process - say, a chess master.

```
    overlay -0 $spawn_id -1 $spawn_id -2 $spawn_id chess
```

This is more efficient than "interact -u", however, it sacrifices the ability to do programmed interaction since the **Expect** process is no longer in control.

Note that no controlling terminal is provided. Thus, if you disconnect or remap standard input, programs that do job control (shells, login, etc) will not function properly.

parity ***[-d] [-i spawn_id] [value]***
defines whether parity should be retained or stripped from the output of spawned processes. If *value* is zero, parity is stripped, otherwise it is not stripped. With no *value* argument, the current value is returned.

With the **-d** flag, the default parity value is set. (The initial default is 1, i.e., parity is not

stripped.) With the **-i** flag, the parity value is set for the named spawn id, otherwise it is set for the current process.

remove_nulls ***[-d] [-i spawn_id] [value]***

defines whether nulls are retained or removed from the output of spawned processes before pattern matching or storing in the variable *expect_out* or *interact_out*. If *value* is 1, nulls are removed. If *value* is 0, nulls are not removed. With no *value* argument, the current value is returned.

With the **-d** flag, the default value is set. (The initial default is 1, i.e., nulls are removed.) With the **-i** flag, the value is set for the named spawn id, otherwise it is set for the current process. Whether or not nulls are removed, **Expect** will record null bytes to the log and stdout.

send ***[-flags] string***

Sends *string* to the current process. For example, the command

```
send "hello world\r"
```

sends the characters, h e l l o <blank> w o r l d <return> to the current process. (Tcl includes a printf-like command (called **format**) which can build arbitrarily complex strings.)

Characters are sent immediately although programs with line-buffered input will not read the characters until a return character is sent. A return character is denoted "\r". The **--** flag forces the next argument to be interpreted as a string rather than a flag. Any string can be preceded by "--" whether or not it actually looks like a flag. This provides a reliable mechanism to specify variable strings without being tripped up by those that accidentally look like flags. (All strings starting with "-" are reserved for future options.) The **-i** flag declares that the string be sent to the named spawn_id. If the spawn_id is *user_spawn_id*, and the terminal is in raw mode, newlines in the string are translated to return-newline sequences so that they appear as if the terminal was in cooked mode. The **-raw** flag disables this translation. The **-null** flag sends null characters (0 bytes). By default, one null is sent. An integer may follow the **-null** to indicate how many nulls to send. The **-break** flag generates a break condition. This only makes sense if the spawn id refers to a tty device opened via "spawn -open". If you have spawned a process such as tip, you should use tip's convention for generating a break. The **-s** flag forces output to be sent "slowly", thus avoid the common situation where a computer outtypes an input buffer that was designed for a human who would never outtype the same buffer. This output is controlled by the value of the variable "send_slow" which takes a two element list. The first element is an integer that describes the number of bytes to send atomically. The second element is a real number that describes the number of seconds by which the atomic sends must be separated. For example, "set send_slow {10 .001}" would force "send -s" to send strings with 1 millisecond in between each 10 characters sent. The **-h** flag forces output to be sent (somewhat) like a human actually typing. Human-like delays appear between the characters. (The algorithm is based upon a Weibull distribution, with modifications to suit this particular application.) This output is controlled by the value of the variable "send_human" which takes a five element list. The first two elements are average interarrival time of characters in seconds. The first is used by default. The second is used at word endings, to simulate the subtle pauses that occasionally occur at such transitions. The third parameter is a measure of variability where .1 is quite variable, 1 is reasonably variable, and 10 is quite invariable. The extremes are 0 to infinity. The last two parameters are, respectively, a minimum and maximum interarrival time. The minimum and maximum are

used last and "clip" the final time. The ultimate average can be quite different from the given average if the minimum and maximum clip enough values. As an example, the following command emulates a fast and consistent typist:

```
set send_human {.1 .3 1 .05 2}
send -h "I'm hungry.  Let's do lunch."
```

while the following might be more suitable after a hangover:

```
set send_human {.4 .4 .2 .5 100}
send -h "Goodd party lash night!"
```

Note that errors are not simulated, although you can set up error correction situations yourself by embedding mistakes and corrections in a send argument.

The flags for sending null characters, for sending breaks, for forcing slow output and for human-style output are mutually exclusive. Only the one specified last will be used. Furthermore, no *string* argument can be specified with the flags for sending null characters or breaks.

It is a good idea to precede the first **send** to a process by an **expect**. **expect** will wait for the process to start, while **send** cannot. In particular, if the first **send** completes before the process starts running, you run the risk of having your data ignored. In situations where interactive programs offer no initial prompt, you can precede **send** by a delay as in:

```
# To avoid giving hackers hints on how to break in,
# this system does not prompt for an external password.
# Wait for 5 seconds for exec to complete
spawn telnet very.secure.gov
sleep 5
send password\r
```

exp_send is an alias for **send**. If you are using Expectk or some other variant of Expect in the Tk environment, **send** is defined by Tk for an entirely different purpose. **exp_send** is provided for compatibility between environments. Similar aliases are provided for other Expect's other send commands.

send_error ***[-flags] string***
: is like **send**, except that the output is sent to stderr rather than the current process.

send_log ***[--] string***
: is like **send**, except that the string is only sent to the log file (see **log_file**.) The arguments are ignored if no log file is open.

send_tty ***[-flags] string***
: is like **send**, except that the output is sent to /dev/tty rather than the current process.

send_user ***[-flags] string***
: is like **send**, except that the output is sent to stdout rather than the current process.

sleep ***seconds***
: causes the script to sleep for the given number of seconds. Seconds may be a decimal number. Interrupts (and Tk events if you are using Expectk) are processed while Expect sleeps.

spawn ***[args] program [args]***

creates a new process running *program args*. Its stdin, stdout and stderr are connected to Expect, so that they may be read and written by other **Expect** commands. The connection is broken by **close** or if the process itself closes any of the file identifiers.

When a process is started by **spawn**, the variable *spawn_id* is set to a descriptor referring to that process. The process described by *spawn_id* is considered the *current process*. *spawn_id* may be read or written, in effect providing job control.

user_spawn_id is a global variable containing a descriptor which refers to the user. For example, when *spawn_id* is set to this value, **expect** behaves like **expect_user**. *error_spawn_id* is a global variable containing a descriptor which refers to the standard error. For example, when *spawn_id* is set to this value, **send** behaves like **send_error**.

tty_spawn_id is a global variable containing a descriptor which refers to /dev/tty. If /dev/tty does not exist (such as in a cron, at, or batch script), then *tty_spawn_id* is not defined. This may be tested as:

```
if {[info vars tty_spawn_id]} {
    # /dev/tty exists
} else {
    # /dev/tty doesn't exist
    # probably in cron, batch, or at script
}
```

spawn returns the UNIX process id. If no process is spawned, 0 is returned. The variable *spawn_out(slave,name)* is set to the name of the pty slave device.

By default, **spawn** echoes the command name and arguments. The **-noecho** flag stops **spawn** from doing this.

The **-console** flag causes console output to be redirected to the spawned process. This is not supported on all systems. Internally, **spawn** uses a pty, initialized the same way as the user's tty. This is further initialized so that all settings are "sane" (according to stty(1)). If the variable *stty_init* is defined, it is interpreted in the style of stty arguments as further configuration. For example, "set stty_init raw" will cause further spawned processes's terminals to start in raw mode. **-nottycopy** skips the initialization based on the user's tty. **-nottyinit** skips the "sane" initialization.

Normally, **spawn** takes little time to execute. If you notice spawn taking a significant amount of time, it is probably encountering ptys that are wedged. A number of tests are run on ptys to avoid entanglements with errant processes. (These take 10 seconds per wedged pty.) Running Expect with the **-d** option will show if **Expect** is encountering many ptys in odd states. If you cannot kill the processes to which these ptys are attached, your only recourse may be to reboot. If *program* cannot be spawned successfully because exec(2) fails (e.g. when *program* doesn't exist), an error message will be returned by the next **interact** or **expect** command as if *program* had run and produced the error message as output. This behavior is a natural consequence of the implementation of **spawn**. Internally, spawn forks,

after which the spawned process has no way to communicate with the original **Expect** process except by communication via the spawn_id. The **-open** flag causes the next argument to be interpreted as a Tcl file identifier (i.e., returned by **open**.) The spawn id can then be used as if it were a spawned process. (The file identifier should no longer be used.) This lets you treat raw devices, files, and pipelines as spawned processes without using a pty. 0 is returned to indicate there is no associated process. When the connection to the spawned process is closed, so is the Tcl file identifier. The **-leaveopen** flag is similar to **-open** except that **-leaveopen** causes the file identifier to be left open even after the spawn id is closed. The **-pty** flag causes a pty to be opened but no process spawned. 0 is returned to indicate there is no associated process. Spawn_id is set as usual. The variable *spawn_out(slave,fd)* is set to a file identifier corresponding to the pty slave. It can be closed using "close -slave". The **-ignore** flag names a signal to be ignored in the spawned process. Otherwise, signals get the default behavior. Signals are named as in the **trap** command, except that each signal requires a separate flag.

strace *level*

causes following statements to be printed before being executed. (Tcl's trace command traces variables.) *level* indicates how far down in the call stack to trace. For example, the following command runs **Expect** while tracing the first 4 levels of calls, but none below that.

```
expect -c "strace 4" script.exp
```

The **-info** flag causes strace to return a description of the most recent non-info arguments given.

stty *args*

changes terminal modes similarly to the external stty command. By default, the controlling terminal is accessed. Other terminals can be accessed by appending "< /dev/tty..." to the command. (Note that the arguments should not be grouped into a single argument.) Requests for status return it as the result of the command. If no status is requested and the controlling terminal is accessed, the previous status of the raw and echo attributes are returned in a form which can later be used by the command. For example, the arguments **raw** or **-cooked** put the terminal into raw mode. The arguments **-raw** or **cooked** put the terminal into cooked mode. The arguments **echo** and **-echo** put the terminal into echo and noecho mode respectively.

The following example illustrates how to temporarily disable echoing. This could be used in otherwise-automatic scripts to avoid embedding passwords in them. (See more discussion on this under EXPECT HINTS below.)

```
stty -echo
send_user "Password: "
expect_user -re "(.*)\n"
set password $expect_out(1,string)
stty echo
```

system *args*

gives *args* to sh(1) as input, just as if it had been typed as a command from a terminal. **Expect** waits until the shell terminates. The return status from sh is handled the same way that **exec** handles its return status.

In contrast to **exec** which redirects stdin and stdout to the script, **system** performs no redirection (other than that indicated by the string itself). Thus, it is possible to use programs which must talk directly to /dev/tty. For the same reason, the results of **system** are not recorded in the log.

timestamp *[args]*

returns a timestamp. With no arguments, the number of seconds since the epoch is returned. The **-format** flag introduces a string which is returned but with substitutions made according to the POSIX rules for strftime. For example %a is replaced by an abbreviated weekday name (i.e., Sat). Others are:

```
%a      abbreviated weekday name
%A      full weekday name
%b      abbreviated month name
%B      full month name
%c      date-time as in: Wed Oct  6 11:45:56 1993
%d      day of the month (01-31)
%H      hour (00-23)
%I      hour (01-12)
%j      day (001-366)
%m      month (01-12)
%M      minute (00-59)
%p      am or pm
%S      second (00-61)
%u      day (1-7, Monday is first day of week)
%U      week (00-53, first Sunday is first day of week one)
%V      week (01-53, ISO 8601 style)
%w      day (0-6)
%W      week (00-53, first Monday is first day of week one)
%x      date-time as in: Wed Oct  6 1993
%X      time as in: 23:59:59
%y      year (00-99)
%Y      year as in: 1993
%Z      timezone (or nothing if not determinable)
%%      a bare percent sign
```

Other % specifications are undefined. Other characters will be passed through untouched. Only the C locale is supported.

The **-seconds** flag introduces a number of seconds since the epoch to be used as a source from which to format. Otherwise, the current time is used.

The **-gmt** flag forces timestamp output to use the GMT timezone. With no flag, the local timezone is used.

trap *[[command] signals]*

causes the given *command* to be executed upon future receipt of any of the given signals. The command is executed in the global scope. If *command* is absent, the signal action is returned. If *command* is the string SIG_IGN, the signals are ignored. If *command* is the string SIG_DFL, the signals are result to the system default. *signals* is either a single signal or a list

of signals. Signals may be specified numerically or symbolically as per signal(3). The "SIG" prefix may be omitted. With no arguments (or the argument -number), **trap** returns the signal number of the trap command currently being executed. The **-code** flag uses the return code of the command in place of whatever code Tcl was about to return when the command originally started running. The **-interp** flag causes the command to be evaluated using the interpreter active at the time the command started running rather than when the trap was declared. The **-name** flag causes the **trap** command to return the signal name of the trap command currently being executed. The **-max** flag causes the **trap** command to return the largest signal number that can be set. For example, the command "trap {send_user "Ouch!"} SIGINT" will print "Ouch!" each time the user presses ^C. By default, SIGINT (which can usually be generated by pressing ^C) and SIGTERM cause Expect to exit. This is due to the following trap, created by default when Expect starts.

```
    trap exit {SIGINT SIGTERM}
```

If you use the -D flag to start the debugger, SIGINT is redefined to start the interactive debugger. This is due to the following trap:

```
    trap {exp_debug 1} SIGINT
```

The debugger trap can be changed by setting the environment variable EXPECT_DEBUG_INIT to a new trap command.

You can, of course, override both of these just by adding trap commands to your script. In particular, if you have your own "trap exit SIGINT", this will override the debugger trap. This is useful if you want to prevent users from getting to the debugger at all.

If you want to define your own trap on SIGINT but still trap to the debugger when it is running, use:

```
    if {![exp_debug]} {trap mystuff SIGINT}
```

Alternatively, you can trap to the debugger using some other signal.

trap will not let you override the action for SIGALRM as this is used internally to **Expect**. The disconnect command sets SIGALRM to SIG_IGN (ignore). You can reenable this as long as you disable it during subsequent spawn commands.

See signal(3) for more info.

wait *[args]*

delays until a spawned process (or the current process if none is named) terminates.

wait normally returns a list of four integers. The first integer is the pid of the process that was waited upon. The second integer is the corresponding spawn id. The third integer is -1 if an operating system error occurred, or 0 otherwise. If the third integer was 0, the fourth integer is the status returned by the spawned process. If the third integer was -1, the fourth integer is the value of errno set by the operating system. The global variable errorCode is also set. Additional elements may appear at the end of the return value from **wait**. An optional fifth element identifies a class of information. Currently, the only possible value for this element is CHILDKILLED in which case the next two values are the C-style signal name and a short textual description.

The **-i** flag declares the process to wait corresponding to the named spawn_id (NOT the process id). Inside a SIGCHLD handler, it is possible to wait for any spawned process by using the spawn id -1. The **-nowait** flag causes the wait to return immediately with the indication of a successful wait. When the process exits (later), it will automatically disappear without the need for an explicit wait. The **wait** command may also be used wait for a forked process using the arguments "-i -1". Unlike its use with spawned processes, this command can be executed at any time. There is no control over which process is reaped. However, the return value can be checked for the process id.

Libraries

Expect automatically knows about two built-in libraries for Expect scripts. These are defined by the directories named in the variables exp_library and exp_exec_library. Both are meant to contain utility files that can be used by other scripts.

exp_library contains architecture-independent files. exp_exec_library contains architecture-dependent files. Depending on your system, both directories may be totally empty. The existence of the file $exp_exec_library/cat-buffers describes whether your /bin/cat buffers by default.

Pretty-printing

A vgrind definition is available for pretty-printing **Expect** scripts. Assuming the vgrind definition supplied with the **Expect** distribution is correctly installed, you can use it as:

```
    vgrind -lexpect file
```

Examples

It many not be apparent how to put everything together that the man page describes. I encourage you to read and try out the examples in the example directory of the **Expect** distribution. Some of them are real programs. Others are simply illustrative of certain techniques, and of course, a couple are just quick hacks. The INSTALL file has a quick overview of these programs.

The **Expect** papers (see SEE ALSO) are also useful. While some papers use syntax corresponding to earlier versions of Expect, the accompanying rationales are still valid and go into a lot more detail than this man page.

Caveats

Extensions may collide with Expect's command names. For example, **send** is defined by Tk for an entirely different purpose. For this reason, most of the **Expect** commands are also available as "exp_XXXX". Commands and variables beginning with "exp", "inter", "spawn", and "timeout" do not have aliases. Use the extended command names if you need this compatibility between environments.

Expect takes a rather liberal view of scoping. In particular, variables read by commands specific to the **Expect** program will be sought first from the local scope, and if not found, in the global scope. For example, this obviates the need to place "global timeout" in every procedure you write that uses **expect**. On the other hand, variables written are always in the local scope (unless a "global" command has been issued). The most common problem this causes is when spawn is executed in a procedure. Outside the procedure, *spawn_id* no longer exists, so the

spawned process is no longer accessible simply because of scoping. Add a "global spawn_id" to such a procedure.

If you cannot enable the multispawning capability (i.e., your system supports neither select (BSD *.*), poll (SVR>2), nor something equivalent), **Expect** will only be able to control a single process at a time. In this case, do not attempt to set *spawn_id*, nor should you execute processes via exec while a spawned process is running. Furthermore, you will not be able to **expect** from multiple processes (including the user as one) at the same time.

Terminal parameters can have a big effect on scripts. For example, if a script is written to look for echoing, it will misbehave if echoing is turned off. For this reason, Expect forces sane terminal parameters by default. Unfortunately, this can make things unpleasant for other programs. As an example, the emacs shell wants to change the "usual" mappings: newlines get mapped to newlines instead of carriage-return newlines, and echoing is disabled. This allows one to use emacs to edit the input line. Unfortunately, Expect cannot possibly guess this.

You can request that Expect not override its default setting of terminal parameters, but you must then be very careful when writing scripts for such environments. In the case of emacs, avoid depending upon things like echoing and end-of-line mappings.

The commands that accepted arguments braced into a single list (the **expect** variants and **interact**) use a heuristic to decide if the list is actually one argument or many. The heuristic can fail only in the case when the list actually does represent a single argument which has multiple embedded \n's with non-whitespace characters between them. This seems sufficiently improbable, however the argument "-nobrace" can be used to force a single argument to be handled as a single argument. This could conceivably be used with machine-generated Expect code. Similarly, -brace forces a single argument to be handle as multiple patterns/actions.

Bugs

It was really tempting to name the program "sex" (for either "Smart EXec" or "Send-EXpect"), but good sense (or perhaps just Puritanism) prevailed.

On some systems, when a shell is spawned, it complains about not being able to access the tty but runs anyway. This means your system has a mechanism for gaining the controlling tty that **Expect** doesn't know about. Please find out what it is, and send this information back to me.

Ultrix 4.1 (at least the latest versions around here) considers timeouts of above 1000000 to be equivalent to 0.

Digital UNIX 4.0A (and probably other versions) refuses to allocate ptys if you define a SIGCHLD handler. See grantpt page for more info.

IRIX 6.0 does not handle pty permissions correctly so that if Expect attempts to allocate a pty previously used by someone else, it fails. Upgrade to IRIX 6.1.

Telnet (verified only under SunOS 4.1.2) hangs if TERM is not set. This is a problem under cron, at and in cgi scripts, which do not define TERM. Thus, you must set it explicitly - to what type is usually irrelevant. It just has to be set to something! The following probably suffices for most cases.

```
    set env(TERM) vt100
```

Tip (verified only under BSDI BSD/OS 3.1 i386) hangs if SHELL and HOME are not set. This is a problem under cron, at and in cgi scripts, which do not define these environment variables.

Thus, you must set them explicitly - to what type is usually irrelevant. It just has to be set to something! The following probably suffices for most cases.

```
    set env(SHELL) /bin/sh
    set env(HOME) /usr/local/bin
```

Some implementations of ptys are designed so that the kernel throws away any unread output after 10 to 15 seconds (actual number is implementation-dependent) after the process has closed the file descriptor. Thus **Expect** programs such as

```
    spawn date
    sleep 20
    expect
```

will fail. To avoid this, invoke non-interactive programs with **exec** rather than **spawn**. While such situations are conceivable, in practice I have never encountered a situation in which the final output of a truly interactive program would be lost due to this behavior.

On the other hand, Cray UNICOS ptys throw away any unread output immediately after the process has closed the file descriptor. I have reported this to Cray and they are working on a fix.

Sometimes a delay is required between a prompt and a response, such as when a tty interface is changing UART settings or matching baud rates by looking for start/stop bits. Usually, all this is require is to sleep for a second or two. A more robust technique is to retry until the hardware is ready to receive input. The following example uses both strategies:

```
    send "speed 9600\r";
    sleep 1
    expect {
        timeout {send "\r"; exp_continue}
        $prompt
    }
```

trap -code will not work with any command that sits in Tcl's event loop, such as sleep. The problem is that in the event loop, Tcl discards the return codes from async event handlers. A workaround is to set a flag in the trap code. Then check the flag immediately after the command (i.e., sleep).

The expect_background command ignores -timeout arguments and has no concept of timeouts in general.

Expect hints

There are a couple of things about **Expect** that may be non-intuitive. This section attempts to address some of these things with a couple of suggestions.

A common expect problem is how to recognize shell prompts. Since these are customized differently by differently people and different shells, portably automating rlogin can be difficult without knowing the prompt. A reasonable convention is to have users store a regular expression describing their prompt (in particular, the end of it) in the environment variable EXPECT_PROMPT. Code like the following can be used. If EXPECT_PROMPT doesn't exist, the code still has a good chance of functioning correctly.

```
    set prompt "(%|#|\\$) $"          ;# default prompt
    catch {set prompt $env(EXPECT_PROMPT)}

    expect -re $prompt
```

I encourage you to write **expect** patterns that include the end of whatever you expect to see. This avoids the possibility of answering a question before seeing the entire thing. In addition, while you may well be able to answer questions before seeing them entirely, if you answer early, your answer may appear echoed back in the middle of the question. In other words, the resulting dialogue will be correct but look scrambled.

Most prompts include a space character at the end. For example, the prompt from ftp is 'f', 't', 'p', '>' and <blank>. To match this prompt, you must account for each of these characters. It is a common mistake not to include the blank. Put the blank in explicitly.

If you use a pattern of the form X*, the * will match all the output received from the end of X to the last thing received. This sounds intuitive but can be somewhat confusing because the phrase "last thing received" can vary depending upon the speed of the computer and the processing of I/O both by the kernel and the device driver.

In particular, humans tend to see program output arriving in huge chunks (atomically) when in reality most programs produce output one line at a time. Assuming this is the case, the * in the pattern of the previous paragraph may only match the end of the current line even though there seems to be more, because at the time of the match that was all the output that had been received.

expect has no way of knowing that further output is coming unless your pattern specifically accounts for it.

Even depending on line-oriented buffering is unwise. Not only do programs rarely make promises about the type of buffering they do, but system indigestion can break output lines up so that lines break at seemingly random places. Thus, if you can express the last few characters of a prompt when writing patterns, it is wise to do so.

If you are waiting for a pattern in the last output of a program and the program emits something else instead, you will not be able to detect that with the **timeout** keyword. The reason is that **expect** will not timeout - instead it will get an **eof** indication. Use that instead. Even better, use both. That way if that line is ever moved around, you won't have to edit the line itself.

Newlines are usually converted to carriage return, linefeed sequences when output by the terminal driver. Thus, if you want a pattern that explicitly matches the two lines, from, say, printf("foo\nbar"), you should use the pattern "foo\r\nbar".

A similar translation occurs when reading from the user, via **expect_user**. In this case, when you press return, it will be translated to a newline. If **Expect** then passes that to a program which sets its terminal to raw mode (like telnet), there is going to be a problem, as the program expects a true return. (Some programs are actually forgiving in that they will automatically translate newlines to returns, but most don't.) Unfortunately, there is no way to find out that a program put its terminal into raw mode.

Rather than manually replacing newlines with returns, the solution is to use the command "stty raw", which will stop the translation. Note, however, that this means that you will no longer get the cooked line-editing features.

interact implicitly sets your terminal to raw mode so this problem will not arise then.

It is often useful to store passwords (or other private information) in **Expect** scripts. This is not recommended since anything that is stored on a computer is susceptible to being accessed by anyone. Thus, interactively prompting for passwords from a script is a smarter idea than embedding them literally. Nonetheless, sometimes such embedding is the only possibility.

Unfortunately, the UNIX file system has no direct way of creating scripts which are executable but unreadable. Systems which support setgid shell scripts may indirectly simulate this as follows:

Create the **Expect** script (that contains the secret data) as usual. Make its permissions be 750 (-rwxr-x---) and owned by a trusted group, i.e., a group which is allowed to read it. If necessary, create a new group for this purpose. Next, create a /bin/sh script with permissions 2751 (-rwxr-s--x) owned by the same group as before.

The result is a script which may be executed (and read) by anyone. When invoked, it runs the **Expect** script.

Author

Don Libes, National Institute of Standards and Technology

Acknowledgments

Thanks to John Ousterhout for Tcl, and Scott Paisley for inspiration. Thanks to Rob Savoye for Expect's autoconfiguration code.

The HISTORY file documents much of the evolution of **expect**. It makes interesting reading and might give you further insight to this software. Thanks to the people mentioned in it who sent me bug fixes and gave other assistance.

Design and implementation of **Expect** was paid for in part by the U.S. government and is therefore in the public domain. However the author and NIST would like credit if this program and documentation or portions of them are used.

Keywords: expect

libexpect

Name

libexpect - programmed dialogue library with interactive programs

Description

This library contains functions that allow Expect to be used as a Tcl extension or to be used directly from C or C++ (without Tcl). Adding Expect as a Tcl extension is very short and simple, so that will be covered first.

Synopsis

#include expect_tcl.h
Expect_Init(interp);

cc files... -lexpect5.20 -ltcl7.5 -lm

Note: library versions may differ in the actual release.

The Expect_Init function adds expect commands to the named interpreter. It avoids overwriting commands that already exist, however aliases beginning with "exp_" are always created for expect commands. So for example, "send" can be used as "exp_send".

Generally, you should only call Expect commands via Tcl_Eval. Certain auxiliary functions may be called directly. They are summarized below. They may be useful in constructing your own main. Look at the file exp_main_exp.c in the Expect distribution as a prototype main. Another prototype is tclAppInit.c in the Tcl source distribution. A prototype for working with Tk is in exp_main_tk.c in the Expect distribution.

```
int exp_cmdlinecmds;
int exp_interactive;
FILE *exp_cmdfile;
char *exp_cmdfilename;
int exp_tcl_debugger_available;

void    exp_parse_argv(Tcl_Interp *,int argc,char **argv);
int exp_interpreter(Tcl_Interp *);
void    exp_interpret_cmdfile(Tcl_Interp *,FILE *);
void    exp_interpret_cmdfilename(Tcl_Interp *,char *);
void    exp_interpret_rcfiles(Tcl_Interp *,int my_rc,int sys_rc);
char *  exp_cook(char *s,int *len);
void    (*exp_app_exit)EXP_PROTO((Tcl_Interp *);
void    exp_exit(Tcl_Interp *,int status);
void    exp_exit_handlers(Tcl_Interp *);
void    exp_error(Tcl_Interp,char *,...);
```

exp_cmdlinecmds is 1 if Expect has been invoked with commands on the program command-line (using "-c" for example). **exp_interactive** is 1 if Expect has been invoked with the -i flag or if no commands or script is being invoked. **exp_cmdfile** is a stream from which Expect will read commands. **exp_cmdfilename** is the name of a file which Expect will open and read commands from. **exp_tcl_debugger_available** is 1 if the debugger has been armed.

exp_parse_argv reads the representation of the command line. Based on what is found, any of the other variables listed here are initialized appropriately. **exp_interpreter** interactively prompts the user for commands and evaluates them. **exp_interpret_cmdfile** reads the given stream and evaluates any commands found. **exp_interpret_cmdfilename** opens the named file and evaluates any commands found. **exp_interpret_rcfiles** reads and evalutes the .rc files. If my_rc is zero, then ~/.expectrc is skipped. If sys_rc is zero, then the system-wide expectrc file is skipped. **exp_cook** returns a static buffer containing the argument reproduced with newlines replaced by carriage-return linefeed sequences. The primary purpose of this is to allow messages to be produced without worrying about whether the terminal is in raw mode or cooked mode. If length is zero, it is computed via strlen. **exp_error is a printf-like function that writes the result** to interp->result.

Synopsis

#include <expect.h>

int

exp_spawnl(file, arg0 [, arg1, ..., argn] (char *)0);
char *file;
char *arg0, *arg1, ... *argn;

int
exp_spawnv(file,argv);
char *file, *argv[ ];

int
exp_spawnfd(fd);
int fd;

FILE *
exp_popen(command);
char *command;

extern int exp_pid;
extern int exp_ttyinit;
extern int exp_ttycopy;
extern int exp_console;
extern char *exp_stty_init;
extern void (*exp_close_in_child)();
extern void (*exp_child_exec_prelude)();
extern void **exp_close_tcl_files**();

cc files... -lexpect -ltcl -lm

Description

exp_spawnl and **exp_spawnv** fork a new process so that its stdin, stdout, and stderr can be written and read by the current process. *file* is the name of a file to be executed. The *arg* pointers are null-terminated strings. Following the style of execve(), *arg0* (or *argv[0]*) is customarily a duplicate of the name of the file.

Four interfaces are available, **exp_spawnl** is useful when the number of arguments is known at compile time. **exp_spawnv** is useful when the number of arguments is not known at compile time. **exp_spawnfd** is useful when an open file descriptor is already available as a source. **exp_popen** is explained later on.

If the process is successfully created, a file descriptor is returned which corresponds to the process's stdin, stdout and stderr. A stream may be associated with the file descriptor by using fdopen(). (This should almost certainly be followed by setbuf() to unbuffer the I/O.)

Closing the file descriptor will typically be detected by the process as an EOF. Once such a process exits, it should be waited upon (via wait) in order to free up the kernel process slot. (Some systems allow you to avoid this if you ignore the SIGCHLD signal).

exp_popen is yet another interface, styled after popen(). It takes a Bourne shell command line, and returns a stream that corresponds to the process's stdin, stdout and stderr. The actual implementation of **exp_popen** below demonstrates **exp_spawnl**.

```
FILE *
exp_popen(program)
char *program;
{
    FILE *fp;
    int ec;

    if (0 > (ec = exp_spawnl("sh","sh","-c",program,(char *)0)))
        return(0);
    if (NULL == (fp = fdopen(ec,"r+")) return(0);
    setbuf(fp,(char *)0);
    return(fp);
}
```

After a process is started, the variable **exp_pid** is set to the process-id of the new process. The variable **exp_pty_slave_name** is set to the name of the slave side of the pty.

The spawn functions uses a pty to communicate with the process. By default, the pty is initialized the same way as the user's tty (if possible, i.e., if the environment has a controlling terminal.) This initialization can be skipped by setting exp_ttycopy to 0.

The pty is further initialized to some system wide defaults if exp_ttyinit is non-zero. The default is generally comparable to "stty sane".

The tty setting can be further modified by setting the variable **exp_stty_init**. This variable is interpreted in the style of stty arguments. For example, exp_stty_init = "sane"; repeats the default initialization.

On some systems, it is possible to redirect console output to ptys. If this is supported, you can force the next spawn to obtain the console output by setting the variable **exp_console** to 1.

Between the time a process is started and the new program is given control, the spawn functions can clean up the environment by closing file descriptors. By default, the only file descriptors closed are ones internal to Expect and any marked "close-on-exec".

If needed, you can close additional file descriptors by creating an appropriate function and assigning it to exp_close_in_child. The function will be called after the fork and before the exec. (This also modifies the behavior of the spawn command in Expect.)

If you are also using Tcl, it may be convenient to use the function exp_close_tcl_files which closes all files between the default standard file descriptors and the highest descriptor known to Tcl. (Expect does this.)

The function exp_child_exec_prelude is the last function called prior to the actual exec in the child. You can redefine this for effects such as manipulating the uid or the signals.

If you want to allocate your own pty

```
extern int exp_autoallocpty;
extern int exp_pty[2];
```

The spawn functions use a pty to communicate with the process. By default, a pty is automatically allocated each time a process is spawned. If you want to allocate ptys yourself, before calling one of the spawn functions, set **exp_autoallocpty** to 0, **exp_pty[0]** to the

master pty file descriptor and **exp_pty[1]** to the slave pty file descriptor. The expect library will not do any pty initializations (e.g., exp_stty_init will not be used). The slave pty file descriptor will be automatically closed when the process is spawned. After the process is started, all further communication takes place with the master pty file descriptor.

exp_spawnl and **exp_spawnv** duplicate the shell's actions in searching for an executable file in a list of directories. The directory list is obtained from the environment.

Expect processing

While it is possible to use read() to read information from a process spawned by **exp_spawnl** or **exp_spawnv**, more convenient functions are provided. They are as follows:

```
int
exp_expectl(fd,type1,pattern1,[re1,],value1,type2,...,exp_end);
int fd;
enum exp_type type;
char *pattern1, *pattern2, ...;
regexp *re1, *re2, ...;
int value1, value2, ...;

int
exp_fexpectl(fp,type1,pattern1,[re1,]value1,type2,...,exp_end);
FILE *fp;
enum exp_type type;
char *pattern1, *pattern2, ...;
regexp *re1, *re2, ...;
int value1, value2, ...;

enum exp_type {
exp_end,
exp_glob,
exp_exact,
exp_regexp,
exp_compiled,
exp_null,
};

struct exp_case {
char *pattern;
regexp *re;
enum exp_type type;
int value;
};

int
exp_expectv(fd,cases);
int fd;
```

```
struct exp_case *cases;

int
exp_fexpectv(fp,cases);
FILE *fp;
struct exp_case *cases;

extern int exp_timeout;
extern char *exp_match;
extern char *exp_match_end;
extern char *exp_buffer;
extern char *exp_buffer_end;
extern int exp_match_max;
extern int exp_full_buffer;
extern int exp_remove_nulls;
```

The functions wait until the output from a process matches one of the patterns, a specified time period has passed, or an EOF is seen.

The first argument to each function is either a file descriptor or a stream. Successive sets of arguments describe patterns and associated integer values to return when the pattern matches.

The type argument is one of four values. exp_end indicates that no more patterns appear. exp_glob indicates that the pattern is a glob-style string pattern. exp_exact indicates that the pattern is an exact string. exp_regexp indicates that the pattern is a regexp-style string pattern. exp_compiled indicates that the pattern is a regexp-style string pattern, and that its compiled form is also provided. exp_null indicates that the pattern is a null (for debugging purposes, a string pattern must also follow).

If the compiled form is not provided with the functions **exp_expectl** and **exp_fexpectl**, any pattern compilation done internally is thrown away after the function returns. The functions **exp_expectv** and **exp_fexpectv** will automatically compile patterns and will not throw them away. Instead, they must be discarded by the user, by calling free on each pattern. It is only necessary to discard them, the last time the cases are used.

Regexp subpatterns matched are stored in the compiled regexp. Assuming "re" contains a compiled regexp, the matched string can be found in re->startp[0]. The match substrings (according to the parentheses) in the original pattern can be found in re->startp[1], re->startp[2], and so on, up to re->startp[9]. The corresponding strings ends are re->endp[x] where x is that same index as for the string start.

The type exp_null matches if a null appears in the input. The variable exp_remove_nulls must be set to 0 to prevent nulls from being automatically stripped. By default, exp_remove_nulls is set to 1 and nulls are automatically stripped.

exp_expectv and **exp_fexpectv** are useful when the number of patterns is not known in advance. In this case, the sets are provided in an array. The end of the array is denoted by a struct exp_case with type exp_end. For the rest of this discussion, these functions will be referred to generically as *expect.*

If a pattern matches, then the corresponding integer value is returned. Values need not be unique, however they should be positive to avoid being mistaken for EXP_EOF, EXP_TIMEOUT, or

EXP_FULLBUFFER. Upon EOF or timeout, the value **EXP_EOF** or **EXP_TIMEOUT** is returned. The default timeout period is 10 seconds but may be changed by setting the variable **exp_timeout**. A value of -1 disables a timeout from occurring. A value of 0 causes the expect function to return immediately (i.e., poll) after one read(). However it must be preceded by a function such as select, poll, or an event manager callback to guarantee that there is data to be read.

If the variable exp_full_buffer is 1, then EXP_FULLBUFFER is returned if exp_buffer fills with no pattern having matched.

When the expect function returns, **exp_buffer** points to the buffer of characters that was being considered for matching. **exp_buffer_end** points to one past the last character in exp_buffer. If a match occurred, **exp_match** points into **exp_buffer** where the match began. **exp_match_end** points to one character past where the match ended.

Each time new input arrives, it is compared to each pattern in the order they are listed. Thus, you may test for absence of a match by making the last pattern something guaranteed to appear, such as a prompt. In situations where there is no prompt, you must check for **EXP_TIMEOUT** (just like you would if you were interacting manually). More philosophy and strategies on specifying **expect** patterns can be found in the documentation on the **expect** program itself. See SEE ALSO below.

Patterns are the usual C-shell-style regular expressions. For example, the following fragment looks for a successful login, such as from a telnet dialogue.

```
    switch (exp_expectl(
        exp_glob,"connected",CONN,
        exp_glob,"busy",BUSY,
        exp_glob,"failed",ABORT,
        exp_glob,"invalid password",ABORT,
        exp_end)) {
    case CONN:  /* logged in successfully */
        break;
    case BUSY:  /* couldn't log in at the moment */
        break;
    case EXP_TIMEOUT:
    case ABORT: /* can't log in at any moment! */
        break;
    default: /* problem with expect */
    }
```

Asterisks (as in the example above) are a useful shorthand for omitting line-termination characters and other detail. Patterns must match the entire output of the current process (since the previous read on the descriptor or stream). More than 2000 bytes of output can force earlier bytes to be "forgotten". This may be changed by setting the variable **exp_match_max**. Note that excessively large values can slow down the pattern matcher.

Running in the background

```
extern int exp_disconnected;
int exp_disconnect();
```

It is possible to move a process into the background after it has begun running. A typical use for this is to read passwords and then go into the background to sleep before using the passwords to do real work.

To move a process into the background, fork, call exp_disconnect() in the child process and exit() in the parent process. This disassociates your process from the controlling terminal. If you wish to move a process into the background in a different way, you must set the variable exp_disconnected to 1. This allows processes spawned after this point to be started correctly.

Multiplexing

By default, the expect functions block inside of a read on a single file descriptor. If you want to wait on patterns from multiple file descriptors, use select, poll, or an event manager. They will tell you what file descriptor is ready to read.

When a file descriptor is ready to read, you can use the expect functions to do one and only read by setting timeout to 0.

Slave control

```
void
exp_slave_control(fd,enable)
int fd;
int enable;
```

Pty trapping is normally done automatically by the expect functions. However, if you want to issue an ioctl, for example, directly on the slave device, you should temporary disable trapping.

Pty trapping can be controlled with exp_slave_control. The first argument is the file descriptor corresponding to the spawned process. The second argument is a 0 if trapping is to be disabled and 1 if it is to be enabled.

Errors

All functions indicate errors by returning -1 and setting errno.

Errors that occur after the spawn functions fork (e.g., attempting to spawn a non-existent program) are written to the process's stderr, and will be read by the first **expect**.

Signals

```
extern int exp_reading;
extern jmp_buf exp_readenv;
```

expect uses alarm() to timeout, thus if you generate alarms during **expect**, it will timeout prematurely.

Internally, **expect** calls read() which can be interrupted by signals. If you define signal handlers, you can choose to restart or abort **expect**'s internal read. The variable, **exp_reading**, is true if (and only if) **expect**'s read has been interrupted. longjmp(exp_readenv,EXP_ABORT) will abort the read. longjmp(exp_readenv,EXP_RESTART) will restart the read.

Logging

```
extern int exp_loguser;
extern int exp_logfile_all
extern FILE *exp_logfile;
```

If **exp_loguser** is nonzero, **expect** sends any output from the spawned process to stdout. Since interactive programs typically echo their input, this usually suffices to show both sides of the conversation. If **exp_logfile** is also nonzero, this same output is written to the stream defined by **exp_logfile**. If **exp_logfile_all** is non-zero, **exp_logfile** is written regardless of the value of **exp_loguser**.

Debugging

While I consider the library to be easy to use, I think that the standalone expect program is much, much, easier to use than working with the C compiler and its usual edit, compile, debug cycle. Unlike typical C programs, most of the debugging isn't getting the C compiler to accept your programs - rather, it is getting the dialogue correct. Also, translating scripts from expect to C is usually not necessary. For example, the speed of interactive dialogues is virtually never an issue. So please try the standalone 'expect' program first. I suspect it is a more appropriate solution for most people than the library.

Nonetheless, if you feel compelled to debug in C, here are some tools to help you.

```
extern int exp_is_debugging;
extern FILE *exp_debugfile;
```

While expect dialogues seem very intuitive, trying to codify them in a program can reveal many surprises in a program's interface. Therefore a variety of debugging aids are available. They are controlled by the above variables, all 0 by default.

Debugging information internal to **expect** is sent to stderr when **exp_is_debugging** is non-zero. The debugging information includes every character received, and every attempt made to match the current input against the patterns. In addition, non-printable characters are translated to a printable form. For example, a control-C appears as a caret followed by a C. If **exp_logfile** is non-zero, this information is also written to that stream.

If **exp_debugfile** is non-zero, all normal and debugging information is written to that stream, regardless of the value of **exp_is_debugging**.

Caveats

The stream versions of the **expect** functions are much slower than the file descriptor versions because there is no way to portably read an unknown number of bytes without the potential of timing out. Thus, characters are read one at a time. You are therefore strongly encouraged to use the file descriptor versions of **expect** (although, automated versions of interactive programs don't usually demand high speed anyway).

You can actually get the best of both worlds, writing with the usual stream functions and reading with the file descriptor versions of **expect** as long as you don't attempt to intermix other stream input functions (e.g., fgetc). To do this, pass fileno(stream) as the file descriptor each time. Fortunately, there is little reason to use anything but the **expect** functions when reading from interactive programs.

There is no matching exp_pclose to exp_popen (unlike popen and pclose). It only takes two

functions to close down a connection (fclose() followed by waiting on the pid), but it is not uncommon to separate these two actions by large time intervals, so the function seems of little value.

If you are running on a Cray running Unicos (all I know for sure from experience), you must run your compiled program as root or setuid. The problem is that the Cray only allows root processes to open ptys. You should observe as much precautions as possible: If you don't need permissions, setuid(0) only immediately before calling one of the spawn functions and immediately set it back afterwards.

Normally, **spawn** takes little time to execute. If you notice spawn taking a significant amount of time, it is probably encountering ptys that are wedged. A number of tests are run on ptys to avoid entanglements with errant processes. (These take 10 seconds per wedged pty.) Running expect with the -d option will show if **expect** is encountering many ptys in odd states. If you cannot kill the processes to which these ptys are attached, your only recourse may be to reboot.

Bugs

The **exp_fexpect** functions don't work at all under HP-UX - it appears to be a bug in getc. Follow the advice (above) about using the **exp_expect** functions (which doesn't need to call getc). If you fix the problem (before I do - please check the latest release) let me know.

Author

Don Libes, libes@nist.gov, National Institute of Standards and Technology

Acknowledgements

Thanks to John Ousterhout (UCBerkeley) for supplying the pattern matcher.

Design and implementation of the **expect** library was paid for by the U.S. government and is therefore in the public domain. However the author and NIST would like credit if this program and documentation or portions of them are used.

Keywords: libexpect

itcl

body

Name

itcl::body - change the body for a class method/proc

Synopsis

itcl::body *className::function args body*

Description

The **body** command is used outside of an **[incr\ Tcl]** class definition to define or redefine the body of a class method or proc. This facility allows a class definition to have separate "interface" and "implementation" parts. The "interface" part is a **class** command with declarations for methods, procs, instance variables and common variables. The "implementation" part is a series of **body** and **configbody** commands. If the "implementation" part is kept in a separate file, it can be sourced again and again as bugs are fixed, to support interactive development. When

using the "tcl" mode in the **emacs** editor, the "interface" and "implementation" parts can be kept in the same file; as bugs are fixed, individual bodies can be highlighted and sent to the test application.

The name "*className*::*function*" identifies the method/proc being changed.

If an *args* list was specified when the *function* was defined in the class definition, the *args* list for the **body** command must match in meaning. Variable names can change, but the argument lists must have the same required arguments and the same default values for optional arguments. The special **args** argument acts as a wildcard when included in the *args* list in the class definition; it will match zero or more arguments of any type when the body is redefined.

If the *body* string starts with "@", it is treated as the symbolic name for a C procedure. The *args* list has little meaning for the C procedure, except to document the expected usage. (The C procedure is not guaranteed to use arguments in this manner.) If *body* does not start with "@", it is treated as a Tcl command script. When the function is invoked, command line arguments are matched against the *args* list, and local variables are created to represent each argument. This is the usual behavior for a Tcl-style proc.

Symbolic names for C procedures are established by registering procedures via **Itcl_RegisterObjC()**. This is usually done in the **Tcl_AppInit()** procedure, which is automatically called when the interpreter starts up. In the following example, the procedure \ffCMy_FooObjCmd() is registered with the symbolic name "foo". This procedure can be referenced in the **body** command as "\ffC@foo".

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter for application. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }

    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

Example

In the following example, a "File" class is defined to represent open files. The method bodies are included below the class definition via the **body** command. Note that the bodies of the constructor/destructor must be included in the class definition, but they can be redefined via the **body** command as well.

```
itcl::class File {
    private variable fid ""
    constructor {name access} {
        set fid [open $name $access]
    }
    destructor {
```

```
        close $fid
    }

    method get {}
    method put {line}
    method eof {}
}

itcl::body File::get {} {
    return [gets $fid]
}
itcl::body File::put {line} {
    puts $fid $line
}
itcl::body File::eof {} {
    return [::eof $fid]
}

#
# See the File class in action:
#
File x /etc/passwd "r"
while {![x eof]} {
    puts "=> [x get]"
}
itcl::delete object x
```

Keywords: body, class, object, procedure

Copyright (c) 1993-1998 Lucent Technologies, Inc.

class

Name

itcl::class - create a class of objects

Synopsis

itcl::class *className* {
inherit *baseClass* ?*baseClass* ...?
constructor *args* ?*init*? *body*
destructor *body*
method *name* ?*args*? ?*body*?
proc *name* ?*args*? ?*body*?
variable *varName* ?*init*? ?*config*?
common *varName* ?*init*?

public *command* ?*arg arg* ...?
protected *command* ?*arg arg* ...?

private *command* ?*arg arg* ...?

set *varName* ?*value*?
array *option* ?*arg arg* ...?
}

className objName ?*arg arg ...*?

objName method ?*arg arg ...*?

className::proc ?*arg arg ...*?

Description

The fundamental construct in **[incr\ Tcl]** is the class definition. Each class acts as a template for actual objects that can be created. The class itself is a namespace which contains things common to all objects. Each object has its own unique bundle of data which contains instances of the "variables" defined in the class definition. Each object also has a built-in variable named "this", which contains the name of the object. Classes can also have "common" data members that are shared by all objects in a class.

Two types of functions can be included in the class definition. "Methods" are functions which operate on a specific object, and therefore have access to both "variables" and "common" data members. "Procs" are ordinary procedures in the class namespace, and only have access to "common" data members.

If the body of any method or proc starts with "@", it is treated as the symbolic name for a C procedure. Otherwise, it is treated as a Tcl code script. See below for details on registering and using C procedures.

A class can only be defined once, although the bodies of class methods and procs can be defined again and again for interactive debugging. See the **body** and **configbody** commands for details.

Each namespace can have its own collection of objects and classes. The list of classes available in the current context can be queried using the "**itcl::find classes**" command, and the list of objects, with the "**itcl::find objects**" command.

A class can be deleted using the "**delete class**" command. Individual objects can be deleted using the "**delete object**" command.

Class definitions

class *className definition*

Provides the definition for a class named *className*. If the class *className* already exists, or if a command called *className* exists in the current namespace context, this command returns an error. If the class definition is successfully parsed, *className* becomes a command in the current context, handling the creation of objects for this class. The class *definition* is evaluated as a series of Tcl statements that define elements within the class. The following class definition commands are recognized:

inherit *baseClass* ?*baseClass*...?

Causes the current class to inherit characteristics from one or more base classes. Classes must have been defined by a previous **class** command, or must be available to the auto-loading facility (see "AUTO-LOADING" below). A single class definition can contain no more than one **inherit** command.

The order of *baseClass* names in the **inherit** list affects the name resolution for class members. When the same member name appears in two or more base classes, the base class that appears first in the **inherit** list takes precedence. For example, if classes "Foo" and "Bar" both contain the member "x", and if another class has the "**inherit**" statement:

```
inherit Foo Bar
```

then the name "x" means "Foo::x". Other inherited members named "x" must be referenced with their explicit name, like "Bar::x".

constructor *args* ?*init*? *body*
Declares the *args* argument list and *body* used for the constructor, which is automatically invoked whenever an object is created.

Before the *body* is executed, the optional *init* statement is used to invoke any base class constructors that require arguments. Variables in the *args* specification can be accessed in the *init* code fragment, and passed to base class constructors. After evaluating the *init* statement, any base class constructors that have not been executed are invoked automatically without arguments. This ensures that all base classes are fully constructed before the constructor *body* is executed. By default, this scheme causes constructors to be invoked in order from least- to most-specific. This is exactly the opposite of the order that classes are reported by the **info heritage** command.

If construction is successful, the constructor always returns the object name-regardless of how the *body* is defined-and the object name becomes a command in the current namespace context. If construction fails, an error message is returned.

destructor *body*
Declares the *body* used for the destructor, which is automatically invoked when an object is deleted. If the destructor is successful, the object data is destroyed and the object name is removed as a command from the interpreter. If destruction fails, an error message is returned and the object remains.

When an object is destroyed, all destructors in its class hierarchy are invoked in order from most- to least-specific. This is the order that the classes are reported by the "**info heritage**" command, and it is exactly the opposite of the default constructor order.

method *name* ?*args*? ?*body*?
Declares a method called *name*. When the method *body* is executed, it will have automatic access to object-specific variables and common data members.

If the *args* list is specified, it establishes the usage information for this method. The **body** command can be used to redefine the method body, but the *args* list must match this specification.

Within the body of another class method, a method can be invoked like any other command-simply by using its name. Outside of the class context, the method name must be prefaced an object name, which provides the context for the data that it manipulates. Methods in a base class that are redefined in the current class, or hidden by another base class, can be qualified using the "*className*::*method*" syntax.

proc *name* ?*args*? ?*body*?
Declares a proc called *name*. A proc is an ordinary procedure within the class namespace.

Unlike a method, a proc is invoked without referring to a specific object. When the proc *body* is executed, it will have automatic access only to common data members.

If the *args* list is specified, it establishes the usage information for this proc. The **body** command can be used to redefine the proc body, but the *args* list must match this specification.

Within the body of another class method or proc, a proc can be invoked like any other command-simply by using its name. In any other namespace context, the proc is invoked using a qualified name like "*className*::*proc*". Procs in a base class that are redefined in the current class, or hidden by another base class, can also be accessed via their qualified name.

variable *varName* ?*init*? ?*config*?

Defines an object-specific variable named *varName*. All object-specific variables are automatically available in class methods. They need not be declared with anything like the **global** command.

If the optional *init* string is specified, it is used as the initial value of the variable when a new object is created. Initialization forces the variable to be a simple scalar value; uninitialized variables, on the other hand, can be set within the constructor and used as arrays.

The optional *config* script is only allowed for public variables. If specified, this code fragment is executed whenever a public variable is modified by the built-in "configure" method. The *config* script can also be specified outside of the class definition using the **configbody** command.

common *varName* ?*init*?

Declares a common variable named *varName*. Common variables reside in the class namespace and are shared by all objects belonging to the class. They are just like global variables, except that they need not be declared with the usual **global** command. They are automatically visible in all class methods and procs.

If the optional *init* string is specified, it is used as the initial value of the variable. Initialization forces the variable to be a simple scalar value; uninitialized variables, on the other hand, can be set with subsequent **set** and **array** commands and used as arrays.

Once a common data member has been defined, it can be set using **set** and **array** commands within the class definition. This allows common data members to be initialized as arrays. For example:

```
itcl::class Foo {
    common boolean
    set boolean(true) 1
    set boolean(false) 0
}
```

Note that if common data members are initialized within the constructor, they get initialized again and again whenever new objects are created.

public *command* ?*arg arg ...*?

...see next...

protected *command* ?*arg arg ...*?

...see next...

private *command* ?*arg arg ...*?

These commands are used to set the protection level for class members that are created

when *command* is evaluated. The *command* is usually **method**, **proc**, **variable** or**common**, and the remaining *arg*'s complete the member definition. However, *command* can also be a script containing many different member definitions, and the protection level will apply to all of the members that are created.

Class usage

Once a class has been defined, the class name can be used as a command to create new objects belonging to the class.

className objName ?args...?

Creates a new object in class *className* with the name *objName*. Remaining arguments are passed to the constructor of the most-specific class. This in turn passes arguments to base class constructors before invoking its own body of commands. If construction is successful, a command called *objName* is created in the current namespace context, and *objName* is returned as the result of this operation. If an error is encountered during construction, the destructors are automatically invoked to free any resources that have been allocated, the object is deleted, and an error is returned.

If *objName* contains the string "**#auto**", that string is replaced with an automatically generated name. Names have the form *className<number>*, where the *className* part is modified to start with a lowercase letter. In class "Toaster", for example, the "**#auto**" specification would produce names like toaster0, toaster1, etc. Note that "**#auto**" can be also be buried within an object name:

```
fileselectiondialog .foo.bar.#auto -background red
```

This would generate an object named ".foo.bar.fileselectiondialog0".

Object usage

Once an object has been created, the object name can be used as a command to invoke methods that operate on the object.

objName method ?args...?

Invokes a method named *method* on an object named *objName*. Remaining arguments are passed to the argument list for the method. The method name can be "constructor", "destructor", any method name appearing in the class definition, or any of the following built-in methods.

Built-in methods

***objName* cget option**

Provides access to public variables as configuration options. This mimics the behavior of the usual "cget" operation for Tk widgets. The *option* argument is a string of the form "**-***varName* ", and this method returns the current value of the public variable *varName*.

***objName* configure ?*option*? ?*value option value ...*?**

Provides access to public variables as configuration options. This mimics the behavior of the usual "configure" operation for Tk widgets. With no arguments, this method returns a list of lists describing all of the public variables. Each list has three elements: the variable name, its initial value and its current value.

If a single *option* of the form "**-***varName*" is specified, then this method returns the information

for that one variable.

Otherwise, the arguments are treated as *option*/*value* pairs assigning new values to public variables. Each variable is assigned its new value, and if it has any "config" code associated with it, it is executed in the context of the class where it was defined. If the "config" code generates an error, the variable is set back to its previous value, and the **configure** method returns an error.

objName* isa *className
Returns non-zero if the given *className* can be found in the object's heritage, and zero otherwise.

***objName* info *option* ?*args...*?**
Returns information related to a particular object named *objName*, or to its class definition. The *option* parameter includes the following things, as well as the options recognized by the usual Tcl "info" command:

***objName* info class**
Returns the name of the most-specific class for object *objName*.

***objName* info inherit**
Returns the list of base classes as they were defined in the "**inherit**" command, or an empty string if this class has no base classes.

***objName* info heritage**
Returns the current class name and the entire list of base classes in the order that they are traversed for member lookup and object destruction.

***objName* info function ?*cmdName*? ?-protection? ?-type? ?-name? ?-args? ?-body?**
With no arguments, this command returns a list of all class methods and procs. If *cmdName* is specified, it returns information for a specific method or proc. If no flags are specified, this command returns a list with the following elements: the protection level, the type (method/proc), the qualified name, the argument list and the body. Flags can be used to request specific elements from this list.

***objName* info variable ?*varName*? ?-protection? ?-type? ?-name? ?-init? ?-value? ?-config? ?-scope?**
With no arguments, this command returns a list of all object-specific variables and common data members. If *varName* is specified, it returns information for a specific data member. Flags can be specified with *varName* in an arbitrary order. The result is a list of the specific information in exactly the same order as the flags are specified. If no flags are given, this command returns a list as if the followings flags have been specified:

-protection -type -name -init -value ?**-config**? The **-config** result is only present if *varName* is a public variable. It contains the code that is executed at initialization of *varName*. The **-scope** flag gives the namespace context of *varName*. Herewith the variable can be accessed from outside the object like any other variable. It is similar to the result of the **itcl::scope** command.

Chaining methods/procs

Sometimes a base class has a method or proc that is redefined with the same name in a derived class. This is a way of making the derived class handle the same operations as the base class, but with its own specialized behavior. For example, suppose we have a Toaster class that looks

like this:

```
itcl::class Toaster {
    variable crumbs 0
    method toast {nslices} {
        if {$crumbs > 50} {
            error "== FIRE! FIRE! =="
        }
        set crumbs [expr {$crumbs+4*$nslices}]
    }
    method clean {} {
        set crumbs 0
    }
}
```

We might create another class like SmartToaster that redefines the "toast" method. If we want to access the base class method, we can qualify it with the base class name, to avoid ambiguity:

```
itcl::class SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [Toaster::toast $nslices]
    }
}
```

Instead of hard-coding the base class name, we can use the "chain" command like this:

```
itcl::class SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [chain $nslices]
    }
}
```

The chain command searches through the class hierarchy for a slightly more generic (base class) implementation of a method or proc, and invokes it with the specified arguments. It starts at the current class context and searches through base classes in the order that they are reported by the "info heritage" command. If another implementation is not found, this command does nothing and returns the null string.

Auto-loading

Class definitions need not be loaded explicitly; they can be loaded as needed by the usual Tcl auto-loading facility. Each directory containing class definition files should have an

accompanying "tclIndex" file. Each line in this file identifies a Tcl procedure or **[incr\ Tcl]** class definition and the file where the definition can be found.

For example, suppose a directory contains the definitions for classes "Toaster" and "SmartToaster". Then the "tclIndex" file for this directory would look like:

```
# Tcl autoload index file, version 2.0 for [incr Tcl]
# This file is generated by the "auto_mkindex" command
# and sourced to set up indexing information for one or
# more commands.  Typically each line is a command that
# sets an element in the auto_index array, where the
# element name is the name of a command and the value is
# a script that loads the command.

set auto_index(::Toaster) "source $dir/Toaster.itcl"
set auto_index(::SmartToaster) "source $dir/SmartToaster.itcl"
```

The **auto_mkindex** command is used to automatically generate "tclIndex" files.

The auto-loader must be made aware of this directory by appending the directory name to the "auto_path" variable. When this is in place, classes will be auto-loaded as needed when used in an application.

C procedures

C procedures can be integrated into an **[incr\ Tcl]** class definition to implement methods, procs, and the "config" code for public variables. Any body that starts with "**@**" is treated as the symbolic name for a C procedure.

Symbolic names are established by registering procedures via **Itcl_RegisterObjC()**. This is usually done in the **Tcl_AppInit()** procedure, which is automatically called when the interpreter starts up. In the following example, the procedure \ffCMy_FooObjCmd() is registered with the symbolic name "foo". This procedure can be referenced in the **body** command as "\ffC@foo".

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter for application. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }

    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

C procedures are implemented just like ordinary Tcl commands. See the **CrtCommand** man page for details. Within the procedure, class data members can be accessed like ordinary variables using **Tcl_SetVar()**, **Tcl_GetVar()**, **Tcl_TraceVar()**, etc. Class methods and procs can be executed like ordinary commands using **Tcl_Eval()**. **[incr\ Tcl]** makes this possible by

automatically setting up the context before executing the C procedure.

This scheme provides a natural migration path for code development. Classes can be developed quickly using Tcl code to implement the bodies. An entire application can be built and tested. When necessary, individual bodies can be implemented with C code to improve performance.

Keywords: class, object, object-oriented

Copyright (c) 1993-1998 Lucent Technologies, Inc.

code

Name

itcl::code - capture the namespace context for a code fragment

Synopsis

itcl::code ?*-namespace name*? *command* ?*arg arg* ...?

Description

Creates a scoped value for the specified *command* and its associated *arg* arguments. A scoped value is a list with three elements: the "\ffC@scope" keyword, a namespace context, and a value string. For example, the command

```
namespace foo {
    code puts "Hello World!"
}
```

produces the scoped value:

```
@scope ::foo {puts {Hello World!}}
```

Note that the **code** command captures the current namespace context. If the **-namespace** flag is specified, then the current context is ignored, and the *name* string is used as the namespace context.

Extensions like Tk execute ordinary code fragments in the global namespace. A scoped value captures a code fragment together with its namespace context in a way that allows it to be executed properly later. It is needed, for example, to wrap up code fragments when a Tk widget is used within a namespace:

```
namespace foo {
    private proc report {mesg} {
        puts "click: $mesg"
    }

    button .b1 -text "Push Me" \
        -command [code report "Hello World!"]
    pack .b1
}
```

The code fragment associated with button \ffC.b1 only makes sense in the context of namespace "foo". Furthermore, the "report" procedure is private, and can only be accessed within that namespace. The **code** command wraps up the code fragment in a way that allows it to be

executed properly when the button is pressed.

Also, note that the **code** command preserves the integrity of arguments on the command line. This makes it a natural replacement for the **list** command, which is often used to format Tcl code fragments. In other words, instead of using the **list** command like this:

```
after 1000 [list puts "Hello $name!"]
```

use the **code** command like this:

```
after 1000 [code puts "Hello $name!"]
```

This not only formats the command correctly, but also captures its namespace context.

Scoped commands can be invoked like ordinary code fragments, with or without the **eval** command. For example, the following statements work properly:

```
set cmd {@scope ::foo .b1}
$cmd configure -background red

set opts {-bg blue -fg white}
eval $cmd configure $opts
```

Note that scoped commands by-pass the usual protection mechanisms; the command:

```
@scope ::foo {report {Hello World!}}
```

can be used to access the "foo::report" proc from any namespace context, even though it is private.

Keywords: code, scope, callback, namespace, public, protected, private

Copyright (c) 1993-1998 Lucent Technologies, Inc.

configbody

Name

itcl::configbody - change the "config" code for a public variable

Synopsis

itcl::configbody *className::varName body*

Description

The **configbody** command is used outside of an **[incr\ Tcl]** class definition to define or redefine the configuration code associated with a public variable. Public variables act like configuration options for an object. They can be modified outside the class scope using the built-in **configure** method. Each variable can have a bit of "config" code associate with it that is automatically executed when the variable is configured. The **configbody** command can be used to define or redefine this body of code.

Like the **body** command, this facility allows a class definition to have separate "interface" and "implementation" parts. The "interface" part is a **class** command with declarations for methods, procs, instance variables and common variables. The "implementation" part is a series of **body** and **configbody** commands. If the "implementation" part is kept in a separate file, it can be

sourced again and again as bugs are fixed, to support interactive development. When using the "tcl" mode in the **emacs** editor, the "interface" and "implementation" parts can be kept in the same file; as bugs are fixed, individual bodies can be highlighted and sent to the test application.

The name "*className*::*varName*" identifies the public variable being updated. If the *body* string starts with "@", it is treated as the symbolic name for a C procedure. Otherwise, it is treated as a Tcl command script.

Symbolic names for C procedures are established by registering procedures via **Itcl_RegisterObjC()**. This is usually done in the **Tcl_AppInit()** procedure, which is automatically called when the interpreter starts up. In the following example, the procedure \ffCMy_FooObjCmd() is registered with the symbolic name "foo". This procedure can be referenced in the **configbody** command as "\ffC@foo".

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter for application. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }

    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

Example

In the following example, a "File" class is defined to represent open files. Whenever the "-name" option is configured, the existing file is closed, and a new file is opened. Note that the "config" code for a public variable is optional. The "-access" option, for example, does not have it.

```
itcl::class File {
    private variable fid ""

    public variable name ""
    public variable access "r"

    constructor {args} {
        eval configure $args
    }
    destructor {
        if {$fid != ""} {
            close $fid
        }
    }
```

```
    method get {}
    method put {line}
    method eof {}
}

itcl::body File::get {} {
    return [gets $fid]
}
itcl::body File::put {line} {
    puts $fid $line
}
itcl::body File::eof {} {
    return [::eof $fid]
}

itcl::configbody File::name {
    if {$fid != ""} {
        close $fid
    }
    set fid [open $name $access]
}

#
# See the File class in action:
#
File x

x configure -name /etc/passwd
while {![x eof]} {
    puts "=> [x get]"
}
itcl::delete object x
```

Keywords: configbody, class, object, variable, configure

Copyright (c) 1993-1998 Lucent Technologies, Inc.

delete

Name

itcl::delete - delete things in the interpreter

Synopsis

itcl::delete *option* ?*arg arg* ...?

Description

The **delete** command is used to delete things in the interpreter. It is implemented as an ensemble, so extensions can add their own options and extend the behavior of this command.

By default, the **delete** command handles the destruction of namespaces.

The *option* argument determines what action is carried out by the command. The legal *options* (which may be abbreviated) are:

delete class *name* ?*name...*?
Deletes one or more **[incr\ Tcl]** classes called *name*. This deletes all objects in the class, and all derived classes as well.

If an error is encountered while destructing an object, it will prevent the destruction of the class and any remaining objects. To destroy the entire class without regard for errors, use the "**delete namespace**" command.

delete object *name* ?*name...*?
Deletes one or more **[incr\ Tcl]** objects called *name*. An object is deleted by invoking all destructors in its class hierarchy, in order from most- to least-specific. If all destructors are successful, data associated with the object is deleted and the *name* is removed as a command from the interpreter.

If the access command for an object resides in another namespace, then its qualified name can be used:

```
itcl::delete object foo::bar::x
```

If an error is encountered while destructing an object, the **delete** command is aborted and the object remains alive. To destroy an object without regard for errors, use the "**rename**" command to destroy the object access command.

delete namespace *name* ?*name...*?
Deletes one or more namespaces called *name*. This deletes all commands and variables in the namespace, and deletes all child namespaces as well. When a namespace is deleted, it is automatically removed from the import lists of all other namespaces.

Keywords: delete, namespace, proc, variable, ensemble

Copyright (c) 1993-1998 Lucent Technologies, Inc.

ensemble

Name

itcl::ensemble - create or modify a composite command

Synopsis

itcl::ensemble *ensName* ?*command arg arg ...*?
ensemble *ensName* {
part *partName args body*
...
ensemble *partName* {
part *subPartName args body*
part *subPartName args body*
...
}
}

or

Description

The **ensemble** command is used to create or modify a composite command. See the section **WHAT IS AN ENSEMBLE?** below for a brief overview of ensembles.

If the **ensemble** command finds an existing ensemble called *ensName*, it updates that ensemble. Otherwise, it creates an ensemble called *ensName*. If the *ensName* is a simple name like "foo", then an ensemble command named "foo" is added to the current namespace context. If a command named "foo" already exists in that context, then it is deleted. If the *ensName* contains namespace qualifiers like "a::b::foo", then the namespace path is resolved, and the ensemble command is added that namespace context. Parent namespaces like "a" and "b" are created automatically, as needed.

If the *ensName* contains spaces like "a::b::foo bar baz", then additional words like "bar" and "baz" are treated as sub-ensembles. Sub-ensembles are merely parts within an ensemble; they do not have a Tcl command associated with them. An ensemble like "foo" can have a sub-ensemble called "foo bar", which in turn can have a sub-ensemble called "foo bar baz". In this case, the sub-ensemble "foo bar" must be created before the sub-ensemble "foo bar baz" that resides within it.

If there are any arguments following *ensName*, then they are treated as commands, and they are executed to update the ensemble. The following commands are recognized in this context: **part** and **ensemble**.

The **part** command defines a new part for the ensemble. Its syntax is identical to the usual **proc** command, but it defines a part within an ensemble, instead of a Tcl command. If a part called *partName* already exists within the ensemble, then the **part** command returns an error.

The **ensemble** command can be nested inside another **ensemble** command to define a sub-ensemble.

What is an ensemble?

The usual "info" command is a composite command--the command name **info** must be followed by a sub-command like **body** or **globals**. We will refer to a command like **info** as an *ensemble*, and to sub-commands like **body** or **globals** as its *parts*.

Ensembles can be nested. For example, the **info** command has an ensemble **info namespace** within it. This ensemble has parts like **info namespace all** and **info namespace children**.

With ensembles, composite commands can be created and extended in an automatic way. Any package can find an existing ensemble and add new parts to it. So extension writers can add their own parts, for example, to the **info** command.

The ensemble facility manages all of the part names and keeps track of unique abbreviations. Normally, you can abbreviate **info complete** to **info comp**. But if an extension adds the part **info complexity**, the minimum abbreviation for **info complete** becomes **info complet**.

The ensemble facility not only automates the construction of composite commands, but it automates the error handling as well. If you invoke an ensemble command without specifying a part name, you get an automatically generated error message that summarizes the usage information. For example, when the **info** command is invoked without any arguments, it produces the following error message:

```
wrong # args: should be one of...
  info args procname
  info body procname
  info cmdcount
  info commands ?pattern?
  info complete command
  info context
  info default procname arg varname
  info exists varName
  info globals ?pattern?
  info level ?number?
  info library
  info locals ?pattern?
  info namespace option ?arg arg ...?
  info patchlevel
  info procs ?pattern?
  info protection ?-command? ?-variable? name
  info script
  info tclversion
  info vars ?pattern?
  info which ?-command? ?-variable? ?-namespace? name
```

You can also customize the way an ensemble responds to errors. When an ensemble encounters an unspecified or ambiguous part name, it looks for a part called **@error**. If it exists, then it is used to handle the error. This part will receive all of the arguments on the command line starting with the offending part name. It can find another way of resolving the command, or generate its own error message.

Example

We could use an ensemble to clean up the syntax of the various "wait" commands in Tcl/Tk. Instead of using a series of strange commands like this:

```
vwait x
tkwait visibility .top
tkwait window .
```

we could use commands with a uniform syntax, like this:

```
wait variable x
wait visibility .top
wait window .
```

The Tcl package could define the following ensemble:

```
itcl::ensemble wait part variable {name} {
    uplevel vwait $name
}
```

The Tk package could add some options to this ensemble, with a command like this:

```
itcl::ensemble wait {
    part visibility {name} {
        tkwait visibility $name
    }
    part window {name} {
        tkwait window $name
    }
}
```

Other extensions could add their own parts to the **wait** command too.

Keywords: ensemble, part, info

Copyright (c) 1993-1998 Lucent Technologies, Inc.

find

Name

itcl::find - search for classes and objects

Synopsis

itcl::find *option* ?*arg arg* ...?

Description

The **find** command is used to find classes and objects that are available in the current interpreter. Classes and objects are reported first in the active namespace, then in all other namespaces in the interpreter.

The *option* argument determines what action is carried out by the command. The legal *options* (which may be abbreviated) are:

find classes ?*pattern*?
: Returns a list of [incr Tcl] classes. Classes in the current namespace are listed first, followed by classes in all other namespaces in the interpreter. If the optional *pattern* is specified, then the reported names are compared using the rules of the "**string match**" command, and only matching names are reported.

If a class resides in the current namespace context, this command reports its simple name--without any qualifiers. However, if the *pattern* contains **::** qualifiers, or if the class resides in another context, this command reports its fully-qualified name. Therefore, you can use the following command to obtain a list where all names are fully-qualified:

```
itcl::find classes ::*
```

find objects ?*pattern*? ?-class *className*? ?-isa *className*?
: Returns a list of [incr Tcl] objects. Objects in the current namespace are listed first, followed by objects in all other namespaces in the interpreter. If the optional *pattern* is specified, then the reported names are compared using the rules of the "**string match**" command, and only matching names are reported. If the optional "**-class**" parameter is specified, this list is restricted to objects whose most-specific class is *className*. If the optional "**-isa**" parameter is specified, this list is further restricted to objects having the given *className* anywhere in

their heritage.

If an object resides in the current namespace context, this command reports its simple name--without any qualifiers. However, if the *pattern* contains **::** qualifiers, or if the object resides in another context, this command reports its fully-qualified name. Therefore, you can use the following command to obtain a list where all names are fully-qualified:

```
itcl::find objects ::*
```

Keywords: find, class, object, search, import

Copyright (c) 1993-1998 Lucent Technologies, Inc.

is

Name

itcl::is - test argument to see if it is a class or an object

Synopsis

itcl::is *option* ?*arg arg* ...?

Description

The **is** command is used to check if the argument given is a class or an object; depending on the option given. If the argument if a class or object, then 1 is returned. Otherwise, 0 is returned. The **is** command also recognizes the commands wrapped in the itcl **code** command.

The *option* argument determines what action is carried out by the command. The legal *options* (which may be abbreviated) are:

is class *command*

Returns 1 if command is a class, and returns 0 otherwise.

The fully qualified name of the class needs to be given as the *command* argument. So, if a class resides in a namespace, then the namespace needs to be specified as well. So, if a class **C** resides in a namespace **N**, then the command should be called like:

```
is N::C
    or
is ::N::C
```

is object ?-class *className*? *command*

Returns 1 if *command* is an object, and returns 0 otherwise.

If the optional "**-class**" parameter is specified, then the *command* will be checked within the context of the class given. Note that *className* has to exist. If not, then an error will be given. So, if *className* is uncertain to be a class, then the programmer will need to check it's existance beforehand, or wrap it in a catch statement.

So, if **c** is an object in the class **C**, in namespace N then these are the possibilities (all return 1):

```
set obj [N::C c]
itcl::is object N::c
itcl::is object c
itcl::is object $obj
itcl::is object [itcl::code c]
```

Keywords: is, class, object

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itcl

Name

itcl - object-oriented extensions to Tcl

Description

[incr\ Tcl] provides object-oriented extensions to Tcl, much as C++ provides object-oriented extensions to C. The emphasis of this work, however, is not to create a whiz-bang object-oriented programming environment. Rather, it is to support more structured programming practices in Tcl without changing the flavor of the language. More than anything else, **[incr\ Tcl]** provides a means of encapsulating related procedures together with their shared data in a namespace that is hidden from the outside world. It encourages better programming by promoting the object-oriented "library" mindset. It also allows for code re-use through inheritance.

Classes

The fundamental construct in **[incr\ Tcl]** is the class definition. Each class acts as a template for actual objects that can be created. Each object has its own unique bundle of data, which contains instances of the "variables" defined in the class. Special procedures called "methods" are used to manipulate individual objects. Methods are just like the operations that are used to manipulate Tk widgets. The "**button**" widget, for example, has methods such as "flash" and "invoke" that cause a particular button to blink and invoke its command.

Within the body of a method, the "variables" defined in the class are automatically available. They need not be declared with anything like the **global** command. Within another class method, a method can be invoked like any other command-simply by using its name. From any other context, the method name must be prefaced by an object name, which provides a context for the data that the method can access.

Each class has its own namespace containing things that are common to all objects which belong to the class. For example, "common" data members are shared by all objects in the class. They are global variables that exist in the class namespace, but since they are included in the class definition, they need not be declared using the **global** command; they are automatically available to any code executing in the class context. A class can also create ordinary global variables, but these must be declared using the **global** command each time they are used.

Classes can also have ordinary procedures declared as "procs". Within another class method or proc, a proc can be invoked like any other command-simply by using its name. From any other context, the procedure name should be qualified with the class namespace like "*className*::*proc*". Class procs execute in the class context, and therefore have automatic access to all

"common" data members. However, they cannot access object-specific "variables", since they are invoked without reference to any specific object. They are usually used to perform generic operations which affect all objects belonging to the class.

Each of the elements in a class can be declared "public", "protected" or "private". Public elements can be accessed by the class, by derived classes (other classes that inherit this class), and by external clients that use the class. Protected elements can be accessed by the class, and by derived classes. Private elements are only accessible in the class where they are defined.

The "public" elements within a class define its interface to the external world. Public methods define the operations that can be used to manipulate an object. Public variables are recognized as configuration options by the "configure" and "cget" methods that are built into each class. The public interface says *what* an object will do but not *how* it will do it. Protected and private members, along with the bodies of class methods and procs, provide the implementation details. Insulating the application developer from these details leaves the class designer free to change them at any time, without warning, and without affecting programs that rely on the class. It is precisely this encapsulation that makes object-oriented programs easier to understand and maintain.

The fact that **[incr\ Tcl]** objects look like Tk widgets is no accident. **[incr\ Tcl]** was designed this way, to blend naturally into a Tcl/Tk application. But **[incr\ Tcl]** extends the Tk paradigm from being merely object-based to being fully object-oriented. An object-oriented system supports inheritance, allowing classes to share common behaviors by inheriting them from an ancestor or base class. Having a base class as a common abstraction allows a programmer to treat related classes in a similar manner. For example, a toaster and a blender perform different (specialized) functions, but both share the abstraction of being appliances. By abstracting common behaviors into a base class, code can be *shared* rather than *copied*. The resulting application is easier to understand and maintain, and derived classes (e.g., specialized appliances) can be added or removed more easily.

This description was merely a brief overview of object-oriented programming and **[incr\ Tcl]**. A more tutorial introduction is presented in the paper included with this distribution. See the **class** command for more details on creating and using classes.

Namespaces

[incr\ Tcl] now includes a complete namespace facility. A namespace is a collection of commands and global variables that is kept apart from the usual global scope. This allows Tcl code libraries to be packaged in a well-defined manner, and prevents unwanted interactions with other libraries. A namespace can also have child namespaces within it, so one library can contain its own private copy of many other libraries. A namespace can also be used to wrap up a group of related classes. The global scope (named "\ffC::") is the root namespace for an interpreter; all other namespaces are contained within it.

See the **namespace** command for details on creating and using namespaces.

Mega-widgets

Mega-widgets are high-level widgets that are constructed using Tk widgets as component parts, usually without any C code. A fileselectionbox, for example, may have a few listboxes, some entry widgets and some control buttons. These individual widgets are put together in a way that makes them act like one big widget.

[incr\ Tk] is a framework for building mega-widgets. It uses **[incr\ Tcl]** to support the object paradigm, and adds base classes which provide default widget behaviors. See the **itk** man page for more details.

[incr\ Widgets] is a library of mega-widgets built using **[incr\ Tk]**. It contains more than 30 different widget classes that can be used right out of the box to build Tcl/Tk applications. Each widget class has its own man page describing the features available.

Keywords: itcl, class, object, object-oriented, namespace, mega-widget

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itclcomponent

Name

itcl::component - define components for extendedclass, widget or widgetadaptor

Parts of this description are "borrowed" from Tcl extension [snit], as the functionality is mostly identical.

Warning!

This is new functionality in [incr Tcl] where the API can still change!!

Synopsis

public component *comp* ?**-inherit**?
protected component *comp* ?**-inherit**?
private component *comp* ?**-inherit**?

Description

The **component** command is used inside an **[incr\ Tcl]** extendedclass/widget/widgetadaptor definition to define components.

Explicitly declares a component called comp, and automatically defines the component's instance variable.

If the *-inherit* option is specified then all unknown methods and options will be delegated to this component. The name -inherit implies that instances of this new type inherit, in a sense, the methods and options of the component. That is, -inherit yes is equivalent to:

```
component mycomp
delegate option * to mycomp
delegate method * to mycomp
```

Keywords: itclcomponent, component, widget, widgetadaptor, extendedclass

Copyright (c) 2008 Arnulf Wiedemann

itcldelegate

Name

itcl::delegation - delegate methods, procs or options to other objects

Parts of this description are "borrowed" from Tcl extension [snit], as the functionality is mostly identical.

Warning!

This is new functionality in [incr Tcl] where the API can still change!!

Synopsis

delegate method *methodName* **to** *componentName* ?*as targetName*?
delegate method *methodName* ?*to componentName*? **using** *pattern*
delegate method * ?*to componentName*? ?*using pattern*? ?*except methodName methodName* ...?

delegate proc *procName* **to** *componentName* ?*as targetName*?
delegate proc *procName* ?*to componentName*? **using** *pattern*
delegate proc * ?*to componentName*? ?*using pattern*? ?*except procName procName* ...?

delegate option *optionSpec* **to** *componentName*
delegate option *optionSpec* **to** *componentName* **as** *targetname?*
delegate option * **to** *componentName*
delegate option * **to** *componentName* **except** *optionName optionname ...*

Description

The **delegate** command is used inside an **[incr\ Tcl]** extendedclass/widget/widgetadaptor definition to delegate methods/procs/options to other objects for handling.

delegate method *methodName* to *componentName* ?as *targetName*?
This form of delegate method delegates method methodName to component componentName. That is, when method methdoNameame is called on an instance of this type, the method and its arguments will be passed to the named component's command instead. That is, the following statement

```
delegate method wag to tail
```

is roughly equivalent to this explicitly defined method:

```
method wag {args} {
    uplevel $tail wag $args
}
```

The optional **as** clause allows you to specify the delegated method name and possibly add some arguments:

```
delegate method wagtail to tail as "wag briskly"
```

A method cannot be both locally defined and delegated.

delegate method *methodName* ?to *componentName*? using *pattern*
In this form of the delegate statement, the **using** clause is used to specify the precise form of the command to which method name name is delegated. The **to** clause is optional, since the chosen command might not involve any particular component.

The value of the using clause is a list that may contain any or all of the following substitution codes; these codes are substituted with the described value to build the delegated command prefix. Note that the following two statements are equivalent:

```
delegate method wag to tail
delegate method wag to tail using "%c %m"
```

Each element of the list becomes a single element of the delegated command --it is never reparsed as a string.

Substitutions:

%%
This is replaced with a single "%". Thus, to pass the string "%c" to the command as an argument, you'd write "%%c".

%c
This is replaced with the named component's command.

%j
This is replaced by the method name; if the name consists of multiple tokens, they are joined by underscores ("_").

%m
This is replaced with the final token of the method name; if the method name has one token, this is identical to **%M**.

%M
This is replaced by the method name; if the name consists of multiple tokens, they are joined by space characters.

%n
This is replaced with the name of the instance's private namespace.

%s
This is replaced with the name of the instance command.

%t
This is replaced with the fully qualified type name.

%w
This is replaced with the original name of the instance command; for Itcl widgets and widget adaptors, it will be the Tk window name. It remains constant, even if the instance command is renamed.

delegate method * ?to *componentName*? ?using *pattern*? ?except *methodName methodName ...*?
In this form all unknown method names are delegeted to the specified component. The except clause can be used to specify a list of exceptions, i.e., method names that will not be so delegated. The using clause is defined as given above. In this form, the statement must contain the to clause, the using clause, or both.

In fact, the "*" can be a list of two or more tokens whose last element is "*", as in the following example:

```
delegate method {tail *} to tail
```

This implicitly defines the method tail whose subcommands will be delegated to the tail component.

The definitions for **delegate proc** ... are the same as for method, the only difference being, that this is for procs.

delegate option *namespec* to *comp*
...see next...

delegate option namespec to comp as target
...see next...

delegate option * to *comp*
...see next...

delegate option * to *comp* except *exceptions*
Defines a delegated option; the namespec is defined as for the option statement. When the configure, configurelist, or cget instance method is used to set or retrieve the option's value, the equivalent configure or cget command will be applied to the component as though the option was defined with the following **-configuremethod** and **-cgetmethod**:

```
method ConfigureMethod {option value} {
    $comp configure $option $value
}

method CgetMethod {option} {
    return [$comp cget $option]
}
```

Note that delegated options never appear in the **itcl_options** array. If the as clause is specified, then the target option name is used in place of name.

delegate *option* * ?except *optionName optionName ...*?
This form delegates all unknown options to the specified component. The except clause can be used to specify a list of exceptions, i.e., option names that will not be so delegated.

Warning: options can only be delegated to a component if it supports the **configure** and **cget** instance methods.

An option cannot be both locally defined and delegated. TBD: Continue from here.

Keywords: itcldelegate, delegation, option, method, proc

Copyright (c) 2008 Arnulf Wiedemann

itclextendedclass

Name
itcl::extendedclass - create a extendedclass of objects

Warning!
This is new functionality in [incr Tcl] where the API can still change!!

Synopsis
itcl::extendedclass *extendedclassName* {
inherit *baseExtendedclass* ?*baseExtendedclass* ...?
constructor *args* ?*init*? *body*
destructor *body*
public method *name* ?*args*? ?*body*?
protected method *name* ?*args*? ?*body*?

private method *name* ?*args*? ?*body*?
public proc *name* ?*args*? ?*body*?
protected proc *name* ?*args*? ?*body*?
private proc *name* ?*args*? ?*body*?
public variable *varName* ?*init*? ?*config*?
protected variable *varName* ?*init*? ?*config*?
private variable *varName* ?*init*? ?*config*?
public common *varName* ?*init*?
protected common *varName* ?*init*?
private common *varName* ?*init*?

public *command* ?*arg arg* ...?
protected *command* ?*arg arg* ...?
private *command* ?*arg arg* ...?

<delegation info> see delegation page

<option info> see option page

set *varName* ?*value*?
array *option* ?*arg arg* ...?
}

extendedclassName objName ?*arg arg ...*?

objName method ?*arg arg ...*?

extendedclassName::proc ?*arg arg ...*?

Description

The fundamental construct in **[incr\ Tcl]** is the extendedclass definition. Each extendedclass acts as a template for actual objects that can be created. The extendedclass itself is a namespace which contains things common to all objects. Each object has its own unique bundle of data which contains instances of the "variables" defined in the extendedclass definition. Each object also has a built-in variable named "this", which contains the name of the object. Extendedclasses can also have "common" data members that are shared by all objects in a extendedclass.

Two types of functions can be included in the extendedclass definition. "Methods" are functions which operate on a specific object, and therefore have access to both "variables" and "common" data members. "Procs" are ordinary procedures in the extendedclass namespace, and only have access to "common" data members.

If the body of any method or proc starts with "@", it is treated as the symbolic name for a C procedure. Otherwise, it is treated as a Tcl code script. See below for details on registering and using C procedures.

A extendedclass can only be defined once, although the bodies of extendedclass methods and procs can be defined again and again for interactive debugging. See the **body** and **configbody** commands for details.

Each namespace can have its own collection of objects and extendedclasses. The list of extendedclasses available in the current context can be queried using the "**itcl::find extendedclasses**" command, and the list of objects, with the "**itcl::find objects**" command.

A extendedclass can be deleted using the "**delete extendedclass**" command. Individual objects can be deleted using the "**delete object**" command.

Class definitions

extendedclass *extendedclassName definition*

Provides the definition for a extendedclass named *extendedclassName*. If the extendedclass *extendedclassName* already exists, or if a command called *extendedclassName* exists in the current namespace context, this command returns an error. If the extendedclass definition is successfully parsed, *extendedclassName* becomes a command in the current context, handling the creation of objects for this extendedclass. The extendedclass *definition* is evaluated as a series of Tcl statements that define elements within the extendedclass. The following extendedclass definition commands are recognized:

inherit *baseExtendedclass* ?*baseExtendedclass*...?

Causes the current extendedclass to inherit characteristics from one or more base extendedclasses. Extendedclasses must have been defined by a previous **extendedclass** command, or must be available to the auto-loading facility (see "AUTO-LOADING" below). A single extendedclass definition can contain no more than one **inherit** command.

The order of *baseExtendedclass* names in the **inherit** list affects the name resolution for extendedclass members. When the same member name appears in two or more base extendedclasses, the base extendedclass that appears first in the **inherit** list takes precedence. For example, if extendedclasses "Foo" and "Bar" both contain the member "x", and if another extendedclass has the "**inherit**" statement:

```
inherit Foo Bar
```

then the name "x" means "Foo::x". Other inherited members named "x" must be referenced with their explicit name, like "Bar::x".

constructor *args* ?*init*? *body*

Declares the *args* argument list and *body* used for the constructor, which is automatically invoked whenever an object is created.

Before the *body* is executed, the optional *init* statement is used to invoke any base extendedclass constructors that require arguments. Variables in the *args* specification can be accessed in the *init* code fragment, and passed to base extendedclass constructors. After evaluating the *init* statement, any base extendedclass constructors that have not been executed are invoked automatically without arguments. This ensures that all base extendedclasses are fully constructed before the constructor *body* is executed. By default, this scheme causes constructors to be invoked in order from least- to most-specific. This is exactly the opposite of the order that extendedclasses are reported by the **info heritage** command.

If construction is successful, the constructor always returns the object name-regardless of how the *body* is defined-and the object name becomes a command in the current namespace context. If construction fails, an error message is returned.

destructor *body*

Declares the *body* used for the destructor, which is automatically invoked when an object is deleted. If the destructor is successful, the object data is destroyed and the object name is removed as a command from the interpreter. If destruction fails, an error message is returned

and the object remains. When an object is destroyed, all destructors in its extendedclass hierarchy are invoked in order from most- to least-specific. This is the order that the extendedclasses are reported by the "**info heritage**" command, and it is exactly the opposite of the default constructor order.

method *name* ?*args*? ?*body*?
Declares a method called *name*. When the method *body* is executed, it will have automatic access to object-specific variables and common data members.

If the *args* list is specified, it establishes the usage information for this method. The **body** command can be used to redefine the method body, but the *args* list must match this specification.

Within the body of another extendedclass method, a method can be invoked like any other command-simply by using its name. Outside of the extendedclass context, the method name must be prefaced an object name, which provides the context for the data that it manipulates. Methods in a base extendedclass that are redefined in the current extendedclass, or hidden by another base extendedclass, can be qualified using the "*extendedclassName*::*method*" syntax.

proc *name* ?*args*? ?*body*?
Declares a proc called *name*. A proc is an ordinary procedure within the extendedclass namespace. Unlike a method, a proc is invoked without referring to a specific object. When the proc *body* is executed, it will have automatic access only to common data members.

If the *args* list is specified, it establishes the usage information for this proc. The **body** command can be used to redefine the proc body, but the *args* list must match this specification.

Within the body of another extendedclass method or proc, a proc can be invoked like any other command-simply by using its name. In any other namespace context, the proc is invoked using a qualified name like "*extendedclassName***::***proc*". Procs in a base extendedclass that are redefined in the current extendedclass, or hidden by another base extendedclass, can also be accessed via their qualified name.

variable *varName* ?*init*? ?*config*?
Defines an object-specific variable named *varName*. All object-specific variables are automatically available in extendedclass methods. They need not be declared with anything like the **global** command.

If the optional *init* string is specified, it is used as the initial value of the variable when a new object is created. Initialization forces the variable to be a simple scalar value; uninitialized variables, on the other hand, can be set within the constructor and used as arrays.

The optional *config* script is only allowed for public variables. If specified, this code fragment is executed whenever a public variable is modified by the built-in "configure" method. The *config* script can also be specified outside of the extendedclass definition using the **configbody** command.

common *varName* ?*init*?
Declares a common variable named *varName*. Common variables reside in the extendedclass namespace and are shared by all objects belonging to the extendedclass. They are just like global variables, except that they need not be declared with the usual **global** command. They are automatically visible in all extendedclass methods and procs.

If the optional *init* string is specified, it is used as the initial value of the variable. Initialization

forces the variable to be a simple scalar value; uninitialized variables, on the other hand, can be set with subsequent **set** and **array** commands and used as arrays.

Once a common data member has been defined, it can be set using **set** and **array** commands within the extendedclass definition. This allows common data members to be initialized as arrays. For example:

```
itcl::extendedclass Foo {
    common boolean
    set boolean(true) 1
    set boolean(false) 0
}
```

Note that if common data members are initialized within the constructor, they get initialized again and again whenever new objects are created.

public *command* ?*arg arg ...*?
...see next...

protected *command* ?*arg arg ...*?
...see next...

private *command* ?*arg arg ...*?
These commands are used to set the protection level for extendedclass members that are created when *command* is evaluated. The *command* is usually **method**, **proc**, **variable** or **common**, and the remaining *arg*'s complete the member definition. However, *command* can also be a script containing many different member definitions, and the protection level will apply to all of the members that are created.

Class usage

Once a extendedclass has been defined, the extendedclass name can be used as a command to create new objects belonging to the extendedclass.

***extendedclassName objName* ?*args...*?**
Creates a new object in extendedclass *extendedclassName* with the name *objName*. Remaining arguments are passed to the constructor of the most-specific extendedclass. This in turn passes arguments to base extendedclass constructors before invoking its own body of commands. If construction is successful, a command called *objName* is created in the current namespace context, and *objName* is returned as the result of this operation. If an error is encountered during construction, the destructors are automatically invoked to free any resources that have been allocated, the object is deleted, and an error is returned.

If *objName* contains the string "**#auto**", that string is replaced with an automatically generated name. Names have the form *extendedclassName*<*number*>, where the *extendedclassName* part is modified to start with a lowercase letter. In extendedclass "Toaster", for example, the "**#auto**" specification would produce names like toaster0, toaster1, etc. Note that "**#auto**" can be also be buried within an object name:

```
fileselectiondialog .foo.bar.#auto -background red
```

This would generate an object named ".foo.bar.fileselectiondialog0".

Object usage

Once an object has been created, the object name can be used as a command to invoke methods that operate on the object.

***objName method* ?*args...*?**
Invokes a method named *method* on an object named *objName*. Remaining arguments are passed to the argument list for the method. The method name can be "constructor", "destructor", any method name appearing in the extendedclass definition, or any of the following built-in methods.

Built-in methods

***objName* cget option**
Provides access to public variables as configuration options. This mimics the behavior of the usual "cget" operation for Tk widgets. The *option* argument is a string of the form "**-***varName* ", and this method returns the current value of the public variable *varName*.

***objName* configure ?*option*? ?*value option value ...*?**
Provides access to public variables as configuration options. This mimics the behavior of the usual "configure" operation for Tk widgets. With no arguments, this method returns a list of lists describing all of the public variables. Each list has three elements: the variable name, its initial value and its current value.

If a single *option* of the form "**-***varName*" is specified, then this method returns the information for that one variable.

Otherwise, the arguments are treated as *option*/*value* pairs assigning new values to public variables. Each variable is assigned its new value, and if it has any "config" code associated with it, it is executed in the context of the extendedclass where it was defined. If the "config" code generates an error, the variable is set back to its previous value, and the **configure** method returns an error.

objName* isa *extendedclassName
Returns non-zero if the given *extendedclassName* can be found in the object's heritage, and zero otherwise.

***objName* info *option* ?*args...*?**
Returns information related to a particular object named *objName*, or to its extendedclass definition. The *option* parameter includes the following things, as well as the options recognized by the usual Tcl "info" command:

***objName* info extendedclass**
Returns the name of the most-specific extendedclass for object *objName*.

***objName* info inherit**
Returns the list of base extendedclasses as they were defined in the "**inherit**" command, or an empty string if this extendedclass has no base extendedclasses.

***objName* info heritage**
Returns the current extendedclass name and the entire list of base extendedclasses in the order that they are traversed for member lookup and object destruction.

***objName* info function ?*cmdName*? ?-protection? ?-type? ?-name? ?-args? ?-body?**
With no arguments, this command returns a list of all extendedclass methods and procs. If *cmdName* is specified, it returns information for a specific method or proc. If no flags are

specified, this command returns a list with the following elements: the protection level, the type (method/proc), the qualified name, the argument list and the body. Flags can be used to request specific elements from this list.

***objName* info variable ?*varName*? ?-protection? ?-type? ?-name? ?-init? ?-value? ?-config?**

With no arguments, this command returns a list of all object-specific variables and common data members. If *varName* is specified, it returns information for a specific data member. If no flags are specified, this command returns a list with the following elements: the protection level, the type (variable/common), the qualified name, the initial value, and the current value. If *varName* is a public variable, the "config" code is included on this list. Flags can be used to request specific elements from this list.

Chaining methods/procs

Sometimes a base extendedclass has a method or proc that is redefined with the same name in a derived extendedclass. This is a way of making the derived extendedclass handle the same operations as the base extendedclass, but with its own specialized behavior. For example, suppose we have a Toaster extendedclass that looks like this:

```
itcl::extendedclass Toaster {
    variable crumbs 0
    method toast {nslices} {
        if {$crumbs > 50} {
            error "== FIRE! FIRE! =="
        }
        set crumbs [expr {$crumbs+4*$nslices}]
    }
    method clean {} {
        set crumbs 0
    }
}
```

We might create another extendedclass like SmartToaster that redefines the "toast" method. If we want to access the base extendedclass method, we can qualify it with the base extendedclass name, to avoid ambiguity:

```
itcl::extendedclass SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [Toaster::toast $nslices]
    }
}
```

Instead of hard-coding the base extendedclass name, we can use the "chain" command like this:

```
itcl::extendedclass SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [chain $nslices]
    }
}
```

The chain command searches through the extendedclass hierarchy for a slightly more generic (base extendedclass) implementation of a method or proc, and invokes it with the specified arguments. It starts at the current extendedclass context and searches through base extendedclasses in the order that they are reported by the "info heritage" command. If another implementation is not found, this command does nothing and returns the null string.

Auto-loading

Extendedclass definitions need not be loaded explicitly; they can be loaded as needed by the usual Tcl auto-loading facility. Each directory containing extendedclass definition files should have an accompanying "tclIndex" file. Each line in this file identifies a Tcl procedure or **[incr\ Tcl]** extendedclass definition and the file where the definition can be found.

For example, suppose a directory contains the definitions for extendedclasses "Toaster" and "SmartToaster". Then the "tclIndex" file for this directory would look like:

```
# Tcl autoload index file, version 2.0 for [incr Tcl]
# This file is generated by the "auto_mkindex" command
# and sourced to set up indexing information for one or
# more commands.  Typically each line is a command that
# sets an element in the auto_index array, where the
# element name is the name of a command and the value is
# a script that loads the command.

set auto_index(::Toaster) "source $dir/Toaster.itcl"
set auto_index(::SmartToaster) "source $dir/SmartToaster.itcl"
```

The **auto_mkindex** command is used to automatically generate "tclIndex" files.

The auto-loader must be made aware of this directory by appending the directory name to the "auto_path" variable. When this is in place, extendedclasses will be auto-loaded as needed when used in an application.

C procedures

C procedures can be integrated into an **[incr\ Tcl]** extendedclass definition to implement methods, procs, and the "config" code for public variables. Any body that starts with "@" is treated as the symbolic name for a C procedure.

Symbolic names are established by registering procedures via **Itcl_RegisterObjC()**. This is usually done in the **Tcl_AppInit()** procedure, which is automatically called when the interpreter starts up. In the following example, the procedure \ffCMy_FooObjCmd() is registered with the

symbolic name "foo". This procedure can be referenced in the **body** command as "\ffC@foo".

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter for application. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }

    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

C procedures are implemented just like ordinary Tcl commands. See the **CrtCommand** man page for details. Within the procedure, extendedclass data members can be accessed like ordinary variables using **Tcl_SetVar()**, **Tcl_GetVar()**, **Tcl_TraceVar()**, etc. Extendedclass methods and procs can be executed like ordinary commands using **Tcl_Eval()**. **[incr\ Tcl]** makes this possible by automatically setting up the context before executing the C procedure.

This scheme provides a natural migration path for code development. Extendedclasses can be developed quickly using Tcl code to implement the bodies. An entire application can be built and tested. When necessary, individual bodies can be implemented with C code to improve performance.

Keywords: itclextendedclass, extendedclass, object, object-oriented

Copyright (c) 2008 Arnulf Wiedemann

itcloption

Name

itcl::option - define options for extendedclass, widget or widgetadaptor

Parts of this description are "borrowed" from Tcl extension [snit], as the functionality is mostly identical.

Warning!

This is new functionality in [incr Tcl] where the API can still change!!

Synopsis

option *optionSpec* ?*defaultValue*?
option *optionSpec* ?*options*?

Description

The **option** command is used inside an **[incr\ Tcl]** extendedclass/widget/widgetadaptor definition to define options.

In the first form defines an option for instances of this type, and optionally gives it an initial value. The initial value defaults to the empty string if no defaultValue is specified.

An option defined in this way is said to be locally defined. The optionSpec is a list defining the option's name, resource name, and class name, e.g.:

```
    option {-font font Font} {Courier 12}
```

The option name must begin with a hyphen, and must not contain any upper case letters. The resource name and class name are optional; if not specified, the resource name defaults to the option name, minus the hyphen, and the class name defaults to the resource name with the first letter capitalized. Thus, the following statement is equivalent to the previous example:

```
    option -font {Courier 12}
```

See The Tk Option Database for more information about resource and class names.

Options are normally set and retrieved using the standard instance methods configure and cget; within instance code (method bodies, etc.), option values are available through the options array:

```
    set myfont $itcl_options(-font)
```

In the second form you can define option handlers (e.g., -configuremethod), then it should probably use configure and cget to access its options to avoid subtle errors.

The option statement may include the following options:

-default *defvalue*
: Defines the option's default value; the option's default value will be "" otherwise.

-readonly
: The option is handled read-only -- it can only be set using configure at creation time, i.e., in the type's constructor.

-cgetmethod *methodName*
: Every locally-defined option may define a -cgetmethod; it is called when the option's value is retrieved using the cget method. Whatever the method's body returns will be the return value of the call to cget.

The named method must take one argument, the option name. For example, this code is equivalent to (though slower than) Itcl's default handling of cget:

```
        option -font -cgetmethod GetOption
        method GetOption {option} {
            return $itcl_options($option)
        }
```

Note that it's possible for any number of options to share a -cgetmethod.

-cgetmethodvar *varName*
: That is very similar to -cgetmethod, the only difference is, one can define a variable, where to find the cgetmethod during runtime.

-configuremethod *methodName*
: Every locally-defined option may define a -configuremethod; it is called when the option's value is set using the configure or configurelist methods. It is the named method's responsibility to save the option's value; in other words, the value will not be saved to the itcl_options() array unless the method saves it there.

The named method must take two arguments, the option name and its new value. For example, this code is equivalent to (though slower than) Itcl's default handling of configure:

```
        option -font -configuremethod SetOption
        method SetOption {option value} {
            set itcl_options($option) $value
        }
```

Note that it's possible for any number of options to share a single -configuremethod.

-configuremethodvar *varName*
That is very similar to -configuremethod, the only difference is, one can define a variable, where to find the configuremethod during runtime.

-validatemethod *methodName*
Every locally-defined option may define a -validatemethod; it is called when the option's value is set using the configure or configurelist methods, just before the -configuremethod (if any). It is the named method's responsibility to validate the option's new value, and to throw an error if the value is invalid.

The named method must take two arguments, the option name and its new value. For example, this code verifies that -flag's value is a valid Boolean value:

```
        option -font -validatemethod CheckBoolean
        method CheckBoolean {option value} {
            if {![string is boolean -strict $value]} {
                error "option $option must have a boolean value."
            }
        }
```

Note that it's possible for any number of options to share a single -validatemethod.

-validatemethodvar *varName*
That is very similar to -validatemethod, the only difference is, one can define a variable, where to find the validatemethod during runtime.

Keywords: itcloption, option, widget, widgetadaptor, extendedclass

Copyright (c) 2008 Arnulf Wiedemann

itclvars

Name

itclvars - variables used by [incr\ Tcl]

Description

The following global variables are created and managed automatically by the **[incr\ Tcl]** library. Except where noted below, these variables should normally be treated as read-only by application-specific code and by users.

itcl::library
When an interpreter is created, **[incr\ Tcl]** initializes this variable to hold the name of a directory containing the system library of **[incr\ Tcl]** scripts. The initial value of **itcl::library** is set from the ITCL_LIBRARY environment variable if it exists, or from a compiled-in value

otherwise.

itcl::patchLevel
When an interpreter is created, **[incr\ Tcl]** initializes this variable to hold the current patch level for **[incr\ Tcl]**. For example, the value "**2.0p1**" indicates **[incr\ Tcl]** version 2.0 with the first set of patches applied.

itcl::purist
When an interpreter is created containing Tcl/Tk and the **[incr\ Tcl]** namespace facility, this variable controls a "backward-compatibility" mode for widget access.

In vanilla Tcl/Tk, there is a single pool of commands, so the access command for a widget is the same as the window name. When a widget is created within a namespace, however, its access command is installed in that namespace, and should be accessed outside of the namespace using a qualified name. For example,

```
namespace foo {
    namespace bar {
        button .b -text "Testing"
    }
}
foo::bar::.b configure -background red
pack .b
```

Note that the window name "\ffC.b" is still used in conjunction with commands like **pack** and **destroy**. However, the access command for the widget (i.e., name that appears as the *first* argument on a command line) must be more specific.

The "**winfo command**" command can be used to query the fully-qualified access command for any widget, so one can write:

```
[winfo command .b] configure -background red
```

and this is good practice when writing library procedures. Also, in conjunction with the **bind** command, the "%q" field can be used in place of "%W" as the access command:

```
bind Button <Key-Return> {%q flash; %q invoke}
```

While this behavior makes sense from the standpoint of encapsulation, it causes problems with existing Tcl/Tk applications. Many existing applications are written with bindings that use "%W". Many library procedures assume that the window name is the access command.

The **itcl::purist** variable controls a backward-compatibility mode. By default, this variable is "0", and the window name can be used as an access command in any context. Whenever the **unknown** procedure stumbles across a widget name, it simply uses "**winfo command**" to determine the appropriate command name. If this variable is set to "1", this backward-compatibility mode is disabled. This gives better encapsulation, but using the window name as the access command may lead to "invalid command" errors.

itcl::version
When an interpreter is created, **[incr\ Tcl]** initializes this variable to hold the version number of the form *x.y*. Changes to *x* represent major changes with probable incompatibilities and changes to *y* represent small enhancements and bug fixes that retain backward compatibility.

Keywords: itclvars, itcl, variables

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itclwidget

Name

itcl::widget - create a widget class of objects

Warning!

This is new functionality in [incr Tcl] where the API can still change!!

Synopsis

itcl::widget *widgetName* {
inherit *baseWidget* ?*baseWidget* ...?
constructor *args* ?*init*? *body*
destructor *body*
public method *name* ?*args*? ?*body*?
protected method *name* ?*args*? ?*body*?
private method *name* ?*args*? ?*body*?
public proc *name* ?*args*? ?*body*?
protected proc *name* ?*args*? ?*body*?
private proc *name* ?*args*? ?*body*?
public variable *varName* ?*init*? ?*config*?
protected variable *varName* ?*init*? ?*config*?
private variable *varName* ?*init*? ?*config*?
public common *varName* ?*init*?
protected common *varName* ?*init*?
private common *varName* ?*init*?

public *command* ?*arg arg* ...?
protected *command* ?*arg arg* ...?
private *command* ?*arg arg* ...?

<delegation info> see delegation page

<option info> see option page

set *varName* ?*value*?
array *option* ?*arg arg* ...?
}

widgetName objName ?arg arg ...?

objName method ?arg arg ...?

widgetName::proc ?arg arg ...?

Description

One of the fundamental constructs in **[incr\ Tcl]** is the widget definition. A widget is like a class with some additional features. Each widget acts as a template for actual objects that can be created. The widget itself is a namespace which contains things common to all objects. Each object has its own unique bundle of data which contains instances of the "variables" defined in

the widget definition. Each object also has a built-in variable named "this", which contains the name of the object. Widgets can also have "common" data members that are shared by all objects in a widget.

Two types of functions can be included in the widget definition. "Methods" are functions which operate on a specific object, and therefore have access to both "variables" and "common" data members. "Procs" are ordinary procedures in the widget namespace, and only have access to "common" data members.

If the body of any method or proc starts with "@", it is treated as the symbolic name for a C procedure. Otherwise, it is treated as a Tcl code script. See below for details on registering and using C procedures.

A widget can only be defined once, although the bodies of widget methods and procs can be defined again and again for interactive debugging. See the **body** and **configbody** commands for details.

Each namespace can have its own collection of objects and widgets. The list of widgets available in the current context can be queried using the "**itcl::find widgets**" command, and the list of objects, with the "**itcl::find objects**" command.

A widget can be deleted using the "**delete widget**" command. Individual objects can be deleted using the "**delete object**" command.

Widget definitions

widget *widgetName definition*

Provides the definition for a widget named *widgetName*. If the widget *widgetName* already exists, or if a command called *widgetName* exists in the current namespace context, this command returns an error. If the widget definition is successfully parsed, *widgetName* becomes a command in the current context, handling the creation of objects for this widget. The widget *definition* is evaluated as a series of Tcl statements that define elements within the widget. The following widget definition commands are recognized:

inherit *baseWidget* ?*baseWidget*...?

Causes the current widget to inherit characteristics from one or more base widgets. Widgets must have been defined by a previous **widget** command, or must be available to the auto-loading facility (see "AUTO-LOADING" below). A single widget definition can contain no more than one **inherit** command.

The order of *baseWidget* names in the **inherit** list affects the name resolution for widget members. When the same member name appears in two or more base widgets, the base widget that appears first in the **inherit** list takes precedence. For example, if widgets "Foo" and "Bar" both contain the member "x", and if another widget has the "**inherit**" statement:

```
inherit Foo Bar
```

then the name "x" means "Foo::x". Other inherited members named "x" must be referenced with their explicit name, like "Bar::x".

constructor *args* ?*init*? *body*

Declares the *args* argument list and *body* used for the constructor, which is automatically invoked whenever an object is created.

Before the *body* is executed, the optional *init* statement is used to invoke any base widget constructors that require arguments. Variables in the *args* specification can be accessed in the *init* code fragment, and passed to base widget constructors. After evaluating the *init* statement, any base widget constructors that have not been executed are invoked automatically without arguments. This ensures that all base widgets are fully constructed before the constructor *body* is executed. By default, this scheme causes constructors to be invoked in order from least- to most-specific. This is exactly the opposite of the order that widgets are reported by the **info heritage** command.

If construction is successful, the constructor always returns the object name-regardless of how the *body* is defined-and the object name becomes a command in the current namespace context. If construction fails, an error message is returned.

destructor *body*

Declares the *body* used for the destructor, which is automatically invoked when an object is deleted. If the destructor is successful, the object data is destroyed and the object name is removed as a command from the interpreter. If destruction fails, an error message is returned and the object remains.

When an object is destroyed, all destructors in its widget hierarchy are invoked in order from most- to least-specific. This is the order that the widgets are reported by the "**info heritage**" command, and it is exactly the opposite of the default constructor order.

method *name* ?*args*? ?*body*?

Declares a method called *name*. When the method *body* is executed, it will have automatic access to object-specific variables and common data members.

If the *args* list is specified, it establishes the usage information for this method. The **body** command can be used to redefine the method body, but the *args* list must match this specification.

Within the body of another widget method, a method can be invoked like any other command-simply by using its name. Outside of the widget context, the method name must be prefaced an object name, which provides the context for the data that it manipulates. Methods in a base widget that are redefined in the current widget, or hidden by another base widget, can be qualified using the "*widgetName*::*method*" syntax.

proc *name* ?*args*? ?*body*?

Declares a proc called *name*. A proc is an ordinary procedure within the widget namespace. Unlike a method, a proc is invoked without referring to a specific object. When the proc *body* is executed, it will have automatic access only to common data members.

If the *args* list is specified, it establishes the usage information for this proc. The **body** command can be used to redefine the proc body, but the *args* list must match this specification.

Within the body of another widget method or proc, a proc can be invoked like any other command-simply by using its name. In any other namespace context, the proc is invoked using a qualified name like "*widgetName***::***proc*". Procs in a base widget that are redefined in the current widget, or hidden by another base widget, can also be accessed via their qualified name.

variable *varName* ?*init*? ?*config*?

Defines an object-specific variable named *varName*. All object-specific variables are automatically available in widget methods. They need not be declared with anything like the

global command.

If the optional *init* string is specified, it is used as the initial value of the variable when a new object is created. Initialization forces the variable to be a simple scalar value; uninitialized variables, on the other hand, can be set within the constructor and used as arrays.

The optional *config* script is only allowed for public variables. If specified, this code fragment is executed whenever a public variable is modified by the built-in "configure" method. The *config* script can also be specified outside of the widget definition using the **configbody** command.

common *varName* ?*init*?
Declares a common variable named *varName*. Common variables reside in the widget namespace and are shared by all objects belonging to the widget. They are just like global variables, except that they need not be declared with the usual **global** command. They are automatically visible in all widget methods and procs.

If the optional *init* string is specified, it is used as the initial value of the variable. Initialization forces the variable to be a simple scalar value; uninitialized variables, on the other hand, can be set with subsequent **set** and **array** commands and used as arrays.

Once a common data member has been defined, it can be set using **set** and **array** commands within the widget definition. This allows common data members to be initialized as arrays. For example:

```
itcl::widget Foo {
    protected common boolean
    set boolean(true) 1
    set boolean(false) 0
}
```

Note that if common data members are initialized within the constructor, they get initialized again and again whenever new objects are created.

public *command* ?*arg arg ...*?
...see next...

protected *command* ?*arg arg ...*?
...see next...

private *command* ?*arg arg ...*?
These commands are used to set the protection level for widget members that are created when *command* is evaluated. The *command* is usually **method**, **proc**, **variable** or**common**, and the remaining *arg*'s complete the member definition. However, *command* can also be a script containing many different member definitions, and the protection level will apply to all of the members that are created.

Widget usage

Once a widget has been defined, the widget name can be used as a command to create new objects belonging to the widget.

widgetName objName ?args...?
Creates a new object in widget *widgetName* with the name *objName*. Remaining arguments are passed to the constructor of the most-specific widget. This in turn passes arguments to base widget constructors before invoking its own body of commands. If construction is

successful, a command called *objName* is created in the current namespace context, and *objName* is returned as the result of this operation. If an error is encountered during construction, the destructors are automatically invoked to free any resources that have been allocated, the object is deleted, and an error is returned.

If *objName* contains the string "**#auto**", that string is replaced with an automatically generated name. Names have the form *widgetName*<*number*>, where the *widgetName* part is modified to start with a lowercase letter. In widget "Toaster", for example, the "**#auto**" specification would produce names like toaster0, toaster1, etc. Note that "**#auto**" can be also be buried within an object name:

```
fileselectiondialog .foo.bar.#auto -background red
```

This would generate an object named ".foo.bar.fileselectiondialog0".

Object usage

Once an object has been created, the object name can be used as a command to invoke methods that operate on the object.

objName method ?args...?

Invokes a method named *method* on an object named *objName*. Remaining arguments are passed to the argument list for the method. The method name can be "constructor", "destructor", any method name appearing in the widget definition, or any of the following built-in methods.

Built-in methods

***objName* cget option**

Provides access to public variables as configuration options. This mimics the behavior of the usual "cget" operation for Tk widgets. The *option* argument is a string of the form "**-***varName* ", and this method returns the current value of the public variable *varName*.

***objName* configure ?*option*? ?*value option value ...*?**

Provides access to public variables as configuration options. This mimics the behavior of the usual "configure" operation for Tk widgets. With no arguments, this method returns a list of lists describing all of the public variables. Each list has three elements: the variable name, its initial value and its current value.

If a single *option* of the form "**-***varName*" is specified, then this method returns the information for that one variable.

Otherwise, the arguments are treated as *option*/*value* pairs assigning new values to public variables. Each variable is assigned its new value, and if it has any "config" code associated with it, it is executed in the context of the widget where it was defined. If the "config" code generates an error, the variable is set back to its previous value, and the **configure** method returns an error.

objName* isa *widgetName

Returns non-zero if the given *widgetName* can be found in the object's heritage, and zero otherwise.

***objName* info *option* ?*args...*?**

Returns information related to a particular object named *objName*, or to its widget definition.

The *option* parameter includes the following things, as well as the options recognized by the usual Tcl "info" command:

***objName* info widget**
Returns the name of the most-specific widget for object *objName*.

***objName* info inherit**
Returns the list of base widgets as they were defined in the "**inherit**" command, or an empty string if this widget has no base widgets.

***objName* info heritage**
Returns the current widget name and the entire list of base widgets in the order that they are traversed for member lookup and object destruction.

***objName* info function ?*cmdName*? ?-protection? ?-type? ?-name? ?-args? ?-body?**
With no arguments, this command returns a list of all widgets methods and procs. If *cmdName* is specified, it returns information for a specific method or proc. If no flags are specified, this command returns a list with the following elements: the protection level, the type (method/proc), the qualified name, the argument list and the body. Flags can be used to request specific elements from this list.

***objName* info variable ?*varName*? ?-protection? ?-type? ?-name? ?-init? ?-value? ?-config?**
With no arguments, this command returns a list of all object-specific variables and common data members. If *varName* is specified, it returns information for a specific data member. If no flags are specified, this command returns a list with the following elements: the protection level, the type (variable/common), the qualified name, the initial value, and the current value. If *varName* is a public variable, the "config" code is included on this list. Flags can be used to request specific elements from this list.

Chaining methods/procs

Sometimes a base widget has a method or proc that is redefined with the same name in a derived widget. This is a way of making the derived widget handle the same operations as the base widget, but with its own specialized behavior. For example, suppose we have a Toaster widget that looks like this:

```
itcl::widget Toaster {
    variable crumbs 0
    method toast {nslices} {
        if {$crumbs > 50} {
            error "== FIRE! FIRE! =="
        }
        set crumbs [expr {$crumbs+4*$nslices}]
    }
    method clean {} {
        set crumbs 0
    }
}
```

We might create another widget like SmartToaster that redefines the "toast" method. If we want to access the base widget method, we can qualify it with the base widget name, to avoid

ambiguity:

```
itcl::widget SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [Toaster::toast $nslices]
    }
}
```

Instead of hard-coding the base widget name, we can use the "chain" command like this:

```
itcl::widget SmartToaster {
    inherit Toaster
    method toast {nslices} {
        if {$crumbs > 40} {
            clean
        }
        return [chain $nslices]
    }
}
```

The chain command searches through the widget hierarchy for a slightly more generic (base widget) implementation of a method or proc, and invokes it with the specified arguments. It starts at the current widget context and searches through base widgets in the order that they are reported by the "info heritage" command. If another implementation is not found, this command does nothing and returns the null string.

Auto-loading

Widget definitions need not be loaded explicitly; they can be loaded as needed by the usual Tcl auto-loading facility. Each directory containing widget definition files should have an accompanying "tclIndex" file. Each line in this file identifies a Tcl procedure or **[incr\ Tcl]** widget definition and the file where the definition can be found.

For example, suppose a directory contains the definitions for widgets "Toaster" and "SmartToaster". Then the "tclIndex" file for this directory would look like:

```
# Tcl autoload index file, version 2.0 for [incr Tcl]
# This file is generated by the "auto_mkindex" command
# and sourced to set up indexing information for one or
# more commands.  Typically each line is a command that
# sets an element in the auto_index array, where the
# element name is the name of a command and the value is
# a script that loads the command.

set auto_index(::Toaster) "source $dir/Toaster.itcl"
set auto_index(::SmartToaster) "source $dir/SmartToaster.itcl"
```

The **auto_mkindex** command is used to automatically generate "tclIndex" files.

The auto-loader must be made aware of this directory by appending the directory name to the "auto_path" variable. When this is in place, widgets will be auto-loaded as needed when used in an application.

C procedures

C procedures can be integrated into an **[incr\ Tcl]** widget definition to implement methods, procs, and the "config" code for public variables. Any body that starts with "@" is treated as the symbolic name for a C procedure.

Symbolic names are established by registering procedures via **Itcl_RegisterObjC()**. This is usually done in the **Tcl_AppInit()** procedure, which is automatically called when the interpreter starts up. In the following example, the procedure \ffCMy_FooObjCmd() is registered with the symbolic name "foo". This procedure can be referenced in the **body** command as "\ffC@foo".

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter for application. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }

    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

C procedures are implemented just like ordinary Tcl commands. See the **CrtCommand** man page for details. Within the procedure, widget data members can be accessed like ordinary variables using **Tcl_SetVar()**, **Tcl_GetVar()**, **Tcl_TraceVar()**, etc. Widget methods and procs can be executed like ordinary commands using **Tcl_Eval()**. **[incr\ Tcl]** makes this possible by automatically setting up the context before executing the C procedure.

This scheme provides a natural migration path for code development. Widgets can be developed quickly using Tcl code to implement the bodies. An entire application can be built and tested. When necessary, individual bodies can be implemented with C code to improve performance.

Keywords: itclwidget, widget, object, object-oriented

Copyright (c) 2008 Arnulf Wiedemann

local

Name

itcl::local - create an object local to a procedure

Synopsis

itcl::local *className objName* ?*arg arg* ...?

Description

The **local** command creates an **[incr\ Tcl]** object that is local to the current call frame. When the call frame goes away, the object is automatically deleted. This command is useful for creating objects that are local to a procedure.

As a side effect, this command creates a variable named "**itcl-local-***xxx*", where *xxx* is the name of the object that is created. This variable detects when the call frame is destroyed and automatically deletes the associated object.

Example

In the following example, a simple "counter" object is used within the procedure "test". The counter is created as a local object, so it is automatically deleted each time the procedure exits. The **puts** statements included in the constructor/destructor show the object coming and going as the procedure is called.

```
itcl::class counter {
    private variable count 0
    constructor {} {
        puts "created: $this"
    }
    destructor {
        puts "deleted: $this"
    }

    method bump {{by 1}} {
        incr count $by
    }
    method get {} {
        return $count
    }
}

proc test {val} {
    local counter x
    for {set i 0} {$i < $val} {incr i} {
        x bump
    }
    return [x get]
}

set result [test 5]
puts "test: $result"

set result [test 10]
puts "test: $result"

puts "objects: [itcl::find objects *]"
```

Keywords: local, class, object, procedure

Copyright (c) 1993-1998 Lucent Technologies, Inc.

scope

Name

itcl::scope - capture the namespace context for a variable

Synopsis

itcl::scope *name*

Description

Creates a scoped value for the specified *name*, which must be a variable name. If the *name* is an instance variable, then the scope command returns a name which will resolve in any context as an instance variable belonging to *object*. The precise format of this name is an internal detail to Itcl. Use of such a scoped value makes it possible to use instance variables in conjunction with widgets. For example, if you have an object with a private variable \ffCx, and you can use \ffCx in conjunction with the \ffC-textvariable option of an entry widget. Before itcl3.0, only common variables could be used in this manner.

If the *name* is not an instance variable, then it must be a common variable or a global variable. In that case, the scope command returns the fully qualified name of the variable, e.g., \ffC::foo::bar::x.

If the *name* is not recognized as a variable, the scope command returns an error.

Ordinary variable names refer to variables in the global namespace. A scoped value captures a variable name together with its namespace context in a way that allows it to be referenced properly later. It is needed, for example, to wrap up variable names when a Tk widget is used within a namespace:

```
namespace foo {
    private variable mode 1

    radiobutton .rb1 -text "Mode #1" \
        -variable [scope mode] -value 1
    pack .rb1

    radiobutton .rb2 -text "Mode #2" \
        -variable [scope mode] -value 2
    pack .rb2
}
```

Radiobuttons \ffC.rb1 and \ffC.rb2 interact via the variable "mode" contained in the namespace "foo". The **scope** command guarantees this by returning the fully qualified variable name \ffC::foo::mode.

You should never attempt to craft your own scoped variable names, even if you believe you've flawlessly reverse-engineered the encoding. Instead, you should always use the scope command to generate the variable name dynamically. Then, you can pass that name to a widget or to any

other bit of code in your program.

Keywords: scope, code, namespace, variable

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itcl-c

Class

Name

Itcl_CreateClass, Itcl_DeleteClass, Itcl_FindClass, Itcl_IsClass, Itcl_IsClassNamespace - Manipulate classes.

Synopsis

#include <itclInt.h>

int
Itcl_CreateClass(*interp*, *path*, *info*, *rPtr*)

int
Itcl_DeleteClass(*interp*, *cdefnPtr*)

ItclClass *
Itcl_FindClass(*interp*, *path*, *autoload*)

int
Itcl_IsClass(*cmd*)

int
Itcl_IsClassNamespace(*namesp*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter to modify.

const char *_path_ (in)
Path of the class.

ItclObjectInfo *_info_ (in)
TODO.

ItclClass **_rPtr_ (in/out)
The address of the pointer to modify.

ItclClass *_cdefnPtr_ (in)
Pointer to class info struct.

int _autoload_ (in)
Flag value for if the class should be autoloaded

Tcl_Command _cmd_ (in)
Command to check.

Tcl_Namespace *_namesp_ (in)
Namespace to check.

Description

Keywords: Class, find

Copyright (c) 1993-1998 Lucent Technologies, Inc.

List

Name

Itcl_InitList, Itcl_DeleteList, Itcl_CreateListElem, Itcl_DeleteListElem, Itcl_InsertList, Itcl_InsertListElem, Itcl_AppendList, Itcl_AppendListElem, Itcl_SetListValue - Manipulate an Itcl list object.

Synopsis

#include <itcl.h>

void
Itcl_InitList(*list*)

void
Itcl_DeleteList(*list*)

Itcl_ListElem *
Itcl_CreateListElem(*list*)

Itcl_ListElem *
Itcl_DeleteListElem(*elem*)

Itcl_ListElem *
Itcl_InsertList(*list*, *clientData*)

Itcl_ListElem *
Itcl_InsertListElem(*elem*, *clientData*)

Itcl_ListElem *
Itcl_AppendList(*list*, *clientData*)

Itcl_ListElem *
Itcl_AppendListElem(*elem*, *clientData*)

void
Itcl_SetListValue(*elem*, *clientData*)

Arguments

Itcl_List *_list_ (in)
List info structure.

Itcl_ListElem *_elem_ (in)
List element info structure.

void *_clientData_ (in)
Arbitrary one-word value to save in the list.

Description

Keywords: List

Copyright (c) 1993-1998 Lucent Technologies, Inc.

Object

Name

Itcl_CreateObject, Itcl_DeleteObject, Itcl_FindObject, Itcl_IsObject, Itcl_IsObjectIsa - Manipulate an class instance.

Synopsis

#include <itclInt.h>

void
Itcl_PreserveData(*cdata*)

void
Itcl_ReleaseData(*cdata*)

void
Itcl_EventuallyFree(*cdata*, *fproc*)

Arguments

Tcl_FreeProc ****fproc* **(in)**
Address of function to call when the block is to be freed.

void ****clientData* **(in)**
Arbitrary one-word value.

Description

Keywords: Object, free, memory

Copyright (c) 1993-1998 Lucent Technologies, Inc.

Preserve

Name

Itcl_Alloc, Itcl_Free, Itcl_PreserveData, Itcl_ReleaseData, Itcl_EventuallyFree - Manipulate an Itcl list object.

Synopsis

#include <itcl.h>

void *
Itcl_Alloc(*size*)

void
Itcl_PreserveData(*ptr*)

void
Itcl_ReleaseData(*ptr*)

void
Itcl_EventuallyFree(*ptr*, *fproc*)

void
Itcl_Free(*ptr*)

Arguments

size_t *size* (in)
Number of bytes to allocate.

void **ptr* (in)
Pointer value allocated by **Itcl_Alloc**.

Tcl_FreeProc **fproc* (in)
Address of function to call when the block is to be freed.

Description

These procedures are used to allocate and release memory, especially blocks of memory that will be used by multiple independent modules. They are similar in function to the routines in the public Tcl interface, **Tcl_Alloc**, **Tcl_Free**, **Tcl_Preserve**, **Tcl_Release**, and **Tcl_EventuallyFree**. The Tcl routines suffer from issues with performance scaling as the number of blocks managed grows large. The facilities of Itcl encounter these performance scaling issues and require an alternative that does not suffer from them.

Itcl_Alloc returns an untyped pointer to an allocated block of memory of at least *size* bytes. All *size* bytes are initialized to zero.

A module calls **Itcl_PreserveData** on a pointer *ptr* allocated by **Itcl_Alloc** to prevent deallocation of that memory while the module remains interested in it.

A module calls **Itcl_ReleaseData** on a pointer *ptr* previously preserved by **Itcl_PreserveData** to indicate the module no longer has an interest in the block of memory, and will not be disturbed by its deallocation.

Itcl_EventuallyFree is called on a pointer *ptr* allocated by **Itcl_Alloc** to register a deallocation routine *fproc* to be called when the number of calls to **Itcl_ReleaseData** on *ptr* matches the number of calls to **Itcl_PreserveData** on *ptr*. This condition indicates all modules have ended their interest in the block of memory and a call to *fproc* with argument *ptr* will deallocate the memory that no module needs anymore.

Itcl_Free is a deallocation routine for a *ptr* value allocated by **Itcl_Alloc**. It may be called on any *ptr* with no history of an **Itcl_PreserveData** call unmatched by an **Itcl_ReleaseData** call. It is best used as an *fproc* argument to **Itcl_EventuallyFree** or as a routine called from within such an *fproc* routine. It can also be used to deallocate a *ptr* value when it can be assured that value has never been passed to **Itcl_PreserveData** or **Itcl_EventuallyFree**.

Keywords: Preserve, free, memory

Copyright (c) 1993-1998 Lucent Technologies, Inc.

RegisterC

Name

Itcl_RegisterC, Itcl_RegisterObjC, Itcl_RegisterObjC2, Itcl_FindC, Itcl_FindC2 - Associate a symbolic name with a C procedure.

Synopsis

#include <itcl.h>

int

Itcl_RegisterC(*interp*, *cmdName*, *argProc*, *clientData*, *deleteProc*)

int
Itcl_RegisterObjC(*interp*, *cmdName*, *objProc*, *clientData*, *deleteProc*)

int
Itcl_RegisterObjC2(*interp*, *cmdName*, *objProc2*, *clientData*, *deleteProc*)

int
Itcl_FindC(*interp*, *cmdName*, *argProcPtr*, *objProcPtr*, *cDataPtr*)

int
Itcl_FindC2(*interp*, *cmdName*, *objProc2Ptr*, *cDataPtr*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter in which to create new command.

const char *_cmdName_ (in)
Name of command.

Tcl_CmdProc *_argProc_ (in)
Implementation of new command: *argProc* will be called whenever

Tcl_CmdProc **_argProcPtr_ (in/out)
The Tcl_CmdProc * to receive the pointer. Can be NULL.

Tcl_ObjCmdProc *_objProc_ (in)
Implementation of the new command: *objProc* will be called whenever

Tcl_ObjCmdProc2 *_objProc2_ (in)
Implementation of the new command: *objProc2* will be called whenever

Tcl_ObjCmdProc **_objProcPtr_ (in/out)
The Tcl_ObjCmdProc * to receive the pointer.

Tcl_ObjCmdProc2 **_objProc2Ptr_ (in/out)
The Tcl_ObjCmdProc2 * to receive the pointer.

void *_clientData_ (in)
Arbitrary one-word value to pass to *proc* and *deleteProc*.

void **_cDataPtr_ (in/out)
The void * to receive the pointer.

Tcl_CmdDeleteProc *_deleteProc_ (in)
Procedure to call before *cmdName* is deleted from the interpreter; allows for command-specific cleanup. If NULL, then no procedure is called before the command is deleted.

Description

Used to associate a symbolic name with an (argc,argv) C procedure that handles a Tcl command. Procedures that are registered in this manner can be referenced in the body of an [incr Tcl] class definition to specify C procedures to acting as methods/procs. Usually invoked in an initialization routine for an extension, called out in Tcl_AppInit() at the start of an application.

Each symbolic procedure can have an arbitrary client data value associated with it. This value is passed into the command handler whenever it is invoked.

A symbolic procedure name can be used only once for a given style (arg/obj) handler. If the name is defined with an arg-style handler, it can be redefined with an obj-style handler; or if the name is defined with an obj-style handler, it can be redefined with an arg-style handler. In either case, any previous client data is discarded and the new client data is remembered. However, if a name is redefined to a different handler of the same style, this procedure returns an error.

Returns TCL_OK on success, or TCL_ERROR (along with an error message in interp->result) if anything goes wrong.

C procedures can be integrated into an **[incr\ Tcl]** class definition to implement methods, procs, and the "config" code for public variables. Any body that starts with "@" is treated as the symbolic name for a C procedure.

Symbolic names are established by registering procedures via **Itcl_RegisterObjC()** or **Itcl_RegisterObjC2()** or **Itcl_RegisterC()**. This is usually done in the **Tcl_AppInit()** procedure, which is automatically called when the interpreter starts up. In the following example, the procedure \ffCMy_FooObjCmd() is registered with the symbolic name "foo". This procedure can be referenced in the **body** command as "\ffC@foo".

```
int
Tcl_AppInit(interp)
    Tcl_Interp *interp;     /* Interpreter for application. */
{
    if (Itcl_Init(interp) == TCL_ERROR) {
        return TCL_ERROR;
    }

    if (Itcl_RegisterObjC(interp, "foo", My_FooObjCmd) != TCL_OK) {
        return TCL_ERROR;
    }
}
```

C procedures are implemented just like ordinary Tcl commands. See the **CrtCommand** man page for details. Within the procedure, class data members can be accessed like ordinary variables using **Tcl_SetVar()**, **Tcl_GetVar()**, **Tcl_TraceVar()**, etc. Class methods and procs can be executed like ordinary commands using **Tcl_Eval()**. **[incr\ Tcl]** makes this possible by automatically setting up the context before executing the C procedure.

This scheme provides a natural migration path for code development. Classes can be developed quickly using Tcl code to implement the bodies. An entire application can be built and tested. When necessary, individual bodies can be implemented with C code to improve performance.

See the Archetype class in **[incr\ Tk]** for an example of how this C linking method is used.

Keywords: RegisterC, class, object

Copyright (c) 1993-1998 Lucent Technologies, Inc.

Stack

Name

Itcl_InitStack, Itcl_DeleteStack, Itcl_PushStack, Itcl_PopStack, Itcl_PeekStack, Itcl_GetStackValue, Itcl_GetStackSize - Manipulate an Itcl stack object.

Synopsis

#include <itcl.h>

int
Itcl_InitStack(*stack*)

int
Itcl_DeleteStack(*stack*)

int
Itcl_PushStack(*cdata*, *stack*)

void *
Itcl_PopStack(*stack*)

void *
Itcl_PeekStack(*stack*)

void *
Itcl_GetStackValue(*stack*, *pos*)

int
Itcl_GetStackSize(*stack*)

Arguments

Itcl_Stack ****stack* **(in)**
Stack info structure.

int *pos* (in)
position in stack order from the top.

void ****clientData* **(in)**
Arbitrary one-word value to save in the stack.

Description

Itcl_InitStack initializes a stack structure and **Itcl_DeleteStack** deletes it. **Itcl_PushStack** pushes the *cdata* value onto the stack. **Itcl_PopStack** removes and returns the top most *cdata* value. **Itcl_PeekStack** returns the top most value, but does not remove it. **Itcl_GetStackValue** gets a value at some index within the stack. Index "0" is the first value pushed onto the stack. **Itcl_GetStackSize** returns the count of entries on the stack.

Keywords: Stack

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itk

Archetype

Name

Archetype - base class for all [incr\ Tk] mega-widgets

Inheritance

none

Widget-specific options

```
Name:           clientData
Class:          ClientData
Command-Line Switch: -clientdata
```

This does not affect the widget operation in any way. It is simply a hook that clients can use to store a bit of data with each widget. This can come in handy when using widgets to build applications.

Description

The **Archetype** class is the basis for all **[incr\ Tk]** mega-widgets. It keeps track of component widgets and provides methods like "configure" and "cget" that are used to access the composite configuration options. Each component widget must be registered with the **Archetype** base class using the "**itk_component add**" method. When the component is registered, its configuration options are integrated into the composite option list. Configuring a composite option like "-background" causes all of the internal components to change their background color.

It is not used as a widget by itself, but is used as a base class for more specialized widgets. The **Widget** base class inherits from **Archetype**, and adds a Tk frame which acts as the "hull" for the mega-widget. The **Toplevel** base class inherits from **Archetype**, but adds a Tk toplevel which acts as the "hull".

Each derived class must invoke the **itk_initialize** method within its constructor, so that all options are properly integrated and initialized in the composite list.

Public methods

The following methods are provided to support the public interface of the mega-widget.

pathName* cget *option

Returns the current value of the configuration option given by *option*. In this case, *option* refers to a composite configuration option for the mega-widget. Individual components integrate their own configuration options onto the composite list when they are registered by the "**itk_component add**" method.

***pathName* component ?*name*? ?*command arg arg ...*?**

Used to query or access component widgets within a mega-widget. With no arguments, this returns a list of symbolic names for component widgets that are accessible in the current scope. The symbolic name for a component is established when it is registered by the "**itk_component add**" method. Note that component widgets obey any public/protected/private access restriction that is in force when the component is created. If a symbolic *name* is specified, this method returns the window path name for that component. Otherwise, the *command* and any remaining *arg* arguments are invoked as a method on the component with the symbolic name *name*. This provides a well-defined way of accessing internal components without relying on specific window path names, which are really details of the implementation.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName* (see **Tk_ConfigureInfo** for information

on the format of this list). If *option* is specified with no *value*, then the command returns a list describing the one named option (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. In this case, the *options* refer to composite configuration options for the mega-widget. Individual components integrate their own configuration options onto the composite list when they are registered by the "**itk_component add**" method.

Protected methods

The following methods are used in derived classes as part of the implementation for a mega-widget.

itk_component add ?-protected? ?-private? ?--? *name createCmds ?optionCmds?*

Creates a component widget by executing the *createCmds* argument and registers the new component with the symbolic name *name*. The **-protected** and **-private** options can be used to keep the component hidden from the outside world. These options have a similar effect on component visibility as they have on class members. The *createCmds* code can contain any number of commands, but it must return the window path name for the new component widget. The *optionCmds* script contains commands that describe how the configuration options for the new component should be integrated into the composite list for the mega-widget. It can contain any of the following commands:

ignore *option ?option option ...?*

Removes one or more configuration *options* from the composite list. All options are ignored by default, so the **ignore** command is only used to negate the effect of a previous **keep** or **rename** command. This is useful, for example, when the some of the options added by the **usual** command should not apply to a particular component, and need to be ignored.

keep *option ?option option ...?*

Integrates one or more configuration *options* into the composite list, keeping the name the same. Whenever the mega-widget option is configured, the new value is also applied to the current component. Options like "-background" and "-cursor" are commonly found on the **keep** list.

rename *option switchName resourceName resourceClass*

Integrates the configuration *option* into the composite list with a different name. The option will be called *switchName* on the composite list. It will also be modified by setting values for *resourceName* and *resourceClass* in the X11 resource database. The "-highlightbackground" option is commonly renamed to "-background", so that when the mega-widget background changes, the background of the focus ring will change as well.

usual ?*tag*?

Finds the usual option-handling commands for the specified *tag* name and executes them. If the *tag* is not specified, then the widget class name is used as the *tag* name. The "usual" option-handling commands are registered via the **usual** command.

If the *optionCmds* script is not specified, the usual option-handling commands associated with the class of the component widget are used by default.

itk_component delete *name ?name name ...?*

Removes the component widget with the symbolic name *name* from the mega-widget. The

component widget will still exist, but it will no longer be accessible as a component of the mega-widget. Also, any options associated with the component are removed from the composite option list. Note that you can destroy a component using the **destroy** command, just as you would destroy any Tk widget. Components automatically detach themselves from their mega-widget parent when destroyed, so "**itk_component delete**" is rarely used.

itk_initialize ?*option value option value...*?

This method must be invoked within the constructor for each class in a mega-widget hierarchy. It makes sure that all options are properly integrated into the composite option list, and synchronizes all components to the initial option values. It is usually invoked near the bottom of the constructor, after all component widgets have been added. If any *option*/*value* pairs are specified, they override settings determined from the X11 resource database. The arguments to the constructor are usually passed along to this method as follows:

```
itcl::class MyWidget {
    inherit Widget

    constructor {args} {
        .
        .
        .
        eval itk_initialize $args
    }
}
```

itk_option add *optName* ?*optName optName ...*?

Adds one or more options to the composite option list for a mega-widget. Here, *optName* can have one of the following forms:

component*.*option

Accesses an *option* belonging to a component with the symbolic name *component*. The *option* name is specified without a leading "**-**" sign.

className*::*option

Accesses an *option* defined by the "**itk_option define**" command in class *className*. The *option* name is specified without a leading "**-**" sign.

Options are normally integrated into the composite option list when a component widget is first created. This method can be used to add options at a later time. For example, the **Widget** and **Toplevel** base classes keep only the bare minimum options for their "hull" component: -background and -cursor. A derived class can override this decision, and add options that control the border of the "hull" component as well:

```
itcl::class MyWidget {
    inherit Widget

    constructor {args} {
        itk_option add hull.borderwidth hull.relief

        itk_component add label {
```

```
            label $itk_interior.l1 -text "Hello World!"
        }
        pack $itk_component(label)

        eval itk_initialize $args
    }
}
```

itk_option define *switchName resourceName resourceClass init* ?*config*?

This command is used at the level of the class definition to define a synthetic mega-widget option. Within the **configure** and **cget** methods, this option is referenced by *switchName*, which must start with a "**-**" sign. It can also be modified by setting values for *resourceName* and *resourceClass* in the X11 resource database. The *init* value string is used as a last resort to initialize the option if no other value can be used from an existing option, or queried from the X11 resource database. If any *config* code is specified, it is executed whenever the option is modified via the **configure** method. The *config* code can also be specified outside of the class definition via the **configbody** command. In the following example, a synthetic "-background" option is added to the class, so that whenever the background changes, the new value is reported to standard output. Note that this synthetic option is integrated with the rest of the "-background" options that have been kept from component widgets:

```
itcl::class MyWidget {
    inherit Widget
    constructor {args} {
        itk_component add label {
            label $itk_interior.l1 -text "Hello World!"
        }
        pack $itk_component(label)

        eval itk_initialize $args
    }
    itk_option define -background background Background #d9d9d9 {
        puts "new background: $itk_option(-background)"
    }
}
```

itk_option remove *optName* ?*optName optName ...*?

Removes one or more options from the composite option list for a mega-widget. Here, *optName* can have one of the forms described above for the "**itk_option add**" command. Options are normally integrated into the composite option list when a component widget is first created. This method can be used to remove options at a later time. For example, a derived class can override an option defined in a base class by removing and redefining the option:

```
itcl::class Base {
    inherit itk::Widget
    constructor {args} {
        eval itk_initialize $args
    }

    itk_option define -foo foo Foo "" {
        puts "Base: $itk_option(-foo)"
    }
}

itcl::class Derived {
    inherit Base

    constructor {args} {
        itk_option remove Base::foo
        eval itk_initialize $args
    }
    itk_option define -foo foo Foo "" {
        puts "Derived: $itk_option(-foo)"
    }
}
```

Without the "**itk_option remove**" command, the code fragments for both of the "-foo" options would be executed each time the composite "-foo" option is configured. In the example above, the \ffCBase::foo option is suppressed in all Derived class widgets, so only the \ffCDerived::foo option remains.

Protected variables

Derived classes can find useful information in the following protected variables.

itk_component(*name*)
: The "itk_component" array returns the real window path name for a component widget with the symbolic name *name*. The same information can be queried using the **component** method, but accessing this array is faster and more convenient.

itk_interior
: This variable contains the name of the window that acts as a parent for internal components. It is initialized to the name of the "hull" component provided by the **Widget** and **Toplevel** classes. Derived classes can override the initial setting to point to another interior window to be used for further-derived classes.

itk_option(*option*)
: The "itk_option" array returns the current option value for the composite widget option named *option*. Here, the *option* name should include a leading "**-**" sign. The same information can be queried using the **cget** method, but accessing this array is faster and more convenient.

Keywords: Archetype, itk, Widget, Toplevel, mega-widget

Copyright (c) 1993-1998 Lucent Technologies, Inc.

Toplevel

Name
Toplevel - base class for mega-widgets in a top-level window

Inheritance
itk::Archetype <- itk::Toplevel

Standard options

```
background cursor
```

See the "options" manual entry for details on the standard options.

Widget-specific options

```
Name:   title
Class:  Title
Command-Line Switch:    -title
```

Sets the title that the window manager displays in the title bar above the window. The default title is the null string.

Description
The **Toplevel** class inherits everything from the **Archetype** class, and adds a Tk toplevel called the "hull" component to represent the body of the mega-widget. The window class name for the hull is set to the most-specific class name for the mega-widget. The protected variable **itk_interior** contains the window path name for the "hull" component. Derived classes specialize this widget by packing other widget components into the hull.

Since the hull for the **Toplevel** class is implemented with a Tk toplevel, mega-widgets in the **Toplevel** class have their own toplevel window. This class is used to create dialog boxes and other pop-up windows.

Components

```
Name:           hull
Class:          Toplevel
```

The "hull" component acts as the body for the entire mega-widget. Other components are packed into the hull to further specialize the widget.

Example
The following example implements a **MessageInfo** mega-widget. It creates a pop-up message that the user can dismiss by pushing the "Dismiss" button.

```
option add *MessageInfo.title "Notice" widgetDefault

itcl::class MessageInfo {
    inherit itk::Toplevel
```

```
    constructor {args} {
        itk_component add dismiss {
            button $itk_interior.dismiss -text "Dismiss" \
                -command "destroy $itk_component(hull)"
        }
        pack $itk_component(dismiss) -side bottom -pady 4

        itk_component add separator {
            frame $itk_interior.sep -height 2 -borderwidth 1 -relief
    sunken
        }
        pack $itk_component(separator) -side bottom -fill x -padx 4

        itk_component add icon {
            label $itk_interior.icon -bitmap info
        }
        pack $itk_component(icon) -side left -padx 8 -pady 8

        itk_component add infoFrame {
            frame $itk_interior.info
        }
        pack $itk_component(infoFrame) -side left -expand yes \
            -fill both -padx 4 -pady 4

        itk_component add message {
            label $itk_interior.mesg -width 20
        } {
            usual
            rename -text -message message Text
        }
        pack $itk_component(message) -expand yes -fill both

        eval itk_initialize $args

        after idle [code $this centerOnScreen]
    }

    protected method centerOnScreen {} {
        update idletasks
        set wd [winfo reqwidth $itk_component(hull)]
        set ht [winfo reqheight $itk_component(hull)]
        set x [expr ([winfo screenwidth $itk_component(hull)]-$wd)/2]
        set y [expr ([winfo screenheight $itk_component(hull)]-$ht)/2]
        wm geometry $itk_component(hull) +$x+$y
    }
}
```

```
itk::usual MessageInfo {
    keep -background -cursor -foreground -font
    keep -activebackground -activeforeground -disabledforeground
    keep -highlightcolor -highlightthickness
}

#
# EXAMPLE:  Create a notice window:
#
MessageInfo .m -message "File not found:\n/usr/local/bin/foo"
```

Keywords: Toplevel, itk, Archetype, Widget, mega-widget

Copyright (c) 1993-1998 Lucent Technologies, Inc.

Widget

Name

Widget - base class for mega-widgets within a frame

Inheritance

itk::Archetype <- itk::Widget

Standard options

```
background cursor
```

See the "options" manual entry for details on the standard options.

Description

The **Widget** class inherits everything from the **Archetype** class, and adds a Tk frame called the "hull" component to represent the body of the mega-widget. The window class name for the hull is set to the most-specific class name for the mega-widget. The protected variable **itk_interior** contains the window path name for the "hull" component. Derived classes specialize this widget by packing other widget components into the hull.

Since the hull for the **Widget** class is implemented with a Tk frame, mega-widgets in the **Widget** class can be packed into other frames and toplevels.

Components

```
Name:           hull
Class:          Frame
```

The "hull" component acts as the body for the entire mega-widget. Other components are packed into the hull to further specialize the widget.

Example

The following example implements a simple **TextDisplay** mega-widget. It creates a read-only display of text with a text widget and a scrollbar.

```
option add *TextDisplay.wrap none widgetDefault
option add *TextDisplay.textBackground ivory widgetDefault
option add *TextDisplay.width 40 widgetDefault
option add *TextDisplay.height 10 widgetDefault

itcl::class TextDisplay {
    inherit itk::Widget

    constructor {args} {
        itk_component add text {
            text $itk_interior.info -state disabled \
                -yscrollcommand [code $itk_interior.sbar set]
        } {
            usual
            keep -tabs -wrap -width -height
            rename -background -textbackground textBackground
    Background
        }
        pack $itk_component(text) -side left -expand yes -fill both

        itk_component add scrollbar {
            scrollbar $itk_interior.sbar \
                -command [code $itk_interior.info yview]
        }
        pack $itk_component(scrollbar) -side right -fill y

        eval itk_initialize $args
    }

    public method display {info}
    public method append {info}
}

itcl::body TextDisplay::display {info} {
    $itk_component(text) configure -state normal
    $itk_component(text) delete 1.0 end
    $itk_component(text) insert 1.0 $info
    $itk_component(text) configure -state disabled
}

itcl::body TextDisplay::append {info} {
    $itk_component(text) configure -state normal
    $itk_component(text) insert end $info
    $itk_component(text) configure -state disabled
}
```

```
itk::usual TextDisplay {
    keep -background -cursor -foreground -font
    keep -activebackground -activerelief
    keep -highlightcolor -highlightthickness
    keep -insertbackground -insertborderwidth -insertwidth
    keep -insertontime -insertofftime
    keep -selectbackground -selectborderwidth -selectforeground
    keep -textbackground -troughcolor
}

#
# EXAMPLE:  Display the /etc/passwd file
#
TextDisplay .file -background red
pack .file

.file display [exec cat /etc/passwd]
```

Keywords: Widget, itk, Archetype, mega-widget

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itk

Name

itk - framework for building mega-widgets in Tcl/Tk

Description

Mega-widgets are high-level widgets that are constructed using Tk widgets as component parts, usually without any C code. A fileselectionbox, for example, may have a few listboxes, some entry widgets and some control buttons. These individual widgets are put together in a way that makes them act like one big widget. A fileselectionbox mega-widget can be created with a command like:

```
fileselectionbox .fsb -background blue -foreground white
```

Once it has been created, it can be reconfigured with a command like:

```
.fsb configure -background green -foreground black
```

and all of its internal components will change color. Each mega-widget has a set of methods that can be used to manipulate it. For example, the current selection can be queried from a fileselectionbox like this:

```
set fileName [.fsb get]
```

In effect, a mega-widget looks and acts exactly like a Tk widget, but is considerably easier to implement.

[incr\ Tk] is a framework for building mega-widgets. It uses **[incr\ Tcl]** to support the object paradigm, and adds base classes which provide default widget behaviors.

All **[incr\ Tk]** widgets are derived from the **Archetype** base class. This class manages internal component widgets, and provides methods like "configure" and "cget" to access configuration options.

The **Widget** base class inherits everything from **Archetype**, and adds a Tk frame which acts as a container for the mega-widget. It is used to build mega-widgets that sit inside of other frames and toplevels. Derived classes create other internal components and pack them into the "hull" frame created by the **Widget** base class.

The **Toplevel** base class inherits everything from **Archetype**, but adds a Tk toplevel which acts as a container for the mega-widget. It is used to build mega-widgets, such as dialog boxes, that have their own toplevel window. Derived classes create other internal components and pack them into the "hull" toplevel created by the **Toplevel** base class.

[incr Widgets] library

[incr\ Widgets] is a mega-widget library built using **[incr\ Tk]**. It can be used right out of the box, and contains more than 30 different widget classes, including:

- fileselectiondialog
- tabnotebook
- panedwindow
- combobox
- optionmenu
- scrolledlistbox
- scrolledframe
- messagedialog
- and many others... The **catalog** demo in the "iwidgets/demos" directory shows all of the available widgets in action. Each widget class has its own man page describing the features available.

Keywords: itk, class, object, object-oriented, mega-widget

Copyright (c) 1993-1998 Lucent Technologies, Inc.

itkvars

Name

itkvars - variables used by [incr\ Tk]

Description

The following global variables are created and managed automatically by the **[incr\ Tk]** library. Except where noted below, these variables should normally be treated as read-only by application-specific code and by users.

itk::library
When an interpreter is created, **[incr\ Tk]** initializes this variable to hold the name of a directory containing the system library of **[incr\ Tk]** scripts. The initial value of **itk::library** is set from the ITK_LIBRARY environment variable if it exists, or from a compiled-in value otherwise. When **[incr\ Tk]** is added to an interpreter, it executes the script "\ffCinit.itk" in this directory. This script, in turn, looks for other script files with the name "\ffCinit.*xxx*". Mega-widget libraries will be automatically initialized if they install a script named "\ffCinit.*xxx* " in this directory, where "*xxx*" is the name of the mega-widget library. For example, the **[incr \ Widgets]** library installs the script "\ffCinit.iwidgets" in this directory. This script establishes the "iwidgets" namespace, and sets up autoloading for all **[incr\ Widgets]** commands.

Keywords: itkvars, itcl, itk, variables

Copyright (c) 1993-1998 Lucent Technologies, Inc.

usual

Name

usual - access default option-handling commands

for a mega-widget component

Synopsis

usual ?*tag*? ?*commands*?

Description

The **usual** command is used outside of an **[incr\ Tcl]** class definition to define the usual set of option-handling commands for a component widget. Option-handling commands are used when a component is registered with the **Archetype** base class via the "**itk_component add**" method. They specify how the component's configuration options should be integrated into the composite option list for the mega-widget. Options can be kept, renamed, or ignored, as described in the **Archetype** man page.

It is tedious to include the same declarations again and again whenever components are added. The **usual** command allows a standard code fragment to be registered for each widget class, which is used by default to handle the options. All of the standard Tk widgets have **usual** declarations defined in the **[incr\ Tk]** library. Similar **usual** declarations should be created whenever a new mega-widget class is conceived. Only the most-generic options should be included in the **usual** declaration.

The *tag* name is usually the name of a widget class, which starts with a capital letter; however, any string registered here can be used later with the **usual** command described on the **Archetype** man page.

If the *commands* argument is specified, it is associated with the *tag* string, and can be accessed later via **itk_component add**.

If only the *tag* argument is specified, this command looks for an existing *tag* name and returns the commands associated with it. If there are no commands associated with *tag*, this command

returns the null string.

If no arguments are specified, this command returns a list of all *tag* names previously registered.

Example

Following is the **usual** declaration for the standard Tk button widget:

```
itk::usual Button {
    keep -background -cursor -foreground -font
    keep -activebackground -activeforeground -disabledforeground
    keep -highlightcolor -highlightthickness
    rename -highlightbackground -background background Background
}
```

Only the options that would be common to all buttons in a single mega-widget are kept or renamed. Options like "-text" that would be unique to a particular button are ignored.

Keywords: usual, itk, Archetype, component, mega-widget

Copyright (c) 1993-1998 Lucent Technologies, Inc.

mclistbox

mclistbox

NAME

mclistbox::mclistbox - Create and manipulate a multi-column listbox

SYNOPSIS

package require mclistbox ?1.02

mclistbox::mclistbox *pathName?options?*

EXPORTS

mclistbox

STANDARD OPTIONS

-background -borderwidth -cursor -exportselection -font -foreground -height -highlightbackground -highlightcolor -highlightthickness -relief -selectbackground -selectborderwidth -selectforeground -setgrid -takefocus -width -xscrollcommand -yscrollcommand

See the *options* manual entry for detailed descriptions of the above options.

WIDGET-SPECIFIC OPTIONS

Command-Line Name: **-columnborderwidth**
Database Name: **columnBorderWidth**
Database Class: **BorderWidth**

Specifies a non-negative value indicating the width of the 3-D border to draw around the outside of the column (if such a border is being drawn; the **columnrelief** option typically determines this). The value may also be used when drawing 3-D effects in the interior of the widget. The value may have any of the forms acceptable to Tk_GetPixels.

Command-Line Name: **-columnrelief**
Database Name: **columnRelief**
Database Class: **Relief**

Specifies the 3-D effect desired for the column. Acceptable values are raised, **sunken**, **flat**, **ridge**, **solid**, and **groove**. The value indicates how the interior of the column should appear relative to its exterior; for example, raised means the interior of the column should appear to protrude from the screen, relative to the exterior of the column.

Command-Line Name: **-fillcolumn**
Database Name: **fillColumn**
Database Class: **FillColumn**

Specifies which column should grow or shrink such that all columns exactly fill the listbox widget.

Command-Line Name: **-labelanchor**
Database Name: **labelAnchor**
Database Class: **Anchor**

Specifies how the information in the column label is to be displayed. Must be one of the values **n** , **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, or **center**. For example, nw means display the information such that its top-left corner is at the top-left corner of the column label.

Command-Line Name: **-labelbackground** or **-labelbg**
Database Name: **labelBackground**
Database Class: **Background**

Specifies the normal background color to use when displaying the label.

Command-Line Name: **-labelborderwidth or -labelbd**
Database Name: **labelBorderWidth**
Database Class: **BorderWidth**

Specifies a non-negative value indicating the width of the 3-D border to draw around the outside of the column label (if such a border is being drawn; the relief option typically determines this). The value may also be used when drawing 3-D effects in the interior of the column label. The value may have any of the forms acceptable to Tk_GetPixels.

Command-Line Name: **-labelfont**
Database Name: **labelFont**
Database Class: **Font**

Specifies the font to use when drawing text inside the column label.

Command-Line Name: **-labelforeground**
Database Name: **labelForeground**
Database Class: **Foreground**

Specifies the normal foreground color to use when displaying the column label.

Command-Line Name: **-labelheight**
Database Name: **labelHeight**
Database Class: **Height**

Specifies a desired height for the label. If an image or bitmap is being displayed in the label then the value is in screen units (i.e. any of the forms acceptable to Tk_GetPixels); for text it is in lines

of text. If this option isn't specified, the label's desired height is computed from the size of the image or bitmap or text being displayed in it.

Command-Line Name: **-labelrelief**
Database Name: **labelRelief**
Database Class: **Relief**

Specifies the 3-D effect desired for the column label. Acceptable values are **raised**, **sunken**, **flat**, **ridge**, **solid**, and **groove**. The value indicates how the interior of the column label should appear relative to its exterior; for example, raised means the interior of the column label should appear to protrude from the screen, relative to the exterior of the column label.

Command-Line Name: **-labels**
Database Name: **labels**
Database Class: **Labels**

A boolean value which determines whether column labels are shown or not.

Command-Line Name: **-selectcommand** *command*
Database Name: **selectCommand**
Database Class: **Command**

Specifies a Tcl command to be run whenever the selection in the mclistbox changes. The command will be called with the result of the curselection command.

Command-Line Name: **-resizablecolumns**
Database Name: **resizableColumns**
Database Class: **ResizableColumns**

Specifies whether the columns can be resized interactively. If set to true, mouse bindings will be defined to allow the columns to be resized. See **DEFAULT BINDINGS** for more information.

Command-Line Name: **-selectmode**
Database Name: **selectMode**
Database Class: **SelectMode**

Specifies one of several styles for manipulating the selection. The value of the option may be arbitrary, but the default bindings expect it to be either **single**, **browse**, **multiple**, or **extended**; the default value is browse.

DESCRIPTION

The **mclistbox** command creates a new window (given by the *pathName* argument) and makes it into a mclistbox widget. Additional options, described above, may be specified on the command line to configure aspects of the mclistbox such as its colors, font, text, and relief. The mclistbox command returns its pathName argument. At the time this command is invoked, there must not exist a window named pathName, but pathName's parent must exist.

A mclistbox is a widget that displays a list of strings, one per line. When first created, a new mclistbox has no elements and no columns. Columns and elementd may be added or deleted using widget commands described below. In addition, one or more elements may be selected as described below. When an element in one column is selected, the elements in all columns are selected.

It is not necessary for all the elements to be displayed in the mclistbox window at once; commands described below may be used to change the view in the window. Mclistboxes allow

scrolling in both directions using the standard xScrollCommand and yScrollCommand options. They also support scanning, as described below.

WIDGET COMMAND

The widget command accepts many of the same arguments as the standard listbox command. The following commands behave identically to the standard listbox commands:

activate bbox cget configure curselection delete index nearest scan see selection size xview yview

In addition, there are a few commands unique to the mclistbox:

pathName column option args

pathName column add name options

Adds a new column with the specified *name*. *options* may be any of the valid column configuration options.

pathName column cget name option

Returns the value for the named *option*

pathName column configure name options

Configures or queries attributes of the *named* column.

Columns may be configured with the standard listbox options (-background, -font, etc). In addition, the following options are available:

-background

Specifies the background color for this column. If the value is an empty string the column will be configured with the default background color.

-bitmap

Specifies a bitmap to display in the column label. If the bitmap option is specified then it overrides the -label option. the bitmap option may be reset to an empty string to re-enable a text display.

-font

Specifies the font for this column. If the value is an empty string the column will be configured with the default font. Note that if fonts with different line heights are used for separate columns, the data in each column will not line up properly.

-foreground

Specifies the foreground color for this column. If the value is an empty string the column will be configured with the default foreground color.

-image

Specifies an image to display in the column label. If the image option is specified then it overrides other options that specify a bitmap or textual value to display in the column label; the image option may be reset to an empty string to re-enable a bitmap or text display.

-label

Specifies the desired text to be used in the column label.

-position

Specifies the position of the column. 0 (zero) is the first column, 1 (one) is the second and so on.

-resizable

Specifies whether the column is resizable. This only takes effect if the global option **-resizablecolumns** is true.

-visible

Specifies whether the column is visible or not.

-width

Specifies the desired width of the column in characters. If the font doesn't have a uniform width then the width of the character ``0'' is used in translating from character units to screen units. If zero or less, then the desired width for the window is made just large enough to hold all the elements in the listbox.

pathName column delete name

Deletes the column identified by *name*

pathName column names

Returns a list containing all of the column *names*. The order of the list is the same as the visual order of the columns, and is the same as the order the data is returned by the get command.

pathName column nearext x

Given a x-coordinate within the mclistbox window, this command returns the name of the column nearest to that x-coordinate.

pathName get first ?last?

If last is omitted, returns a list containing the element from each column of the listbox element indicated by *first*, or an empty list if first refers to a non-existent element. If *last* is specified, the command returns a list of lists. Each of the sublists will contain an element from each column of the mclistbox element. Both first and last may have any of the standard forms for indices.

The data will be returned in the order that the columns appear visually, and is the same order as reported by the **column names** command.

pathName insert index list ?list ...?

Inserts zero or more new elements in the mclistbox just before the element given by index. If index is specified as end then the new elements are added to the end of the list. Returns an empty string. Each *list* argument becomes one row in the mclistbox. If there are not enough elements to fill all of the columns, blank values will be used to fill in the missing columns.

pathName label bind name sequence command

This command associates *command* with a column label such that whenever the event sequence given by *sequence* occurs over the label for the column named *name* the *command* will be invoked. This widget command is similar to the bind command except that it operates on column labels rather than the entire widget. See the bind manual entry for complete details on the syntax of *sequence* and the substitutions performed on *command* before invoking it.

EXAMPLE

The following example creates a listbox with three columns:

```
    mclistbox .listbox -width 55
```

```
    .listbox column add name -label "First Name"    -width 20
    .listbox column add size -label "Last Name"     -width 20
    .listbox column add mod  -label "Organization"  -width 15
```

To add items to the list, do the following:

```
    .listbox insert end [list "Bryan" "Oakley" "ChannelPoint"]
    .listbox insert end [list "John"  "Ousterhout" "Ajuba Solutions"]
```

... or more concisely:

```
    .listbox insert end \
            [list "Bryan" "Oakley" "ChannelPoint"] \
            [list "John"  "Ousterhout" "Ajuba Solutions"]
```

To cause a proc to be called when the user clicks on the column label of the column named ``date'', use ``label bind'' as in the following example:

```
    .listbox label bind date <ButtonPress-1> "sortByDate %W"
```

```
    proc sortByDate {w} {
        # code to sort the contents of the mclistbox named $w
    }
```

DEFAULT BINDINGS

Bindings in the mclistbox widget are the same as for a standard listbox widget. In addition, if **-resizableColumns** is set to true, the following additional bindings are set:

ButtonPress-1 starts a column resize if the cursor is over a column header and within a few pixels of the left or right edge of the label. **B1-Motion** causes the columns on either side of the cursor to be resized based on the amount the cursor has moved. **ButtonRelease-1** ends the column resize.

If a binding is set on a label via the **label bind** command and **-resizablecolumns** is set, the binding will not fire the event happens in the few pixels inside the left and right edges of the label where a resize can occur. This is to prevent both the label bindings and resize events to happen with a single button press.

BUGS

If you use different fonts for different columns, the data will not line up properly if those fonts don't have the same line height. There is currently no workaround other than to use fonts with the same line height.

Mixing labels and images in column headers can produce unevenly sized headers, which in turn will cause the data in the columns not to line up properly.

Event bindings that fire while the mouse cursor is over a column return wrong values for %W, %x and %y. The returned values represent the internal widgets clicked over rather than relative to the mclistbox widget (there's no way to work around this in tk as of 8.1b2). To convert a value returned for the `%W` keyword, use the function ::mclistbox::convert. Sorry, no real documentation here; see the example (example.tcl) that comes with this distribution

The command ``<pathName> xview index'' doesn't work.

I'm sure there are others...

Keywords: mclistbox

mpexpr

mpexpr

Table of Contents

NAME

mpexpr - Evaluate an expression with multiple precision math

SYNOPSIS

package require Mpexpr
mpexpr arg ?*arg arg ...* ?
mpformat formatString ?*arg arg ...* ?
global mp_precision

DESCRIPTION

Mpexpr is based on Tcl's native *expr* command, and shares many similarities with *expr*. Mpexpr performs all of its calculations using an arbitrary precision math package.

Mpexpr concatenates *arg* 's (adding separator spaces between them), evaluates the result as a Tcl expression, and returns the value. The operators permitted in Tcl expressions are a subset of the operators permitted in C expressions, and they have the same meaning and precedence as the corresponding C operators. Expressions almost always yield numeric results (integer or floating-point values). For example, the expression

mpexpr 8.2 + 6

evaluates to 14.2. Tcl expressions differ from C expressions in the way that operands are specified. Also, Tcl expressions support non-numeric operands and string comparisons.

OPERANDS

A Tcl expression consists of a combination of operands, operators, and parentheses. White space may be used between the operands and operators and parentheses; it is ignored by the expression processor. Where possible, operands are interpreted as integer values. Integer values may be specified in decimal (the normal case), in octal (if the first character of the operand is **0**), or in hexadecimal (if the first two characters of the operand are **0x**). If an operand does not have one of the integer formats given above, then it is treated as a floating-point number if that is possible. Floating-point numbers may be specified in any of the ways accepted by an ANSI-compliant C compiler (except that the ``f'', ``F'', ``l'', and ``L'' suffixes will not be permitted in most installations). For example, all of the following are valid floating-point numbers: 2.1, 3., 6e4, 7.91e+16. If no numeric interpretation is possible, then an operand is left as a string (and only a limited set of operators may be applied to it).

Operands may be specified in any of the following ways:

1. As an numeric value, either integer or floating-point.
2. As a Tcl variable, using standard **$** notation. The variable's value will be used as the operand.
3. As a string enclosed in double-quotes. The expression parser will perform backslash, variable,

and command substitutions on the information between the quotes, and use the resulting value as the operand

4. As a string enclosed in braces. The characters between the open brace and matching close brace will be used as the operand without any substitutions.
5. As a Tcl command enclosed in brackets. The command will be executed and its result will be used as the operand.
6. As a mathematical function whose arguments have any of the above forms for operands, such as ``**sin($x)** ''. See below for a list of defined functions.

Where substitutions occur above (e.g. inside quoted strings), they are performed by the expression processor. However, an additional layer of substitution may already have been performed by the command parser before the expression processor was called. As discussed below, it is usually best to enclose expressions in braces to prevent the command parser from performing substitutions on the contents.

For some examples of simple expressions, suppose the variable **a** has the value 3 and the variable **b** has the value 6. Then the command on the left side of each of the lines below will produce the value on the following line:

mpexpr 3.1 + $a
6.1

mpexpr 2 + "$a.$b"
5.6

mpexpr 4*[llength "6 2"]
8

mpexpr {{word one} < "word $a"}
0

OPERATORS

The valid operators are listed below, grouped in decreasing order of precedence:

- + ~!
Unary minus, unary plus, bit-wise NOT, logical NOT. None of these operands may be applied to string operands, and bit-wise NOT may be applied only to integers.

*** / %**
Multiply, divide, remainder. None of these operands may be applied to string operands, and remainder may be applied only to integers. The remainder will always have the same sign as the divisor and an absolute value smaller than the divisor.

+ -
Add and subtract. Valid for any numeric operands.

<< >>
Left and right shift. Valid for integer operands only. Integers in mpexpr are not limited to a machine word and do not use two's complement format. Therefore shifting will not include a sign bit.

< > <= >=
Boolean less, greater, less than or equal, and greater than or equal. Each operator produces 1 if the condition is true, 0 otherwise. These operators may be applied to strings as well as numeric operands, in which case string comparison is used.

==!=
Boolean equal and not equal. Each operator produces a zero/one result. Valid for all operand types.

&
Bit-wise AND. Valid for integer operands only.

^
Bit-wise exclusive OR. Valid for integer operands only.

|
Bit-wise OR. Valid for integer operands only.

&&
Logical AND. Produces a 1 result if both operands are non-zero, 0 otherwise. Valid for numeric operands only (integers or floating-point).

||
Logical OR. Produces a 0 result if both operands are zero, 1 otherwise. Valid for numeric operands only (integers or floating-point).

x?y:z
If-then-else, as in C. If *x* evaluates to non-zero, then the result is the value of *y*. Otherwise the result is the value of *z*. The *x* operand must have a numeric value.

See the C manual for more details on the results produced by each operator. All of the binary operators group left-to-right within the same precedence level. For example, the command

mpexpr 4*2 < 7

returns 0.

The **&&**, ||, and**?:** operators have ``lazy evaluation'', just as in C, which means that operands are not evaluated if they are not needed to determine the outcome. For example, in the command

mpexpr {$v ? [a]: [b]}

only one of **[a]** or **[b]** will actually be evaluated, depending on the value of **$v**. Note, however, that this is only true if the entire expression is enclosed in braces; otherwise the Tcl parser will evaluate both **[a]** and **[b]** before invoking the **expr** command.

MATH FUNCTIONS

Mpexpr supports the following mathematical functions in expressions. *x* and *y* are integer or floating point values; *i*, *j* and *c* are integer values;

Math functions compatible with *expr*:

acos(x)
Arc cosine of *x*.

asin(x)
Arc sine of *x*.

atan(x)
Arc tangent of *x*.

atan2(x,y)
Arc tangent of *x* / *y*.

ceil(x)

Least integral value greater than or equal to *x*.

cos(x)
Cosine of *x*.

cosh(x)
Hyperbolic cosine of *x*.

exp(x)
Exponential function e ** *x*.

floor(x)
Greatest integral value less than or equal to *x*.

fmod(x,y)
Remainder of *x* divided by *y*.

hypot(x,y)
Euclidean distance of sqrt(*x* * *x* + *y* * *y*).

log(x)
Natural logarithm of *x*.

log10(x)
Base-10 logarithm of *x*.

pow(x,y)
x raised to the *y* power.

sin(x)
Sine of *x*.

sinh(x)
Hyperbolic sine of *x*.

sqrt(x)
Square root of *x*.

tan(x)
Tangent of *x*.

tanh(x)
Hyperbolic tangent of *x*.

abs(x)
Returns the absolute value of *x*. *x* may be either integer or floating-point, and the result is returned in the same form.

double(x)
If *x* is a floating value, returns *x*, otherwise converts *x* to floating and returns the converted value.

int(x)
If *x* is an integer value, returns *x*, otherwise converts *x* to integer by truncation and returns the converted value.

round(x)
If *x* is an integer value, returns *x*, otherwise converts *x* to integer by rounding and returns the converted value.

Additional *mpexpr* functions:

root(x,y)

The *yth* root of *x*.

frem(x,y)
Remove all occurance of factory from number *x*.

minv(x,y)
Inverse of *x* modulo *y*.

gcd(x,y)
Greatest common divisor of *x* and *y*.

lcm(x,y)
Least common multiple of *x* and *y*.

max(x,y)
Maximum of *x* and *y*.

min(x,y)
Minimum of *x* and *y*.

pi()
Value of pi.

fib(i)
Fibonacci number of integer *i*.

fact(i)
Factorial of integer *i*.

pfact(i)
Product of prime numbers up to integer *i*.

lfactor(i,c)
Lowest prime factor of integer *i*, trying count *c* times.

iroot(i,j)
Integer root *j* of integer *i*.

gcdrem(i,j)
Relatively prime of greatest common divisor of *i* divided by *j*.

perm(i,j)
Permutations of *i* taking *j* at a time: *i*! / (*i* - *j*)!.

comb(i,j)
Combinations of *i* taking *j* at a time: *i*! / (*j*! * (*i* - *j*)!).

prime(i,c)
Return 0 if *i* is not prime, return 1 if *i* probably is prime. Test for primality count *c* times. The chance of a non-prime passing this test is less than (1/4)^count. For example, a count of 100 fails for only 1 in 10^60 numbers.

relprime(i,j)
Return 1 if *i* and *j* are relatively prime to each other, 0 otherwise.

TYPES, OVERFLOW, AND PRECISION

Computations are performed using arbitrary fixed and floating point values. Native machine values (*int, long, IEEE 754 floating point, etc.*) and instructions are not used. Conversion among internal representations for integer, floating-point, and string operands is done automatically as needed. For arithmetic computations, integers are used until some floating-point number is introduced, after which floating-point is used. For example,

mpexpr 5 / 4

returns 1, while

mpexpr 5 / 4.0
mpexpr 5 / ([string length "abcd"] + 0.0)

both return 1.25. Floating-point values are always returned with a ``.'' or an ``e'' so that they will not look like integer values. For example,

mpexpr 20.0/5.0

returns ``4.0'', not ``4''.

The global variable **mp_precision** determines the number of significant digits that are retained during evaluation. If **mp_precision** is unset then 17 digits of precision are used. The maximum value of **mp_precision** is 10000. Note that larger values for **mp_precision** will require increasingly longer execution times. Setting **mp_precision** to an illegal value will generate an error.

STRING OPERATIONS

String values may be used as operands of the comparison operators, although the expression evaluator tries to do comparisons as integer or floating-point when it can. If one of the operands of a comparison is a string and the other has a numeric value, the numeric operand is converted back to a string using the C *sprintf* format specifier **%d** for integers and **%g** for floating-point values. For example, the commands

mpexpr {"0x03" > "2"} mpexpr {"0y" < "0x12"}

both return 1. The first comparison is done using integer comparison, and the second is done using string comparison after the second operand is converted to the string ``18''. Because of Tcl's tendency to treat values as numbers whenever possible, it isn't generally a good idea to use operators like **==** when you really want string comparison and the values of the operands could be arbitrary; it's better in these cases to use the **string compare** command instead.

mpformat formats a string in the style of Tcl's native *format* command. Mpformat will interpret numeric arguments as arbitrary precision numbers. Mpformat performs limited % substitution on the output string. The following may be specified:

% [-] [width[.precision]] formatChar

-
Specifies left justification; right justification is the default.

width.precision
Specifies optional width and precision. Default precision is 8. Width and/or precision may be specified as *, in which the next argument will be used for the width or precision value.

Format character and result

d
Format next argument as integer, truncating after the decimal point.

f
Format next argument in decimal floating point.

e
Format next argument in scientific notation.

r, R
Format next argument as rational fraction x / y.

N
Format next argument as numerator only of rational fraction x / y.

D
Format next argument as denominator only of rational fraction x / y.

o
Format next argument in octal format, with leading '0'; floating point argument formatted as octal rational fraction x / y.

x
Format next argument in hexadecimal format, with leading '0x'; floating point formatted argument as hexadecimal rational fraction x / y.

b
Format next argument in binary format, with leading '0b'; floating point argument formatted as binary rational fraction x / y.

s
Format next argument as string.

c
Format next argument as single character value.

%
Format single literal %.

Other characters in format string

\n
Format ASCII newline.

\r
Format ASCII carriage return.

\t
Format ASCII tab.

\f
Format ASCII form feed.

\v
Format ASCII vertical tab.

\b
Format ASCII backspace.

NOTES

Mpexpr is based on Tcl 7.6 'tclExpr.c' and David Bell's 'Calc' program. This man page is largely borrowed from Tcl 7.6 as well, as is the mpexpr test suite.

See the files README and INSTALL for additional information.

Tcl 7.6 is Copyright (c) 1987-1994 The Regents of the University of California and Copyright (c) 1994 Sun Microsystems, Inc.

Calc is Copyright (c) 1994 David I. Bell.

AUTHOR

Tom Poindexter, tpoindex@nyx.net, Talus Technologies, Inc., Highlands Ranch, CO.
http://www.nyx.net/~tpoindex

Version 1.0 released November, 1998.

Copyright 1998 Tom Poindexter. See the file 'LICENSE.TERMS' for additional copyright and licensing terms.

Table of Contents

- NAME
- SYNOPSIS
- DESCRIPTION
- OPERANDS
- OPERATORS
- MATH FUNCTIONS
- TYPES, OVERFLOW, AND PRECISION
- STRING OPERATIONS
- NOTES
- AUTHOR

Keywords: mpexpr

tclhttpd

httpd

Name

tclhttpd - Tcl Web Server

Synopsis

tclsh httpd.tcl ?*options*?

Options

-help
Displays usage information, then exit without doing anything.

-config *filename*
Name of the configuration file (e.g. tclpro/bin/tclhttpd.rc).

-main *filename*
Name of the per-thread main script (e.g. tclpro/bin/httpdthread.tcl).

-docRoot *directory*
The root directory of your web pages (e.g., tclpro/tclhttpd/htdocs).

-port *value*
HTTP listening port. Defaults to 8015.

-host *value*
The hostname for the HTTP listening socket.

-ipaddr *value*

Interface the server should bind to.

-webmaster *email*
Email contact for webmaster.

-uid *userid*
User name or ID for server process user ID.

-gid *groupid*
Group name or ID for server process group ID.

-threads *num*
Run with *num* worker threads. Requires a thread safe Tcl shell.

-library *directory*
Directory to add to the auto_path.

-verbose
Causes extra print statements during startup.

Description

TclHttpd is a simple, extensible, embeddable Web Server. The best source of documentation is in HTML distributed with the server.

To start the server, simply run the httpd.tcl script with *tclsh* or *wish*. For example, this starts the server on the standard Web server port, 80. tclsh <installdir>/bin/httpd.tcl -port 80 Note that you must start the server as root if you use port numbers less than 1024 on UNIX systems. If you want the server process to run under a different user than root, which is strongly recommended, then use the **-uid** and **-gid** options. This way the server can start as root, open the socket, and then switch to a less privileged account.

Configuration and customization

The main script depends on a per-thread Tcl script, httpdthread.tcl, and a configuration file, tclhttpd.rc. These have configuration settings and the start up code for the web server.

The configuration file can be used to set the port, user ID, and other values described in the Options list above. You can configure additional features such as log file location, and more, by editting the configuration file. There is an explanation about each option, so you can make a copy of the configuration file and try out new settings.

```
tclsh httpd.tcl -config myserver.rc
```

If you plan to extend Tcl Httpd with your own code, you may need to add initialization code to bin/httpd.tcl and bin/httpdthread.tcl. This code is typically a "package require" for your module and one or two calls to initialize it. For example, this code the httpdthread.tcl enables a /debug URL implementation that lets you examine the state of the server.

```
package require httpd::debug
Debug_Url /debug Debug
```

The web server should have access to any Tcl package installed along with your Tcl installation. Consult the on-line HTML documentation for a more indepth discussion of programming the server.

Web page templates

TclHttpd supports a flexible template system that embeds Tcl code into your HTML pages. The Web Server processes the Tcl, which typically generates bits and pieces of your HTML page, and delivers the result to the client transparently. You can cache the results of processing your templates, or you can have pages that are processed dynamically on each access.

Any page that ends in ".tml" is treated like an HTML+Tcl template page. The Web Server uses the Tcl **subst** command to replace commands within brackets, [and], and variable references, like $Phone, with their value. Backslash processing is also done. The main thing you need to watch out for is putting literal dollar amounts in your templates. You'll need to protect your $ with a backslash:

```
The price is \$10.00.
```

The ".tml" files in the sample htdocs directory structure should give you examples to work from.

Try to limit the Tcl code in your pages to simple procedure calls, and put the procedure definitions in per-directory files named ".tml". The name of this file is confusing: each directory can contain a file named "dot-t-m-l" (.tml) that should contain Tcl code. These files are automatically loaded before any templates in that directory (or subdirectories) is processed.

For example, first create a new directory of the htdocs directory that comes with TclHttpd.

```
mkdir htdocs/mystuff
```

Next, put the following into htdocs/mystuff/.tml

```
package require htmlutils

# A procedure to format the date the way you like it
proc MyDate {{seconds {}}} {
    if {[string length $seconds] == 0} {
    set seconds [clock seconds]
    }
    return [clock format $seconds -format "%B %m, %Y"]
}
# Some page settings
set bgcolor pink
```

Now, any page in the htdocs/mystuff directory can use the **MyDate** procedure in a template. Finally, put the following into htodcs/mystuff/index.tml

```
<title>My Stuff</title>
<body text=black bgcolor=$bgcolor>
<h2>My Stuff</h2>
[MyDate]
<br>
Page content here.
<p>
Send email to [Mailto [Doc_Webmaster]].
```

The **bgcolor** variable is set in the .tml file and used in the BODY tag. The **Mailto** is part of the **htmlutils** package that was required by the .tml file. The **Doc_Webmaster** procedure is built

into TclHttpd. The MyDate procedure was added by you, and is shared by any page in or below the htdocs/mystuff directory.

Keywords: httpd, Web Server, HTTP, TclHttpd

Copyright (c) 1998-2000 by Ajuba Solutions
All rights reserved.

tcltls

tls

Name

tls - binding to the OpenSSL library for encrypted socket and I/O channel communications

Synopsis

package require Tcl 8.5-
package require tls 2.0.1
tls::init ?*-option*? ?*value*? ?*-option value* ...?
tls::socket ?*-option*? ?*value*? ?*-option value* ...? *host port*
tls::socket -server *command* ?*-option*? ?*value*? ?*-option value* ...? *port*
tls::import *channel* ?*-option*? ?*value*? ?*-option value* ...?
tls::starttls *channel* ?*-option*? ?*value*? ?*-option value* ...?
tls::handshake *channel*
tls::shutdown *channel*
tls::unimport *channel*
tls::unstack *channel*
tls::status ?**-local**? *channel*
tls::connection *channel*
tls::ciphers ?*protocol*? ?*verbose*? ?*supported*?
tls::protocols
tls::version

Description

This extension provides TCL script access to Secure Socket Layer (SSL) communications using the Transport Layer Security (TLS) protocol. It provides a generic binding to OpenSSL, utilizing the **Tcl_StackChannel** API in TCL 8.4 or later. These sockets behave exactly the same as channels created using the built-in **socket** command, but provide additional options for controlling the SSL/TLS session.

Compatibility

This extension is compatible with OpenSSL 1.1.1 or later including the 3.x and 4.0 releases. It requires Tcl version 8.5 or later and will work with Tcl 9.0.

Commands

The following are the commands provided by the TcLTLS package. See **Examples** for example usage and the "*demos*" directory for more example usage.

tls::init ?*-option*? ?*value*? ?*-option value ...*?

Optional function to set the default options used by **tls::socket**. If you call **tls::import** directly, the values set by this command have no effect. This command supports all of the same options as the **tls::socket** command, though you should limit your options to only the TLS related ones.

tls::socket ?*-option*? ?*value*? ?*-option value ...*? *host port*

This is a helper function that utilizes the underlying commands **socket** and **tls::import** to create the connection. It behaves the same as the native TCL **socket** command, but also supports the **tls::import** command options with one additional option. It returns the channel handle id for the new socket. Additional options are:

-autoservername *bool*

If **true**, automatically set the **-servername** argument to the *host* argument. Prior to TclTLS 2.0, the default is **false**. Starting in TclTLS 2.0, the default is **true** unless **-servername** is also specified.

tls::socket -server *command* ?*-option*? ?*value*? ?*-option value ...*? *port*

Same as previous command, but instead creates a server socket for clients to connect to just like the Tcl **socket -server** command. It returns the channel handle id for the new socket.

tls::import *channel* ?*-option*? ?*value*? ?*-option value ...*?

...see next...

tls::starttls *channel* ?*-option*? ?*value*? ?*-option value ...*?

Start TLS encryption on TCL channel *channel* via a stacked channel. It need not be a socket, but must provide bi-directional flow. Also sets session parameters for SSL handshake. Valid options are:

-alpn *list*

List of protocols to offer during Application-Layer Protocol Negotiation (ALPN). For example: **h2** and **http/1.1**, but not **h3** or **quic**. This option is new for TclTLS 1.8.

-cadir *directory*

Specifies the directory where the Certificate Authority (CA) certificates are stored. The default is platform specific and can be set at compile time. The default location can be overridden by the **SSL_CERT_DIR** environment variable. See **Certificate Validation** for more details.

-cafile *filename*

Specifies the file with the Certificate Authority (CA) certificates to use in **PEM** file format. The default is "*cert.pem*", in the OpenSSL directory. The default file can be overridden by the **SSL_CERT_FILE** environment variable. See **Certificate Validation** for more details.

-castore *URI*

Specifies the Uniform Resource Identifier (URI) for the Certificate Authority (CA) store, which may be a single container or a catalog of containers. Starting with OpenSSL 3.2 on MS Windows, set to "**org.openssl.winstore://**" to use the built-in MS Windows Certificate Store. See **Certificate Validation** for more details. This option is new for TclTLS 1.8.

-certfile *filename*

Specifies the name of the file with the certificate to use in PEM format as the local (client or server) certificate. It also contains the public key.

-cert *string*

Specifies the certificate to use as a DER encoded string (X.509 DER).

-cipher *string*

Specifies the list of ciphers to use for TLS 1.2 and earlier connections. String is a colon "**:**" separated list of ciphers. Ciphers can be combined using the "**+**" character. Prefixes can be used to permanently remove "**!**", delete "**-**", or move to the end "**+**" a specified cipher. Keywords **@STRENGTH** (sort by algorithm key length), **@SECLEVEL=***n* (set security level to n), and **DEFAULT** (use default cipher list, at start only) can also be specified. See the OpenSSL documentation for the full list of valid values.

-ciphersuites *string*
Specifies the list of cipher suites to use for TLS 1.3 as a colon "**:**" separated list of cipher suite names. See the OpenSSL documentation for the full list of valid values. This option is new for TclTLS 1.8.

-command *callback*
Specifies the callback command to be invoked at several points during the handshake to pass errors, tracing information, and protocol messages. See **Callback Options** for more info.

-dhparams *filename*
Specifies the Diffie-Hellman (DH) parameters file.

-keyfile *filename*
Specifies the private key file. The default is to use the file specified by the **-certfile** option.

-key *string*
Specifies the private key to use as a DER encoded string (PKCS#1 DER).

-model *channel*
Force this channel to share the same *SSL_CTX* structure as the specified *channel*, and therefore share config, callbacks, etc.

-password *callback*
Specifies the callback command to invoke when OpenSSL needs to obtain a password. This is typically used to unlock the private key of a certificate. The callback should return a password string. This option has changed for TclTLS 1.8. See **Callback Options** for more info.

-post_handshake *bool*
Allow post-handshake session ticket updates. This option is new for TclTLS 1.8.

-request *bool*
Request a certificate from the peer during the SSL handshake. This is needed to do Certificate Validation. Starting in TclTLS 1.8, the default is **true** for client connections. Starting in TclTLS 2.0, if set to **false** and **-require** is **true**, then this will be overridden to **true**. See **Certificate Validation** for more details.

-require *bool*
Require a valid certificate from the peer during the SSL handshake. If this is set to true, then **-request** must also be set to true and a either **-cadir**, **-cafile**, **-castore**, or a platform default must be provided in order to validate against. The default in TclTLS 1.8 and earlier versions is **false** since not all platforms have certificates to validate against in a form compatible with OpenSSL. Starting in TclTLS 2.0, the default is **true** for client connections. See **Certificate Validation** for more details.

-security_level *integer*
Specifies the security level (value from 0 to 5). The security level affects the allowed cipher suite encryption algorithms, supported ECC curves, supported signature algorithms, DH parameter sizes, certificate key sizes and signature algorithms. The default is 1 prior to OpenSSL 3.2 and 2 thereafter. Level 3 and higher disable support for session tickets and only

accept cipher suites that provide forward secrecy. This option is new for TclTLS 1.8.

-server *bool*
Specifies whether to act as a server and respond with a server handshake when a client connects and provides a client handshake. The default is **false**.

-servername *hostname*
Specify the peer's hostname. This is used to set the TLS Server Name Indication (SNI) extension. Set this to the expected servername in the server's certificate or one of the Subject Alternate Names (SAN). Starting in TclTLS 2.0, this will default to the host from the **tls::socket** command.

-session_id *binary_string*
Specifies the session id to resume a session. Not supported yet. This option is new for TclTLS 1.8.

-ssl2 *bool*
Enable use of SSL v2.The default is **false**. OpenSSL 1.1+ no longer supports SSL v2, so this may not have any effect. See the **tls::protocols** command for supported protocols.

-ssl3 *bool*
Enable use of SSL v3. The default is **false**. Starting in TclTLS 1.8, use of SSL v3 if only available via a compile time option. OpenSSL 4.0+ no longer supports SSL v3, so this may not have any effect. See the **tls::protocols** command for supported protocols.

-tls1 *bool*
Enable use of TLS v1. Starting in TclTLS 2.0, the default is **false**. Note: TLS 1.0 needs SHA1 to operate, which is only available in security level 0 for Open SSL 3.0+. See the **-security_level** option.

-tls1.1 *bool*
Enable use of TLS v1.1. Starting in TclTLS 2.0, the default is **false**. Note: TLS 1.1 needs SHA1 to operate, which is only available in security level 0 for Open SSL 3.0+. See the **-security_level** option.

-tls1.2 *bool*
Enable use of TLS v1.2. The default is **true**.

-tls1.3 *bool*
Enable use of TLS v1.3. The default is **true**. This is only available starting with OpenSSL 1.1.1 and TclTLS 1.7.

-validatecommand *callback*
Specifies the callback command to invoke to validate the peer certificates and other config info during the protocol negotiation phase. This can be used by TCL scripts to perform their own Certificate Validation to supplement the default validation provided by OpenSSL. The script must return a boolean true to continue the negotiation. See **Callback Options** for more info. This option is new for TclTLS 1.8.

tls::handshake *channel*
Forces the TLS negotiation handshake to take place immediately, and returns 0 if handshake is still in progress (non-blocking), or 1 if the handshake was successful. If the handshake failed, an error will be returned.

tls::shutdown *channel*
...see next...

tls::unimport *channel*
...see next...

tls::unstack *channel*
This terminates the SSL/TLS session by sending the "close_notify" message and removes the top level stacked channel from *channel*, but it does not close the socket. It is the compliment to **tls::import** by ending encryption of a TCL channel. An error is thrown if TLS is not the top stacked channel type.

tls::status ?-local? *channel*
Returns the current status of an SSL channel. The result is a list of key-value pairs describing the SSL, certificate, and certificate verification status. If the SSL handshake has not yet completed, an empty list is returned. If the **-local** option is specified, then the local certificate is used. Returned values include: SSL Status

alpn *protocol*
The protocol selected after Application-Layer Protocol Negotiation (ALPN). This value is new for TclTLS 1.8.

cipher *cipher*
The current cipher in use for the session.

peername *name*
The peername from the certificate. This value is new for TclTLS 1.8.

protocol *version*
The protocol version used for the connection: SSL2, SSL3, TLS1, TLS1.1, TLS1.2, TLS1.3, or unknown. This value is new for TclTLS 1.8.

sbits *n*
The number of bits used for the session key.

signatureHashAlgorithm *algorithm*
The signature hash algorithm. This value is new for TclTLS 1.8.

signatureType *type*
The signature type value. This value is new for TclTLS 1.8.

verifyDepth *n*
Maximum depth for the certificate chain verification. Default is -1, to check all. This value is new for TclTLS 1.8.

verifyMode *list*
List of certificate verification modes. This value is new for TclTLS 1.8.

verifyResult *result*
Certificate verification result. This value is new for TclTLS 1.8.

ca_names *list*
List of the Certificate Authorities used to create the certificate. This value is new for TclTLS 1.8.

Certificate Status

all *string*
Dump of all certificate info. This value is new for TclTLS 1.8.

version *value*
The certificate version.

serialNumber *string*
The serial number of the certificate as a hex string. This value was changed from serial in TclTLS 1.8.

signature *algorithm*
Cipher algorithm used for certificate signature. This value is new for TclTLS 1.8.

issuer *string*
The distinguished name (DN) of the certificate issuer.

notBefore *date*
The beginning date of the certificate validity.

notAfter *date*
The expiration date of the certificate validity.

subject *string*
The distinguished name (DN) of the certificate subject. Fields include: Common Name (CN), Organization (O), Locality or City (L), State or Province (S), and Country Name (C).

issuerUniqueID *string*
The issuer unique id. This value is new for TclTLS 1.8.

subjectUniqueID *string*
The subject unique id. This value is new for TclTLS 1.8.

num_extensions *n*
Number of certificate extensions. This value is new for TclTLS 1.8.

extensions *list*
List of certificate extension names. This value is new for TclTLS 1.8.

authorityKeyIdentifier *string*
Authority Key Identifier (AKI) of the Issuing CA certificate that signed the SSL certificate as a hex string. This value matches the SKI value of the Intermediate CA certificate. This value is new for TclTLS 1.8.

subjectKeyIdentifier *string*
Subject Key Identifier (SKI) hash of the public key inside the certificate as a hex string. Used to identify certificates that contain a particular public key. This value is new for TclTLS 1.8.

subjectAltName *list*
List of all of the Subject Alternative Names (SAN) including domain names, sub domains, and IP addresses that are secured by the certificate. This value is new for TclTLS 1.8.

ocsp *list*
List of all Online Certificate Status Protocol (OCSP) URLs that can be used to check the validity of this certificate. This value is new for TclTLS 1.8.

certificate *cert*
The PEM encoded certificate.

signatureAlgorithm *algorithm*
Cipher algorithm used for the certificate signature. This value is new for TclTLS 1.8.

signatureValue *string*
Certificate signature as a hex string. This value is new for TclTLS 1.8.

signatureDigest *version*
Certificate signing digest as a hex string. This value is new for TclTLS 1.8.

publicKeyAlgorithm *algorithm*

Certificate signature public key algorithm. This value is new for TclTLS 1.8.

publicKey *string*
Certificate signature public key as a hex string. This value is new for TclTLS 1.8.

bits *n*
Number of bits used for certificate signature key. This value is new for TclTLS 1.8.

self_signed *boolean*
Whether the certificate signature is self signed. This value is new for TclTLS 1.8.

sha1_hash *hash*
The SHA1 hash of the certificate as a hex string. This value is new for TclTLS 1.8.

sha256_hash *hash*
The SHA256 hash of the certificate as a hex string. This value is new for TclTLS 1.8.

tls::connection *channel*
Returns the current connection status of an SSL channel. The result is a list of key-value pairs describing the connection. This command is new for TclTLS 1.8. Returned values include: SSL Status

state *state*
State of the connection.

servername *name*
The name of the connected to server.

protocol *version*
The protocol version used for the connection: SSL2, SSL3, TLS1, TLS1.1, TLS1.2, TLS1.3, or unknown.

renegotiation_allowed *boolean*
Whether protocol renegotiation is supported or not.

security_level *level*
The security level used for selection of ciphers, key size, etc.

session_reused *boolean*
Whether the session has been reused or not.

is_server *boolean*
Whether the connection is configured as a server (1) or client (0).

compression *mode*
Compression method.

expansion *mode*
Expansion method.

caList *list*
List of Certificate Authorities (CA) for X.509 certificate.

Cipher Info

cipher *cipher*
The current cipher in use for the connection.

standard_name *name*
The standard RFC name of cipher.

algorithm_bits *n*

The number of processed bits used for cipher.

secret_bits *n*
The number of secret bits used for cipher.

min_version *version*
The minimum protocol version for cipher.

cipher_is_aead *boolean*
Whether the cipher is Authenticated Encryption with Associated Data (AEAD).

cipher_id *id*
The OpenSSL cipher id.

description *string*
A text description of the cipher.

handshake_digest *boolean*
Digest used during handshake.

Session Info

alpn *protocol*
The protocol selected after Application-Layer Protocol Negotiation (ALPN).

resumable *boolean*
Whether the session can be resumed or not.

start_time *seconds*
Time since session started in seconds since epoch.

timeout *seconds*
Max duration of session in seconds before time-out.

lifetime *seconds*
Session ticket lifetime hint in seconds.

session_id *binary_string*
Unique session id for use in resuming the session.

session_ticket *binary_string*
Unique session ticket for use in resuming the session.

ticket_app_data *binary_string*
Unique session ticket application data.

master_key *binary_string*
Unique session master key.

session_cache_mode *mode*
Server cache mode (client, server, or both).

tls::ciphers ?*protocol*? ?*verbose*? ?*supported*?
Without any options, it returns a list of all symmetric ciphers for use with the *-cipher* option. With *protocol*, only the ciphers supported for that protocol are returned. See the **tls::protocols** command for the supported protocols. If *verbose* is specified as true then a verbose, human readable list is returned with additional information on the cipher. If *supported* is specified as true, then only the ciphers supported for protocol will be listed. The *supported* arg is new for TclTLS 1.8.

tls::protocols
Returns a list of the supported SSL/TLS protocols. Valid values are: **ssl2**, **ssl3**, **tls1**, **tls1.1**,

tls1.2, and **tls1.3**. Exact list depends on OpenSSL version and compile time flags. This command is new for TclTLS 1.8.

tls::version
Returns the OpenSSL version string.

Certificate validation

Pki and certificates

Using the Public Key Infrastructure (PKI), each user creates a private key that only they know about and a public key they can exchange with others for use in encrypting and decrypting data. The process is the sender encrypts their data using their private key and the receiver's public key. The data is then sent to the receiver. In a similar manner, the receiver uses their private key and the sender's public key to decrypt the data. This provides data integrity, to ensure the data can't be viewed or altered during transport. See the **-key** and **-keyfile** options for how to specify the private key. Also see the **-password** option for how to provide the password.

In order to provide authentication, i.e. ensuring someone is who they say they are, the public key and user identification info is stored in a X.509 certificate and that certificate is authenticated (i.e. signed) by a Certificate Authority (CA). Users can then exchange these certificates during the TLS initialization process and check them against the root CA certificates to ensure they are valid. This is handled by OpenSSL via the **-request** and **-require** options. See the **-cadir**, **-cadir**, and **-castore** options for how to specify where to find the CA certificates. Optionally, in a future release, they can also be checked against the Certificate Revocation List (CRL) of revoked certificates. Certificates can also be self-signed, but they are by default not trusted unless you add them to your certificate store.

Typically when visiting web sites, only the client needs to check the server's certificate to ensure it is valid. The server doesn't need to check the client certificate unless you need to authenticate with them to login, etc. See the **-cert** and **-certfile** options if you need to provide a certificate.

Summary of command line options

The following options are used for peer certificate validation:

-cadir *directory*
Specifies the directory where the Certificate Authority (CA) certificates are stored. The default is platform specific, but is usually "*/etc/ssl/certs*" on Linux/Unix systems. The default location can be overridden by the **SSL_CERT_DIR** environment variable.

-cafile *filename*
Specifies the file with the Certificate Authority (CA) certificates to use in **PEM** file format. The default is "*cert.pem*", in the OpenSSL directory. On Linux/Unix systems, this is usually "*/etc/ssl/ca-bundle.pem*". The default file can be overridden by the **SSL_CERT_FILE** environment variable.

-castore *URI*
Specifies the Uniform Resource Identifier (URI) for the Certificate Authority (CA) store, which may be a single container or a catalog of containers. Starting with OpenSSL 3.2 on MS Windows, set to "**org.openssl.winstore://**" to use the built-in MS Windows Certificate Store. Starting in TclTLS 2.0, this is the default if **-cadir**, **-cadir**, and **-castore** are not specified. This store only supports root certificate stores.

-request *bool*

Request a certificate from the peer during the SSL handshake. This is needed to do Certificate Validation. Starting in TclTLS 1.8, the default is **true** for client connections. Starting in TclTLS 2.0, if set to **false** and **-require** is **true**, then this will be overridden to **true**. In addition, the client can manually inspect and accept or reject each certificate using the **-validatecommand** option.

-require ***bool***
Require a valid certificate from the peer during the SSL handshake. If this is set to true, then **-request** must also be set to true and a either **-cadir**, **-cafile**, **-castore**, or a platform default must be provided in order to validate against. The default in TclTLS 1.8 and earlier versions is **false** since not all platforms have certificates to validate against in a form compatible with OpenSSL. Starting in TclTLS 2.0, the default is **true** for client connections.

When are command line options needed?

In TclTLS 1.8 and earlier versions, certificate validation is *NOT* enabled by default. This limitation is due to the lack of a common cross platform database of Certificate Authority (CA) provided certificates to validate against. Many Linux systems natively support OpenSSL and thus have these certificates installed as part of the OS, but MacOS and MS Windows do not. Staring in TclTLS 2.0, the default for client connections has been changed to require certificate validation by default. In order to use the **-require** option, one of the following must be true:

- On Linux and Unix systems with OpenSSL already installed or if the CA certificates are available in PEM format, and if they are stored in the standard locations, or if the **SSL_CERT_DIR** or **SSL_CERT_FILE** environment variables are set, then **-cadir**, **-cadir**, and **-castore** aren't needed.
- If OpenSSL is not installed in the default location, or when using Mac OS or MS Windows and OpenSSL is installed, the **SSL_CERT_DIR** and/or **SSL_CERT_FILE** environment variables or the one of the **-cadir**, **-cadir**, or **-castore** options must be defined.
- On MS Windows, starting in OpenSSL 3.2, it is now possible to access the built-in Windows Certificate Store from OpenSSL. This can be utilized by setting the **-castore** option to "**org.openssl.winstore://**". In TclTLS 2.0, this is the default value if **-cadir**, **-cadir**, and **-castore** are not specified.
- If OpenSSL is not installed or the CA certificates are not available in PEM format, the CA certificates must be downloaded and installed with the user software. The CURL team makes them available at CA certificates extracted from Mozilla in the "*cacert.pem*" file. You must then either set the **SSL_CERT_DIR** and/or **SSL_CERT_FILE** environment variables or the **-cadir** or **-cafile** options to the CA cert file's install location. It is your responsibility to keep this file up to date.

Callback options

As previously described, each channel can be given their own callbacks to handle intermediate processing by the OpenSSL library, using the **-command**, **-password**, and **-validate_command** options passed to either of **tls::socket** or **tls::import**. Unlike previous versions of TclTLS, only if the callback generates an error, will the **bgerror** command be invoked

with the error information.

Values for command callback

The callback for the **-command** option is invoked at several points during the OpenSSL handshake and during routine operations. See below for the possible arguments passed to the callback script. Values returned from the callback are ignored.

error *channelId message*

This form of callback is invoked whenever an error occurs during the initial connection, handshake, or I/O operations. The *message* argument can be from the Tcl_ErrnoMsg, OpenSSL function **ERR_reason_error_string()**, or a custom message. This callback is new for TclTLS 1.8.

info *channelId major minor message type*

This form of callback is invoked by the OpenSSL function **SSL_set_info_callback()** during the initial connection and handshake operations. The arguments are:

major

Major category for error. Valid enums are: **handshake**, **alert**, **connect**, **accept**.

minor

Minor category for error. Valid enums are: **start**, **done**, **read**, **write**, **loop**, **exit**.

message

Descriptive message string which may be generated either by **SSL_state_string_long()** or **SSL_alert_desc_string_long()**, depending on the context.

type

For alerts, the possible values are: **warning**, **fatal**, and **unknown**. For others, **info** is used. This argument is new for TclTLS 1.8.

message *channelId direction version content_type message*

This form of callback is invoked by the OpenSSL function **SSL_set_msg_callback()** whenever a message is sent or received during the initial connection, handshake, or I/O operations. It is only available when OpenSSL is complied with the **enable-ssl-trace** option. This callback is new for TclTLS 1.8. The arguments are:

direction

Direction is either **Sent** or **Received**.

version

Version is the protocol version.

content_type

Content type is the message content type.

message

Message is more info from the **SSL_trace** API.

session *channelId session_id session_ticket lifetime*

This form of callback is invoked by the OpenSSL function **SSL_CTX_sess_set_new_cb()** whenever a new session id is sent by the server during the initial connection and handshake and also during the session if the **-post_handshake** option is set to true. This callback is new for TclTLS 1.8. The arguments are:

session_id

Session Id is the current session identifier

session_ticket

Ticket is the session ticket info

lifetime

Lifetime is the ticket lifetime in seconds.

verify *channelId depth cert status error*

This callback was moved to **-validatecommand** in TclTLS 1.8.

Values for password callback

The callback for the **-password** option is invoked by TclTLS whenever OpenSSL needs to obtain a password. See below for the possible arguments passed to the callback script. The user provided password is expected to be returned by the callback.

password *rwflag size*

Invoked when loading or storing an encrypted PEM certificate. The arguments are:

rwflag

The read/write flag is 0 for reading/decryption or 1 for writing/encryption. The latter can be used to determine when to prompt the user to confirm. This argument is new for TclTLS 1.8.

size

The size is the maximum length of the password in bytes. This argument is new for TclTLS 1.8.

Values for validate command callback

The callback for the **-validatecommand** option is invoked during the handshake process in order for the application to validate the provided value(s). See below for the possible arguments passed to the callback script. If not specified, OpenSSL will accept all valid certificates and extensions. To reject the value and abort the connection, the callback should return 0. To accept the value and continue the connection, it should return 1. To reject the value, but continue the connection, it should return 2. This callback is new for TclTLS 1.8.

alpn *channelId protocol match*

For servers, this form of callback is invoked when the client ALPN extension is received. If *match* is true, then *protocol* is the first **-alpn** protocol option in common to both the client and server. If not, the first client specified protocol is used. This callback is called after the Hello and SNI callbacks.

hello *channelId servername session_id*

For servers, this form of callback is invoked during client hello message processing. The purpose is so the server can select the appropriate certificate to present to the client, and to make other configuration adjustments relevant to that server name and its configuration. It is called before the SNI and ALPN callbacks.

sni *channelId servername*

For servers, this form of callback is invoked when the Server Name Indication (SNI) extension is received. The *servername* argument is the client provided server name specified in the **-servername** option. The purpose is so when a server supports multiple names, the right certificate can be used. It is called after the Hello callback but before the ALPN callback.

verify *channelId depth cert status error*

This form of callback is invoked by OpenSSL when a new certificate is received from the peer.

It allows the client to check the certificate verification results and choose whether to continue or not. It is called for each certificate in the certificate chain. This callback was moved from **-command** in TclTLS 1.8. The arguments are:

depth
The depth is the integer depth of the certificate in the certificate chain, where 0 is the peer certificate and higher values going up to the Certificate Authority (CA).

cert
The cert argument is a list of key-value pairs similar to those returned by **tls::status**.

status
The status argument is the boolean validity of the current certificate where 0 is invalid and 1 is valid.

error
The error argument is the error message, if any, generated by **X509_STORE_CTX_get_error()**.

Reference implementations of these callbacks are provided in "*tls.tcl*" as **tls::callback**, **tls::password**, and **tls::validate_command** respectively. Note that these are only *sample* implementations. In a more realistic deployment you would specify your own callback scripts on each TLS channel using the **-command**, **-password**, and **-validate_command** options.

The default behavior when the **-command** and **-validate_command** options are not specified, is for TclTLS to process the associated library callbacks internally. The default behavior when the **-password** option is not specified is for TclTLS to process the associated library callbacks by attempting to call **tls::password**. The difference between these two behaviors is a consequence of maintaining compatibility with earlier implementations.

The use of the reference callbacks **tls::callback**, **tls::password**, and **tls::validate_command** is not recommended. They may be removed from future releases.

Debug

For most debugging needs, the **-callback** option can be used to provide sufficient insight and information on the TLS handshake and progress. If further troubleshooting insight is needed, the compile time option **--enable-debug** can be used to get detailed execution flow status.

TLS key logging can be enabled by setting the environment variable **SSLKEYLOGFILE** to the name of the file to log to. Then whenever TLS key material is generated or received it will be logged to the file. This is useful for logging key data for network logging tools to use to decrypt the data.

The **tls::debug** variable provides some additional control over the debug logging in the **tls::callback**, **tls::password**, and **tls::validate_command** default handlers in "*tls.tcl*". The default value is 0 with higher values producing more diagnostic output, and will also force the verify method in **tls::callback** to accept the certificate, even if it is invalid when the **-validatecommand** option is set to **tls::validate_command**.

The use of the variable **tls::debug** is not recommended. It may be removed from future releases.

Examples

The following are example scripts to download a webpage and file using the http package. See

Certificate Validation for when the **-cadir**, **-cafile**, and **-castore** options are also needed. See the "*demos*" directory for more example scripts.

Example #1: Download a web page

```
package require http
package require tls

set url "https://www.tcl.tk/"
http::register https 443 [list ::tls::socket -autoservername 1
    -require 1]

# Get URL
set token [http::geturl $url]

# Check for error
if {[http::status $token] ne "ok"} {
    puts [format "Error %s" [http::status $token]]
}

# Save web page to file
set ch [open example.html wb]
puts $ch [http::data $token]
close $ch

# Cleanup
::http::cleanup $token
```

Example #2: Download a file

```
package require http
package require tls

set url "https://wiki.tcl-lang.org/sitemap.xml"
http::register https 443 [list ::tls::socket -autoservername 1
    -require 1]

# Open output file
set filename [file tail $url]
set ch [open $filename wb]

# Get file
set token [::http::geturl $url -blocksize 65536 -channel $ch]

# Check for error
if {[http::status $token] ne "ok"} {
    puts [format "Error %s" [http::status $token]]
}
```

```
# Cleanup
close $ch
::http::cleanup $token
```

Special considerations

The capabilities of this package can vary enormously based upon how the linked to OpenSSL library was configured and built. New versions may obsolete older protocol versions, add or remove ciphers, change default values, etc. Use the **tls::protocols** command to obtain the supported protocol versions.

Error messages

Some OpsnSSl error messages have cryptic meanings. This is a list of messages along with their true meaning.

handshake failed: certificate verify failed due to "unable to get local issuer certificate"

The certificates in the CA file or certificate store either do not have one or more issuers of the certificates you are validating or they have expired. Usually this means you need an updated CAcert file.

packet length too long

Client has tried to connect to a HTTP server on the plain-text port instead of the SSL/TLS port.

unexpected eof while reading

The peer has closed the connection without sending the "close notify" shutdown alert. Some servers will terminate the connection after the file or webpage has been sent without sending the "close notify" message. In this case, it should not result in a loss of data.

wrong version number

Client has tried to connect to a non-HTTP server on a non-TLS (i.e. plain text) port.

Category

tls

Copyright


```
Copyright (c) 1999 Matt Newman
Copyright (c) 2004 Starfish Systems
Copyright (c) 2024 Brian O'Hagan
```


Keywords: tls, I/O, IP Address, OpenSSL, SSL, TCP, TclTLS, asynchronous I/O, bind, certificate, channel, connection, domain name, host, https, network, network address, socket

tcludp

udp

Name

udp - Create UDP sockets in Tcl

Synopsis

package require Tcl 8.5-
package require udp 1.0.14
udp_open ?*localport*? ?**reuse**? ?**ipv6**?
udp_conf *channel host port*
udp_conf *channel* ?*optionName*? ?*value*? ?*optionName value* ...?
udp_peek *channel* ?*buffersize*?
::udp::build-info
::udp::getaddrinfo -hostname *name* ?*optionName value* ...?
::udp::getnameinfo *address* ?**ipv6**?

Description

This package provides support for using UDP through Tcl. The package provides a new channel type and attempts to permit the use of packet oriented UDP over stream oriented Tcl channels. The package defined three commands but **udp_conf** should be considered depreciated in favor of the standard Tcl command **fconfigure**.

Commands

udp_open ?*localport*? ?reuse? ?ipv6?
: **udp_open** will open a UDP socket. If a *localport* is specified the UDP socket will be opened on that port. Otherwise the system will choose a port and the user can use the **udp_conf** command to obtain the port number if required. The following keywords can be used to specify options on the opened socket.

reuse
: Using this keyword sets the SO_REUSEADDR socket option which permits multiple sockets to be bound to the same address/port combination.

ipv6
: By default a IPv4 socket is created. When keyword **ipv6** is specified, an IPv6 socket is opened.

udp_conf *channel host port*
: *Deprecated* in favor of the standard Tcl **fconfigure** or **chan configure** commands. **udp_conf** in this configuration is used to specify the remote destination for packets written to this *channel*. You must call this command before writing data to the UDP socket.

udp_conf *channel?optionName??value??optionName value ...?*
: *Deprecated* in favor of the standard Tcl **fconfigure** or **chan configure** commands. In addition to being used to configure the remote host, the **udp_conf** command is used to obtain information about the UDP socket. NOTE all these options are now available using the standard Tcl **fconfigure** or **chan configure** command.

-myport
: Returns the local port number of the socket. Read-only option.

-remote*?address port?*
: Specifies or returns the remote hostname and port number. Can also be set using **udp_conf** *channel host port*.

-peer
: Returns the remote hostname and port number for the packet most recently received by this socket. Read-only option.

-family
Returns whether socket is configured for IPv4 or IPv6. Read-only option.

-broadcast*?boolean?*
Specifies or returns whether can listen and send on the broadcast address. For some systems a flag must be set on the socket to use broadcast. This option is not permitted when using IPv6, instead use multicast.

-ttl*?count?*
The time-to-live is given as the number of router hops the packet may do. For multicast packets this is important in specifying the distribution of the packet. The system default for multicast is 1 which restricts the packet to the local subnet. To permit packets to pass routers, you must increase the ttl. A value of 31 should keep it within a site, while 255 is global.

-mcastadd groupaddr
...see next...

-mcastadd "groupaddr netwif"
...see next...

-mcastdrop groupaddr
...see next...

-mcastdrop "groupaddr netwif"
...see next...

-mcastgroups
tcludp sockets can support IPv4 and IPv6 multicast operations. To receive multicast packets the application has to notify the operating system that it should join a particular multicast group. For IPv4 these are specified as addresses in the range 224.0.0.0 to 239.255.255.255. When specifying only the *groupaddr* the system will determine the network interface to use. Specifying the *netwif* will join a multicast group on a specific network interface. This is useful on a multihomed system with multiple network interfaces. On windows you must specify the network interface index. For other platforms the network interface (e.g. 'eth0') name can be specified. To view the current set of multicast groups for a channel use *-mcastgroups*

-mcastif
Returns which interface is used for outgoing multicast packets. UNIX only.

-mcastloop*?boolean?*
With multicast udp the system can choose to receive packets that it has sent or it can drop them. This is known as multicast loopback and can be controlled using this option. By default the value is true and your application will receive its own transmissions.

udp_peek *channel* ?*buffersize*?
Examine a packet without removing it from the buffer. Option *buffersize* specifies the maximum buffer size. Value must be between 0 and 16. This function is not available on windows.

::udp::build-info
Return information on the build environment.

::udp::getaddrinfo -hostname *name?optionName value ...?*
Returns a list with info on the IP address matching the specified parameters. Valid options are:

-hostname *name*
Specifies which IP address to use for lookup. Can use IP address or name. Required option.

-port *number*
Specifies which port number to use for lookup.

-service *name*
Specifies which well-known service (i.e. http, https, etc.) to use for port in lookup.

-ipv4
Only return IP v4 addresses.

-ipv6
Only return IP v6 addresses.

-server
Only return services for which hostname is a server.

-tcp
Only return TCP services.

-udp
Only return UDP services.

::udp::getnameinfo *address*?ipv6?
::udp::getnameinfo will return the name(s) corresponding to IP address *address*. With *ipv6*, address is an IPv6 address.

Examples

```
# Send data to a remote UDP socket
proc udp_puts {host port} {
    set s [udp_open]
    fconfigure $s -remote [list $host $port]
    puts $s "Hello, World"
    close $f
}
```

```
# A simple UDP server
package require udp

proc udpEventHandler {sock} {
    set pkt [read $sock]
    set peer [fconfigure $sock -peer]
    puts "$peer: [string length $pkt] {$pkt}"
    return
}

proc udp_listen {port} {
    set srv [udp_open $port]
    fconfigure $srv -buffering none -translation binary
    fileevent $srv readable [list ::udpEventHandler $srv]
    puts "Listening on udp port: [fconfigure $srv -myport]"
    return $srv
}
```

```
set sock [udp_listen 53530]
vwait forever
close $sock
```

```
# A multicast demo.
proc udpEvent {chan} {
    set data [read $chan]
    set peer [fconfigure $chan -peer]
    puts "$peer [string length $data] '$data'"
    if {[string match "QUIT*" $data]} {
        close $chan
        set ::forever 1
    }
    return
}

set group 224.5.1.21
set port  7771
set s [udp_open $port]
fconfigure $s -buffering none -blocking 0
fconfigure $s -mcastadd $group -remote [list $group $port]
fileevent $s readable [list udpEvent $s]
puts -nonewline $s "hello, world"
set ::forever 0
vwait ::forever
exit
```

History

Some of the code in this extension is copied from Michael Miller's tcludp package. (http://www.neosoft.com/tcl/ftparchive/sorted/comm/tcludp-1.0/) Compared with Michael's UDP extension, this extension provides Windows support and provides the ability of using 'gets/puts' to read/write the socket. In addition, it provides more configuration ability.

Enhancements to support binary data and to setup the package for the Tcl Extension Architecture by Pat Thoyts.

Support for IPv6 and allowing a multicast join on a specific network interface is added by Huub Eikens.

Copyright


```
Copyright (c) 1999-2000 Columbia University; all rights reserved
```


Keywords: udp, I/O, IP Address, TclUDP, asynchronous I/O, bind, channel, connection, domain, host, name, network, network address, networking, socket

tclvfs

vfs-filesystems

Name

vfs-filesystems - Filesystems provided by tclvfs

Synopsis

package require Tcl 8.4
package require vfs ?**1.2.1**?
package require vfs::zip ?**1.0**?
package require vfs::mk4 ?**1.6**?
package require vfs::tar ?**0.9**?
package require vfs::ftp ?**1.0**?
package require vfs::ns ?**1.0**?
package require vfs::webdav ?**0.1**?
package require vfs::http ?**0.5**?
package require vfs::urltype ?**1.0**?
vfs::zip::Mount *path to*
vfs::mk4::Mount *path to*
vfs::tar::Mount *path to*
vfs::ftp::Mount *path to*
vfs::ns::Mount *path to*
vfs::webdav::Mount *path to*
vfs::http::Mount *path to*
vfs::urltype::Mount *path to*

Description

The package **vfs** provides not only the means to implement a virtual filesystem at the tcl level, but also a number of ready to be used filesystems as well.

Each of these filesystem exists as its own package, and can be accessed through **package require vfs::NAME**. The whole set of these virtual filesystems is known informally as \'*vfslib*'.

Supported vfs types

Currently supported are ftp, tar, http, zip, mk4, ns, and webdav. In addition there is the ability to mount any 'urltype' as a new directory, provided an appropriate vfs is supported. This means that you can treat urls based on the schemes *ftp://*, *http://* and *file://* as files. To do this, simply evaluate the command **vfs::urltype::Mount ftp** for example. Any access inside the new volume will result in an attempt to require a package through **package require vfs::${type}**, which must therefore exist, or errors will be thrown. If a filesystem is loaded, use the associated command listed below to mount a source for that filesystem as a tcl directory.

vfs::zip::Mount *path to*
Mount the zip file *path* as directory *to*.

vfs::mk4::Mount *path to*
Mount the metakit database file file *path* as directory *to*.

vfs::tar::Mount *path to*
Mount the tar file *path* as directory *to*.

vfs::ftp::Mount *path to*

Mount the ftp url *path* as directory *to*.

vfs::ns::Mount *path to*
Mount the tcl namespace *path* as directory *to*.

vfs::webdav::Mount *path to*
Mount the webdav url *path* as directory *to*.

vfs::http::Mount *path to*
Mount the http url *path* as directory *to*.

vfs::urltype::Mount *path to*
Mount the url *path*, of type *urltype* as directory *to*.

Limitations
Most of the vfs types listed above have not been very well debugged as yet. Please test them!

Copyright

```
Copyright (c) 2001-2003 Vince Darley
    <vincentdarley@users.sourceforge.net>
Copyright (c) 2003 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```


Keywords: vfs-filesystems, file, filesystem, ftp, http, metakit, namespace, tar, vfs, vfslib, webdav, zip

vfs-fsapi

Name
vfs-fsapi - API for the implementation of a filesystem in Tcl

Synopsis
package require Tcl 8.4
package require vfs ?**1.2.1**?
vfshandler *subcmd root relative actualpath args...*
vfshandler access *root relative actualpath mode*
vfshandler createdirectory *root relative actualpath*
vfshandler deletefile *root relative actualpath*
vfshandler fileattributes *root relative actualpath* ?*index*? ?*value*?
vfshandler matchindirectory *root relative actualpath pattern types*
vfshandler open *root relative actualpath mode permissions*
vfshandler removedirectory *root relative actualpath recursive*
vfshandler stat *root relative actualpath*
vfshandler utime *root relative actualpath actime mtime*
vfs::accessMode *mode*
vfs::matchDirectories *types*
vfs::matchFiles *types*
vfs::matchCorrectTypes *types filelist* ?*inDir*?

Description
This document explains the API used by the package **vfs** to communicate with filesystem

implementations written in tcl.

Handler overview

The package **vfs** intercepts every filesystem operation which falls within a given mount point, and passes the operation on to the mount point's **vfshandler** command in the interpreter which registered it.

If the handler takes appropriate action for each of the cases it is called for, a complete, perfect virtual filesystem will be achieved, indistinguishable to Tcl from the native filesystem. (CAVEATS: Right now **vfs** does not expose to Tcl all the permission-related flags of **glob**).

vfshandler *subcmd root relative actualpath args...*

The first argument specifies the operation to perform on behalf of the filesystem code in the tcl core, the remainder specify the file path on which to operate, in different forms, and parts, and any additional arguments which may be required to carry out the action. To demonstrate the treatment of a path by the generic layer we use "*C:/foo/bar/mount.zip/xxx/yyy*" as an example and additionally assume that the following conditions are true:

[1]

"*mount.zip*" is a zip archive which has been mounted on top of itself,

[2]

said zip archive contains a file with path "*xxx/yyy*",

[3]

the current working directory of the application is inside of directory "*xxx*",

[4]

and the command executed is **file exists yyy**.

The file separator between *root* and *relative* is omitted. Most filesystem operations need only the *relative* argument for their correct operation, but some actually require the other parts of the path.

subcmd

This argument of the handler can be one of the following **access**, **createdirectory**, **deletefile**, **fileattributes**, **matchindirectory**, **open**, **removedirectory**, **stat**, or **utime**. The generic layer expects that the subcommands of a handler signal error conditions by calling **vfs::filesystem posixerror** with the appropriate posix error code instead of throwing a tcl error. If the latter is done nevertheless it will be treated as an unknown posix error. There are three exceptions to the rule above: If any of **open** (when an interpreter is given), **matchindirectory**, and **fileattributes** (for a set or get operation only) throw a tcl error, this error will be passed up to the caller of the filesystem command which invoked the handler. Note that this does not preclude the ability of these subcommands to use the command **vfs::filesystem posixerror** to report more regular filesystem errors.

root

Part of the specification of the path to operate upon. It contains the part of the path which lies outside this filesystem's mount point. For example outlined above its value will be "*C:/foo/bar/mount.zip*".

relative

Part of the specification of the path to operate upon. It contains the part of the path which lies inside this filesystem. For example outlined above its value will be "*xxx/yyy*".

actualpath
Part of the specification of the path to operate upon. It contains the original (unnormalized) name of the path which was used in the current command wherever it originated (in Tcl or C). For example outlined above its value will be "*yyy*".

Handler methods

vfshandler access *root relative actualpath mode*
Signal a posix error if the specified access *mode* (an integer number) is not compatible with the file or directory described by the path. The generic layer will ignore any non-empty return value. The command **vfs::accessMode** (see section **HANDLER ENVIRONMENT**) can be used to convert the integer *mode* into an easier to check string value.

vfshandler createdirectory *root relative actualpath*
Create a directory with the given name. The command can assume that all sub-directories in the path exist and are valid, and that the actual desired path does not yet exist (Tcl takes care of all of that for us).

vfshandler deletefile *root relative actualpath*
Delete the given file.

vfshandler fileattributes *root relative actualpath* ?*index*? ?*value*?
The command has to return a list containing the names of all acceptable attributes, if neither *index* nor *value* were specified. The command has to return the value of the *index*'th attribute if the *index* is specified, but not the *value*. The attributes are counted in the same order as their names appear in the list returned by a call where neither *index* nor *value* were specified. The first attribute is has the index **0**. The command has to set the value of the *index*'th attribute to *value* if both *index* and *value* were specified for the call.

vfshandler matchindirectory *root relative actualpath pattern types*
Return the list of files or directories in the given path which match the glob *pattern* and are compatible with the specified list of *types*. The specified path is always the name of an existing directory. *Note:* As Tcl generates requests for directory-only matches from the filesystems involved when performing any type of recursive globbing this subcommand absolutely has to handle such (and file-only) requests correctly or bad things (TM) will happen. The commands **vfs::matchDirectories** and **vfs::matchFiles** (see section **HANDLER ENVIRONMENT**) can aid the implementation greatly in this task.

vfshandler open *root relative actualpath mode permissions*
Either returns a list describing the successfully opened file, or throws an error describing how the operation failed. The list returned upon success contains at least one and at most two elements. The first, obligatory, element is always the handle of the channel which was created to allow access to the contents of the file. If specified the second element will be interpreted as a callback, i.e. a command prefix. This prefix will always be executed as is, i.e. without additional arguments. Any required arguments have to be returned as part of the result of the call to **open**. If present the specified callback will be evaluated just before the channel is closed *by the generic filesystem layer*. The callback itself *must not* call **close**. The channel however is live enough to allow **seek** and **read** operations. In addition all available data will have been flushed into it already. This means, for example, that the callback can seek to the beginning of the said channel, read its contents and then store the gathered data elsewhere. In other words, this callback is not only crucial to the cleanup of any resources associated with an opened file, but also for the ability to implement a filesystem which can be

written to. Under normal circumstances return code and any errors thrown by the callback itself are ignored. In that case errors have to be signaled asychronously, for example by calling **bgerror**. However if, through a call of the subcommand **internalerror**, an error handling script has been specified for the file system, all errors thrown here will be passed to that script for further action.

mode
can be any of **r**, **w**, **a**, **w+**, or **a+**.

permissions
determines the native mode the openend file is created with. Relevant only of the open *mode* actually requests the creation of a non-existing file, i.e. is not **r**.

vfshandler removedirectory *root relative actualpath recursive*
Delete the given directory. Argument *recursive* is a boolean. If the specified value is **true** then even if the directory is non-empty, an attempt has to be made to recursively delete it and its contents. If the spcified value is **false** and the directory is non-empty, a posix error (**EEXIST**) has to be thrown.

vfshandler stat *root relative actualpath*
The result has to be a list of keys and values, in a format acceptable to the builtin command **array set**. It describes the contents of a stat structure. The order of the keys in the list is not important. Given this the subcommand should use something like

```
return [list dev 0 type file mtime 1234 ...].
```

as the last command of its implementation. The following keys and their values have to be supplied by the filesystem:

dev
A long integer number, the device number of the path stat was called for.

ino
A long integer number, the inode number of the path stat was called for. Each path handled by the filesystem should be uniquely identified by the combination of device and inode number. Violating this principle will cause higher-level algorithms which(have to) keep track of device and inode information to fail in all manners possible. An example of such an algorithm would be a directory walker using device/inode information to keep itself out of infinite loops generated through symbolic links. Returning non-unique device/inode information will most likely cause such a walker to skip over paths under the wrong assumption of having them seen already.

mode
An integer number, the access mode of the path. It is this mode which is checked by the subcommand **access**.

nlink
A long integer number, the number of hard links to the specified path.

uid
A long integer number, the id of the user owning the virtual path.

gid
A long integer number, the id of the user group the virtual path belongs to.

size

A long integer number, the true size of the virtual path, in bytes.

atime
A long integer number, the time of the latest access to the path, in seconds since the epoch. Convertible into a readable date/time by the command **clock format**.

mtime
A long integer number, the time of the latest modification of the path, in seconds since the epoch. Convertible into a readable date/time by the command **clock format**.

ctime
A long integer number, the time of the path was created, in seconds since the epoch. Convertible into a readable date/time by the command **clock format**.

type
A string, either **directory**, or **file**, describing the type of the given path.

vfshandler utime *root relative actualpath actime mtime*
Set the access and modification times of the given file (these are read with **stat**).

Handler environment

The implementation of a filesystem handler can rely on the existence of the following utility commands:

vfs::accessMode *mode*
This commands converts an access *mode* given as integer into a string, one of **F**, **X**, **W**, **XW**, **R**, **RX**, and **RW**.

vfs::matchDirectories *types*
Checks if the glob types specification ask for the inclusion of directories. Returns a boolean result. **true** is returned if types does ask for directories, else **false**.

vfs::matchFiles *types*
Checks if the glob types specification ask for the inclusion of files. Returns a boolean result. **true** is returned if types does ask for directories, else **false**.

vfs::matchCorrectTypes *types filelist* ?*inDir*?
Returns that subset of the *filelist* which are compatible with the *types* given. The elements of *filelist* are either absolute paths, or names of files in the directory *indir*. The latter interpretation is taken if and only if the argument *indir* is specified.

Filesystem debugging

To debug a problem in the implementation of a filesystem use code as shown below. This registers the command **report** as the error handler for the filesystem, which in turn prints out the error stack provided by tcl.

```
vfs::filesystem internalerror report

proc report {} {
    puts stderr $::errorInfo
}
```

Copyright

```
Copyright (c) 2001-2003 Vince Darley
    <vincentdarley@users.sourceforge.net>
Copyright (c) 2003 Andreas Kupries
    <andreas_kupries@users.sourceforge.net>
```

Keywords: vfs-fsapi, file, filesystem, vfs

vfs

Name

::vfs - Commands and Procedures to create virtual filesystems

Synopsis

package require Tcl

package require vfs ?**1.4**?

vfs::filesystem *info*

vfs::filesystem *mount*

vfs::filesystem *unmount*

vfs::accessMode *mode*

vfs::matchDirectories *types*

vfs::matchFiles *types*

vfs::matchCorrectTypes *types filelist* ?*inDir*?

Description

The **::vfs** package provides commands to query, mount and unmount virtual filesystems, and provides as Tcl libraries some facilities for helping the writing of new virtual filesystems in Tcl. Once a virtual filesystem is in place, the standard Tcl **file**, **glob**, **cd**, **pwd**, **open** commands, including all their C APIs in the Tcl library (e.g. **Tcl_FSOpenFileChannel**, **Tcl_FSMatchInDirectory**,...), can be used within the filesystem (and indeed, properly written extensions such as Tk which may open or read files will also transparently access the virtual filesystem). Because all of Tcl's FS activity passes through a single layer, it can all be intercepted. This package does just that. Notice that this is quite different to overloading the **file** command in Tcl. We are actually providing vfs replacements for C commands like **access**, **stat**. By implementing just a handful of commands at this low level, we ensure that all commands at higher levels function irrespective of what is going on inside the FS layer.

Tcl's filesystem hooks operate on a per-process basis. This means every Tcl interpreter in the same process/application sees the same filesystem, including any virtual filesystems.

The **package require vfs** command should be used to access this library. It automatically registers the vfs hooks into Tcl's filesystem, and these will not be removed until Tcl exits (if desired, control over this could be exposed to Tcl in the future). However, the vfs package will at that stage not have any new filesystems mounted, so it will have little effect. Note that **package require vfs** has two effects. First of all, when it is issued in **any** Tcl interpreter it will ensure the vfs hooks have been registered with Tcl's core just once (and if any of those interpreters are later deleted, the vfs hooks will still remain registered - they remain until Tcl

exits). The second effect is to provide the command **vfs::filesystem** which allows the interpreter to intercept filesystem commands and handle them with Tcl code in that interpreter.

There are three somewhat unsupported subcommands of **vfs::filesystem**, **fullynormalize path**, **posixerror int**, **internalerror ?script?**, which are used to normalize a path (including any final symlink), to register a posix error code with a Tcl error, and to trap/report internal errors in tclvfs implementations respectively.

vfs::filesystem *mount?-volume? path command*
: To use a virtual filesystem, it must be 'mounted'. Mounting involves declaring to the vfs package that any subdirectories of a given *path* in the filesystem should be handled by the given *command* which should be a Tcl command or procedure in the interpreter in which the **vfs::filesystem** is executed. If the*?-volume?* flag is given, the given mount point is also registered with Tcl as a new volume (like a new drive which will appear in *file volumes*). This is useful (and required for reasonable operation) for mounts like *ftp://*. For paths mounted inside the native filesystem, it should of course not be given. The new filesystem mounts will be observed immediately in all interpreters in the current process. If the interpreter is later deleted, all mounts which are intercepted by it will be automatically removed (and will therefore affect the view of the filesystem seen by all interpreters).

vfs::filesystem *unmount path*
: This unmounts the virtual filesystem which was mounted at *path* (hence removing it from Tcl's filesystem), or throws an error if no filesystem was mounted there.

vfs::filesystem *info?path?*
: If no arguments are given, this returns a list of all filesystems mounted (in all interpreters). If a path argument is given, then the *command* to be used for that path is returned, or an error is thrown if no vfs is mounted for that path. There is currently no facility for examining in which interpreter each command will be evaluated.

vfs::filesystem *fullynormalize path*
: Performs a full expansion of *path*, (as per 'file normalize'), but including following any links in the last element of path.

Implementing a tcl only vfs

The vfs package will intercept every filesystem operation which falls within a given mount point, and pass the operation on to the mount point's *command* in the interpreter which registered it. In general this occurs by the C equivalent of an evaluation like this: *eval $command [list $subcmd $root $relative $actualpath] $args*.

Here *subcmd* may be any of the following: *access*, *createdirectory*, *deletefile*, *fileattributes*, *matchindirectory*, *open*, *removedirectory*, *stat*, *utime*. If *command* takes appropriate action for each of these cases, a complete, perfect virtual filesystem will be achieved, indistinguishable to Tcl from the native filesystem. (CAVEATS: right now I don't expose to Tcl all the permission-related flags of 'glob').

The remaining arguments specify a file path on which to operate (all commands operate on one of these), and any additional arguments which may be required to carry out the action. The file path is specified by three arguments: *root* is the part of the path which lies outside this filesystem's mount point, *relative* is the part of the path which lies inside this filesytem, and *actualpath* is the original (unnormalized) name of the path which was used in the current command wherever it originated (in Tcl or C). For example, if *C:/foo/bar/mount.zip/xxx/yyy* is a

path in your filesystem, where *mount.zip* is a zip archive which has been mounted (on top of itself) and contains *xxx/yyy*, and the current working directory is inside *xxx*, and we evaluate a command like *file exists yyy*, then *root\R will be C:/foo/bar/mount.zip*, *relative* will be *xxx/yyy*, and *actualpath* will be *yyy*. The file separator between the *root* and *relative* is omitted.

Note that most filesystem operations will only require the *relative* argument to work correctly, but the other arguments are actually required for correct operation of some subcommands.

Almost all of these commands should either return correctly (i.e. with a TCL_OK result at the C level) or they should use vfs::filesystem posixerror to signal the appropriate posix error code. If a Tcl error is thrown, that should be considered a bug, but it will be interpreted as an unknown posix error in the filesystem call. The exceptions to these rules are those filesystem commands which are able to specify a Tcl error message directly: open (when an interpreter is given), matchindirectory and fileattributes (for a set or get operation only). These three commands are allowed to throw any Tcl error message which will be passed along to the caller, or they may throw a posix error which will be handled appropriately.

The actual commands are as follows (where *r-r-a* represents the standard argument triplet of *root*, *relative* and *actualpath*):

command access r-r-a mode
: Return TCL_OK or throw a posix error depending on whether the given access mode (which is an integer) is compatible with the file.

command createdirectory r-r-a
: Create a directory with the given name. The command can assume that all sub-directories in the path exist and are valid, and that the actual desired path does not yet exist (Tcl takes care of all of that for us).

command deletefile r-r-a
: Delete the given file.

command fileattributes r-r-a?index??value?
: If neither index nor value is given, then return a list of all acceptable attribute names. If *index* is given, but no value, then retrieve the value of the *index*'th attribute (counting in order over the list returned when no argument is given) for the given file. If a value is also given then set the *index*'th attribute of the given file to that value.

command matchindirectory r-r-a pattern types
: Return the list of files or directories in the given path (which is always the name of an existing directory), which match the *pattern* and are compatible with the *types* given. It is very important that the command correctly handle *types* requests for directories only (and files only), because to handle any kind of recursive globbing, Tcl will actually generate requests for directory-only matches from the filesystem. See **vfs::matchDirectories** below for help.

command open r-r-a mode permissions
: For this command, *mode* is any of "r", "w", "a", "w+", "a+". If the open involves creating a file, then *permissions* dictates what modes to create it with. If the open operation was not successful, an error should be thrown. If the open operation is successful, the command should return a list of either one or two items. The first item (which is obligatory) is the name of the channel which has been created. The second item, if given, is a Tcl-callback to be used when the channel is closed, so that the vfs can clean up as appropriate. This callback will be evaluated by Tcl just before the channel is closed. The channel will still exist, and all available

data will have been flushed into it. The callback can, for example, seek to the beginning of the channel, read its contents and store that contents elsewhere (e.g. compressed or on a remote ftp site, etc). The return code or any errors returned by the callback are ignored (if the callback wishes to signal an error, it must do so asycnhronously, with bgerror, for example), unless the 'internalerror' script has been specified, when they are passed to that script for further action.

command removedirectory r-r-a recursive
Delete the given directory. *recursive* is either 0 or 1. If it is 1 then even if the directory is non-empty, an attempt should be made to recursively delete it and its contents. If it is 0 and the directory is non-empty, a posix error (EEXIST) should be thrown.

command stat r-r-a
Return a list of even length containing field-name and value pairs for the contents of a stat structure. The order is not important. The option names are dev (long), ino (long), mode (int), nlink (long), uid (long), gid (long), size (long), atime (long), mtime (long), ctime (long), type (string which is either "directory" or "file"), where the type of each argument is given in brackets. The procedure should therefore return with something like *return [list dev 0 type file mtime 1234 ...]*.

command utime r-r-a actime mtime
Set the access and modification times of the given file (these are read with 'stat').

Vfs helpers

The vfslib provides a number of Tcl procedures which can help with writing command procedures to handle the above possibilities. These are:

vfs::accessMode *mode*
converts an integer *access* mode to a somewhat more preferable string, any of F X W XW R RX RW.

vfs::matchDirectories *types*
Does *types* want directories included?

vfs::matchFiles *types*
Does *types* want files included?

vfs::matchCorrectTypes *types filelist?inDir?*
Returns that subset of the *filelist* (which are either absolute paths or names of files in *inDir*) which are compatible with the *types* given.

Vfs debugging

Use something like this to debug problems in your implementation: vfs::filesystem internalerror report; proc report {} { puts stderr $::errorInfo }

Limitations

There are very few limitations to the vfs code. One subtlety that you may encounter is if you mount a case-sensitive virtual filesystem into a case-insensitive system (e.g. the standard Windows or MacOS fs) and your code relies on case-insensitivity, then it will not run properly in the virtual filesystem. Of course if your code relies on case-insensitivity, it wouldn't run under Tcl on Unix either, so the best solution is to fix your code!

We may add *link* and *lstat* commands in the future to allow virtual filesystems to support reading

and writing links - this is supported by the C API, but has simply not been exposed to Tcl in this extension, yet.

The Tcl 'Tcl_FSMatchInDirectory' function takes a variety of type information in a Tcl_GlobTypeData structure. We currently only expose the 'type' field from that structure (so the 'permissions' and MacOS type/creator fields are ignored).

Keywords: vfs, filesystem, file

Copyright (c) 2001-2003, Vince Darley

vfslib

Name

::vfslib - Procedures to interact with virtual filesystems

Synopsis

package require Tcl

package require vfs ?**1.2.1**?

package require vfs::zip ?**1.0**?

package require vfs::mk4 ?**1.6**?

package require vfs::tar ?**0.9**?

package require vfs::ftp ?**1.0**?

package require vfs::ns ?**1.0**?

package require vfs::webdav ?**0.1**?

package require vfs::http ?**0.5**?

package require vfs::urltype ?**1.0**?

vfs::zip::Mount *path to*

vfs::ftp::Mount *path to*

vfs::tar::Mount *path to*

vfs::http::Mount *path to*

vfs::mk4::Mount *path to*

vfs::ns::Mount *path to*

vfs::urltype::Mount *type*

Description

The **::vfs** package includes a library of Tcl code, implementing a number of different virtual filesystems. Each of these exists as its own package, and can be accessed through **package require vfs::NAME**. The whole set of virtual filesystems is known informally as 'vfslib'

Supported vfs types

The current supported types are ftp, tar, http, zip, mk4, ns, webdav. In addition there is the ability to mount any 'urltype' as a new volume, provided an appropriate vfs is supported. This means that you can treat 'ftp://', 'http://' and 'file://' urls as files. To do this, simply evaluate the command

vfs::urltype::Mount ftp

for instance. Any access inside the new volume will result in an attempt to require a package through 'package require vfs::${type}', which must therefore exist, or errors will be thrown.

Limitations

Most of the vfs types listed above have not been very well debugged as yet. Please test them!

Keywords: vfslib, vfs, filesystem, zip, tar, webdav, namespace, ftp, http, file

Copyright (c) 2001, Vince Darley

tclws

Calling_a_Web_Service

Calling a Web Service from Tcl

Contents

- 1 Overview
- 2 Loading the Webservices Client Package
- 3 Quering a remote Web Services Server for its WSDL and parsing it
- 4 Parsing a saved WSDL
- 5 Loading a pre-parsed WSDL
- 6 Defining a REST based service by hand
- 7 Defining Transforms
- 8 Synchronous Call returning a dictionary object
- 9 Asynchronous Call with separate success and error callbacks
- 10 Creation of stub Tcl procedures to make synchronous calls
- 11 Synchronous Call returning the raw XML
- 12 Configuring Services
- 13 Dealing With Casting Abstract Types to Concrete Types

Overview

The Webservices Client package provides a several ways to define what operations a remote Web Services Server provides and how to call those operations. It also includes several ways to call an operation.

The following ways are provided to define remote operations:

- Loading a pre-parsed WSDL
- Quering a remote Web Services Server for its WSDL and parsing it
- Parsing a saved WSDL
- Defining a REST based service by hand

The parsed format is much more compact than the XML of a WSDL.

The following ways are provided to directly call an operation of a Web Service Server:

- Synchronous Call returning a dictionary object
- Synchronous Call returning the raw XML
- Asynchronous Call with separate success and error callbacks

- Creation of stub Tcl procedures to make synchronous calls

This package makes use of the **log** package from **TclLib**. In particular the following levels are used:

- error/warning -- errors encountered when parsing a WSDL. Actual level depends on options that are set in the **::WS::Utils** package.
- info -- HTTP calls, including the XML, made to invoke operations and the replies received. *Introduced in 2.2.8.*
- debug -- detailed internal information. This should only be used if you want to code dive into the TclWs package internals.

Loading the Webservices Client Package

To load the webservices server package, do:

```
package require WS::Client
```

This command will only load the utilities the first time it is used, so it causes no ill effects to put this in each file using the utilties.

Quering a remote Web Services Server for its WSDL and parsing it

Procedure Name : ::WS::Client::GetAndParseWsdl

Description : Fetch the WSDL file from the given URL, aprse it and create the service.

Arguments :

```
    url     - The url of the WSDL
    headers - Extra headers to add to the HTTP request. This
              is a key value list argument. It must be a list with
              an even number of elements that alternate between
              keys and values. The keys become header field names.
              Newlines are stripped from the values so the header
              cannot be corrupted.
              This is an optional argument and defaults to {}.
    serviceAlias - Alias (unique) name for service.
                   This is an optional argument and defaults to the
   name of the
                   service in serviceInfo.
    serviceNumber - Number of service within the WSDL to assign the
                    serviceAlias to. Only recognized with a
   serviceAlias.
                    First service (default) is addressed by value "1".
```

The following example WSDL snipped defines two services:

```
 >definitions ...<
   >service name="service1"<
     ...
   >/service<
   >service name="service1"<
```

```
    ...
  >/service<
>/definitions<
```

Using an empty or no *serviceAlias* would result in the creation of the services "service1" and "service2".

Using *serviceAlias*="SE" and *serviceNumber*=2 would result in the creation of the service "SE" containing the "service2" of the WSDL.

Returns : The parsed service definition

Side-Effects : None

Exception Conditions : None

Pre-requisite Conditions : None

Parsing a saved WSDL

Procedure Name : ::WS::Client::ParseWsdl

Description : Parse a WSDL and create the service. Create also the stubs if specified.

Arguments :

```
 wsdlXML - XML of the WSDL
```

Optional Arguments:

```
 -createStubs 0|1 - create stub routines for the service
 -headers         - Extra headers to add to the HTTP request. This
                    is a key value list argument. It must be a
list with
                    an even number of elements that alternate
between
                    keys and values. The keys become header field
names.
                    Newlines are stripped from the values so the
header
                    cannot be corrupted.
                    This is an optional argument and defaults to
{}.
 -serviceAlias    - Alias (unique) name for service.
                    This is an optional argument and defaults to
the name of the
                    service in serviceInfo.
 serviceNumber - Number of service within the WSDL to assign the
                 serviceAlias to. Only recognized with a
serviceAlias.
                 First service (default) is addressed by value "1".
```

The arguments are position independent.

For an example use of *serviceAlias* and *serviceNumber*, see the chapter above.

Returns : The parsed service definition

Side-Effects : None

Exception Conditions :None

Pre-requisite Conditions : None

Loading a pre-parsed WSDL

Procedure Name : ::WS::Client::LoadParsedWsdl

Description : Load a saved service definition in

Arguments :

```
     serviceInfo - parsed service definition, as returned from
                   ::WS::Client::ParseWsdl or
    ::WS::Client::GetAndParseWsdl
     headers     - Extra headers to add to the HTTP request. This
                     is a key value list argument. It must be a list
    with
                     an even number of elements that alternate between
                     keys and values. The keys become header field
    names.
                     Newlines are stripped from the values so the
    header
                     cannot be corrupted.
                     This is an optional argument and defaults to {}.
     serviceAlias - Alias (unique) name for service.
                     This is an optional argument and defaults to the
    name of the
                     service in serviceInfo.
```

Returns : The name of the service loaded

Side-Effects : None

Exception Conditions : None

Pre-requisite Conditions : None

Defining a REST based service by hand

Service Definition

Procedure Name : ::WS::Client::CreateService

Description : Define a REST service

Arguments :

```
     serviceName - Service name to add namespace to
     type        - The type of service, currently only REST is
    supported.
     url          - URL of namespace.
     args         - Optional arguments:
                             This is an optional argument and defaults
    to the name of the
                                  -header httpHeaderList.
```

Returns : The local alias (tns)

Side-Effects : None

Exception Conditions : None

Pre-requisite Conditions : None

Method Definition
Procedure Name : ::WS::Client::DefineRestMethod

Description : Define a method on a REST service

Arguments :

```
     serviceName - Service name to add namespace to
     methodName  - The name of the method to add.
     returnType  - The type, if any returned by the procedure.  Format
    is:
                           xmlTag typeInfo.
     inputArgs   - List of input argument definitions where each
    argument
                           definition is of the format: name typeInfo.

    where, typeInfo is of the format {type typeName comment
    commentString}
```

Returns : The current service definition

Side-Effects : None

Exception Conditions : None

Pre-requisite Conditions : None

Defining Transforms

Procedure Name : ::WS::Client::SetServiceTransforms

Description : Define a service's transforms

```
            Transform signature is:
                cmd serviceName operationName transformType xml
{url {}} {argList {}}
            where transformType is REQUEST or REPLY
            and url and argList will only be present for
transformType == REQUEST
```

Arguments :

```
 serviceName  - The name of the Webservice
 inTransform  - Input transform cmd, defaults to {}.
 The inTransform is the proc which allows to transform the SOAP
output message, which will be input in the server.
 outTransform - Output transformcmd, defaults to {}.
 The outTransform is the proc which allows to transform the SOAP
input message, which is the answer of the server.
```

Returns : None

Side-Effects : None

Exception Conditions : None

Pre-requisite Conditions : Service must have been defined.

Synchronous Call returning a dictionary object

Procedure Name : ::WS::Client::DoCall

Description : Call an operation of a web service

Arguments :

```
 serviceName     - The name of the Webservice
 operationName   - The name of the Operation to call
 argList         - The arguements to the operation as a dictionary
object
                   This is for both the Soap Header and Body
messages.
 headers         - Extra headers to add to the HTTP request. This
                   is a key value list argument. It must be a list
with
                   an even number of elements that alternate
between
                   keys and values. The keys become header field
names.
                   Newlines are stripped from the values so the
header
                   cannot be corrupted.
                   This is an optional argument and defaults to {}.
```

Returns :

```
    The return value of the operation as a dictionary object.
      This includes both the return result and any return headers.
```

Side-Effects : None

Exception Conditions :

```
    WSCLIENT HTTPERROR      - if an HTTP error occured
    others                  - as raised by called Operation
```

Pre-requisite Conditions : Service must have been defined.

Asynchronous Call with separate success and error callbacks

Procedure Name : ::WS::Client::DoAsyncCall

Description : Call an operation of a web service asynchronously

Arguments :

```
 serviceName     - The name of the Webservice
 operationName   - The name of the Operation to call
 argList         - The arguements to the operation as a dictionary
object
                   This is for both the Soap Header and Body
messages.
 succesCmd       - A command prefix to be called if the operations
                   does not raise an error.  The results, as a
dictionary
                   object are concatinated to the prefix. This
includes
                   both the return result and any return headers.
Leave
                   empty to not call any function.

 errorCmd        - A command prefix to be called if the operations
                   raises an error.  The error code, stack trace
and
                   error message are concatinated to the prefix.
Leave
                   empty to not call any function.
 headers         - Extra headers to add to the HTTP request. This
                   is a key value list argument. It must be a list
with
                   an even number of elements that alternate
between
                   keys and values. The keys become header field
names.
                   Newlines are stripped from the values so the
header
                   cannot be corrupted.
                   This is an optional argument and defaults to {}.
```

Returns : Nothing.

Side-Effects : None

Exception Conditions :

```
 WSCLIENT HTTPERROR      - if an HTTP error occured
 others                  - as raised by called Operation
```

Pre-requisite Conditions : Service must have been defined.

Creation of stub Tcl procedures to make synchronous calls

Procedure Name : ::WS::Client::CreateStubs

Description : Create stubs routines to make calls to Webservice Operations.

```
          All routines will be create in a namespace that is the
    same
          as the service name.  The procedure name will be the same
          as the operation name.
```

```
          NOTE -- Webservice arguments are position independent,
    thus
                  the proc arguments will be defined in
    alphabetical order.
```

Arguments :

```
     serviceName     - The service to create stubs for
```

Returns : A string describing the created procedures.

Side-Effects : Existing namespace is deleted.

Exception Conditions : None

Pre-requisite Conditions : Service must have been defined.

Synchronous Call returning the raw XML

Procedure Name : ::WS::Client::DoRawCall

Description : Call an operation of a web service

Arguments :

```
     serviceName     - The name of the Webservice
     operationName   - The name of the Operation to call
     argList         - The arguements to the operation as a dictionary
    object
                       This is for both the Soap Header and Body
    messages.
     headers         - Extra headers to add to the HTTP request. This
                       is a key value list argument. It must be a list
    with
                       an even number of elements that alternate
    between
                       keys and values. The keys become header field
    names.
                       Newlines are stripped from the values so the
    header
                       cannot be corrupted.
                       This is an optional argument and defaults to {}.
```

Returns :

```
     The XML of the operation.
```

Side-Effects : None

Exception Conditions :

```
     WSCLIENT HTTPERROR      - if an HTTP error occured
```

Pre-requisite Conditions : Service must have been defined.

Generating a Template Dictionary

Procedure Name : ::WS::Utils::GenerateTemplateDict

Description : Generate a template dictionary object for a given type.

Arguments :

```
     mode            - The mode, Client or Server
     serviceName     - The name of the Webservice
     type            - The name of the type
     arraySize       - Number of elements to generate in an array.
    Default is 2
```

Returns :

```
A dictionary object for a given type.  If any circular references
    exist, they will have the value of <** Circular Reference **>
Side-Effects : None
Exception Conditions  : None
Pre-requisite Conditions : Service must have been defined.
```

Configuring a Service

There are two procedures to configure a service:

- ::WS::Client::SetOption
- ::WS::Client::Config

The first procedure contains the default options of the package. The default options are used on service creation and are then copied to the service.

The second procedure contains the options of each service. They are copied on service creation from the default options.

Most option items may be accessed by both functions. Some options are only used on service creation phase, which do not exist as service option. Other options do not exist as default option, as they are initialized from the WSDL file.

In the following, first the two access routines are described. Then, a list of options for both functions are given, with remarks, if they are only valid for one of the two procedures.

Procedure Name : ::WS::Client::SetOption

Description : Get or set the default options of the package

Arguments :

```
     -globalonly   - Return a list of global-only options and their
    values.
                    Global-only options are not copied to the service.
     -defaultonly  - Return a list of default-only options and their
    values.
                    default-only options are copied to the service.
     --            - End of options
     item          - The option item to get or configure.
                    Return a list of all item/value pairs if
    ommitted.
     value         - The value to set the option item to.
                    Return current value if omitted.
```

Procedure Name : ::WS::Client::Config

Description : Get or set the options local to a service definition

Arguments :

```
     serviceName - The name of the Webservice.
                   Return a list of default items/values paires if not
    given.
     item        - The option item to get or configure.
                   Return a list of all item/value pairs, if not
    given.
     value       - The value to set the option item to.
                   Return current value if omitted.
```

Option List:

-

`allowOperOverloading`

An overloaded operation is an operation with the same name but different may exist with different input parameter sets.

This option throws an error, if a WSDL is parsed with an overloaded operation.

Default: 1

-

`contentType`

The http content type of the http request sent to call the web service.
Default: "text/xml;charset=utf-8"

-

`errorOnRedefine`

Throw an error, if a service is created (CreateService etc) for an already existing service.

Default value: 0

This item may not be used with *::WS::Client::Config*.

-

`genOutAttr`

generate attributes on outbound tags, see here for details

-

`inlineElementNS`

Namespace prefixes for types may be defined within the WSDL root element.

This item may not be used with *::WS::Client::Config*.

Activate this option, to also search namespace prefixes in the type definition. As those are seen as global prefixes, there might be a double-used prefix which will cause a processing error, if different URI's are assigned.

The error would be caused by a WSDL as follows

```
<wsdl:definitions targetNamespace="http://www.webserviceX.NET/"
xmlns:wsdl="http://schemas.xmlsoap.org/wsdl/ xmlns:q1="myURI1" ...> ...
<xs:element xmlns:q1="myURI2" type="q1:MessageQ1"/>
```

-

`location`

The URL of the service. This is initialized on the value in the WSDL file, when the WSDL file is parsed. The value may be overwritten setting this option.

This item may not be used with *::WS::Client::SetOption*.

-

`noTargetNs`

The target namespace URI is normally included twice in the envelope of the webservice call:

```
<SOAP-ENV:Envelope ... xmlns="http://targeturi.org/"
xmlns:tns1="http://targeturi.org/" ...>
```

Setting this option to 1 suppresses the line with "xmlns=".
This option was set to call a service published by SAP.
Default value: 0

-

`nsOnChangeOnly`

only put namespace prefix when namespaces change

-

`parseInAttr`

parse attributes on inbound tags, see here for details

-

`queryTimeout`

Timeout to any network query in ms. Default value: 60000 (1 minuite). The utility package has an option with the same functionality, which is used, when there is no call option context.

-

`skipHeaderLevel`

boolean indicating the first level of the XML in a request header *shall be* skipped. Derived from options. Default is 0 (do not skip). *(Introduced in 2.2.8)*

-

`skipLevelOnReply`

boolean indicating the first level of the XML in a reply *may be* skipped. Derived from options. Default is 0 (do not skip). *(Introduced in 2.2.8)*

-

`skipLevelWhenActionPresent`

boolean indicating if the first level of the XML is to be skipped. Derived from options. Default is 0 (do not skip).

-

`suppressNS (default: empty string)`

do not put a particular namespace prefix

-

`suppressTargetNS`

Do not add the Target Namespace URI prefix "tns1" to all parameters in the webservice call XML.

As an example, the XML is modified from (option not set):

```
<SOAP-ENV:Envelope ... xmlns:tns1="http://targeturi.org/" ... >
<SOAP-ENV:Body> <tns1:CalledMethod>
<tns1:Parameter1>Value;/tns1:Parameter1> </tns1:CalledMethod>
</SOAP-ENV:Body> </SOAP-ENV:Envelope>
```

to (option set)

```
<SOAP-ENV:Envelope ... xmlns:tns1="http://targeturi.org/" ... >
<SOAP-ENV:Envelope ... <SOAP-ENV:Body> <tns1:CalledMethod>
<Parameter1>Value;/Parameter1> </tns1:CalledMethod> </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Derived from options.
Internally, this option sets the option "suppressNS" to "tns1".
This option was set to call a service published by SAP.
This option made a call to a certain MS Web Service fail with the error message: "Input parameter 'Parameter1' can not be NULL or Empty.".
Default is 0 (do not suppress).

-

`targetNamespace (default: empty string)`

the target namespace of the service, derived from the WSDL.

This item may not be used with *::WS::Client::SetOption*.

-

`UseNS (default: empty string)`

See here

-

`useTypeNS (default: empty string)`

use type's namespace prefix as prefix of elements

-

`valueAttrCompatiblityMode (default: 1)`

If this and genOutAttr/parseInAttr are set, then values are specified in the dictionary as {}. Otherwise if genOutAttr/parseInAttr is set this is not set, then the values are specified in the dictionary as ::value.

```
     value         - Optional, the new value.
```

Returns :

```
     The value of the item.
Side-Effects : None
Exception Conditions  : None
Pre-requisite Conditions : Service must have been defined.
```

Dealing With Casting Abstract Types to Concrete Types

If you turn on *parseInAttr* and *genOutAttr*, the system will semi-automatically deal with casting of elements declared as a being of a type that is an abstract type to/from the concrete type actually to be used in a message. On an element that is decleared to be a type which is an abstract type, the value of the **::type** key in the dictionary will specify the concrete type to be actually used (or for a reply message the concrete type that was actually used).

NOTE: While in the WSDL the concreate type must be an extention of the abstract type, the package does not enforce this restriction, so caution must be taken.

Keywords: Calling_a_Web_Service

Creating_a_Tcl_Web_Service

Creating a Tcl Web Service

Contents

- 1 Loading the Webservices Server Package
- 2 Defining a Service
- 3 Defining an Operation (aka a Service Procedure)
- 4 Declaring Complex Types

Loading the Webservices Server Package

To load the webservices server package, do:

```
 package require WS::Server
```

This command will only load the server the first time it is used, so it causes no ill effects to put this in each file declaring a service or service procedure.

Using as part of TclHttpd

The Web Services package, WS::Server, is not a standalone application, but rather is designed to be a "module" of TclHttpd. The following command is normally placed in httpdthread.tcl:

Embedding in a Standalone Application

To embed a Web Service into an application, the application needs to be event driven and you also need to use the **WS::Embeded** package. You also must define the service with the *-mode=embedded* option.

See also Embedding a Web Service into an application.

Using with Apache Rivet

Apache Rivet is a module (mod_rivet) that can be loaded by Apache httpd server to allow web pages to run embedded Tcl commands in a way similar to PHP. To create a Web Service in Rivet, use the example EchoRivetService.rvt as a starting point by simply copying it into any directory served by your Apache instance. You should be able to immediately access that new location at the following URLs:

```
/path/to/EchoRivetService.rvt/doc
      Displays an HTML page describing the service
/path/to/EchoRivetService.rvt/wsdl
      Returns a WSDL describing the service
/path/to/EchoRivetService.rvt/op
      Invoke an operation
```

If you would prefer to expose the published URLs of your service differently, you can use the standard Apache mod_rewrite or mod_alias modules to transparently map any other URL to those locations.

Defining a Service

The code that defines a service is normally placed in one or more files in the custom directory.

Procedure Name : ::WS::Server::Service

Description: Declare a Web Service, the following URLs will exist

```
/service/<ServiceName>
      Displays an HTML page describing the service
/service/<ServiceName>/wsdl
      Returns a WSDL describing the service
/service/<ServiceName>/op
      Invoke an operation
```

Arguments: this procedure uses position independent arguments, they are:

```
          -hostcompatibility32 bool - Activate version 3.2.0
   compatibility
                            mode for -host parameter.
                            Defaults to true.
          -host           - The host specification within XML
   namespaces
                            of the transmitted XML files.
                            This should be unique.
                            Defaults to localhost.
```

```
                              If 3.2 compatibility is activated, the
default
                              value is changed to ip:port in embedded
mode.
         -hostlocation   - The host name, which is promoted within
the
                              generated WSDL file. Defaults to
localhost.
                              If 3.2 compatibility is activated, the
                              default value is equal to the -host
parameter.
         -hostlocationserver bool - If true, the host location is
set by
                              the current server settings.
                              In case of httpd server, this value is
imported.
                              For other servers or if this fails, the
value
                              is the current ip:port.
                              The default value is true.
                              In case of 3.2 compatibility, the
default
                              value is true for tclhttpd, false
otherwise.
         -hostProtocol   - Define the host protocol (http, https)
for the
                              WSDL location URL. The special value
"server"
                              (default) follows the TCP/IP server
specification.
                              This is implemented for Embedded server
and tclhttpd.
                              Remark that the protocol for XML
namespaces
                              is always "http".
         -description    - The HTML description for this service
         -htmlhead       - The title string of the service
description
         -author         - The author property in the service
description
         -xmlnamespace   - Extra XML namespaces used by the service
         -service        - The service name (this will also be
used for
                               the Tcl namespace of the procedures
that implement
                               the operations.
```

```
         -premonitor     - This is a command prefix to be called
before
                             an operation is called.  The
following arguments are
                             added to the command prefix:
                                PRE serviceName operationName
operArgList
         -postmonitor    - This is a command prefix to be called
after
                             an operation is called.  The
following arguments are
                             added to the command prefix:
                                POST serviceName operationName
OK|ERROR results
         -inheaders      - List of input header types.
         -outheaders     - List of output header types.
         -intransform    - Inbound (request) transform procedure
(2.0.3 and later).
                            The signature of the command must be:
                                 cmd \
                                     mode (REQUEST) \
                                     xml \
                                     notUsed_1 \
                                     notUsed_2
         -outtransform   - Outbound (reply) transform procedure
(2.0.3 and later).
                            The signature of the command must be:
                                 cmd \
                                     mode (REPLY) \
                                     xml \
                                     operation \
                                     resultDict
         -checkheader    - Command prefix to check headers.
                               If the call is not to be allowed,
this command
                               should raise an error.
                               The signature of the command must
be:
                                 cmd \
                                     service \
                                     operation \
                                     caller_ipaddr \
                                     http_header_list \
                                     soap_header_list
        -mode           - Mode that service is running in.  Must
be one of:
```

```
                                tclhttpd  -- running inside of
tclhttpd or an
                                          environment that
supplies a
                                          compatible
Url_PrefixInstall
                                          and Httpd_ReturnData
commands
                                embedded  -- using the
::WS::Embedded package
                                aolserver -- using the
::WS::AolServer package
                                wub       -- using the ::WS::Wub
package
                                wibble    -- running inside wibble
                                rivet     -- running inside Apache
Rivet (mod_rivet)
        -ports          - List of ports for embedded mode.
Default: 80
                                 NOTE -- a call should be to
                                         ::WS::Embedded::Listen for
each port
                                         in this list prior to
calling ::WS::Embeded::Start
        -prefix         - Path prefix used for the namespace and
endpoint
                          Defaults to "/service/" plus the service
name
        -traceEnabled   - Boolean to enable/disable trace being
passed back in exception
                          Defaults to "Y"
        -docFormat      - Format of the documentation for
operations ("text" or "html").
                          Defaults to "text"
        -stylesheet     - The CSS stylesheet URL used in the HTML
documentation

        -errorCallback  - Callback to be invoked in the event of
an error being produced
        -verifyUserArgs - Boolean to enable/disable validating
user supplied arguments
                          Defaults to "N"
        -enforceRequired - Throw an error if a required field is
not included in the
                           response.
                           Defaults to "N"
```

Returns: Nothing

Side-Effects: None

Exception Conditions:

```
     MISSREQARG -- Missing required arguments
```

Pre-requisite Conditions: None

Defining an Operation (aka a Service Procedure)
Procedure Name : ::WS::Server::ServiceProc

Description: Register an operation for a service and declare the procedure to handle the operations.

Arguments:

```
     ServiceName     -- Name of the service this operation is for
     NameInfo        -- List of two elements:
                             1) OperationName -- the name of the
    operation
                             2) ReturnType    -- the type of the
    procedure return,
                                                 this can be a simple
    or complex type
     Arglist         -- List of argument definitions,
                         each list element must be of the form:
                             1) ArgumentName -- the name of the
    argument
                             2) ArgumentTypeInfo -- -- A list of:
                                    {type typeName comment
    commentString}
                                         typeName can be any simple or
    defined type.
                                         commentString is a quoted
    string describing the field
     Documentation   -- HTML describing what this operation does
     Body            -- The tcl code to be called when the operation
    is invoked. This
                           code should return a dictionary with
    <OperationName>Result as a
                           key and the operation's result as the
    value.
```

Available simple types are:

- anyType, string, boolean, decimal, float, double, duration, dateTime, time, date, gYearMonth, gYear, gMonthDay, gDay, gMonth, hexBinary, base64Binary, anyURI, QName, NOTATION, normalizedString, token, language, NMTOKEN, NMTOKENS, Name, NCName, ID, IDREF, IDREFS, ENTITY, ENTITIES, integer, nonPositiveInteger, negativeInteger, long, int, short, byte,

nonNegativeInteger, unsignedLong, unsignedInt, unsignedShort, unsignedByte, positiveInteger

The

typeName

may contain the following suffixes:

- *()*: type is an array
- *?*: type is an optional parameter

Returns: Nothing

Side-Effects:

```
   A procedure named "<ServiceName>::<OperationName>" defined
   A type name with the name <OperationName>Result is defined.
```

Exception Conditions: None

Pre-requisite Conditions: ::WS::Server::Server must have been called for the ServiceName

---

Declaring Complex Types

See: Creating a Web Service Type from Tcl

Keywords: Creating_a_Tcl_Web_Service

Creating_a_Web_Service_Type

Creating a Web Service Type from Tcl

Contents

- 1 Overview
- 2 Loading the Webservices Utility Package
- 3 Defining a type
- 3 Defining a derived type
- 4 Getting a type definition
- 5 Generating a template dictionary for a type definition

Overview

Webservice Type declaration is part of the Webservices Utility package.

When writing a web service it is often requried to write a complex type definition for an argument containing structured data.

When calling an operation on a web service it is sometimes convient to define a complex type to return structured data as an XML fragment even though the sevice may state that it is only expecting a string.

Loading the Webservices Utility Package

To load the webservices server package, do:

```
package require WS::Utils
```

This command will only load the utilities the first time it is used, so it causes no ill effects to put this in each file using the utilties.

Defining a type

Procedure Name: *::WS::Utils::ServiceTypeDef*

Description: Define a type for a service.

Arguments:

```
 mode            - Client or Server
 service         - The name of the service this type definition is
for
 type            - The type to be defined/redefined
 definition      - The definition of the type's fields.  This
consist of one
                       or more occurance of a field definition.
Each field definition
                       consist of:  fieldName fieldInfo
                       Where field info is: {type typeName comment
commentString}
                          typeName can be any simple or defined
type.
                          commentString is a quoted string
describing the field.
```

Returns: Nothing

Side-Effects: None

Exception Conditions: None

Pre-requisite Conditions: None

Defining a derived type

Procedure Name: *::WS::Utils::ServiceSimpleTypeDef*

Description: Define a derived type for a service.

Arguments:

```
 mode            - Client or Server
 service         - The name of the service this type definition is
for
 type            - The type to be defined/redefined
 definition      - The definition of the type's fields.  This
consist of one
                       or more occurance of a field definition.
Each field definition
                       consist of:  fieldName fieldInfo
                       Where: {type typeName comment commentString}
                          baseType typeName - any simple or
defined type.
```

```
                              comment commentString - a quoted string
    describing the field.
                              pattern value
                              length value
                              fixed "true"|"false"
                              maxLength value
                              minLength value
                              minInclusive value
                              maxInclusive value
                              enumeration value
```

Returns: Nothing

Side-Effects: None

Exception Conditions: None

Pre-requisite Conditions: None

Getting a type definition

Procedure Name: *::WS::Utils::GetServiceTypeDef*

Description: Query for type definitions.

Arguments:

```
     mode            - Client or Server
     service         - The name of the service this query is for
     type            - The type to be retrieved (optional)
```

Returns:

```
     If type not provided, a dictionary object describing all of the
    types
     for the service.
     If type provided, a dictionary object describing the type.
       A definition consist of a dictionary object with the following
    key/values:
         xns         - The namespace for this type.
         definition  - The definition of the type's fields.  This
    consist of one
                       or more occurance of a field definition.  Each
    field definition
                       consist of:  fieldName fieldInfo
                       Where field info is: {type typeName comment
    commentString}
                         typeName can be any simple or defined type.
                         commentString is a quoted string describing
    the field.
```

Side-Effects: None

Exception Conditions: None

Pre-requisite Conditions: The service must be defined.

Generating a template dictionary for a type definition

Procedure Name: *::WS::Utils::GenerateTemplateDict*

Description: Generate a template dictionary object for a given type.

Arguments:

```
     mode            - Client or Server
     serviceName     - The service name the type is defined in
     type            - The name of the type
     arraySize       - Number of elements to generate in an array.
    Default is 2.
```

Returns:

```
      A dictionary object for a given type.  If any circular
    references exist, they will have the value of <** Circular
    Reference **>
```

Side-Effects: None

Exception Conditions: None

Pre-requisite Conditions: The type and service must be defined.

Keywords: Creating_a_Web_Service_Type

Defining_Types

Contents

- 1 Overview
- 2 Loading the Webservices Utility Package
- 3 Defining a type
- 3 Defining a derived type
- 4 Getting a type definition
- 5 Generating a template dictionary for a type definition

Overview

Webservice Type declaration is part of the Webservices Utility package.

When writing a web service it is often requried to write a complex type definition for an argument containing structured data.

When calling an operation on a web service it is sometimes convient to define a complex type to return structured data as an XML fragment even though the sevice may state that it is only expecting a string.

Loading the Webservices Utility Package

To load the webservices server package, do:

```
package require WS::Utils
```

This command will only load the utilities the first time it is used, so it causes no ill effects to put this in each file using the utilties.

Defining a type

Procedure Name: *::WS::Utils::ServiceTypeDef*

Description: Define a type for a service.

Arguments:

```
 mode            - Client or Server
 service         - The name of the service this type definition is
for
 type            - The type to be defined/redefined
 definition      - The definition of the type's fields.  This
consist of one
                       or more occurance of a field definition.
Each field definition
                       consist of:  fieldName fieldInfo
                       Where field info is: {type typeName comment
commentString}
                          typeName can be any simple or defined
type.
                          commentString is a quoted string
describing the field.
```

Returns: Nothing

Side-Effects: None

Exception Conditions: None

Pre-requisite Conditions: None

Defining a derived type

Procedure Name: *::WS::Utils::ServiceSimpleTypeDef*

Description: Define a derived type for a service.

Arguments:

```
 mode            - Client or Server
 service         - The name of the service this type definition is
for
 type            - The type to be defined/redefined
 definition      - The definition of the type's fields.  This
consist of one
                       or more occurance of a field definition.
Each field definition
```

```
                          consist of:  fieldName fieldInfo
                          Where: {type typeName comment commentString}
                             baseType typeName - any simple or
   defined type.
                             comment commentString - a quoted string
   describing the field.
                             pattern value
                             length value
                             fixed "true"|"false"
                             maxLength value
                             minLength value
                             minInclusive value
                             maxInclusive value
                             enumeration value
```

Returns: Nothing

Side-Effects: None

Exception Conditions: None

Pre-requisite Conditions: None

Getting a type definition

Procedure Name: *::WS::Utils::GetServiceTypeDef*

Description: Query for type definitions.

Arguments:

```
    mode            - Client or Server
    service         - The name of the service this query is for
    type            - The type to be retrieved (optional)
```

Returns:

```
    If type not provided, a dictionary object describing all of the
   types
    for the service.
    If type provided, a dictionary object describing the type.
      A definition consist of a dictionary object with the following
   key/values:
        xns         - The namespace for this type.
        definition  - The definition of the type's fields.  This
   consist of one
                      or more occurance of a field definition.  Each
   field definition
                      consist of:  fieldName fieldInfo
                      Where field info is: {type typeName comment
   commentString}
```

```
                        typeName can be any simple or defined type.
                        commentString is a quoted string describing
    the field.
```

Side-Effects: None

Exception Conditions: None

Pre-requisite Conditions: The service must be defined.

Generating a template dictionary for a type definition

Procedure Name: *::WS::Utils::GenerateTemplateDict*

Description: Generate a template dictionary object for a given type.

Arguments:

```
     mode            - Client or Server
     serviceName     - The service name the type is defined in
     type            - The name of the type
     arraySize       - Number of elements to generate in an array.
    Default is 2.
```

Returns:

```
      A dictionary object for a given type.  If any circular
    references exist, they will have the value of <** Circular
    Reference **>
```

Side-Effects: None

Exception Conditions: None

Pre-requisite Conditions: The type and service must be defined.

Keywords: Defining_Types

Dictionary_Representation_of_XML_Arrays

XML arrays are represented in dictionary format as a list of values. Lets consider what this looks like for a simple type and for a complex type;.

Array of Simple Type

Lets assume we have an element with the following definition:

```
    <xs:element minOccurs="0" maxOccurs="unbounded" name="Primes"
    type="xs:integer" />
```

Lets also assume that we will have that element in our dictionary with the first four prime numbers, thus the dictionary representation for that element would look like:

```
    Primes {2 3 5 7}
```

Or, if we have are using attributes (i.e. parseInAttr and/or genOutAttr are set), it would look like:

```
    Primes {{} {2 3 5 7}}
```

Array of Complex Type
Lets assume we have the type definition:

```
<xs:element name="Person">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="FristName" type="xs:string"/>
      <xs:element name="LastName" type="xs:integer"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Lets assume we have the following definition:

```
    <xs:element minOccurs="0" maxOccurs="unbounded" name="Attendees"
    type="Person" />
```

Now lets assume the following people are are attending:

- John Doe
- Jane Doe

Thus the dictionary representation for that element would look like:

```
    Attendees {
        {FirstName {John} LastName {Doe}}
        {FirstName {Jane} LastName {Doe}}
    }
```

Or, if we have are using attributes (i.e. parseInAttr and/or genOutAttr are set), it would look like:

```
    Attendees {
        {{} {FirstName {{} {John}} LastName {{} {Doe}}}}
        {{} {FirstName {{} {Jane}} LastName {{} {Doe}}}}
    }
```

Keywords: Dictionary_Representation_of_XML_Arrays

Embedded_Web_Service

Embeding a Web Service

Contents

- 1 Loading the Webservices Embeded Package
- 2 Specify a Port to Receive Request on
- 3 Run the event queue
- 4 Close a port
- 4 Close all ports

Loading the Webservices Server Package

To load the webservices server package, do:

```
 package require WS::Embeded
```

This command will only load the server the first time it is used, so it causes no ill effects to put this in each file declaring a service or service procedure.

Specify a Port to Receive Request on

The following command opens a listener socket in the specified port. The webservice functionality may be added by a call to *::WS::Server::Service* with the *-mode* parameter set to *embedded*.

Procedure Name : ::WS::Embeded::Listen

Description: Instruct the module to listen on a Port, security information.

Arguments: this procedure uses position dependent arguments, they are:

```
 port     -- Port number to listen on.
 certfile -- Name of the certificate file or a pfx archive for
twapi.
             Defaults to {}.
 keyfile  -- Name of the key file. Defaults to {}.
             To use twapi TLS, specify a list with the following
elements:
             -- "-twapi": Flag, that TWAPI TLS should be used
             -- password: password of PFX file passed by
                [::twapi::conceal]. The concealing makes sure that
the
                password is not readable in the error stack trace
             -- ?subject?: optional search string in pfx file, if
                multiple certificates are included.
 userpwds -- A list of username:password. Defaults to {}.
 realm    -- The seucrity realm. Defaults to {}.
 timeout  -- A time in ms the sender may use to send the request.
             If a sender sends wrong data (Example: TLS if no TLS
is
             used), the process will just stand and a timeout is
required
             to clear the connection. Set to 0 to not use a
timeout.
             Default: 60000 (1 Minuit).
```

Returns: Handle of socket

Side-Effects: None

Exception Conditions: : None

Pre-requisite Conditions: None

Run the event queue

To serve any requests, the interpreter must run the event queue using. If this is not anyway the case (Tk present etc.), one may call:

```
    vwait waitVariable
```

To stop the event queue after server shutdown, one may execute:

```
    set waitVariable 1
```

Close a port

Procedure Name : ::WS::Embeded::Close

Description: Close a formerly opened listener port and stop all running requests on this port.

Arguments: this procedure uses position dependent arguments, they are:

```
    port     -- Port number to close.
```

Returns: None

Side-Effects: None

Exception Conditions: : None

Pre-requisite Conditions: None

Close all ports

Procedure Name : ::WS::Embeded::CloseAll

Description: Close all formerly opened listener port and stop all running requests.

Arguments: this procedure uses no arguments

Returns: None

Side-Effects: None

Exception Conditions: : None

Pre-requisite Conditions: None

Keywords: Embedded_Web_Service

Rest_flavor_service_response

Rest-flavor service reply

Contents

- 1 Overview
- 2 Rivet example

Overview

Since TCLWS 2.4, it is possible to return a response in *REST style*. This means, that a JSON reply is returned instead of an XML document.

Our use case has only required us to accept FORM arguments and return JSON responses for everything, so we haven't implemented logic to parse any input arguments that are passed in as JSON serialized data, but this might be an area of future exploration for someone.

Rivet Example

Here's a bit of code showing how we initially start up this mode in Apache Rivet, which is actually pretty similar to how you'd use tclws in SOAP mode from Apache Rivet:

```
        # Capture the info from the request into an array.
        load_headers hdrArray
        set sock [pid];         # an arbitrary value
        array unset ::Httpd$sock

        # Prepare the CGI style arguments into a list
        load_response formArray
        set opname $formArray(call)
        unset formArray(call)
        set queryarg [list $opname [array get formArray]]

        # Invoke the the method
        array set ::Httpd$sock [list query $queryarg ipaddr [env
    REMOTE_ADDR] headerlist [array get hdrArray]]

        # Invoke the method in REST mode.
        set result [catch {::WS::Server::callOperation $svcname $sock
    -rest} error]
        array unset ::Httpd$sock
        if {$result} {
                headers numeric 500
                puts "Operation failed: $error"
                abort_page
        }
```

Keywords: Rest_flavor_service_response

Tcl_Web_Service_Example

Tcl Web Service Example

Server Side

The following is placed in the httpdthread.tcl:

```
    package require WS::Server
    package require WS::Utils
```

The following is placed in the a file in the custom directory:

```
##
## Define the service
##
::WS::Server::Service \
    -service wsExamples \
    -description  {Tcl Example Web Services} \
    -host         $::Config(host):$::Config(port)
```

```
##
## Define any special types
##
::WS::Utils::ServiceTypeDef Server wsExamples echoReply {
    echoBack     {type string}
    echoTS       {type dateTime}
}
```

```
##
## Define the operations available
##
::WS::Server::ServiceProc \
    wsExamples \
    {SimpleEcho {type string comment {Requested Echo}}} \
    {
        TestString      {type string comment {The text to echo
 back}}
    } \
    {Echo a string back} {
```

```
    return [list SimpleEchoResult $TestString]
}
```

```
::WS::Server::ServiceProc \
    wsExamples \
    {ComplexEcho {type echoReply comment {Requested Echo -- text
 and timestamp}}} \
    {
        TestString      {type string comment {The text to echo
 back}}
    } \
    {Echo a string and a timestamp back} {
```

```
    set timeStamp [clock format [clock seconds] -format
 {%Y-%m-%dT%H:%M:%SZ} -gmt yes]
    return [list ComplexEchoResult [list echoBack $TestString
 echoTS $timeStamp]  ]
}
```

Client Side

```
package require WS::Client
```

```
##
## Get Definition of the offered services
##
::WS::Client::GetAndParseWsdl
 http://localhost:8015/service/wsExamples/wsdl
```

```
set testString "This is a test"
set inputs [list TestString $testString]
```

```
##
## Call synchronously
##
puts stdout "Calling SimpleEcho via DoCalls!"
set results [::WS::Client::DoCall wsExamples SimpleEcho $inputs]
puts stdout "\t Received: {$results}"
```

```
puts stdout "Calling ComplexEcho via DoCalls!"
set results [::WS::Client::DoCall wsExamples ComplexEcho $inputs]
puts stdout "\t Received: {$results}"
```

```
##
## Generate stubs and use them for the calls
##
::WS::Client::CreateStubs wsExamples
puts stdout "Calling SimpleEcho via Stubs!"
set results [::wsExamples::SimpleEcho $testString]
puts stdout "\t Received: {$results}"
```

```
puts stdout "Calling ComplexEcho via Stubs!"
set results [::wsExamples::ComplexEcho $testString]
puts stdout "\t Received: {$results}"
```

```
##
## Call asynchronously
##
proc success {service operation result} {
    global waitVar
```

```
    puts stdout "A call to $operation of $service was successful
 and returned $result"
    set waitVar 1
}
```

```
proc hadError {service operation errorCode errorInfo} {
    global waitVar
```

```
    puts stdout "A call to $operation of $service was failed with
 {$errorCode} {$errorInfo}"
    set waitVar 1
}
```

```
set waitVar 0
puts stdout "Calling SimpleEcho via DoAsyncCall!"
::WS::Client::DoAsyncCall wsExamples SimpleEcho $inputs \
        [list success wsExamples SimpleEcho] \
        [list hadError wsExamples SimpleEcho]
vwait waitVar
```

```
puts stdout "Calling ComplexEcho via DoAsyncCall!"
::WS::Client::DoAsyncCall wsExamples ComplexEcho $inputs \
        [list success wsExamples SimpleEcho] \
        [list hadError wsExamples SimpleEcho]
vwait waitVar
```

```
exit
```

Keywords: Tcl_Web_Service_Example

Tcl_Web_Service_Math_Example

Tcl Web Service Math Example

Server Side

The following is placed in the httpdthread.tcl:

```
package require WS::Server
package require WS::Utils
```

The following is placed in the a file in the custom directory:

```
 ##
 ## Define the service
 ##
 ::WS::Server::Service \
     -service wsMathExample \
     -description  {Tcl Web Services Math Example} \
     -host         $::Config(host):$::Config(port)

 ##
 ## Define any special types
 ##
 ::WS::Utils::ServiceTypeDef Server wsMathExample Term {
```

```
       `coef         {type float}
        powerTerms   {type PowerTerm()}
    }
    ::WS::Utils::ServiceTypeDef Server wsMathExample PowerTerm {
        var          {type string}
        exponet      {type float}
    }
    ::WS::Utils::ServiceTypeDef Server wsMathExample Variables {
        var          {type string}
        value        {type float}
    }

   ##
   ## Define the operations available
   ##
   ::WS::Server::ServiceProc \
        wsMathExample \
        {EvaluatePolynomial {type float comment {Result of evaluating
    a polynomial}}} \
        {
            varList       {type Variables() comment {The variables to
    be substitued into the polynomial}}
            polynomial    {type Term() comment {The polynomial}}
        } \
        {Evaluate a polynomial} {
        set equation {0 }
        foreach varDict $varList {
            set var [dict get $varDict var]
            set val [dict get $varDict value]
            set vars($var) $val
        }
        foreach term $polynomial {
            if {[dict exists $term coef]} {
                set coef [dict get $term coef]
            } else {
                set coef 1
            }
            append equation "+ ($coef"
            foreach pow [dict get $term powerTerms] {
                if {[dict exists $pow exponet]} {
                    set exp [dict get $pow exponet]
                } else {
                    set exp 1
                }
                append equation [format { * pow($vars(%s),%s} [dict
    get $pow var] $exp]
```

```
            }
            append equation ")"
        }
        set result [expr $equation]
        return [list SimpleEchoResult $result]
    }
```

Client Side

```
    package require WS::Client
    ##
    ## Get Definition of the offered services
    ##
    ::WS::Client::GetAndParseWsdl
    http://localhost:8015/service/wsMathExamples/wsdl

    dict set term var X
    dict set term value 2.0
    dict lappend varList $term
    dict set term var Y
    dict set term value 3.0
    dict lappend varList $term

    set term {}
    set powerTerm {}
    dict set powerTerm coef 2.0
    dict set term var X
    dict set term pow 2.0
    dict lappend terms $term
    dict set term var Y
    dict set term pow 3.0
    dict lappend terms $term
    dict set powerTerm powerTerms $terms

    dict set powerTerm coef -2.0
    dict set term var X
    dict set term pow 3.0
    dict lappend terms $term
    dict set term var Y
    dict set term pow 2.0
    dict lappend terms $term
    dict set powerTerm powerTerms $terms
    dict lappend polynomial powerTerms $powerTerm

    dict set input [list varList $varList polynomial $polynomial]
    ##
```

```
    ## Call service
    ##
    puts stdout "Calling EvaluatePolynomial wiht {$input}"
    set resultsDict [::WS::Client::DoCall wsMathExample
    EvaluatePolynomial $input]
    puts stdout "Results are {$resultsDict}"
```

Keywords: Tcl_Web_Service_Math_Example

Using_Options

Using Web Service Options

Contents

- Overview
- Loading the Webservices Utilities Package
- Access Routine
- Options
  - genOutAttr - generate attributes on outbound tags
  - includeDirectory - disk directory to use for XSD includes when they can not be accessed via the Web.
  - nsOnChangeOnly - only put namespace prefix when namespaces change
  - parseInAttr - parse attributes on inbound tags
  - queryTimeout - set http(s) fetch timeout.
  - StrictMode - WSDL processing mode
  - UseNS - put namespaces on field tags
  - useTypeNs - use type's namespace prefix as prefix of elements
  - suppressNS - do not put a particular namespace prefix

Overview

The Webservices Client and Server packages make use of the following options:

- UseNS, when set the following options become relevant:
- StrictMode
- parseInAttr
- genOutAttr
- includeDirectory
  - useTypeNs
  - nsOnChangeOnly
  - suppressNS

The attributes can be retrieved and set using ::WS::Utils::SetOption.

If called by the client side, the following options are overwritten and restored on any call:

- genOutAttr
- nsOnChangeOnly
- parseInAttr
- suppressNS
- UseNS

- useTypeNs
- valueAttrCompatiblityMode

Loading the Webservices Utilities Package

To load the webservices server package, do:

```
package require WS::Utils
```

This command will only load the utilities the first time it is used, so it causes no ill effects to put this in each file using the utilties.

Access Routine

Procedure Name : ::WS::Utils::SetOption

Description : Retrieve or set an option

Arguments :

```
    option - name of the option
    value - value of the option (optional)
```

Returns : The value of the option

Side-Effects : None

Exception Conditions :None

Pre-requisite Conditions : None

---

genOutAttr - generate attributes on outbound tags

The *genOutAttr* option, if set to a "true" value, will convert all dictionary keys of the entry for a given field tag to attribute value pairs of the tag in the outbound XML. For attributes in the "http://www.w3.org/2001/XMLSchema-instance" url, the key will be the attribute name prepended with two colons (e.g. **::type**) and the value will be the value of the attribute. For attributes other than those in the "http://www.w3.org/2001/XMLSchema-instance" url, the key will be the attribute name and the value will be the value of the attribute. The value of the tag will have a key determined by the valueAttrCompatiblityMode.

The default value, "0", is for this option to be turned off.

---

includeDirectory - disk directory to use for XSD includes when they can not be accessed via the Web.

The *includeDirectory* option, if set, instructs TclWs to look in the specified directory for any XSD includes that can not be found via the web.

The default value, "{}", is for this option to be turned off.

---

nsOnChangeOnly - only put namespace prefix when namespaces change

The *nsOnChangeOnly* option, if set to a "true" value, will only place namespace prefixes when

the namespaces change.

This option is only relevant, if the option UseNS is set.

The default value, "0", is for this option to be turned off.

---

parseInAttr - parse attributes on inbound tags

The *parseInAttr* option, if set to a "true" value, will convert all attributes of inbound field tags to dictionary entries for that tag. For attributes in the "http://www.w3.org/2001/XMLSchema-instance" url, the key will be the attribute name prepended with two colons (e.g. **::type**) and the value will be the value of the attribute. For attributes other than those in the "http://www.w3.org/2001/XMLSchema-instance" url, the key will be the attribute name and the value will be the value of the attribute. The value of the tag will have a key determined by the valueAttrCompatiblityMode.

The default value, "0", is for this option to be turned off.

---

queryTimeout - set http(s) query timeout

Timeout to any network query in ms. The client side package has an option with the same functionality, which is used, when there is a call option context.

Default value: 60000 (1 minuite).

StrictMode - WSDL processing mode

The *StrictMode* option determines what happens when an error is detected in parsing a WSDL. The legal values are:

- **debug**
- **warning**
- **error**

If the *StrictMode* is set to **debug** or **warning**, a message is logged using the ::log package at that level and the error is then ignored.

If the *StrictMode* is set to any value other than **debug** or **warning**, a message is logged using the ::log package at the **error** level and exception is generated.

The default value is **error**.

A major use of this is to ignore namespace imports in a WDSL that do not actually import any definitions.

UseNS - put namespaces on field tags

The *UseNS* option, if set to a "true" value, will put a namespace alias on all field tags.

The default value, "1", is for this option to be turned on.

---

useTypeNs - use type's namespace prefix as prefix of elements

The *useTypeNs* option, if set to a "true" value, will use the prefix of the type's namespace instead of the prefix of the element's namespace.

This option is only relevant, if the option UseNS is set.

The default value, "0", is for this option to be turned off.

---

suppressNS - do not put a given namespace prefix.

The *suppressNS* option, if set, will cause the given namespace to never be used as a prefix (i.e. tags that would normally have had the given prefix will not have any prefix).

This option is only relevant, if the option UseNS is set.

The default value, "{}", is for this option to be turned off.

---

valueAttrCompatiblityMode - specify dictionary key for value when attributes are in use

This option is only meaningful when the *parseInAttr* or *genOutAttr* option is set to a "true" value. When set to a "true" value, the value of the tag will have a key of the null string (i.e. **{}**). When set to a "false" value, the value of the tag will have a key of **::value**.

The default value, "0", is for this option to be turned off.

Keywords: Using_Options

index

Tcl Web Services

Summary

The distribution provides both client side access to Web Services and server side creation of Web Services. Currently only document/literal and rpc/encoded with HTTP Soap transport are supported on the client side. The server side currently works only with TclHttpd or embedded into an application. The server side provides all services as document/literal over HTTP Soap transport. Documentation for the package, including examples can be found here.

- Calling a Web Service from Tcl
- Creating a Tcl Web Service
- Creating a Web Service Type
- REST flavor service response
- Dictionary Representation of XML Arrays
- Using Web Service Options
- Embeding a Web Service into an application
- Tcl Web Service Example
- Tcl Web Service Math Example

The client is known to work with #C and Java based Web Services (your mileage may very).

License

Standard BSD.

Web Servers

The server side works with the following web servers:

- TclHttpd
- Rivet
- AOLserver
- WUB
- wibble
- Embedded mode (see Embedding)

Packages Required

The following packages are used:

- Tcl 8.6
- tdom 0.8.1
- tcltls or TWAPI 4.4.0 for client and embedded server TLS support
- log from TclLib
- uri from TclLib
- struct::set from TclLib
- http from Tcl itself
- yajl-tcl from flightaware github (only for rest-flavour requests)

If you are running the TclHttpd on Windows, it is highly recommended that you use the iocpsock extension.

The following packages are additionally used in Embedded Server mode:

- base64 from TclLib (also channel server)
- html from TclLib (also channel server)
- ncgi from TclLib
- fileutil from TclLib

Keywords: index

tdbc

tdbc

Name

tdbc - Tcl Database Connectivity

Synopsis

package require tdbc 1.0
package require tdbc::driver version

tdbc::driver::connection create *db* ?*-option value* ...?

Description

Tcl Database Connectivity (TDBC) is a common interface for Tcl programs to access SQL databases. It is implemented by a series of database *drivers*: separate modules, each of which adapts Tcl to the interface of one particular database system. All of the drivers implement a common series of commands for manipulating the database. These commands are all named dynamically, since they all represent objects in the database system. They include **connections,** which represent connections to a database; **statements,** which represent SQL

statements, and **result sets,** which represent the sets of rows that result from executing statements. All of these have manual pages of their own, listed under **SEE ALSO**.

In addition, TDBC itself has a few service procedures that are chiefly of interest to driver writers. **SEE ALSO** also enumerates them.

Copyright

Copyright (c) 2008 by Kevin B. Kenny.

Keywords: tdbc, SQL, database, connectivity, connection, resultset, statement

tdbc_connection

Name

tdbc::connection - TDBC connection object

Synopsis

package require tdbc 1.0
package require tdbc::driver version

tdbc::driver::connection create *db* ?*-option value* ...?

db **configure** ?*-option value* ...?
db **close**
db **foreignkeys** ?*-primary tableName*? ?*-foreign tableName*?
db **prepare** *sql-code*
db **preparecall** *call*
db **primarykeys** *tableName*
db **statements**
db **resultsets**
db **tables** ?*pattern*?
db **columns** *table* ?*pattern*?
db **begintransaction**
db **commit**
db **rollback**
db **transaction** *script*
db **foreach** ?*-as lists|dicts*? ?*-columnsvariable name*? ?*--*? *varName sqlcode* ?*dictionary*? *script*

.in 14 *db* **allrows** ?**-as lists**|**dicts**? ?**-columnsvariable** *name*? ?**--**? *sql-code* ?*dictionary*?

Description

Every database driver for TDBC (Tcl DataBase Connectivity) implements a *connection* object that represents a connection to a database. By convention, this object is created by the command, **tdbc::***driver***::connection create**. This command accepts the name of a Tcl command that will represent the connection and a possible set of options (see **CONFIGURATION OPTIONS**). It establishes a connection to the database and returns the name of the newly-created Tcl command.

The **configure** object command on a database connection, if presented with no arguments, returns a list of alternating keywords and values representing the connection's current configuration. If presented with a single argument *-option*, it returns the configured value of the

given option. Otherwise, it must be given an even number of arguments which are alternating options and values. The specified options receive the specified values, and nothing is returned.

The **close** object command on a database connection closes the connection. All active statements and result sets on the connection are closed. Any uncommitted transaction is rolled back. The object command is deleted.

The **prepare** object command on a database connection prepares a SQL statement for execution. The *sql-code* argument must contain a single SQL statement to be executed. Bound variables may be included. The return value is a newly-created Tcl command that represents the statement. See **tdbc::statement** for more detailed discussion of the SQL accepted by the **prepare** object command and the interface accepted by a statement.

On a database connection where the underlying database and driver support stored procedures, the **preparecall** object command prepares a call to a stored procedure for execution. The syntax of the stored procedure call is:

```
?resultvar =? procname(?arg ?, arg...?)
```

The return value is a newly-created Tcl command that represents the statement. See **tdbc::statement** for the interface accepted by a statement.

The **statements** object command returns a list of statements that have been created by **prepare** and **preparecall** statements against the given connection and have not yet been closed.

The **resultsets** object command returns a list of result sets that have been obtained by executing statements prepared using the given connection and not yet closed.

The **tables** object command allows the program to query the connection for the names of tables that exist in the database. The optional *pattern* parameter is a pattern to match the name of a table. It may contain the SQL wild-card characters '**%**' and '**_**'. The return value is a dictionary whose keys are table names and whose values are subdictionaries. See the documentation for the individual database driver for the interpretation of the values.

The **columns** object command allows the program to query the connection for the names of columns that exist in a given table. The optional **pattern** parameter is a pattern to match the name of a column. It may contain the SQL wild-card characters '**%**' and '**_**'. The return value is a dictionary whose keys are column names and whose values are dictionaries. Each of the subdictionaries will contain at least the following keys and values (and may contain others whose usage is determined by a specific database driver).

type
: Contains the data type of the column, and will generally be chosen from the set, **bigint**, **binary**, **bit**, **char**, **date**, **decimal**, **double**, **float**, **integer**, **longvarbinary**, **longvarchar**, **numeric**, **real**, **time**, **timestamp**, **smallint**, **tinyint**, **varbinary**, and **varchar**. (If the column has a type that cannot be represented as one of the above, **type** will contain a driver-dependent description of the type.)

precision
: Contains the precision of the column in bits, decimal digits, or the width in characters, according to the type.

scale
: Contains the scale of the column (the number of digits after the radix point), for types that

support the concept.

nullable
Contains 1 if the column can contain NULL values, and 0 otherwise. The **primarykeys** object command allows the program to query the connection for the primary keys belonging to a given table. The *tableName* parameter identifies the table being interrogated. The result is a list of dictionaries enumerating the keys (in a similar format to the list returned by *$connection* **allrows -as dicts**). The keys of the dictionary may include at least the following. Values that are NULL or meaningless in a given database are omitted.

tableCatalog
Name of the catalog in which the table appears.

tableSchema
Name of the schema in which the table appears.

tableName
Name of the table owning the primary key.

constraintCatalog
Name of the catalog in which the primary key constraint appears. In some database systems, this may not be the same as the table's catalog.

constraintSchema
Name of the schema in which the primary key constraint appears. In some database systems, this may not be the same as the table's schema.

constraintName
Name of the primary key constraint,

columnName
Name of a column that is a member of the primary key.

ordinalPosition
Ordinal position of the column within the primary key. To these columns may be added additional ones that are specific to a particular database system. The **foreignkeys** object command allows the program to query the connection for foreign key relationships that apply to a particular table. The relationships may be constrained to the keys that appear in a particular table (**-foreign** *tableName*), the keys that refer to a particular table (**-primary** *tableName*), or both. At least one of **-primary** and **-foreign** should be specified, although some drivers will enumerate all foreign keys in the current catalog if both options are omitted. The result of the **foreignkeys** object command is a list of dictionaries, with one list element per key (in a similar format to the list returned by *$connection* **allrows -as dicts**). The keys of the dictionary may include at least the following. Values that are NULL or meaningless in a given database are omitted.

foreignConstraintCatalog
Catalog in which the foreign key constraint appears.

foreignConstraintSchema
Schema in which the foreign key constraint appears.

foreignConstraintName
Name of the foreign key constraint.

primaryConstraintCatalog
Catalog holding the primary key constraint (or unique key constraint) on the column to which

the foreign key refers.

primaryConstraintSchema
Schema holding the primary key constraint (or unique key constraint) on the column to which the foreign key refers.

primaryConstraintName
Name of the primary key constraint (or unique key constraint) on the column to which the foreign key refers.

updateAction
Action to take when an UPDATE statement invalidates the constraint. The value will be **CASCADE**, **SET DEFAULT**, **SET NULL**, **RESTRICT**, or **NO ACTION**.

deleteAction
Action to take when a DELETE statement invalidates the constraint. The value will be **CASCADE**, **SET DEFAULT**, **SET NULL**, **RESTRICT**, or **NO ACTION**.

primaryCatalog
Catalog name in which the primary table (the one to which the foreign key refers) appears.

primarySchema
Schema name in which the primary table (the one to which the foreign key refers) appears.

primaryTable
Table name of the primary table (the one to which the foreign key refers).

primaryColumn
Name of the column to which the foreign key refers.

foreignCatalog
Name of the catalog in which the table containing the foreign key appears.

foreignSchema
Name of the schema in which the table containing the foreign key appears.

foreignTable
Name of the table containing the foreign key.

foreignColumn
Name of the column appearing in the foreign key.

ordinalPosition
Position of the column in the foreign key, if the key is a compound key. The **begintransaction** object command on a database connection begins a transaction on the database. If the underlying database does not support atomic, consistent, isolated, durable transactions, the **begintransaction** object command returns an error reporting the fact. Similarly, if multiple **begintransaction** commands are executed withough an intervening **commit** or **rollback** command, an error is returned unless the underlying database supports nested transactions. The **commit** object command on a database connection ends the most recent transaction started by **begintransaction** and commits changes to the database. The **rollback** object command on a database connection rolls back the most recent transaction started by **begintransaction**. The state of the database is as if nothing happened during the transaction. The **transaction** object command on a database connection presents a simple way of bundling a database transaction. It begins a transaction, and evaluates the supplied *script* argument as a Tcl script in the caller's scope. If *script* terminates normally, or by **break**, **continue**, or **return**, the transaction is committed (and any action requested by **break**,

continue, or **return** takes place). If the commit fails for any reason, the error in the commit is treated as an error in the *script*. In the case of an error in *script* or in the commit, the transaction is rolled back and the error is rethrown. Any nonstandard return code from the script causes the transaction to be rolled back and then is rethrown. The **allrows** object command prepares a SQL statement (given by the *sql-code* parameter) to execute against the database. It then executes it (see **tdbc::statement** for details) with the optional *dictionary* parameter giving bind variables. Finally, it uses the *allrows* object command on the result set (see **tdbc::resultset**) to construct a list of the results. Finally, both result set and statement are closed. The return value is the list of results. The **foreach** object command prepares a SQL statement (given by the *sql-code* parameter) to execute against the database. It then executes it (see **tdbc::statement** for details) with the optional *dictionary* parameter giving bind variables. Finally, it uses the *foreach* object command on the result set (see **tdbc::resultset**) to evaluate the given *script* for each row of the results. Finally, both result set and statement are closed, even if the given *script* results in a **return**, an error, or an unusual return code.

Configuration options

The configuration options accepted when the connection is created and on the connection's **configure** object command include the following, and may include others specific to a database driver.

-encoding *name*
: Specifies the encoding to be used in connecting to the database. The *name* should be one of the names accepted by the **encoding** command. This option is usually unnecessary; most database drivers can figure out the encoding in use by themselves.

-isolation *level*
: Specifies the transaction isolation level needed for transactions on the database. The acceptable values for *level* are shown under **TRANSACTION ISOLATION LEVELS**.

-timeout *ms*
: Specifies the maximum time to wait for a an operation database engine before reporting an error to the caller. The *ms* argument gives the maximum time in milliseconds. A value of zero (the default) specifies that the calling process is to wait indefinitely for database operations.

-readonly *flag*
: Specifies that the connection will not modify the database (if the Boolean parameter *flag* is true), or that it may modify the database (if *flag* is false). If *flag* is true, this option may have the effect of raising the transaction isolation level to *readonly*.

Transaction isolation levels

The acceptable values for the **-isolation** configuration option are as follows:

readuncommitted
: Allows the transaction to read "dirty", that is, uncommitted data. This isolation level may compromise data integrity, does not guarantee that foreign keys or uniqueness constraints are satisfied, and in general does not guarantee data consistency.

readcommitted
: Forbids the transaction from reading "dirty" data, but does not guarantee repeatable reads; if a transaction reads a row of a database at a given time, there is no guarantee that the same

row will be available at a later time in the same transaction.

repeatableread
Guarantees that any row of the database, once read, will have the same values for the life of a transaction. Still permits "phantom reads" (that is, newly-added rows appearing if a table is queried a second time).

serializable
The most restrictive (and most expensive) level of transaction isolation. Any query to the database, if repeated, will return precisely the same results for the life of the transaction, exactly as if the transaction is the only user of the database.

readonly
Behaves like **serializable** in that the only results visible to the transaction are those that were committed prior to the start of the transaction, but forbids the transaction from modifying the database. A database that does not implement one of these isolation levels will instead use the next more restrictive isolation level. If the given level of isolation cannot be obtained, the database interface throws an error reporting the fact. The default isolation level is **readcommitted**. A script should not the isolation level when a transaction is in progress.

Copyright

Copyright (c) 2008 by Kevin B. Kenny.

Keywords: tdbc_connection, TDBC, SQL, database, connectivity, connection, resultset, statement

tdbc_mapSqlState

Name

tdbc::mapSqlState - Map SQLSTATE to error class

Synopsis

package require tdbc 1.0

tdbc::mapSqlState *sqlstate*

Description

The **tdbc::mapSqlState** command accepts a string that is expected to be a five-character 'SQL state' as returned from a SQL database when an error occurs. It examines the first two characters of the string, and returns an error class as a human- and machine-readable name (for example, **FEATURE_NOT_SUPPORTED**, **DATA_EXCEPTION** or **INVALID_CURSOR_STATE**).

The TDBC specification requires database drivers to return a description of an error in the error code when an error occurs. The description is a string that has at least four elements: "**TDBC** *errorClass sqlstate driverName details...*". The **tdbc::mapSqlState** command gives a convenient way for a TDBC driver to generate the *errorClass* element given the SQL state returned from a database.

Copyright

Copyright (c) 2009 by Kevin B. Kenny.

Keywords: tdbc_mapSqlState, TDBC, SQL, database, state

tdbc_resultset

Name

tdbc::resultset - TDBC result set object

Synopsis

package require tdbc 1.0
package require tdbc::driver version

tdbc::driver::connection create *db* ?*-option value* ...?

set *stmt* **[db prepare** *sql-code]*
set *resultset* **[$stmt execute** ?*args...?]*?*

$resultset **columns**
$resultset **rowcount**
$resultset **nextrow** ?*-as lists|dicts*? ?**--**? *varname*
$resultset **nextlist** *varname*
$resultset **nextdict** *varname*
$resultset **nextresults**
$resultset **foreach** ?*-as lists|dicts*? ?*-columnsvariable name*? ?**--**? *varname script*
$resultset **close**

.in 14 *$resultset* **allrows** ?**-as lists|dicts**? ?**-columnsvariable** *name*? ?**--**?

Description

Every database driver for TDBC (Tcl DataBase Connectivity) implements a *result set* object that represents a the results returned from executing SQL statement in a database. Instances of this object are created by executing the **execute** object command on a statement object.

The **columns** object command returns a list of the names of the columns in the result set. The columns will appear in the same order as they appeared in the SQL statement that performed the database query. If the SQL statement does not return a set of columns (for instance, if it is an INSERT, UPDATE, or DELETE statement), the **columns** command will return an empty list.

The **rowcount** object command returns the number of rows in the database that were affected by the execution of an INSERT, UPDATE or DELETE statement. For a SELECT statement, the row count is unspecified.

The **nextlist** object command sets the variable given by *varname* in the caller's scope to the next row of the results, expressed as a list of column values. NULL values are replaced by empty strings. The columns of the result row appear in the same order in which they appeared on the SELECT statement. The return of **nextlist** is **1** if the operation succeeded, and **0** if the end of the result set was reached.

The **nextdict** object command sets the variable given by *varname* in the caller's scope to the next row of the results, expressed as a dictionary. The dictionary's keys are column names, and the values are the values of those columns in the row. If a column's value in the row is NULL, its key is omitted from the dictionary. The keys appear in the dictionary in the same order in which the columns appeared on the SELECT statement. The return of **nextdict** is **1** if the operation succeeded, and **0** if the end of the result set was reached.

The **nextrow** object command is precisely equivalent to the **nextdict** or **nextlist** object

command, depending on whether **-as dicts** (the default) or **-as lists** is specified.

Some databases support the idea of a single statement that returns multiple sets of results. The **nextresults** object command is executed, typically after the **nextlist** of **nextdict** object command has returned **0**, to advance to the next result set. It returns **1** if there is another result set to process, and **0** if the result set just processed was the last. After calling **nextresults** and getting the return value of **1**, the caller may once again call **columns** to get the column descriptions of the next result set, and then return to calling **nextdict** or **nextlist** to process the rows of the next result set. It is an error to call **columns**, **nextdict**, **nextlist** or **nextrow** after **nextresults** has returned **0**.

The **allrows** object command sets the variable designated by the **-columnsvariable** option (if present) to the result of the **columns** object command. It then executes the **nextrow** object command repeatedly until the end of the result set is reached. If **nextresults** returns a nonzero value, it executes the above two steps (**columns** followed by iterated **nextrow** calls) as long as further results are available. The rows returned by **nextrow** are assembled into a Tcl list and become the return value of the **allrows** command; the last value returned from **columns** is what the application will see in **-columnsvariable**.

The **foreach** object command sets the variable designated by the **-columnsvariable** option (if present) to the result of the **columns** object command. It then executes the **nextrow** object command repeatedly until the end of the result set is reached, storing the successive rows in the variable designated by *varName*. For each row, it executes the given *script*. If the script terminates with an error, the error is reported by the **foreach** command, and iteration stops. If the script performs a **break** operation, the iteration terminates prematurely. If the script performs a **continue** operation, the iteration recommences with the next row. If the script performs a **return**, results are the same as if a script outside the control of **foreach** had returned. Any other unusual return code terminates the iteration and is reported from the **foreach**.

Once **nextrow** returns **0**, the **foreach** object command tries to advance to the next result set using **nextresults**. If **nextresults** returns **1**, the above steps (**columns** and **nextrow**, with script invocation) are repeated as long as more result sets remain. The *script* will always see the correct description of the columns of the current result set in the variable designated byt **-columnsvariable**. At the end of the call, the variable designated by **-columnsvariable** will have the description of the columns of the last result set.

The **close** object command deletes the result set and frees any associated system resources.

Copyright

Copyright (c) 2008 by Kevin B. Kenny.

Keywords: tdbc_resultset, TDBC, SQL, database, connectivity, connection, resultset, statement, bound variable, stored procedure, call

tdbc_statement

Name

tdbc::statement - TDBC statement object

Synopsis

package require tdbc 1.0
package require tdbc::driver version

tdbc::driver::connection create *db* ?*-option value* ...?

set *stmt* **[db prepare** *sql-code]*
set *stmt* **[db preparecall** *call]*

$stmt **params**
$stmt **paramtype** ?*direction*? *type* ?*precision*? ?*scale*?
$stmt **execute** ?*dict*?
$stmt **resultsets**
$stmt **foreach** ?*-as lists|dicts*? ?*-columnsvariable name*? ?**--**? *varName* ?*dict*? *script*
$stmt **close**

.in 14 *$stmt* **allrows** ?**-as lists|dicts**? ?**-columnsvariable** *name*? ?**--**? ?*dict*

Description

Every database driver for TDBC (Tcl DataBase Connectivity) implements a *statement* object that represents a SQL statement in a database. Instances of this object are created by executing the **prepare** or **preparecall** object command on a database connection.

The **prepare** object command against the connection accepts arbitrary SQL code to be executed against the database. The SQL code may contain *bound variables*, which are strings of alphanumeric characters or underscores (the first character of the string may not be numeric), prefixed with a colon (**:**). If a bound variable appears in the SQL statement, and is not in a string set off by single or double quotes, nor in a comment introduced by **--**, it becomes a value that is substituted when the statement is executed. A bound variable becomes a single value (string or numeric) in the resulting statement. *Drivers are responsible for ensuring that the mechanism for binding variables prevents SQL injection.*

The **preparecall** object command against the connection accepts a stylized statement in the form:

```
procname (?:varname? ?,:varname...?)
```

or

```
varname = procname (?:varname? ?,:varname...?)
```

This statement represents a call to a stored procedure *procname* in the database. The variable name to the left of the equal sign (if present), and all variable names that are parameters inside parentheses, become bound variables.

The **params** method against a statement object enumerates the bound variables that appear in the statement. The result returned from the **params** method is a dictionary whose keys are the names of bound variables (listed in the order in which the variables first appear in the statement), and whose values are dictionaries. The subdictionaries include at least the following keys (database drivers may add additional keys that are not in this list).

direction
: Contains one of the keywords, **in**, **out** or **inout** according to whether the variable is an input to or output from the statement. Only stored procedure calls will have **out** or **inout** parameters.

type
Contains the data type of the column, and will generally be chosen from the set, **bigint**, **binary**, **bit**, **char**, **date**, **decimal**, **double**, **float**, **integer**, **longvarbinary**, **longvarchar**, **numeric**, **real**, **time**, **timestamp**, **smallint**, **tinyint**, **varbinary**, and **varchar**. (If the variable has a type that cannot be represented as one of the above, **type** will contain a driver-dependent description of the type.)

precision
Contains the precision of the column in bits, decimal digits, or the width in characters, according to the type.

scale
Contains the scale of the column (the number of digits after the radix point), for types that support the concept.

nullable
Contains 1 if the column can contain NULL values, and 0 otherwise. The **paramtype** object command allows the script to specify the type and direction of parameter transmission of a variable in a statement. (Some databases provide no method to determine this information automatically and place the burden on the caller to do so.) The *direction*, *type*, *precision*, *scale*, and *nullable* arguments have the same meaning as the corresponding dictionary values in the **params** object command. The **execute** object command executes the statement. Prior to executing the statement, values are provided for the bound variables that appear in it. If the *dict* parameter is supplied, it is searched for a key whose name matches the name of the bound variable. If the key is present, its value becomes the substituted variable. If not, the value of the substituted variable becomes a SQL NULL. If the *dict* parameter is *not* supplied, the **execute** object command searches for a variable in the caller's scope whose name matches the name of the bound variable. If one is found, its value becomes the bound variable's value. If none is found, the bound variable is assigned a SQL NULL as its value. Once substitution is finished, the resulting statement is executed. The return value is a result set object (see **tdbc::resultset** for details). The **resultsets** method returns a list of all the result sets that have been returned by executing the statement and have not yet been closed. The **allrows** object command executes the statement as with the **execute** object command, accepting an optional *dict* parameter giving bind variables. After executing the statement, it uses the *allrows* object command on the result set (see **tdbc::resultset**) to construct a list of the results. Finally, the result set is closed. The return value is the list of results. The **foreach** object command executes the statement as with the **execute** object command, accepting an optional *dict* parameter giving bind variables. After executing the statement, it uses the *foreach* object command on the result set (see **tdbc::resultset**) to evaluate the given *script* for each row of the results. Finally, the result set is closed, even if the given *script* results in a **return**, an error, or an unusual return code. The **close** object command removes a statement and any result sets that it has created. All system resources associated with the objects are freed.

Examples

The following code would look up a telephone number in a directory, assuming an appropriate SQL schema:

```
package require tdbc::sqlite3
tdbc::sqlite3::connection create db phonebook.sqlite3
set statement [db prepare {
    select phone_num from directory
    where first_name = :firstname and last_name = :lastname
}]
set firstname Fred
set lastname Flintstone
$statement foreach row {
    puts [dict get $row phone_num]
}
$statement close
db close
```

Copyright

Copyright (c) 2008 by Kevin B. Kenny.

Keywords: tdbc_statement, TDBC, SQL, database, connectivity, connection, resultset, statement, bound variable, stored procedure, call

tdbc_tokenize

Name

tdbc::tokenize - TDBC SQL tokenizer

Synopsis

package require tdbc 1.0

tdbc::tokenize *string*

Description

As a convenience to database drivers, Tcl Database Connectivity (TDBC) provides a command to break SQL code apart into tokens so that bound variables can readily be identified and substituted.

The **tdbc::tokenize** command accepts as its parameter a string that is expected to contain one or more SQL statements. It returns a list of substrings; concatenating these substrings together will yield the original string. Each substring is one of the following:

[1]
A bound variable, which begins with one of the characters '**:**', '**@**', or '**$**'. The remainder of the string is the variable name and will consist of alphanumeric characters and underscores. (The leading character will be be non-numeric.)

[2]
A semicolon that separates two SQL statements.

[3]
Something else in a SQL statement. The tokenizer does not attempt to parse SQL; it merely identifies bound variables (distinguishing them from similar strings appearing inside quotes or comments) and statement delimiters.

Copyright

Copyright (c) 2008 by Kevin B. Kenny.

Keywords: tdbc_tokenize, TDBC, SQL, database, tokenize

tdbc-c

Tdbc_Init

Name

Tdbc_Init, Tdbc_MapSqlState, Tdbc_TokenizeSql - C procedures to facilitate writing TDBC drivers

Synopsis

#include <tdbc.h>

int
Tdbc_Init(*interp*)

Tcl_Obj *
Tdbc_TokenizeSql(*interp*, *sqlcode*)

const char *
Tdbc_MapSqlState(*state*)

Arguments

Tcl_Interp ****interp* **(in/out)
Pointer to a Tcl interpreter.

const char ****state* **(in)
Pointer to a character string containing a 'SQL state' from a database error.

const char ****sqlcode* **(in)
Pointer to a character string containing a SQL statement.

Description

The TDBC library provides several C procedures that simplify writing a TDBC driver. They include a procedure that tokenizes a SQL statement, locating variables to be substituted, and a procedure that accepts a SQL state and returns an error class for the interpreter error information.

Tdbc_Init must be invoked prior to any other TDBC call. It accepts a pointer to a Tcl interpreter, and arranges to load the TDBC library. It returns **TCL_OK** if the Tcl library was loaded successfully, and **TCL_ERROR** otherwise. If **TCL_ERROR** is returned, the interpreter's result contains the error message.

Tdbc_TokenizeSql accepts a pointer to a Tcl interpreter, and a pointer to a character string containing one or more SQL statements. It tokenizes the SQL statements, and returns a pointer to a Tcl_Obj that contains a list of the tokens that make up the statement. Concatenating the tokens together will yield the original SQL code. The returned Tcl_Obj has a reference count of zero. The caller is responsible for managing the reference count as needed. See **TOKENS** below for a description of what may be in the returned list of tokens.

Tdbc_MapSqlState accepts a pointer to a string, usually five characters long, that is the 'SQL

state' that resulted from a database error. It returns a character string that is suitable for inclusion as the error class when constructing the error code for an error in a TDBC driver. (By convention, the error code is a list having at least four elements: "**TDBC** *errorClass sqlstate driverName details...*".)

Tokens

Each token returned from **Tdbc_TokenizeSql** may be one of the following:

[1]
A bound variable, which begins with one of the characters '**:**', '**@**', or '**$**'. The remainder of the string is the variable name and will consist of alphanumeric characters and underscores. (The leading character will be be non-numeric.)

[2]
A semicolon that separates two SQL statements.

[3]
Something else in a SQL statement. The tokenizer does not attempt to parse SQL; it merely identifies bound variables (distinguishing them from similar strings appearing inside quotes or comments) and statement delimiters.

Copyright

Copyright (c) 2009 by Kevin B. Kenny.

Keywords: Tdbc_Init, TDBC, SQL, database, tokenize

tdbcmysql

tdbc_mysql

Name

tdbc::mysql - TDBC-MYSQL bridge

Synopsis

package require tdbc::mysql 1.0
tdbc::mysql::connection create *db* ?*-option value* ...?
tdbc::mysql::connection new ?*-option value* ...?
tdbc::mysql::datasources ?**-system|-user**?
tdbc::mysql::drivers
tdbc::mysql::datasource *command driverName* ?*keyword-value?* ...?

Description

The **tdbc::mysql** driver provides a database interface that conforms to Tcl DataBase Connectivity (TDBC) and allows a Tcl script to connect to a MySQL database.

Connection to an MYSQL database is established by invoking **tdbc::mysql::connection create**, passing it the name to give the database handle and a set of *-option-value* pairs. The available options are enumerated under CONNECTION OPTIONS below. As an alternative, **tdbc::mysql::connection new** may be used to create a database connection with an automatically assigned name. The return value from **tdbc::mysql::connection new** is the name that was chosen for the connection handle.

The side effect of **tdbc::mysql::connection create** is to create a new database connection.. See **tdbc::connection(n)** for the details of how to use the connection to manipulate a database.

Connection options

The **tdbc::mysql::connection create** object command supports the **-encoding**, **-isolation**, **-readonly** and **-timeout** options common to all TDBC drivers. The **-encoding** option will always fail unless the encoding is **utf-8**; the database connection always uses UTF-8 encoding to be able to transfer arbitrary Unicode characters. The **-readonly** option must be **0**, because MySQL does not offer read-only connections.

In addition, the following options are recognized:

-host *hostname*
: Connects to the host specified by *hostname*. This option must be set on the initial creation of the connection; it cannot be changed after connecting. Default is to connect to the local host.

-port *number*
: Connects to a MySQL server listening on the port specified by *number*. This option may not be changed after connecting. It is used only when *host* is specified and is not **localhost**.

-socket *path*
: Connects to a MySQL server listening on the Unix socket or named pipe specified by *path*. This option may not be changed after connecting. It is used only when *-host* is not specified or is **localhost**.

-user *name*
: Presents *name* as the user name to the MySQL server. Default is the current user ID.

-passwd *password*
: ...see next...

-password *password*
: These two options are synonymous. They present the given *password* as the user's password to the MySQL server. Default is not to present a password.

-database *name*
: ...see next...

-db *name*
: These two options are synonymous. They present the given *name* as the name of the default database to use in MySQL queries. If not specified, the default database for the current user is used.

-interactive *flag*
: The *flag* value must be a Boolean value. If it is **true** (or any equivalent), the default timeout is set for an interactive user, otherwise, the default timeout is set for a batch user. This option is meaningful only on initial connection. When using the **configure** method on a MySQL connection use the **-timeout** option to set the timeout desired.

-ssl_ca *string*
: ...see next...

-ssl_capath *string*
: ...see next...

-ssl_cert *string*

...see next...

-ssl_cipher *string*

...see next...

-ssl_key *string*

These five options set the certificate authority, certificate authority search path, SSL certificate, transfer cipher, and SSL key to the given *string* arguments. These options may be specified only on initial connection to a database, not in the **configure** method of an existing connection. Default is not to use SSL.

Examples

```
tdbc::mysql::connection -user joe -passwd sesame -db joes_database
```

Connects to the MySQL server on the local host using the default connection method, presenting user ID 'joe' and password 'sesame'. Uses 'joes_database' as the default database name.

Additional connection methods

In addition to the usual methods on the tdbc::connection(n) object, connections to a MySQL database support one additional method:

$connection* evaldirect *sqlStatement

This method takes the given *sqlStatement* and interprets as MySQL native SQL code and evaluates it without preparing it. The statement may not contain variable substitutions. The result set is returned as a list of lists, with each sublist being the list of columns of a result row formatted as character strings. Note that the string formatting is done by MySQL and not by Tcl, so details like the appearance of floating point numbers may differ. *This command is not recommended* for anything where the usual *prepare* or *preparecall* methods work correctly. It is provided so that data management language statements that are not implemented in MySQL's prepared statement API, such as **CREATE DATABASE** or **CREATE PROCEDURE**, can be executed.

Copyright

Copyright (c) 2009 by Kevin B. Kenny.

Keywords: tdbc_mysql, TDBC, SQL, MySQL, database, connectivity, connection

Copyright (c) 2008 by Kevin B. Kenny.

tdbcodbc

tdbc_odbc

Name

tdbc::odbc - TDBC-ODBC bridge

Synopsis

package require tdbc::odbc 1.0
tdbc::odbc::connection create *db connectionString* ?*-option value* ...?
tdbc::odbc::connection new *connectionString* ?*-option value* ...?
tdbc::odbc::datasources ?**-system|-user**?

tdbc::odbc::drivers
tdbc::odbc::datasource *command driverName* ?*keyword-value?* ...?

Description

The **tdbc::odbc** driver provides a database interface that conforms to Tcl DataBase Connectivity (TDBC) and allows a Tcl script to connect to any SQL database presenting an ODBC interface. It is also provided as a worked example of how to write a database driver in C, so that driver authors have a starting point for further development.

Connection to an ODBC database is established by invoking **tdbc::odbc::connection create**, passing it the name to be used as a connection handle, followed by a standard ODBC connection string. As an alternative, **tdbc::odbc::connection new** may be used to create a database connection with an automatically assigned name. The return value from **tdbc::odbc::connection new** is the name that was chosen for the connection handle.

The connection string will include at least a **DRIVER** or **DSN** keyword, and may include others that are defined by a particular ODBC driver. (If the local ODBC system supports a graphical user interface, the **-parent** option (see below) may allow calling **tdbc::odbc::connection create** with an empty connection string.)

The side effect of **tdbc::odbc::connection create** is to create a new database connection.. See **tdbc::connection(n)** for the details of how to use the connection to manipulate a database.

In addition to a standard TDBC interface, **tdbc::odbc** supports three additional ccommands. The first of these, **tdbc::odbc::datasources**, which returns a Tcl list enumerating the named data sources available to the program (for connection with the **DSN** keyword in the connection string). The result of **tdbc::odbc::datasources** may be constrained to only system data sources or only user data sources by including the **-system** or **-user** options, respectively.

The **tdbc::odbc::drivers** command returns a dictionary whose keys are the names of drivers available for the **DRIVER** keyword in the connection string, and whose values are descriptions of the drivers.

The **tdbc::odbc::datasource** command allows configuration of named data sources on those systems that support the ODBC Installer application programming interface. It accepts a *command*, which specifies the operation to be performed, the name of a *driver* for the database in question, and a set of keyword-value pairs that are interpreted by the given driver. The *command* must be one of the following:

add
: Adds a user data source. The keyword-value pairs must include at least a **DSN** option naming the data source

add_system
: Adds a system data source. The keyword-value pairs must include at least a **DSN** option naming the data source

configure
: Configures a user data source. The keyword-value pairs will usually include a **DSN** option naming the data source. Some drivers will support other options, such as the **CREATE_DB** option to the Microsoft Access driver on Windows.

configure_system

Configures a system data source.

remove
Removes a user data source. The keyword-value pairs must include a **DSN** option specifying the data source to remove.

remove_system
Removes a system data source. The keyword-value pairs must include a **DSN** option specifying the data source to remove.

Connection options

The **tdbc::odbc::connection create** object command supports the **-encoding**, **-isolation**, **-readonly** and **-timeout** options common to all TDBC drivers. The **-encoding** option will succeed only if the requested encoding is the same as the system encoding; **tdbc::odbc** does not attempt to specify alternative encodings to an ODBC driver. (Some drivers accept encoding specifications in the connection string.)

In addition, if Tk is present in the requesting interpreter, and the local system's ODBC driver manager supports a graphical user interface, the **tdbc::odbc::connection create** object command supports a **-parent** option, whose value is the path name of a Tk window. If this option is specified, and a connection string does not specify all the information needed to connect to an interface, the ODBC driver manager will display a dialog box to request whatever additional information is required. The requesting interpreter will block until the user dismisses the dialog, at which point the connection is made.

Examples

Sincs ODBC connection strings are driver specific, it is often difficult to find the documentation needed to compose them. The following examples are known to work on most Windows systems and provide at least a few useful things that a program can do.

```
tdbc::odbc::connection create db \
    "DSN={PAYROLL};UID={aladdin};PWD={Sesame}"
```

Connects to a named data source "PAYROLL", providing "aladdin" as a user name and "Sesame" as a password. Uses **db** as the name of the connection.

```
set connString {DRIVER={Microsoft Access Driver (*.mdb)};}
append connString {FIL={MS Access}\;}
append connString {DBQ=} \
    [file nativename [file normalize $fileName]]
tdbc::odbc::connection create db2 -readonly 1 $connString
```

Opens a connection to a Microsoft Access database file whose name is in *$fileName*. The database is opened in read-only mode. The resulting connection is called "db2".

```
tdbc::odbc::connection create db3 \
    "DRIVER=SQLite3;DATABASE=$fileName"
```

Opens a connection to a SQLite3 database whose name is in "$fileName".

```
tdbc::odbc::datasource add \
    {Microsoft Access Driver (*.mdb)} \
    DSN=MyTestDatabase \
    DBQ=[file native [file normalize $fileName]]
```

Creates a new user data source with the name, "MyTestDatabase" bound to a Microsoft Access file whose path name is in "$fileName". No connection is made to the data source until the program calls **tdbc::odbc::connection create**.

```
tdbc::odbc::datasource configure \
    {Microsoft Access Driver (*.mdb)} \
    CREATE_DB=[file native [file normalize $fileName]] \
    General
```

Creates a new, empty Microsoft Access database in the file identified by "$fileName". No connection is made to the database until the program calls **tdbc::odbc::connection create**.

Additional connection methods

In addition to the usual methods on the tdbc::connection(n) object, connections to an ODBC database support one additional method:

$connection* evaldirect *sqlStatement

This method takes the given driver-native SQL code *sqlStatement* and evaluates it without preparing it. The statement is not tokenized and must not contain variable substitutions. Evaluating the *sqlStatement* produces a result set of zero or more rows. The result of the command is a list of dictionaries, with one list element per row in the result set (in a similar format to the list returned by *$connection allrows -as dicts*). *This command is not recommended* for anything where the usual *prepare* or *preparecall* methods work correctly. It is provided so that data management language statements that are not implemented by the driver's prepared statement API (such as **CREATE DATABASE** or **CREATE PROCEDURE**), or that contain characters that are reserved by the tokenizer, can be executed.

Copyright

Copyright (c) 2008 by Kevin B. Kenny.

Keywords: tdbc_odbc, TDBC, SQL, ODBC, database, connectivity, connection

tdbcpostgres

tdbc_postgres

Name

tdbc::postgres - TDBC-POSTGRES bridge

Synopsis

package require tdbc::postgres 1.0
tdbc::postgres::connection create *db* ?*-option value* ...?
tdbc::postgres::connection new ?*-option value* ...?

Description

The **tdbc::postgres** driver provides a database interface that conforms to Tcl DataBase Connectivity (TDBC) and allows a Tcl script to connect to a Postgres database.

Connection to a POSTGRES database is established by invoking **tdbc::postgres::connection create**, passing it the name to give the database handle and a set of *-option-value* pairs. The available options are enumerated under CONNECTION OPTIONS below. As an alternative, **tdbc::postgres::connection new** may be used to create a database connection with an automatically assigned name. The return value from **tdbc::postgres::connection new** is the name that was chosen for the connection handle.

The side effect of **tdbc::postgres::connection create** is to create a new database connection.. See **tdbc::connection(n)** for the details of how to use the connection to manipulate a database.

Connection options

The **tdbc::postgres::connection create** object command supports the **-encoding**, **-isolation** , **-readonly** and **-timeout** options common to all TDBC drivers. The **-timeout** option will only affect connection process, once connected this value will be ignored and cannot be changed after connecting.

In addition, the following options are recognized (these options must be set on the initial creation of the connection; they cannot be changed after connecting):

-host *hostname*
: Connects to the host specified by *hostname*. Default is to connect using a local Unix domain socket.

-hostaddr *address*
: Connects to the host specified by given IP *address*. If both **-host** and **-hostaddr** are given, the value of **-host** is ignored. Default is to connect using a local Unix domain socket.

-port *number*
: Connects to a Postgres server listening on the port specified by *number*. It is used only when *host* or *hostaddr* is specified.

-user *name*
: Presents *name* as the user name to the Postgres server. Default is the current user ID.

-passwd *password*
: ...see next...

-password *password*
: These two options are synonymous. They present the given *password* as the user's password to the Postgres server. Default is not to present a password.

-database *name*
: ...see next...

-db *name*
: These two options are synonymous. They present the given *name* as the name of the default database to use in Postgres queries. If not specified, the default database for the current user is used.

-options *opts*
: This sets *opts* as additional command line options send to the server.

-tty *file*
This option is ignored on never servers. Formerly this specified where to send debug output. This option is left for compatibility with older servers.

-sslmode *mode*
This option determines whether or with what priority an SSL connection will be negotiated with the server. There are four *modes*: **disable** will attempt only an unencrypted SSL connection; **allow** will negotiate, trying first a non-SSL connection, then if that fails, trying an SSL connection; **prefer** (the default) will negotiate, trying first an SSL connection, then if that fails, trying a regular non-SSL connection; **require** will try only an SSL connection. If PostgreSQL is compiled without SSL support, using option **require** will cause an error, and options **allow** and **prefer** will be tolerated but the driver will be unable to negotiate an SSL connection.

-requiressl *flag*
This option is deprecated in favor of the **-sslmode** setting. The *flag* value must be a Boolean value. If it is **true** (or any equivalent), driver will then refuse to connect if the server does not accept an SSL connection. The default value is **false** (or any equivalent), and acts the same like **-sslmode preffered**

-service *name*
It specifies a service *name* in pg_service.conf file that holds additional connection parameters. This allows applications to specify only a service name so connection parameters can be centrally maintained. Refer to PostgreSQL Documentation or PREFIX/share/pg_service.conf.sample file for details.

Examples

```
tdbc::postgres::connection -user joe -passwd sesame -db joes_database
```

Connects to the Postgres server on the local host using the default connection method, presenting user ID 'joe' and password 'sesame'. Uses 'joes_database' as the default database name.

Copyright

Copyright (c) 2009 by Slawomir Cygan

Keywords: tdbc_postgres, TDBC, SQL, Postgres, database, connectivity, connection

tdbcsqlite3

tdbc_sqlite3

Name

tdbc::sqlite3 - TDBC driver for the SQLite3 database manager

Synopsis

package require tdbc::sqlite3 1.0
tdbc::sqlite3::connection create *db fileName* ?*-option value* ...?

Description

The **tdbc::sqlite3** driver provides a database interface that conforms to Tcl DataBase Connectivity (TDBC) and allows a Tcl script to connect to a SQLite3 database. It is also provided as a worked example of how to write a database driver in Tcl, so that driver authors have a starting point for further development.

Connection to a SQLite3 database is established by invoking **tdbc::sqlite3::connection create** , passing it a string to be used as the connection handle followed by the file name of the database. The side effect of **tdbc::sqlite3::connection create** is to create a new database connection.. As an alternative, **tdbc::sqlite::connection new** may be used to create a database connection with an automatically assigned name. The return value from **tdbc::sqlite::connection new** is the name that was chosen for the connection handle. See **tdbc::connection(n)** for the details of how to use the connection to manipulate a database.

Configuration options

The standard configuration options **-encoding**, **-isolation**, **-readonly** and **-timeout** are all recognized, both on **tdbc::sqlite3::connection create** and on the **configure** method of the resulting connection.

Since the encoding of a SQLite3 database is always well known, the **-encoding** option accepts only **utf-8** as an encoding and always returns **utf-8** for an encoding. The actual encoding may be set using a SQLite3 **PRAGMA** statement when creating a new database.

Only the isolation levels **readuncommitted** and **serializable** are implemented. Other isolation levels are promoted to **serializable**.

The **-readonly** flag is not implemented. A **false** boolean value is accepted silently, while any other value reports an error.

The **-keepcase** flag is special and controls how table and column names are reported in the various methods to retrieve database metadata. By default the flag is zero, i.e. table and column names are reported all lower case. Otherwise, table and column names are reported as defined in the database.

Bugs

If any column name is not unique among the columns in a result set, the results of **-as dicts** returns will be missing all but the rightmost of the duplicated columns. This limitation can be worked around by adding appropriate **AS** clauses to **SELECT** statements to ensure that all returned column names are unique. Plans are to fix this bug by using a C implementation of the driver, which will also improve performance significantly.

Copyright

Copyright (c) 2008 by Kevin B. Kenny.

Keywords: tdbc_sqlite3, TDBC, SQL, SQLite3, database, connectivity, connection

tdom

dom

Name

dom - Create an in-memory DOM tree from XML

Synopsis

package require tdom

dom *method* ?*arg arg* ...?

Description

This command provides the creation of DOM trees in memory. In the usual case a string containing an XML document is parsed and converted into a DOM tree. Other possible parse input may be HTML or JSON. The *method* indicates a specific subcommand.

The valid methods are:

dom parse ?*options*? ?*data*? ?objvar?

Parses the *data* and builds up a DOM tree from that in memory providing a Tcl object command to this DOM document object. Example:

```
dom parse $xml doc
$doc documentElement root
```

This parses the XML data in the variable xml, creates the DOM tree in memory, make a reference to the document object, visible in Tcl as a document object command and assigns this new object command to the variable doc. If the variable doc gets gets out of scope, the DOM tree and the associated Tcl command object (document and all node objects) are freed automatically.

Instead of using the *objvar* argument you may just use the returned object command:

```
set document [dom parse $xml]
set root     [$document documentElement]
```

This creates a persitent DOM tree which is not freed automatically. To free the DOM tree and the related Tcl object commands (document + nodes + fragment nodes) the document object command has to be explicitly deleted by:

```
$document delete
```

or

```
rename $document ""
```

By default no white space only text nodes are inserted into the tree with respecting *xml:space*="*preserve*" attributes (and reverting to default inside such a subtree with *xml:space*="*default*").

The valid options are:

-simple

If *-simple* is specified, a simple but fast parser is used (conforms not fully to XML recommendation). That should double parsing and DOM generation speed. The encoding of the data is not transformed inside the parser. The simple parser does not respect any encoding information in the XML declaration. It skips over the internal DTD subset and ignores any information in it. Therefore, it doesn't include defaulted attribute values into the tree, even if the according attribute declaration is in the internal subset. It also doesn't expand internal or external entity references other than the predefined entities and character references

-html

If *-html* is specified, a fast HTML parser is used, which tries to even parse badly formed HTML into a DOM tree. If the HTML document given to parse does not have a single root element (as it was legal up to HTML 4.01) and the -forest option is not used then a html node will be inserted as document element, with the HTML input data top level elements as children.

-html5

This option is only available if tDOM was build with --enable-html5. Use the *featureinfo* method if you need to know if this feature is build in. If *-html5* is specified, the gumbo lib html5 parser (https://github.com/google/gumbo-parser) is used to build the DOM tree. This is, as far as it goes, XML namespace-aware (which means for example that all HTML elements are in the html5 namespace). Since this probably isn't wanted by a lot of users and adds only burden for no good in a lot of use cases *-html5* can be combined with *-ignorexmlns*, in which case all nodes and attributes in the DOM tree are not in an XML namespace. All tag and attribute names in the DOM tree will be lower case, even for foreign elements not in the xhtml, svg or mathml namespace. The DOM tree may include nodes, that the parser inserted because they are implied by the context (as <head>, <tbody>, etc.). Input longer than 4 GByte byte length is not supported by the underlying gumbo parser.

-json

If *-json* is specified, the *data* is expected to be a valid JSON string (according to RFC 7159). The command returns an ordinary DOM document with nesting token inside the JSON data translated into tree hierarchy. If a JSON array value is itself an object or array then container element nodes named (in a default build) arraycontainer or objectcontainer, respectively, are inserted into the tree. The JSON serialization of this document (with the domDoc method *asJSON*) is the same JSON information as the *data*, preserving JSON datatypes, allowing non-unique member names of objects while preserving their order and the full range of JSON string values. JSON datatype handling is done with an additional property "sticking" at the doc and tree nodes. This property isn't contained in an XML serialization of the document. If you need to store the JSON data represented by a document, store the JSON serialization and parse it back from there. Apart from this JSON type information the returned doc command or handle is an ordinary DOM doc, which may be investigated or modified with the full range of the doc and node methods. Please note that the element node names and the text node values within the tree may be outside of what the appropriate XML productions allow.

-jsonroot <document element name>

If given makes the given element name the document element of the resulting doc. The parsed content of the JSON string will be the children of this document element node.

-jsonmaxnesting *integer*

This option only has effect if used together with the *-json* option. The current implementation uses a recursive descent JSON parser. In order to avoid using excess stack space, any JSON input that has more than a certain levels of nesting is considered invalid. The default maximum nesting is 2000. The option -jsonmaxnesting allows the user to adjust that.

--

The option -- marks the end of options. To give this option isn't strictly necessary even in the case of JSON parsing, for which valid data may start with a "-". If parsing json and if the second to last or last argument start with a "-" and isn't a known option name it will be treated as JSON data.

-keepEmpties
If *-keepEmpties* is specified then text nodes which contain only whitespaces will be part of the resulting DOM tree. In default case (*-keepEmpties* not given) those empty text nodes are removed at parsing time.

-keepCDATA
If *-keepCDATA* is specified then CDATA sections aren't added to the tree as text nodes (and, if necessary, combined with sibling text nodes into one text node) as without this option but are added as CDATA_SECTION_NODEs to the tree. Please note that the resulting tree isn't prepared for XPath selects or to be the source or the stylesheet of an XSLT transformation. If not combined with *-keepEmpties* only not whitespace only CDATA sections will be added to the resulting DOM tree.

-channel <*channel-ID*>
If *-channel <channel-ID>* is specified, the input to be parsed is read from the specified channel. The encoding setting of the channel (via fconfigure -encoding) is respected, ie the data read from the channel are converted to UTF-8 according to the encoding settings before the data is parsed.

-baseurl <*baseURI*>
If *-baseurl <baseURI>* is specified, the baseURI is used as the base URI of the document. External entities references in the document are resolved relative to this base URI. This base URI is also stored within the DOM tree.

-feedbackAfter <*#bytes*>
If *-feedbackAfter <#bytes>* is specified, the Tcl command given by *-feedbackcmd* is evaluated at the first element start within the document (or an external entity) after the start of the document or external entity or the last such call after #bytes. For backward compatibility if no -feedbackcmd is given but there is a Tcl proc named ::dom::domParseFeedback this proc is used as -feedbackcmd. If there isn't such a proc and -feedbackAfter is used it is an error to not also use -feedbackcmd. If the called script raises error, then parsing will be aborted, the *dom parse* call returns error, with the script error msg as error msg. If the called script *return -code break*, the parsing will abort and the *dom parse* call will return the empty string.

-feedbackcmd <*script*>
If *-feedbackcmd <script>* is specified, the script *script* is evaluated at the first element start within the document (or an external entity) after the start of the document or external entity or the last such call after #bytes value given by the *-feedbackAfter* option. If *-feedbackAfter* isn't given, using this option doesn't has any effect. If the called script raises error, then parsing will be aborted, the *dom parse* call returns error, with the script error msg as error msg. If the called script *return -code break*, the parsing will abort and the *dom parse* call will return the empty string.

-externalentitycommand <*script*>
If *-externalentitycommand <script>* is specified, the specified Tcl script is called to resolve any external entities of the document. The actual evaluated command consists of this option followed by three arguments: the base uri, the system identifier of the entity and the public identifier of the entity. The base uri and the public identifier may be the empty list. The script has to return a Tcl list consisting of three elements. The first element of this list signals how the external entity is returned to the processor. Currently the two allowed types are "string"

and "channel". The second element of the list has to be the (absolute) base URI of the external entity to be parsed. The third element of the list are data, either the already read data out of the external entity as string in the case of type "string", or the name of a Tcl channel, in the case of type "channel". Note that if the script returns a Tcl channel, it will not be closed by the processor. It must be closed separately if it is no longer needed.

-useForeignDTD <*boolean*>

If <boolean> is true and the document does not have an external subset, the parser will call the -externalentitycommand script with empty values for the systemId and publicID arguments. Please note that if the document also doesn't have an internal subset, the -startdoctypedeclcommand and -enddoctypedeclcommand scripts, if set, are not called.

-paramentityparsing <*always*|*never*|*notstandalone*>

The *-paramentityparsing* option controls, if the parser tries to resolve the external entities (including the external DTD subset) of the document while building the DOM tree. *-paramentityparsing* requires an argument, which must be either "always", "never", or "notstandalone". The value "always" means that the parser tries to resolves (recursively) all external entities of the XML source. This is the default in case *-paramentityparsing* is omitted. The value "never" means that only the given XML source is parsed and no external entity (including the external subset) will be resolved and parsed. The value "notstandalone" means, that all external entities will be resolved and parsed, with the exception of documents, which explicitly states standalone="yes" in their XML declaration.

-forest

If this option is given, there is no need for a single root; any sequence of well-formed, balanced subtrees will be parsed into a DOM tree. This works for the expat DOM builder, the simple xml parser enabled with *-simple* and the simple HTML parser enabled -with *-html*. If used together with *-json* or *-html5* this option is ignored.

-ignorexmlns

It is recommended, that you only use this option with the *-html5* option. If this option is given, no node within the created DOM tree will be internally marked as placed into an XML Namespace, even if there is a default namespace in scope for un-prefixed elements or even if the element has a defined namespace prefix. One consequence is that XPath node expressions on such a DOM tree doesn't work as may be expected. Prefixed element nodes can't be selected naively and element nodes without prefix will be seen by XPath expressions as if they are not in any namespace (no matter if they are in fact should be in a default namespace). If you need to inject prefixed node names into an XPath expression use the '%' syntax described in the documentation of the of the

domNode

command method **selectNodes**.

-billionLaughsAttackProtectionMaximumAmplification <*float*>

<URL: https://en.wikipedia.org/wiki/Billion_laughs_attack> "" This option together with *-billionLaughsAttackProtectionActivationThreshold* gives control over the parser limits that protects against billion laugh attacks (). This option expects a float >= 1.0 as argument. You should never need to use this option, because the default value (100.0) should work for any real data. If you ever need to increase this value for non-attack payload, please report.

-billionLaughsAttackProtectionActivationThreshold <*long*>

<URL: https://en.wikipedia.org/wiki/Billion_laughs_attack> "" This option together with

-billionLaughsAttackProtectionMaximumAmplification gives control over the parser limits that protects against billion laugh attacks (). This option expects a positiv integer as argument. You should never need to use this option, because the default value (8388608) should work for any real data. If you ever need to increase this value for non-attack payload, please report.

-setReparseDeferralEnabled <*boolean*>
Expat protects itself against quadratic parsing time due to long tokens in malicious crafted documents. This protection may have the side effect that a parsing event is not triggered directly after the end of the respective token but only after the parser has processed some more bytes of the input. This option allows disabling this behavior.

-allocTrackerActivationThreshold <*float*>
This option together with *-allocTrackerMaximumAmplification* gives control for the parser limits to protect against the use of disproportional amounts of dynamic memory due to malicious crafted documents. This option expects a positive integer as argument. You should never need to use this option, because the default value (100.0) should work for any real data. If you ever need to increase this value for non-attack payload, please report.

-allocTrackerMaximumAmplification <*long*>
This option together with *-allocTrackerActivationThreshold* gives control for the parser limits to protect against the use of disproportional amounts of dynamic memory due to malicious crafted documents. This option expects a positive integer as argument. You should never need to use this option, because the default value (67108864, that is 64 MiB) should work for any real data. If you ever need to increase this value for non-attack payload, please report.

dom createDocument *docElemName*?*objVar*?
Creates a new DOM document object with one element node with node name *docElemName*. The *objVar* controls the memory handling as explained above.

dom createDocumentNS *uri docElemName*?*objVar*?
Creates a new DOM document object with one element node with node name *docElemName*. *Uri* gives the namespace of the document element to create. The *objVar* controls the memory handling as explained above.

dom createDocumentNode ?*-jsonType jsonType*? ?*objVar*?
Creates a new 'empty' DOM document object without any element node. *objVar* controls the memory handling as explained above. If the option -jsonType is given the created document node will be of the given JSON type.

dom createFromTypedList *typedList*?*objVar*?
Creates a new DOM document from the argument *typedList*. The *objVar* argument controls the memory handling as explained above.

The *typedList* argument must be a Tcl list and must follow the format of the output of the document command method *asTypedList*, see there.

**dom createNodeCmd*?-returnNodeCmd??-tagName name??-jsonType jsonType?*
?-namespace URI??-noNamespacedAttributes??-notempty?
(element|comment|text|cdata|pi)Node commandName
This method creates Tcl commands, which in turn create tDOM nodes. Tcl commands created by this command are only available inside a script given to the domNode methods *appendFromScript* or *insertBeforeFromScript*. If a command created with *createNodeCmd* is invoked in any other context, it will return error. The created command *commandName*

replaces any existing command or procedure with that name. If the *commandName* includes any Tcl namespace qualifiers, it is created in the specified namespace. The *-tagName* option is only allowed for the elementNode type. The *-jsonType* option is only allowed for elementNode and textNode types.

If such command is invoked inside a script given as argument to the domNode method *appendFromScript* or *insertBeforeFromScript* it creates a new node and appends this node at the end of the child list of the invoking element node. If the option *-returnNodeCmd* was given, the command returns the created node as Tcl command. If this option was omitted, the command returns nothing. Each command creates always the same type of node. Which type of node is created by the command is determined by the first argument to the *createNodeCmd*. The syntax of the created command depends on the type of the node it creates.

If the command type to create is *elementNode*, the created command will create an element node, if called. Though, if the *-notempty* flag was used in creating the node command the element node will only be created if it is not empty. Without the *-tagName* option the tag name of the created node is *commandName* without Tcl namespace qualifiers. If the *-tagName* option was given then the created elements will have the value of this option as tag name. If the *-jsonType* option was given then the created node elements will have the given JSON type. If the *-namespace* option is given the created element node will be XML namespaced and in the namespace given by the option. The element name will be literal as given either by the command name or the *-tagname* option, if that was given. An appropriate XML namespace declaration will be automatically added, to bind the prefix (if the element name has one) or the default namespace (if the element name hasn't a prefix) to the namespace if such a binding isn't in scope.

The syntax of the created command is:

```
elementNodeCmd ?attributeName attributeValue ...? ?script?
elementNodeCmd ?-attributeName attributeValue ...? ?script?
elementNodeCmd ?name_value_list? ?script?
```

The command syntax allows three different ways to specify the attributes of the resulting element. These could be specified with *attributeName attributeValue* argument pairs, in an "option style" way with *-attriubteName attributeValue* argument pairs (the '-' character is only syntactical sugar and will be stripped off) or as a Tcl list with elements interpreted as attribute name and the corresponding attribute value. The attribute name elements in the list may have a leading '-' character, which will be stripped off. If the *elementNodeCmd* is called with only one argument this will be interpreted as content script.

If an *attributeName* is prefixed then by default the prefix will be first looked up in the *selectNodesNamespaces* prefix URI list and, if not found, looked up in the XML namespace scope of the node. If the prefix does not resolve error is raisen. If the prefix resolve then the attribute is created in this namespace (adding XML namespace declarations, if necessary). If *elementNodeCmd* was created with the flag *-noNamespacedAttributes* then no prefix lookup is done and the attribute is created with the name as given and in no XML namespace. It is recommended to not create XML namespace declarations with this.

Every *elementNodeCmd* accepts an optional Tcl script as last argument. This script is evaluated as recursive *appendFromScript* script with the node created by the *elementNodeCmd* as parent of all nodes created by the script.

If the first argument of the method is *textNode*, the command will create a text node. If the *-jsonType* option was given then the created text node will have that JSON type. The syntax of the created command is:

```
textNodeCmd ?-disableOutputEscaping? ?data?
```

If the json type of the created text node is NULL, TRUE or FALSE then the *data* argument is optional, otherwise it this argument must be given.

If the json type of the created text node is the virtual type BOOLEAN then if the text value is a boolean as recognized by Tcl_GetBooleanFromObj() the json value will be the according JSON boolean value. If the text node value is not a boolean value understood by Tcl the value will be written as json string.

If the optional flag *-disableOutputEscaping* is given, the escaping of the ampersand character (&) and the left angle bracket (<) inside the data is disabled. You should use this flag carefully.

If the first argument of the method is *commentNode* or *cdataNode* the command will create an comment node or CDATA section node. The syntax of the created command is:

```
nodeCmd data
```

If the first argument of the method is *piNode*, the command will create a processing instruction node. The syntax of the created command is:

```
piNodeCmd target data
```

Beside the with **dom createNodeCmd** calls created node commands there are two more commands which automatically insert nodes into the tree inside an **appendFromScript** script.

```
tdom::fsnewNode ?-jsonType <jsonType>? ?-namespace <namespace>?
    ?-noNamespacedAttributes? -notempty tagName ?attributes? ?script?
```

If called inside a fromScript context this command creates a new node *tagName* in the XML namespace *namespace* if the *-namespace* option was given and with the JSON type *jsonType* if the *-jsonType* option was given and appends this node at the end of the child list of the invoking element node. The other flags will also work as if given at the creation of an element creating command; see there. The *attributes* and *script* arguments will also be processed as if given to an element creating node command. If called outside a fromScript context this command will raise error.

```
tdom::fsinsertNode node
```

If called inside a fromScript context this command instead of creating a new node appends the as argument given node at the end of the child list of the invoking element node. The node is unlinked from its previous place. If called outside a fromScript context this command will raise error.

dom fromScriptContext ?*objVar*?

If called inside a fromScript context this method returns the node on which the script context appends nodes to the child list. Otherwise it returns error.

If you create nodeCmds with the -notempty the returned node may have vanished outside the evaluation context if it is still empty at the end of the evaluation.

dom setStoreLineColumn*?boolean*?
If switched on, the DOM nodes will contain line and column position information for the original XML document after parsing. The default is not to store line and column position information.

dom setNameCheck*?boolean*?
If NameCheck is true, every method which expects an XML Name, a full qualified name or a processing instructing target will check, if the given string is valid according to its production rule. For commands created with the *createNodeCmd* method to be used in the context of *appendFromScript* the status of the flag at creation time decides. If NameCheck is true at creation time, the command will check its arguments, otherwise not. The *setNameCheck* set this flag. It returns the current NameCheck flag state. The default state for NameCheck is true.

dom setTextCheck*?boolean*?
If TextCheck is true, every command which expects XML Chars, a comment, a CDATA section value or a processing instructing value will check, if the given string is valid according to its production rule. For commands created with the *createNodeCmd* method to be used in the context of *appendFromScript* the status of the flag at creation time decides. If TextCheck is true at creation time, the command will check its arguments, otherwise not.The *setTextCheck* method sets this flag. It returns the current TextCheck flag state. The default state for TextCheck is true.

dom setObjectCommands ?*(automatic|token|command)*?
Controls if documents and nodes are created as Tcl commands or as token to be used with the domNode and domDoc commands. If the mode is 'automatic', then methods used at Tcl commands will create Tcl commands and methods used at doc or node tokes will create tokens. If the mode is 'command' then always Tcl commands will be created. If the mode is 'token', then always token will be created. The method returns the current mode. This method is an experimental interface.

dom isName *name*
Returns 1 if *name* is a valid XML Name according to production 5 of the XML 1.0 recommendation. This means that *name* is a valid XML element or attribute name. Otherwise it returns 0.

dom isPIName *name*
Returns 1 if *name* is a valid XML processing instruction target according to production 17 of the XML 1.0 recommendation. Otherwise it returns 0.

dom isNCName *name*
Returns 1 if *name* is a valid NCName according to production 4 of the of the Namespaces in XML recommendation. Otherwise it returns 0.

dom isQName *name*
Returns 1 if *name* is a valid QName according to production 6 of the of the Namespaces in XML recommendation. Otherwise it returns 0.

dom isCharData *string*
Returns 1 if every character in *string* is a valid XML Char according to production 2 of the XML 1.0 recommendation. Otherwise it returns 0.

dom isHTML5CustomName *string*
Returns 1 if *string* is a HTML5 valid custom element name (according to

https://html.spec.whatwg.org/#valid-custom-element-name at 26 June 2024, it's a living standard). Otherwise it returns 0.

dom clearString ?*-replace*? *string*
Returns the string given as argument cleared out from any characters not allowed as XML parsed character data. If the *-replace* option is given then instead of removing such characters are replaced with \uFFFD.

dom isBMPCharData *string*
Returns 1 if every character in *string* is a valid XML Char with a Unicode code point within the Basic Multilingual Plane (that means, that every character within the string is at most 3 bytes long). Otherwise it returns 0.

dom isComment *string*
Returns 1 if *string* is a valid comment according to production 15 of the XML 1.0 recommendation. Otherwise it returns 0.

dom isCDATA *string*
Returns 1 if *string* is valid according to production 20 of the XML 1.0 recommendation. Otherwise it returns 0.

dom isPIValue *string*
Returns 1 if *string* is valid according to production 16 of the XML 1.0 recommendation. Otherwise it returns 0.

dom jsonEscape *string*
Returns the given *string* argument escaped in a way that if the returned string is used literary in a JSON document it is read by any confirming JSON parser as the original string.

dom featureinfo *feature*
This method provides information about the used build options and the expat version. The valid values for the *feature* argument are:

expatversion
Returns the version of the underlyling expat version as string, something like "exapt_2.1.0". This is what the expat API function XML_ExpatVersion() returns.

expatmajorversion
Returns the major version of the at build time used expat version as integer.

expatminorversion
Returns the minor version of the at build time used expat version as integer.

expatmicroversion
Returns the micro version of the at build time used expat version as integer.

dtd
Returns as boolean if build with *--enable-dtd*.

ns
Returns as boolean if build with *--enable-ns*.

unknown
Returns as boolean if build with *--enable-unknown*.

tdomalloc
Returns as boolean if build with *--enable-tdomalloc*.

lessns
Returns as boolean if build with *--enable-lessns*.

TCL_UTF_MAX
Returns the TCL_UTF_MAX value of the Tcl core, tDOM was build with as integer

html5
Returns as boolean, if build with *--enable-html5*.

versionhash
Returns the fossil repository version hash.

pullparser
Returns as boolean if the pullparser command is build in.

schema
Returns as boolean if the tDOM schema features are build in.

largedata
Returns as boolean if the expat library configuration allows pcdata content of an XML element greater than 2 GByte. Could be true only if build with Tcl 9 for a 64-bit architecture and then depends if the build-in expat version is used with default configuration or if the used system expat library is capable enough for this (typically not).

Keywords: dom, XML, document, node, parsing

domDoc

Name

domDoc - Manipulates an instance of a DOM document object

Synopsis

```
domDocObjCmd method ?arg arg ...?
```

Description

This command manipulates one particular instance of a document object. *method* indicates a specific method of the document class. These methods should closely conform to the W3C recommendation "Document Object Model (Core) Level 1" (http://www.w3.org/TR/REC-DOM-Level-1/level-one-core.html). Look at these documents for a deeper understanding of the functionality.

The valid methods are:

documentElement ?*objVar*?
Returns the top most element in the document (the root element).

getElementsByTagName *name*
Returns a list of all elements in the document matching (glob style) *name*.

getElementsByTagNameNS *uri localname*
Returns a list of all elements in the subtree matching (glob style) *localname* and having the given namespace *uri*.

createElement *tagName*?*objVar*?
Creates (allocates) a new element node with node name *tagName*, append it to the hidden fragment list in the document object and returns the node object. If *objVar* is given the new node object is stored in this variable.

createElementNS *url tagName*?*objVar*?

Creates (allocates) a new element node within a namespace having *uri* as the URI and node name *tagName*, which could include the namespace prefix, append it to the hidden fragment list in the document object and returns the node object. If *objVar* is given the new node object is stored in this variable.

createTextNode *text*?*objVar*?

Creates (allocates) a new text node with node value *text*, appends it to the hidden fragment list in the document object and returns the node object. If *objVar* is given, the new node object is stored in this variable.

createComment *text*?*objVar*?

Creates (allocates) a new comment node with value *text*, appends it to the hidden fragment list in the document object and returns the node object. If *objVar* is given, the new comment node object is stored in this variable.

createCDATASection *data*?*objVar*?

Creates (allocates) a new CDATA node with node value *data*, appends it to the hidden fragment list in the document object and returns the node object. If *objVar* is given, the new node object is stored in this variable.

createProcessingInstruction *target data*?*objVar*?

Creates a process instruction, appends it to the hidden fragment list in the document object and returns the node object. If *objVar* is given, the new node object is stored in this variable.

delete

Explicitly deletes the document, including the associated Tcl object commands (for nodes, fragment/new nodes, the document object itself) and the underlying DOM tree.

getDefaultOutputMethod

Returns the default output method of the document. This is usually a result of a XSLT transformation.

asXML?-indent none/tabs/1..8??-channel channelId??-escapeNonASCII?
?-doctypeDeclaration <boolean>? -xmlDeclaration <boolean>? -encString <string>
?-escapeAllQuot??-indentAttrs??-nogtescape??-noEmptyElementTag??-escapeCR?
?-escapeTab?

Returns the DOM tree as an (optional indented) XML string or sends the output directly to the given channelId.

The -indent option requires "no", "none", "tabs" or a natural number betwenn 0 and 8, both included, as value. With the values "no" or "none" no additional white space outside of markup will be added to the serialization. I. Otherwise, it's a "pretty-print" serialization, due to inserting white space between end and the next start tag according to the nesting level. The level indentation wide is given with the number. If the value is "tabs", then indentation is done with tabs, one tab per level.

If the option *-escapeNonASCII* is given, every non 7 bit ASCII character in attribute values or element PCDATA content will be escaped as character reference in decimal representation.

The flag *-doctypeDeclaration* determines whether there will be a DOCTYPE declaration emitted before the first node of the document. The default is not to emit it. The DOCTYPE name will always be the element name of the document element. An external entity declaration of the external subset is only emitted if the document has a system identifier.

The flag *-xmlDeclaration* determines whether there will be an XML Declaration and a newline

emitted before anything else. The default is not to emit one. If this flag is given with a true argument then

-encString sets the encoding value in the XML Declaration. Otherwise this option is ignored. Please note that this option just enhances the string representation of the generated XML Declaration with an encoding information string, nothing more. It's up to the user to handle encoding in case of writing to a channel or reparsing.

If the option *-escapeAllQuot* is given, quotation marks will be escaped with " even in text content of elements.

If the option *-indentAttrs* is given, then attributes will each be separated with newlines and indented to the same level as the parent node plus the value given as argument to *-indentAttrs* (0..8).

If the option *-nogtescape* is given then the character '>' won't get escaped in attribute values and text content of elements. The default is to escape this character.

If the option *-noEmptyElementTag* is given then no empty tag syntax will be used. Instead, if an element has empty content it will be serialized with an element start tag and an immediately following element end tag.

If the option *-escapeCR* is given then the character '\r' will be escaped as character reference in attribute values and text content of elements. The default is to not do this.

If the option *-escapeTab* is given then the character '\t' will be escaped as character reference in attribute values and text content of elements. The default is to not do this.

asCanonicalXML?-channel channelId??-comments <boolean>?

Returns the DOM tree as canonical XML string according to the "Canonical XML Version 1.0 W3C Recommendation 15 March 2001" or sends the output directly to the given channelId.

If the goal is to get a canonical XML serialization of the XML file from which the DOM tree was parsed there are a few prerequisites. The XML data must be parsed with the *-keepEmpties* option. If the XML data includes a DTD which defines attribute defaults or external parsed entity references it is necessary to use the expat parser (not the *-simple* one). For any supported Tcl version lesser then 9.0 if the XML data includes characters outside the BMP a Tcl build with TCL_UTF_MAX defined to 6 (and a tDOM build with this Tcl) is necessary.

If the -channel option is given then the output is send directly to the Tcl channel given as argument. It is the up to the caller to ensure that the channel is correctly fconfigured. If this option is not given then the command returns the serialization as string.

If the option *-comments* is given with a true value then the serialization includes comments according to the rules of the recommendation. If the value is false or this option is omitted then comments are removed from the serialization.

asHTML?-channel channelId??-escapeNonASCII??-htmlEntities??-doctypeDeclaration <boolean>??-breakLines??-onlyContents?

Returns the DOM tree serialized according to HTML rules (HTML elements are recognized regardless of case, without end tags for empty HTML elements etc.) as string or sends the output directly to the given channelId.

If the option *-escapeNonASCII* is given, every non 7 bit ASCII character in attribute values or element PCDATA content will be escaped as character reference in decimal representation.

If the option *-htmlEntities* is given, a character or a pair of characters is written using its HTML 5 character entity reference, if it has one. Some HTML 5 character entity references encode the same character or code points. From the possible entity names the shortest is chosen. If there are more than one shortest name and this names differ only in case then the lowercase alternative is chosen. Otherwise frist name in lexical ASCII order is chosen. There is one HTML5 entitiy (ThickSpace), which escapes two characters for which the first and the second character have an entity name by itself. If the two characters are to be serialized then the one two-characters entity ThickSpace will be chosen.

If the option *-breakLines* is given the serialization outputs "\n>" instead of ">" for the opening tags of elements.

If the option *-onlyContents* is given only all child nodes are serialized. This option is ignored by document nodes.

If the flag *-doctypeDeclaration* is given there will be a DOCTYPE declaration emitted before the first node of the document. The default is, to do not. The DOCTYPE name will always be the element name of the document element without case normalization. An external entity declaration of the external subset is only emitted, if the document has a system identifier. The doctype declaration will be written from the available information, without check, if this is a known (w3c) HTML version information or if the document confirms to the given HTML version. All nodes types other than document nodes ignore this option.

asText
: The asText method returns the tree by serializing the string-value of every text node in document order without any escaping. In effect, this is what the xslt output method "text" (XSLT 1.0 recommendation, section 16.3) does.

asJSON?-indent none/0..8??-channel channelId?
: The asJSON method serializes the tree into a valid JSON data string. In general, this may be a lossy serialization. For this serialization all comment, character data sections and processing instruction nodes, all attributes and all XML namespaces are ignored. Only element and text nodes may be reflected in the generated JSON serialization. Appropriate JSON data type information of a node will be respected.

If an element node has the JSON type OBJECT, then every element node child of this element will be serialized as member of that object, with the node name of the child as the member name and the relevant children of that child as the value. Every other child nodes will be ignored.

If an element node has the JSON type ARRAY, then the text and element node children of that element node are serialized as the consecutive values of the array. Element node children of an ARRAY element will be container nodes for nested ARRAY or OBJECT values.

Text nodes with the JSON types TRUE, FALSE or NULL will be serialized to the corresponding JSON token without looking at the value of the text node. A text node without JSON type will always be serialized as a JSON string token. A text node with JSON type NUMBER will be serialized as JSON number token if the text node value is in fact a valid JSON number and as a JSON string if not.

If an element node doesn't has a JSON type then the serialization of its children is determined by the following rules:

Only text and element node child are relevant. If the element node to serialize is the member of a JSON object and there is no relevant child node the value of that member will be an empty JSON string. If the only relevant child node of this element node is a text node then the JSON

value of that text node will be the value of the object member. If the element has more than one relevant child nodes and the first one is a text node then the relevant children will be serialized as JSON array. If the only relevant child node is an element node or the first relevant child is an element node and the node name of that only or first relevant child isn't equal to the array container node name all element node children will be serialized as the members of a JSON object (while ignoring any intermixed text nodes). If the only or first relevant child is an element node and the node name of this child is equal to the array container element name then all relevant children will be serialized as the values of a JSON array.

If the element to serialize is a value of a JSON array and the node name of this element isn't equal to the array container node name that element will be seen as a container node for a JSON object and all element node children will be serialized as the members of that array while ignoring any text node children. If the element to serialize is a value of a JSON array and the node name of this element is equal to the array container node name, all relevant children will be serialized as JSON array.

If the *-channel* option is given the serialization isn't returned as string but send directly to the channel, given as argument to the option.

If the *-indent* option is given and the argument given to this option isn't "none" then the returned JSON string is "pretty-printed". The numeric argument to this option defines the number of spaces for any indentation level. The default is to not emit any additional white space.

asTclValue?typevariable?

In case the DOM tree includes JSON type information this method returns the JSON data as nested Tcl data structure.

The returned value may be a Tcl dict, a Tcl list or a string. If the optional argument *typevariable* is given then the variable with that name is set to the value *dict*, *list* or *string* respectively to signal the type of the result.

A JSON object is returned as Tcl dict, a JSON array is returned as list and JSON strings and numbers as well as the symbolic JSON values null, true and false are returned as string (with the strings null, true and false for the respectively JSON symbol). The value of a member of a JSON object may be also a Tcl dict, or a Tcl list or a string and the elements of a JSON array list may be a Tcl dict or a Tcl list or a string.

asTypedList

In case the DOM tree includes JSON type information this method returns the JSON data as a nested Tcl list.

The first element of every of this lists describes the type of the value. The types are: OBJECT, ARRAY, STRING, NUMBER, TRUE, FALSE or NULL.

If the type is NUMBER or STRING, then the second (and last) element is the value. If the type is NULL, TRUE or FALSE the list does not have any other elements.

If the type is OBJECT the second value will be a Tcl list of property name and value pairs, which means the second element could be used as dict. The value will be a Tcl list build by the rules of the *asTypedList* method.

If the type is ARRAY the second value will be a Tcl list of the JSON array values, each one build by the rules of the *asTypedList* method.

publicId*?publicId?*

Returns the public identifier of the doctype declaration of the document, if there is one, otherwise the empty string. If there is a value given to the method, the public identifier of the document is set to this value.

systemId*?systemId?*
Returns the system identifier of the doctype declaration of the document, if there is one, otherwise the empty string. If there is a value given to the method, the system identifier of the document is set to this value.

internalSubset*?internalSubset?*
Returns the internal subset of the doctype declaration of the document, if there is one, otherwise the empty string. If there is a value given to the method, the internal subset of the document is set to this value. Note that none of the parsing methods preserve the internal subset of a document; a freshly parsed document will always have an empty internal subset. Also note that the method doesn't do any syntactical check on a given internal subset.

cdataSectionElements *(?URI:?localname|*) ?<boolean>?*
This method allows one to control for which element nodes the text node children will be serialized as CDATA sections (this affects only serialization with the asXML method, no text node is altered in any way by this method). IF the method is called with an element name as first argument and a boolean with value true as second argument, every text node child of every element node in the document with the same name as the first argument will be serialized as CDATA section. If the second argument is a boolean with value false, all text nodes of all elements with the same name as the first argument will be serialized as usual. Namespaced element names have to be given in the form namespace_URI:localname, not in the otherwise usual prefix:localname form. With two arguments called, the method returns the used boolean value. If the method is called with only an element name, it will return a boolean value, indicating that the text node children of all elements with that name in the document will be serialized as CDATA section elements (return value 1) or not (return value 0). If the method is called with only one argument and that argument is an asterisk ('*'), then the method returns an unordered list of all element names of the document, for which the text node children will be serialized as CDATA section nodes.

selectNodesNamespaces?prefixUriList?
This method gives control to a document global prefix to namespace URI mapping, which will be used for selectNodes method calls (on document as well as on all nodes, which belongs to the document) if it is not overwritten by using the -namespaces option of the selectNodes method. Any namespace prefix within an xpath expression will be first resolved against this list. If the list binds the same prefix to different namespaces, then the first binding will win. If a prefix could not resolved against the document global prefix / namespaces list, then the namespace definitions in scope of the context node will be used to resolve the prefix, as usual. If the optional argument *prefixUriList* is given, then the global prefix / namespace list is set to this list and returns it. Without the optional argument the method returns the current list. The default is the empty list.

xslt?-parameters parameterList??-ignoreUndeclaredParameters??-maxApplyDepth int??-xsltmessagecmd script? *stylesheet?outputVar?*
Applies an XSLT transformation on the whole document of the node object using the XSLT *stylesheet* (given as domDoc). Returns a document object containing the result document of the transformation and stores that document object in the optional *outputVar*, if that was given.

The optional *-parameters* option sets top level <xsl:param> to string values. The *parameterList* has to be a tcl list consisting of parameter name and value pairs.

If the option *-ignoreUndeclaredParameters* is given, then parameter names in the *parameterList* given to the *-parameters* options that are not declared as top-level parameters in the stylesheet are silently ignored. Without this option, an error is raised if the user tries to set a top-level parameter that is not declared in the stylesheet.

The option *-maxApplyDepth* expects a positiv integer as argument. By default, the XSLT engine allows XSLT templates to nest up to 3000 levels (and raises error if they nest deeper). This limit can be set by the *-maxApplyDepth* option.

The *-xsltmessagecmd* option sets a callback for xslt:message elements in the stylesheet. The actual command consists of the script, given as argument to the option, appended with the XML Fragment from instantiating the xsl:message element content as string (as if the XPath string() function would have been applied to the XML Fragment) and a flag, which indicates, if the xsl:message has an attribute "terminate" with the value "yes". If the called script returns anything else then TCL_OK then the XSLT transformation will be aborted, returning error. If the called script returns -code break, the error message is empty, otherwise the result code is reported. In case of terminated transformation, the outputVar, if given, is set to the empty string.

toXSLTcmd ?*objVar*?

If the DOM tree represents a valid XSLT stylesheet, this method transforms the DOM tree into an XSLT command, otherwise it returns error. The created xsltCmd is returned and stored in the *objVar*, if a var name was given. A successful transformation of the DOM tree to an xsltCmd removes the domDoc cmd and all nodeCmds of the document.

The syntax of the created xsltCmd is:

```
xsltCmd method ?arg ...?
```

The valid methods are:

transform?-parameters parameterList??-ignoreUndeclaredParameters?
?-maxApplyDepth int??-xsltmessagecmd script? *domDoc?outputVar?*

Applies XSLT transformation on the document *domDoc*. Returns a document object containing the result document of that transformation and stores it in the optional *outputVar*.

The optional *-parameters* option sets top level <xsl:param> to string values. The *parameterList* has to be a tcl list consisting of parameter name and value pairs.

If the option *-ignoreUndeclaredParameters* is given, then parameter names in the *parameterList* given to the *-parameters* options that are not declared as top-level parameters in the stylesheet are silently ignored. Without this option, an error is raised if the user tries to set a top-level parameter, which is not declared in the stylesheet.

The option *-maxApplyDepth* expects a positiv integer as argument. By default, the XSLT engine allows XSLT templates to nest up to 3000 levels (and raises error if they nest deeper). This limit can be set by the *-maxApplyDepth* option.

The *-xsltmessagecmd* option sets a callback for xslt:message elements in the stylesheet. The actual command consists of the script, given as argument to the option, appended with the XML Fragment from instantiating the xsl:message element content as string (as if the XPath string() function would have been applied to the XML Fragment) and a flag, which indicates, if the

xsl:message has an attribute "terminate" with the value "yes".

delete
Deletes the xsltCmd and cleans up all used recourses If the first argument to an xsltCmd is a domDoc or starts with a "-", then the command is processed in the same way as *<xsltCmd> transform*.

normalize*?-forXPath?*
Puts all text nodes in the document into a "normal" form where only structure (e.g., elements, comments, processing instructions and CDATA sections) separates text nodes, i.e., there are neither adjacent text nodes nor empty text nodes. If the option *-forXPath* is given, all CDATA sections in the nodes are converted to text nodes, as a first step before the normalization.

nodeType
Returns the node type of the document node. This is always DOCUMENT_NODE.

getElementById *id*
Returns the node having a id attribute with value *id* or the empty string, if no node has an id attribute with that value.

firstChild?objVar?
Returns the first top level node of the document.

lastChild?objVar?
Returns the last top level node of the document.

appendChild *newChild*
Append *newChild* to the end of the list of top level nodes of the document.

removeChild *child*
Removes *child* from the list of top level nodes of the document. *child* will be part of the document fragment list after this operation. It is not physically deleted.

hasChildNodes
Returns 1 if the document has any nodes in the tree. Otherwise 0 is returned.

childNodes
Returns a list of the top level nodes of the document.

ownerDocument?domObjVar?
Returns the document itself.

insertBefore *newChild refChild*
Insert *newChild* before the *refChild* into the list of top level nodes of the document. If *refChild* is the empty string, inserts *newChild* at the end of the top level nodes.

replaceChild *newChild oldChild*
Replaces *oldChild* with *newChild* in the list of children of that node. The *oldChild* node will be part of the document fragment list after this operation.

appendFromList *list*
Parses *list*, creates an according DOM subtree and appends this subtree at the end of the current list of top level nodes of the document.

appendXML *XMLstring*
Parses *XMLstring*, creates an according DOM subtree and appends this subtree at the end of the current list of top level nodes of the document.

selectNodes?-namespaces prefixUriList??-cache <boolean>??-list? *xpathQuery* ?typeVar?

Returns the result of applying the XPath query *xpathQuery* to the document. The context node of the query is the root node in the sense of the XPath recommendation (not the document element). The result can be a string/value, a list of strings, a list of nodes or a list of attribute name / value pairs. If *typeVar* is given the result type name is stored into that variable (empty, bool, number, string, nodes, attrnodes or mixed).

See the documentation of the of the

domNode
command method **selectNodes** for a detailed description of the arguments.

baseURI*?URI?*
Returns the present baseURI of the document. If the optional argument URI is given, sets the base URI of the document to the given URI.

appendFromScript *tclScript*
Appends the nodes created by the *tclScript* by Tcl functions, which have been built using *dom createNodeCmd*, at the end of the current list of top level nodes of the document.

insertBeforeFromScript *tclScript refChild*
Inserts the nodes created in the *tclScript* by Tcl functions, which have been built using *dom createNodeCmd*, before the *refChild* into to the list of top level nodes of the document. If *refChild* is the empty string, the new nodes will be appended.

deleteXPathCache*?xpathQuery?*
If called without the optional argument, all cached XPath expressions of the document are freed. If called with the optional argument *xpathQuery*, this single XPath query will be removed from the cache, if it is there. The method always returns an empty string. Otherwise, if an unknown method name is given, the command with the same name as the given method within the namespace **::dom::domDoc** is tried to be executed. This allows quick method additions on Tcl level. Newly created nodes are appended to a hidden fragment list. If they are not moved into the tree they are automatically deleted as soon as the whole document gets deleted.

Keywords: domDoc, DOM node creation, document element

domNode

Name

domNode - Manipulates an instance of a DOM node object

Synopsis

```
$nodeObject method  arg arg ...
```

Description

This command manipulates one particular instance of a DOM node object. *method* indicates a specific method of the node class. These methods should closely conform to the W3C recommendation "Document Object Model (Core) Level 1" (http://www.w3.org/TR/REC-DOM-Level-1/level-one-core.html) as well to parts of the W3C draft "XML Pointer Language (XPointer)" (http://www.w3.org/TR/1998/WD-xptr-19980303). Please note, that the XPointer methods are deprecated. Use DOM methods or XPath expressions instead of

them.

The selectNodes method implements the "XML Path Language (XPath) Version 1.0" W3C recommendation 16 November 1999 (http://www.w3.org/TR/1999/REC-xpath-19991116). Look at these documents for a deeper understanding of the functionality.

The valid methods are:

nodeType
Returns the node type of that node object. This can be: ELEMENT_NODE, TEXT_NODE, CDATA_SECTION_NODE, COMMENT_NODE or PROCESSING_INSTRUCTION_NODE.

nodeName
Returns the node name of that node object. This is the element (tag) name for element nodes (type ELEMENT_NODE), the processing-instruction target for processing-instructions, "#text" for text node, "#comment" for comment nodes or "#cdata" for cdata section nodes.

nodeValue*?newValue?*
Returns the value of that node object. This is the text or the data for element nodes of type TEXT_NODE, COMMENT_NODE, PROCESSING_INSTRUCTION_NODE or CDATA_SECTION_NODE). Otherwise it is empty. If the node is a TEXT_NODE, COMMENT_NODE or PROCESSING_INSTRUCTION_NODE and the optional argument *newValue* is given, the node is set to that value.

hasChildNodes
Returns 1 if the node has children. Otherwise 0 is returned.

parentNode?objVar?
Returns the parent node.

childNodes
Returns a list of direct children node objects.

childNodesLive
Returns a "live" nodeList object of the child nodes of the node in the sense of the DOM recommendation. This nodeList object is "live" in the sense that, for instance, changes to the children of the node object that it was created from are immediately reflected in the nodes returned by the NodeList accessors; it is not a static snapshot of the content of the node. The two accessors known by the nodeList object are "item <index>", which returns the indexth item in the collection, and "length", which returns the number of nodes in the list.

firstChild?objVar?
Returns the first child as a node object.

lastChild?objVar?
Returns the last child as a node object.

nextSibling?objVar?
Returns the next sibling relative to the current node as a node object.

previousSibling?objVar?
Returns the next sibling relative to the current node as a node object.

getElementsByTagName *name*
Returns a list of all elements in the subtree matching (glob style) *name*.

getElementsByTagNameNS *uri localname*
Returns a list of all elements in the subtree matching (glob style) *localname* and having the given namespace *uri*.

getElementById *id*
Returns the node having an id attribute with value *id* or the empty string if no node has an id attribute with that value.

hasAttribute *attributeName*
Returns 1 if the object node contains an attribute with name *attributeName*. Otherwise 0 is returned.

getAttribute *attributeName ?defaultValue?*
Returns the value of the attribute *attributeName*. If the attribute is not available *defaultValue* is returned.

setAttribute *attributeName newValue ?attributeName newValue ...?*
Sets the value for one or more attributes. Every *attributeName* is set to the corresponding *newValue*. If there isn't an attribute for one or more of the *attributeName*, this will create that attribute. It is not recommended to set attributes that look like XML namespace declarations.

removeAttribute *attributeName*
Removes the attribute *attributeName*.

hasAttributeNS *uri localName*
Returns 1 if the object node contains an attribute with the local name *localName* within the namespace *uri*. Otherwise 0 is returned.

getAttributeNS *uri localName ?defaultValue?*
Returns the value of the attribute with the local name *localName* within the namespace URI *uri*. If the node dosn't have that attribute the *defaultValue* is returned.

setAttributeNS *uri qualifiedName newValue ?uri qualifiedName newValue ...?*
Sets the value for one or more full qualified attributes. Every attribute *qualifiedName* with the namespace URI *uri* will be set to *newValue*. This will create a new attribute, if it wasn't available before. If you want to set an attribute within a namespace you must specify the attribute name with prefix, even if you want to set an already existing attribute to a new value. While searching, if the attribute already exists, only the given *uri* and the localname of the *qualifiedName* is used.

```
$node setAttributeNS "http://some.uri.com/wow" prefix:attr1 attrValue
```

If the uri is the empty string and the attribute name has no prefix, this method has the same effect as the method **setAttribute**.

```
$node setAttributeNS "" attri "some Value"
```

With the exceptions of the special prefixes "xmlns" and "xml" you always must provide a non empty *uri* if your *qualifiedName* has a prefix. It is not recommended to set XML namespace declarations. The effects are complicated and not always obvious up to resulting a not well-formed serializations after further processing.

removeAttributeNS *uri localName*
Removes the attribute with the local name *localName* within the namespace *uri*.

attributes?attributeNamePattern?
Returns information about the attributes matching the *attributeNamePattern*. If *attributeNamePattern* isn't given, information about all attributes are returned. The return value is a Tcl list, the elements just the attribute name in case of non namespaced attributes and three element sublists for namespaced attributes. In case of an "ordinary" namespaced

attribute, the sublist elements are {<localname> <prefix> <namespace_uri>}. In the special case of an XML namespace declaration it is {<the prefix defined> <localname> ""}.

attributeNames?attributeNamePattern?
Returns a flat list of all attributes names (as found in the XML source) matching the *attributeNamePattern*. If *attributeNamePattern* isn't given, all attribute names are returned as a Tcl list.

appendChild *newChild*
Appends *newChild* to the end of the child list of the node.

insertBefore *newChild refChild*
Inserts *newChild* before the *refChild* into the list of children of node. If *refChild* is the empty string, insert *newChild* at the end of the child nodes list of that node.

replaceChild *newChild oldChild*
Replaces *oldChild* with *newChild* in the list of children of that node. The *oldChild* node will be part of the document fragment list after this operation.

removeChild *child*
Removes *child* from the list of children of that node. *child* will be part of the document fragment list after this operation.

delete
Deletes the given node and its complete child tree and frees the complete internal memory. The affected nodes are not accessible through the document fragment list.

cloneNode?-deep?
Clones this node and adds the new create node into the document fragment list. If the *-deep* option is specified, all descendant nodes are also cloned.

ownerDocument?domObjVar?
Returns the document object of the document this node belongs to.

find *attrName attrVal*?objVar?
Finds the node with the attribute name *attrName*, and attribute value *attrVal* in the subtree starting the current node.

child *number*|*all* type attrName attrValue
(XPointer) child

descendant *number*|*all* type attrName attrValue
(XPointer) descendant

ancestor *number*|*all* type attrName attrValue
(XPointer) ancestor

fsibling *number*|*all* type attrName attrValue
(XPointer) fsibling

psibling *number*|*all* type attrName attrValue
(XPointer) psibling

root objVar
(XPointer) root

text
Returns all text node children of that current node combined, i.e. appended into one string.

target
For a processing instruction node the target part is returned. Otherwise an error is generated.

data
For a processing instruction node the data part is returned. For a text node, comment node or cdata section node the value is returned. Otherwise an error is generated.

prefix
Returns the namespace prefix.

namespaceURI
Returns the namespace URI.

localName
Returns the localName from the tag name of the given node.

selectNodes?-namespaces prefixUriList??-cache <boolean>??-list? *xpathQuery* ?typeVar?
Returns the result of applying the XPath query *xpathQuery* to the subtree. This result can be a string/value, a list of strings, a list of nodes or a list of attribute name / value pairs. If *typeVar* argument is given the result type name is stored into that variable (empty, bool, number, string, nodes, attrnodes or mixed).

The argument *xpathQuery* has to be a valid XPath 1.0 expression. However there are a few exceptions to that rule. Tcl variable references (in the usual tcl syntax: $varname) may appear in the XPath statement at any position where it is legal according to the rules of the XPath syntax to put an XPath variable. Ignoring the syntax rules of XPath the Tcl variable name may be any legal Tcl var name: local variables, global variables, array entries and so on. The value will always be seen as string literal by the xpath engine. Cast the value explicitly with the according xpath functions (number(), boolean()) to another data type, if needed.

Similar to the way described above to inject literals in a secure way into the XPath expression using tcl variable references there is a syntax to inject element names from tcl variables. At every place where the XPath syntax allows a node test there could be a Tcl variable reference (in any form), just the leading $ replaced with %. This allows one to select nodes with 'strange' (invalid, according to the appropriate XML production rule) node names which may be needed in case of working with JSON data.

The option *-namespaces* expects a Tcl list with prefix / namespace pairs as argument. If this option is not given, then any namespace prefix within the xpath expression will be first resolved against the list of prefix / namespace pairs set with the selectNodesNamespaces method for the document, the node belongs to. If this fails, then the namespace definitions in scope of the context node will be used to resolve the prefix. If this option is given, any namespace prefix within the xpath expression will be first resolved against that given list (and ignoring the document global prefix / namespace list). If the list binds the same prefix to different namespaces, then the first binding will win. If this fails, then the namespace definitions in scope of the context node will be used to resolve the prefix, as usual.

If the *-cache* option is used with a true value, then the *xpathQuery* will be looked up in a document specific cache. If the query is found, then the stored pre-compiled query will be used. If the query isn't found, it will be compiled and stored in the cache, for use in further calls. Please note that the *xpathQuery* given as string is used as key for the cache. This means, that equal XPath expressions, which differ only in white space are treated as different cache entries. Special care is needed, if the XPath expression includes namespace prefixes or references to Tcl variables. Both namespace prefixes and Tcl variable references will be resolved according to the

XML prefix namespace mappings and Tcl variable values at expression compilation time. If the same XPath expression is used later on in a context with other XML prefix namespace mappings or values of the used Tcl variables, make sure to first remove the compiled expression from the cache with the help of the **deleteXPathCache** method, to force a recompilation. Without using the *-cache* option such consideration is never needed.

If the *-list* option is given then the *xpathQuery* argument is expected to be a Tcl list of XPath queries. Every XPath query in this list other than the last one must return a node set result. Using the context and namespace resolution rules as without the *-list* option the first query out of the list is run. Every node out of the result set of this query is used as context node for the next XPath query out of the list and so on. It returns the result sets of the last query in the query list concatenated together.

Examples:

```
set paragraphNodes [$node selectNodes
    {chapter[3]//para[@type='warning' or @type='error'} ]
foreach paragraph $paragraphNodes {
    lappend  values [$paragraph selectNodes attribute::type]
}

set doc [dom parse {<doc
    xmlns="http://www.defaultnamespace.org"><child/></doc>}]
set root [$doc documentElement]
set childNodes [$root selectNodes -namespaces {default
    http://www.defaultnamespace.org} default:child]
```

getLine
Returns the line number of that node in the originally parsed XML. The counting starts with 1

getColumn
Returns the column number of that node in the originally parsed XML. The counting starts with 0

getByteIndex
Returns the byte position of that node in the originally parsed XML. The counting starts with 0.

asList
Returns the DOM substree starting from the current node as a nested Tcl list.

asXML?-indent none/tabs/1..8??-channel channelId??-escapeNonASCII? ?-doctypeDeclaration <boolean>? -xmlDeclaration <boolean>? -encString <string> ?-escapeAllQuot??-indentAttrs??-nogtescape??-noEmptyElementTag??-escapeCR? ?-escapeTab?
Returns the DOM substree starting from the current node as the root node of the result as an (optional indented) XML string or sends the output directly to the given channelId.

See the documentation of the

domDoc
command method **asXML** for a detailed description of the arguments.

asCanonicalXML?-channel channelId??-comment?
Returns the DOM tree as canonical XML string according to the "Canonical XML Version 1.0

W3C Recommendation 15 March 2001" or sends the output directly to the given channelId.

See the documentation of the

domDoc
command method **asCanonicalXML** for a detailed description of the arguments.

asHTML?-channel channelId? b?-escapeNonASCII??-htmlEntities??-doctypeDeclaration <boolean>??-breakLines??-onlyContents?
Returns the DOM substree starting from the current node as the root node of the result serialized according to HTML rules (HTML elements are recognized regardless of case, without end tags for empty HTML elements etc.), as string or sends the output directly to the given channelId.

See the documentation of the

domDoc
method **asHTML** for a detailed description of the arguments.

asText
For ELEMENT_NODEs, the asText method outputs the string-value of every text node descendant of node in document order without any escaping. For every other node type, this method outputs the XPath string value of that node.

asJSON?-indent none/0..8??-channel channelId?
The asJSON method serializes the subtree starting with the node the method was called on into a valid JSON data string.

See the documentation of the

domDoc
method **asJSON** for a detailed description of the method

asTclValue?typevariable?
In case the subtree starting with the node the method was called on includes JSON type information this method returns the JSON data as nested Tcl data structure.

See the documentation of the

domDoc
method **asTclValue** for a detailed description of the method

appendFromList *list*
Parses *list*, creates an according DOM subtree and appends this subtree to the current node.

appendFromScript *tclScript*
Appends the nodes created in the *tclScript* by Tcl functions, which have been built using *dom createNodeCmd*, to the given node.

insertBeforeFromScript *tclScript refChild*
Inserts the nodes created in the *tclScript* by Tcl functions, which have been built using *dom createNodeCmd*, before the *refChild* into the list of children of node. If *refChild* is the empty string, the new nodes will be appended.

appendXML *XMLstring*
Parses *XMLstring*, creates an according DOM subtree and appends this subtree to the current node.

simpleTranslate *outputVar specifications*

Translates the subtree starting at the object node according to the specifications in *specifications* and outputs the result in the variable *outputVar*. The translation is very similar to Cost Simple mode.

toXPath*?-legacy?*

Returns an XPath, which exactly addresses the given node in its document. This XPath is only valid as there are no changes to DOM tree made later one. With the -legacy option, other XPath expressions are returned, which doesn't work in all cases.

getBaseURI

Returns the baseURI of the node. This method is deprecated in favor of the *baseURI* method.

baseURI*?URI?*

Returns the present baseURI of the node. If the optional argument URI is given, it sets the base URI of the node and of all of its child nodes out of the same entity as node to the given URI.

disableOutputEscaping*?boolean?*

This method works only for text nodes; for every other node it returns error. Without the optional argument it returns, if disabling output escaping is on. The return value 0 means, the characters of the text node will be escaped, to generate valid XML, if serialized. This is the default for every parsed or created text node (with the exception of that text nodes in a result tree of an XSLT transformation, for which disabling output escaping was requested explicitly in the stylesheet). The return value 1 means, that output escaping is disabled for this text node. If such a text node is serialized (with asXML or asHTML), it is literally written, without escaping of the special XML characters. If the optional boolean value *boolean* is given, the flag is set accordingly. You should not set this flag to 1 until you really know what you do.

precedes *refnode*

Compares the relative order of the node and *refnode*. Both nodes must be part of the same documents and not out of the fragment list of the document. Returns true if node is in document order (in the sense of the XPath 1.0 recommendation) before *refnode*, and false otherwise.

normalize*?-forXPath?*

Puts all Text nodes in the full depth of the sub-tree underneath this Node into a "normal" form where only structure (e.g., elements, comments, processing instructions and CDATA sections) separates Text nodes, i.e., there are neither adjacent Text nodes nor empty Text nodes. If the option *-forXPath* is given, all CDATA sections in the nodes are converted to text nodes, as a first step before the normalization.

xslt?-parameters parameterList??-ignoreUndeclaredParameters??-maxApplyDepth int??-xsltmessagecmd script? *stylesheet?outputVar?*

Applies an XSLT transformation on the document using the XSLT *stylesheet* (given as domDoc). Returns a document object containing the result document of that transformation and stores it in the optional *outputVar*.

The optional *-parameters* option sets top level <xsl:param> to string values. The *parameterList* has to be a Tcl list consisting of parameter name and value pairs.

If the option *-ignoreUndeclaredParameters* is given, then parameter names in the *parameterList* given to the *-parameters* options that are not declared as top-level parameters in the stylesheet are silently ignored. Without this option, an error is raised if the user tries to set a top-level

parameter which is not declared in the stylesheet.

The option *-maxApplyDepth* expects a positive integer as argument. By default, the XSLT engine allows XSLT templates to nest up to 3000 levels (and raises error if they nest deeper). This limit can be set by the *-maxApplyDepth* option.

The *-xsltmessagecmd* option sets a callback for xslt:message elements in the stylesheet. The actual command consists of the script, given as argument to the option, appended with the XML Fragment from instantiating the xsl:message element content as string (as if the XPath string() function would have been applied to the XML Fragment) and a flag, which indicates whether the xsl:message has an attribute "terminate" with the value "yes". If the called script returns anything else then TCL_OK then the XSLT transformation will be aborted, returning error. If the called script returns -code break the error message is empty, otherwise the result code is reported. In case of terminated transformation the outputVar, if given, is set to the empty string.

@attrName
Returns the value of the attribute *attrName*. Short cut for *getAttribute*.

jsonType*?(OBJECT|ARRAY|NONE)|(STRING|NUMBER|TRUE|FALSE|NULL|NONE)?*
Only element and text nodes may have a JSON type and only this types of nodes support the *jsonType* method; the other node types return error if called with this method. Returns the jsonType of the node. If the optional argument is given, the JSON type of the node is set to the given type and returned. Valid type arguments for element nodes are OBJECT, ARRAY and NONE. Valid type arguments for text nodes are STRING, NUMBER, TRUE, FALSE, NULL and NONE. Otherwise, if an unknown method name is given, the command with the same name as the given method within the namespace **::dom::domNode** is tried to be executed. This allows quick method additions on Tcl level.

Keywords: domNode, XML, DOM, document, node, parsing

expat

Name

expat - Creates an instance of an expat parser object

Synopsis

package require tdom

expat ?*parsername*? ?*-namespace*? ?*arg arg..*?

xml::parser ?*parsername*? ?*-namespace*? ?*arg arg..*?

Description

The parser created with *expat* or *xml::parser* (which is just another name for the same command in an own namespace) are able to parse any kind of well-formed XML. The parsers are stream oriented XML parser. This means that you register handler scripts with the parser prior to starting the parse. These handler scripts are called when the parser discovers the associated structures in the document being parsed. A start tag is an example of the kind of structures for which you may register a handler script.

The parsers always check for XML well-formedness of the input (and report error, if the input isn't well-formed). They parse the internal DTD and, at request, external DTD and external entities, if you resolve the identifier of the external entities with the -externalentitycommand

script (see there). If you use the -validateCmd option (see there), the input is additionally validated.

Additionly, the Tcl extension code that implements this command provides an API for adding C level coded handlers. Up to now, there exists the parser extension command "tdom". The handler set installed by this extension build an in memory "tDOM" DOM tree, while the parser is parsing the input.

It is possible to register an arbitrary amount of different handler scripts and C level handlers for most of the events. If the event occurs, they are called in turn.

Command options

-namespace
: Enables namespace parsing. You must use this option while creating the parser with the **expat** or **xml::parser** command. You can't enable (nor disable) namespace parsing with **<parserobj> configure ...**.

-namespaceseparator *char*
: This option has only effect, if used together with the option *-namespace*. If given, this option determines the character inserted between namespace URI and the local name, while reporting an XML element name to a handler script. The default is the character ':'. The value must be a one-character string less or equal to \u00FF, preferably a 7-bit ASCII character or the empty string. If the value is the empty string (as well, as if the value is \x00) the namespace URI and the local name will be concatenated without any separator.

-final *boolean*
: This option indicates whether the document data next presented to the parse method is the final part of the document. A value of "0" indicates that more data is expected. A value of "1" indicates that no more is expected. The default value is "1".

If this option is set to "0" then the parser will not report certain errors if the XML data is not well-formed upon end of input, such as unclosed or unbalanced start or end tags. Instead some data may be saved by the parser until the next call to the parse method, thus delaying the reporting of some of the data.

If this option is set to "1" then documents which are not well-formed upon end of input will generate an error.

-validateCmd <*tdom schema cmd*>
: This option expects the name of a tDOM schema command. If this option is given, then the input is also validated. If the schema command hasn't set a reportcmd then the first validation error will stop further parsing (as a well-formedness error).

-baseurl *url*
: Reports the base url of the document to the parser.

-fastcall *boolean*
: By default this option is 0 (off). If this option is on then any handler installed by the options *-elementstartcommand*, *-elementendcommand* and *-characterdatacommand* from this moment on is installed in a way that calling it has lesser overhead as usual. This option may be switched between callback handler (re-)installion as desired.

However this has some requirenments. The handler proc has to be defined before used as callback. If it is not then the callback is installed as ususal. And the handler proc must not be

removed or re-defined as long as it is used as callback. If this is done the program probably will crash but also may execute arbitrary code.

The callback handler installed while this option is on will not be traced by executing them.

-keepTextStart *boolean*
By default this option is 0 (off). If this option is on then the position information of the start of a text or CDATA node is keeped over collecting the sometimes by expat delivered cdata pieces. With this option on the position information returned by the parser in a -characterdatacommand proc will be correct, otherwise not. Called in all other handler code the parser always return the correct position results, no matter what value this option have. It is off by default because it is rarely needed and saves a few cpu cyles this way.

-elementstartcommand *script*
Specifies a Tcl command to associate with the start tag of an element. The actual command consists of this option followed by at least two arguments: the element type name and the attribute list.

The attribute list is a Tcl list consisting of name/value pairs, suitable for passing to the array set Tcl command.

Example:

```
proc HandleStart {name attlist} {
    puts stderr "Element start ==> $name has attributes $attlist"
}

$parser configure -elementstartcommand HandleStart

$parser parse {<test id="123"></test>}
```

This would result in the following command being invoked:

```
HandleStart text {id 123}
```

-elementendcommand *script*
Specifies a Tcl command to associate with the end tag of an element. The actual command consists of this option followed by at least one argument: the element type name. In addition, if the -reportempty option is set then the command may be invoked with the -empty configuration option to indicate whether it is an empty element. See the description of the -reportempty option for an example.

Example:

```
proc HandleEnd {name} {
    puts stderr "Element end ==> $name"
}

$parser configure -elementendcommand HandleEnd

$parser parse {<test id="123"></test>}
```

This would result in the following command being invoked:

```
HandleEnd test
```

-characterdatacommand *script*
Specifies a Tcl command to associate with character data in the document, ie. text. The actual command consists of this option followed by one argument: the text.

Other than with the C API of the expat parser it is guaranteed that character data will be passed to the application in a single call to this command.

Example:

```
proc HandleText {data} {
    puts stderr "Character data ==> $data"
}

$parser configure -characterdatacommand HandleText

$parser parse {<test>this is a test document</test>}
```

This would result in the following command being invoked:

```
HandleText {this is a test document}
```

-processinginstructioncommand *script*
Specifies a Tcl command to associate with processing instructions in the document. The actual command consists of this option followed by two arguments: the PI target and the PI data.

Example:

```
proc HandlePI {target data} {
    puts stderr "Processing instruction ==> $target $data"
}

$parser configure -processinginstructioncommand HandlePI

$parser parse {<test><?special this is a processing
    instruction?></test>}
```

This would result in the following command being invoked:

```
HandlePI special {this is a processing instruction}
```

-notationdeclcommand *script*
Specifies a Tcl command to associate with notation declaration in the document. The actual command consists of this option followed by four arguments: the notation name, the base uri of the document (this means, whatever was set by the -baseurl option), the system identifier and the public identifier. The notation name is never empty, the other arguments may be.

-externalentitycommand *script*
Specifies a Tcl command to associate with references to external entities in the document. The actual command consists of this option followed by three arguments: the base uri, the system identifier of the entity and the public identifier of the entity. The base uri and the public identifier may be the empty list.

This handler script has to return a tcl list consisting of three elements. The first element of this list signals, how the external entity is returned to the processor. At the moment, the three allowed types are "string", "channel" and "filename". The second element of the list has to be the (absolute) base URI of the external entity to be parsed. The third element of the list are data, either the already read data out of the external entity as string in the case of type "string", or the name of a tcl channel, in the case of type "channel", or the path to the external entity to be read in case of type "filename". Behind the scene, the external entity referenced by the returned Tcl channel, string or file name will be parsed with an expat external entity parser with the same handler sets as the main parser. If parsing of the external entity fails, the whole parsing is stopped with an error message. If a Tcl command registered as externalentitycommand isn't able to resolve an external entity it is allowed to return TCL_CONTINUE. In this case, the wrapper give the next registered externalentitycommand a try. If no externalentitycommand is able to handle the external entity parsing stops with an error.

Example:

```
proc externalEntityRefHandler {base systemId publicId} {
    if {![regexp {^[a-zA-Z]+:/} $systemId]}  {
        regsub {^[a-zA-Z]+:} $base {} base
        set basedir [file dirname $base]
        set systemId "[set basedir]/[set systemId]"
    } else {
        regsub {^[a-zA-Z]+:} $systemId systemId
    }
    if {[catch {set fd [open $systemId]}]} {
        return -code error \
                -errorinfo "Failed to open external entity $systemId"
    }
    return [list channel $systemId $fd]
}

set parser [expat -externalentitycommand externalEntityRefHandler \
                  -baseurl "file:///local/doc/doc.xml" \
                  -paramentityparsing notstandalone]
$parser parse {<?xml version='1.0'?>
<!DOCTYPE test SYSTEM "test.dtd">
<test/>}
```

This would result in the following command being invoked:

```
externalEntityRefHandler file:///local/doc/doc.xml test.dtd {}
```

External entities are only tried to resolve via this handler script, if necessary. This means, external parameter entities triggers this handler only, if -paramentityparsing is used with argument "always" or if -paramentityparsing is used with argument "notstandalone" and the document isn't marked as standalone.

-unknownencodingcommand *script*
Not implemented at Tcl level.

-startnamespacedeclcommand *script*
Specifies a Tcl command to associate with start scope of namespace declarations in the document. The actual command consists of this option followed by two arguments: the namespace prefix and the namespace URI. For an xmlns attribute, prefix will be the empty list. For an xmlns="" attribute, uri will be the empty list. The call to the start and end element handlers occur between the calls to the start and end namespace declaration handlers.

-endnamespacedeclcommand *script*
Specifies a Tcl command to associate with end scope of namespace declarations in the document. The actual command consists of this option followed by the namespace prefix as argument. In case of an xmlns attribute, prefix will be the empty list. The call to the start and end element handlers occur between the calls to the start and end namespace declaration handlers.

-commentcommand *script*
Specifies a Tcl command to associate with comments in the document. The actual command consists of this option followed by one argument: the comment data.

Example:

```
proc HandleComment {data} {
    puts stderr "Comment ==> $data"
}

$parser configure -commentcommand HandleComment

$parser parse {<test><!-- this is <obviously> a comment --></test>}
```

This would result in the following command being invoked:

```
HandleComment { this is <obviously> a comment }
```

-notstandalonecommand *script*
This Tcl command is called, if the document is not standalone (it has an external subset or a reference to a parameter entity, but does not have standalone="yes"). It is called with no additional arguments.

-startcdatasectioncommand *script*
Specifies a Tcl command to associate with the start of a CDATA section. It is called with no additional arguments.

-endcdatasectioncommand *script*
Specifies a Tcl command to associate with the end of a CDATA section. It is called with no additional arguments.

-elementdeclcommand *script*
Specifies a Tcl command to associate with element declarations. The actual command consists of this option followed by two arguments: the name of the element and the content model. The content model arg is a tcl list of four elements. The first list element specifies the type of the XML element; the six different possible types are reported as "MIXED", "NAME", "EMPTY", "CHOICE", "SEQ" or "ANY". The second list element reports the quantifier to the content model in XML Syntax ("?", "*" or "+") or is the empty list. If the type is "MIXED", then the quantifier will be "{}", indicating an PCDATA only element, or "*", with the allowed

elements to intermix with PCDATA as tcl list as the fourth argument. If the type is "NAME", the name is the third arg; otherwise the third argument is the empty list. If the type is "CHOICE" or "SEQ" the fourth argument will contain a list of content models build like this one. The "EMPTY", "ANY", and "MIXED" types will only occur at top level.

Examples:

```
proc elDeclHandler {name content} {
     puts "$name $content"
}

set parser [expat -elementdeclcommand elDeclHandler]
$parser parse {<?xml version='1.0'?>
<!DOCTYPE test [
<!ELEMENT test (#PCDATA)>
]>
<test>foo</test>}
```

This would result in the following command being invoked:

```
test {MIXED {} {} {}}

$parser reset
$parser parse {<?xml version='1.0'?>
<!DOCTYPE test [
<!ELEMENT test (a|b)>
]>
<test><a/></test>}
```

This would result in the following command being invoked:

```
elDeclHandler test {CHOICE {} {} {{NAME {} a {}} {NAME {} b {}}}}
```

-attlistdeclcommand *script*

Specifies a Tcl command to associate with attlist declarations. The actual command consists of this option followed by five arguments. The Attlist declaration handler is called for *each* attribute. So a single Attlist declaration with multiple attributes declared will generate multiple calls to this handler. The arguments are the element name this attribute belongs to, the name of the attribute, the type of the attribute, the default value (may be the empty list) and a required flag. If this flag is true and the default value is not the empty list, then this is a "#FIXED" default.

Example:

```
proc attlistHandler {elname name type default isRequired} {
    puts "$elname $name $type $default $isRequired"
}

set parser [expat -attlistdeclcommand attlistHandler]
$parser parse {<?xml version='1.0'?>
```

```
<!DOCTYPE test [
<!ELEMENT test EMPTY>
<!ATTLIST test
          id      ID      #REQUIRED
          name    CDATA   #IMPLIED>
]>
<test/>}
```

This would result in the following commands being invoked:

```
attlistHandler test id ID {} 1
attlistHandler test name CDATA {} 0
```

-startdoctypedeclcommand *script*
Specifies a Tcl command to associate with the start of the DOCTYPE declaration. This command is called before any DTD or internal subset is parsed. The actual command consists of this option followed by four arguments: the doctype name, the system identifier, the public identifier and a boolean, that shows if the DOCTYPE has an internal subset.

-enddoctypedeclcommand *script*
Specifies a Tcl command to associate with the end of the DOCTYPE declaration. This command is called after processing any external subset. It is called with no additional arguments.

-paramentityparsing *never*|*notstandalone*|*always*
"never" disables expansion of parameter entities, "always" expands always and "notstandalone" only, if the document isn't "standalone='no'". The default ist "never"

-entitydeclcommand *script*
Specifies a Tcl command to associate with any entity declaration. The actual command consists of this option followed by seven arguments: the entity name, a boolean identifying parameter entities, the value of the entity, the base uri, the system identifier, the public identifier and the notation name. According to the type of entity declaration some of this arguments may be the empty list.

-ignorewhitecdata *boolean*
If this flag is set, element content which contain only whitespaces isn't reported with the **-characterdatacommand**.

-ignorewhitespace *boolean*
Another name for *-ignorewhitecdata*; see there.

-handlerset *name*
This option sets the Tcl handler set scope for the configure options. Any option value pair following this option in the same call to the parser are modifying the named Tcl handler set. If you don't use this option, you are modifying the default Tcl handler set, named "default".

-noexpand *boolean*
Normally, the parser will try to expand references to entities defined in the internal subset. If this option is set to a true value this entities are not expanded, but reported literal via the default handler. **Warning:** If you set this option to true and doesn't install a default handler (with the -defaultcommand option) for every handler set of the parser all internal entities are silent lost for the handler sets without a default handler.

-useForeignDTD <*boolean*>
If <boolean> is true and the document does not have an external subset, the parser will call the -externalentitycommand script with empty values for the systemId and publicID arguments. This option must be set, before the first piece of data is parsed. Setting this option, after the parsing has started has no effect. The default is not to use a foreign DTD. The default is restored, after resetting the parser. Peace notice, that a -paramentityparsing value of "never" (which is the default) suppresses any call to the -externalentitycommand script. Peace notice, that, if the document also doesn't have an internal subset, the -startdoctypedeclcommand and enddoctypedeclcommand scripts, if set, are not called.

-billionLaughsAttackProtectionMaximumAmplification <*float*>
<URL: https://en.wikipedia.org/wiki/Billion_laughs_attack> "" This option together with *-billionLaughsAttackProtectionActivationThreshold* gives control over the parser limits that protects against billion laugh attacks (). This option expects a float >= 1.0 as argument. You should never need to use this option, because the default value (100.0) should work for any real data. If you ever need to increase this value for non-attack payload, please report.

-billionLaughsAttackProtectionActivationThreshold <*long*>
<URL: https://en.wikipedia.org/wiki/Billion_laughs_attack> "" This option together with *-billionLaughsAttackProtectionMaximumAmplification* gives control over the parser limits that protects against billion laugh attacks (). This option expects a positiv integer as argument. You should never need to use this option, because the default value (8388608) should work for any real data. If you ever need to increase this value for non-attack payload, please report.

-setReparseDeferralEnabled <*boolean*>
Expat protects itself against quadratic parsing time due to long tokens in malicious crafted documents. This protection may have the side effect that a parsing event is not triggered directly after the end of the respective token but only after the parser has processed some more bytes of the input. This option allows disabling this behavior.

-allocTrackerActivationThreshold <*float*>
This option together with *-allocTrackerMaximumAmplification* gives control for the parser limits to protect against the use of disproportional amounts of dynamic memory due to malicious crafted documents. This option expects a positive integer as argument. You should never need to use this option, because the default value (100.0) should work for any real data. If you ever need to increase this value for non-attack payload, please report.

-allocTrackerMaximumAmplification <*long*>
This option together with *-allocTrackerActivationThreshold* gives control for the parser limits to protect against the use of disproportional amounts of dynamic memory due to malicious crafted documents. This option expects a positive integer as argument. You should never need to use this option, because the default value (67108864, that is 64 MiB) should work for any real data. If you ever need to increase this value for non-attack payload, please report.

Command methods

parser configure *option value ?option value?*
Sets configuration options for the parser. Every command option, except *-namespace* can be set or modified with this method.

parser cget*?-handlerset name? option*
Return the current configuration value option for the parser.

If the -handlerset option is used, the configuration for the named handler set is returned.

parser currentmarkup
Returns the current markup as found in the XML, if called from within one of its markup event handler script (-elementstartcommand, -elementendcommand, -commentcommand and -processinginstructioncommand). Otherwise it return the empty string.

parser delete
Deletes the parser and the parser command. A parser cannot be deleted from within one of its handler callbacks (neither directly nor indirectly) and will raise a tcl error in this case.

parser free
Another name to call the method *delete*, see there.

parser get
***-specifiedattributecount*|*-idattributeindex*|*-currentbytecount*|*-currentlinenumber*|*-currentcolumn**

-specifiedattributecount
Returns the number of the attribute/value pairs passed in last call to the elementstartcommand that were specified in the start-tag rather than defaulted. Each attribute/value pair counts as 2; thus this corresponds to an index into the attribute list passed to the elementstartcommand.

-idattributeindex
Returns the index of the ID attribute passed in the last call to XML_StartElementHandler, or -1 if there is no ID attribute. Each attribute/value pair counts as 2; thus this corresponds to an index into the attributes list passed to the elementstartcommand.

-currentbytecount
Return the number of bytes in the current event. Returns 0 if the event is in an internal entity.

If you use this option consider if you may need the parser option *-keepTextStart*.

-currentlinenumber
Returns the line number of the current parse location.

If you use this option consider if you may need the parser option *-keepTextStart*.

-currentcolumnnumber
Returns the column number of the current parse location.

If you use this option consider if you may need the parser option *-keepTextStart*.

-currentbyteindex
Returns the byte index of the current parse location.

If you use this option consider if you may need the parser option *-keepTextStart*.

Only one value may be requested at a time.

parser parse *data*
Parses the XML string *data*. The event callback scripts will be called, as there triggering events happens. This method cannot be used from within a callback (neither directly nor indirectly) of the parser to be used and will raise an error in this case.

parser parsechannel *channelID*
Reads the XML data out of the tcl channel *channelID* (starting at the current access position, without any seek) up to the end of file condition and parses that data. The channel encoding is respected. Use the helper proc tDOM::xmlOpenFile out of the tDOM script library to open a

file, if you want to use this method. This method cannot be used from within a callback (neither directly nor indirectly) of the parser to be used and will raise an error in this case.

parser parsefile *filename*
Reads the XML data directly out of the file with the filename *filename* and parses that data. This is done with low-level file operations. The XML data must be in US-ASCII, ISO-8859-1, UTF-8 or UTF-16 encoding. If applicable, this is the fastest way, to parse XML data. This method cannot be used from within a callback (neither directly nor indirectly) of the parser to be used and will raise an error in this case.

parser reset
Resets the parser in preparation for parsing another document. A parser cannot be reset from within one of its handler callbacks (neither directly nor indirectly) and will raise a tcl error in this cases.

Callback Command Return Codes

A script invoked for any of the parser callback commands, such as -elementstartcommand, -elementendcommand, etc, may return an error code other than "ok" or "error". All callbacks may in addition return "break" or "continue".

If a callback script returns an "error" error code then processing of the document is terminated and the error is propagated in the usual fashion.

If a callback script returns a "break" error code then all further processing of every handler script out of this Tcl handler set is suppressed for the further parsing. This does not influence any other handler set.

If a callback script returns a "continue" error code then processing of the current element, and its children, ceases for every handler script out of this Tcl handler set and processing continues with the next (sibling) element. This does not influence any other handler set.

If a callback script returns a "return" error code then parsing is canceled altogether, but no error is raised.

Keywords: expat, SAX, push, pushparser

expatapi

Name

CheckExpatParserObj, CHandlerSetInstall, CHandlerSetRemove, CHandlerSetCreate, CHandlerSetGetUserData, GetExpatInfo - Functions to create, install and remove expat parser object extensions.

Synopsis

#include <tclexpat.h>

int
CheckExpatParserObj (*interp*, *nameObj*)

int
CHandlerSetInstall (*interp*, *expatObj*, *handlerSet*)

int
CHandlerSetRemove (*interp*, *expatObj*, *handlerSetName*)

CHandlerSet*
CHandlerSetCreate (*handlerSetName*)

CHandlerSet*
CHandlerSetGet (*interp*, *expatObj*, *handlerSetName*)

void*
CHandlerSetGetUserData (*interp*, *expatObj*, *handlerSetName*)

TclGenExpatInfo*
GetExpatInfo (*interp*, *expatObj*)

Arguments

Tcl_Interp *_interp_ (in)
Interpreter with the expat parser object.

Tcl_Obj *_expatObj_ (in)
A Tcl Object containing the command name of the expat parser object to be queried or modified.

char *_handlerSetName_ (in)
Identifier of the handler set.

CHandlerSet *_handlerSet_ (in)
Pointer to a C handler set.

Tcl_Obj *_nameObj_
A Tcl Object containing the name of a expat parser object

Description

The functions described in this manual allows one to add C level coded event handler to an tDOM Tcl expat parser objects. A tDOM Tcl expat parser object is able to have several Tcl scripts and C functions associated with a specific event. If the event occurs, first the Tcl scripts then the C functions associated with the event are called in turn.

A tDOM Tcl expat parser extension is an ordinary Tcl extension and loaded like every other Tcl extension. It must install at least one new Tcl Level command, that manipulates a tDOM Tcl expat parser object.

A C level handler set is a data structure like this:

```
typedef struct CHandlerSet {
    struct CHandlerSet *nextHandlerSet;
    char *name;                     /* refname of the handler set */
    int ignoreWhiteCDATAs;          /* ignore 'white' CDATA sections */

    void *userData;                 /* Handler set specific Data
    Structure;
                                       the C handler set extension has
    to
                                       malloc the needed structure in
    his
                                       init func and has to provide a
                                       cleanup func (to free it). */
```

```
    CHandlerSet_userDataReset        resetProc;
    CHandlerSet_userDataFree         freeProc;

    /* C func for element start */
    XML_StartElementHandler          elementstartcommand;
    /* C func for element end */
    XML_EndElementHandler            elementendcommand;
    /* C func for character data */
    XML_CharacterDataHandler         datacommand;
    /* C func for namespace decl start */
    XML_StartNamespaceDeclHandler    startnsdeclcommand;
    /* C func for namespace decl end */
    XML_EndNamespaceDeclHandler      endnsdeclcommand;
    /* C func for processing instruction */
    XML_ProcessingInstructionHandler picommand;
    /* C func for default data */
    XML_DefaultHandler               defaultcommand;
    /* C func for unparsed entity declaration */
    XML_NotationDeclHandler          notationcommand;
    /* C func for external entity */
    XML_ExternalEntityRefHandler     externalentitycommand;
    /* C func for unknown encoding */
    XML_UnknownEncodingHandler       unknownencodingcommand;
    /* C func for comments */
    XML_CommentHandler               commentCommand;
    /* C func for "not standalone" docs */
    XML_NotStandaloneHandler         notStandaloneCommand;
    /* C func for CDATA section start */
    XML_StartCdataSectionHandler     startCdataSectionCommand;
    /* C func for CDATA section end */
    XML_EndCdataSectionHandler       endCdataSectionCommand;
    /* C func for !ELEMENT decl's */
    XML_ElementDeclHandler           elementDeclCommand;
    /* C func for !ATTLIST decl's */
    XML_AttlistDeclHandler           attlistDeclCommand;
    /* C func for !DOCTYPE decl's */
    XML_StartDoctypeDeclHandler      startDoctypeDeclCommand;
    /* C func for !DOCTYPE decl ends */
    XML_EndDoctypeDeclHandler        endDoctypeDeclCommand;
    /* C func for !ENTITY decls's */
    XML_EntityDeclHandler            entityDeclCommand;
} CHandlerSet;
```

The types and the arguments of the event functions (XML_*) are exactly the same like the expat lib handler functions and described in detail in expat.h, see there. The extension has only to

provided the handler functions needed; it's perfectly OK to leave unused handler slots as the are (they are initialized to NULL by CHandlerSetCreate).

The *name* of this structure is used to identify the handler set.

If the flag *ignoreWhiteCDATAs* is set, element content which contain only whitespace isn't reported with the datacommand.

The *userData* element points to the handler set specific data. The event handler functions are called with this pointer as userData argument.

resetProc and *freeProc* must have arguments that match the type

```
void (Tcl_Interp *interp, void *userData)
```

resetProc is called in case the parser is reset with **<parserObj> reset** and should do any necessary cleanup and reinitializing to prepare the C handler set for a new XML document. The *freeProc* is called, if the parser is deleted and should do memory cleanup etc.

CHandlerSetCreate create a C handler set, gives it the name *name* and initializes any other element with NULL.

CHandlerSetInstall adds the *handlerSet* to the parser *expatObj*. The parser has to be a tDOM Tcl expat parser object in the interpreter *interp*. The name of the C handler set has to be unique for the parser. Returns 0 in case of success. Returns 1 if *expatObj* isn't a parser object in the interpreter *interp*. Returns 2 if this parser has already a C handler set with the handlerSet name.

CHandlerSetRemove removes the C handler set referenced by the *handlerSetName* from the parser *expatObj*. The parser has to be a tDOM Tcl expat parser object in the interpreter *interp*. *CHandlerSetRemove* calls the freeProc function of the C handler set structure, removes the handler set from the C handler set list and frees the handler structure. Returns 0 in case of success. Returns 1 if *expatObj* isn't a parser object in the interpreter *interp*. Returns 2 if this parser hasn't a C handler set named *handlerSetName*.

CheckExpatParserObj returns 1, if *nameObj* is a tDOM Tcl expat parser object in the interpreter *interp*, otherwise it returns 0. Example:

```
int
TclExampleObjCmd(dummy, interp, objc, objv)
     ClientData dummy;
     Tcl_Interp *interp;
     int objc;
     Tcl_Obj *const objv[];
{
    char          *method;
    CHandlerSet   *handlerSet;
    int            methodIndex, result;
    simpleCounter *counter;


    static char *exampleMethods[] = {
        "enable", "getresult", "remove",
        NULL
```

```
    };
    enum exampleMethod {
        m_enable, m_getresult, m_remove
    };

    if (objc != 3) {
        Tcl_WrongNumArgs (interp, 1, objv, example_usage);
        return TCL_ERROR;
    }

    if (!CheckExpatParserObj (interp, objv[1])) {
        Tcl_SetResult (interp, "First argument has to be a expat
    parser object", NULL);
        return TCL_ERROR;
    }
    /* ... */
```

CHandlerSetGet returns a pointer to the C handler Set referenced by the name *handlerSetName* of the parser object *expatObj*. *expatObj* has to be an expat parser object in the interpreter *interp* . Returns NULL in case of error.

CHandlerSetGetUserData returns a pointer to the userData of the C handler set referenced by the name *handlerSetName* of the parser object *expatObj*. *expatObj* has to be an expat parser object in the interpreter *interp*. Returns NULL in case of error.

GetExpatInfo Is a helper function that returns a pointer to the TclGenExpatInfo structure of the tDOM Tcl expat parser object *expatObj*. The *expatObj* has to be a tDOM Tcl expat parser object in the interpreter *interp*. This is most useful, to set the application status of the parser object.

See the simple but full functionally example in the extensions/example dir or the more complex example tnc in the extensions/tnc dir (a simple DTD validation extension).

Keywords: expatapi, C handler set

pullparser

Name

tdom::pullparser - Create an XML pull parser command

Synopsis

package require tdom

tdom::pullparser *cmdName* ?*-ignorewhitecdata*?

Description

This command creates XML pull parser commands with a simple API, along the lines of a simple StAX parser. After creation, you've to set an input source, to do anything useful with the pull parser. For this see the methods *input*, *inputchannel* and *inputfile*.

The parser has always a *state*. You start parsing the XML data until some next state, do what has to be done and skip again to the next state. XML well-formedness errors along the way will be

reported as TCL_ERROR with additional info in the error message.

The pull parsers don't follow external entities and are XML 1.0 only, they know nothing about XML Namespaces. You get the tags and attribute names as in the source. You aren't noticed about comments, processing instructions and external entities; they are silently ignored for you. CDATA Sections are handled as if their content would have been provided without using a CDATA Section.

On the brighter side is that character entity and attribute default declarations in the internal subset are respected (because of using expat as underlying parser). It is probably somewhat faster than a comperable implementation with the SAX interface. It's a nice programming model. It's a slim interface.

If the option *-ignorewhitecdata* is given, the created XML pull parser command will ignore any white space only (' ', \t, \n and \r) text content between START_TAG and START_TAG / END_TAG. The parser won't stop at such input and will create TEXT state events only for not white space only text.

Not all methods are valid in every state. The parser will raise TCL_ERROR if a method is called in a state the method isn't valid for. Valid methods of the created commands are:

state
This method is valid in all parser states. The possible return values and their meanings are:

- *READY* - The parser is created or reset, but no input is set.
- *START_DOCUMENT* - Input is set, parser is ready to start parsing.
- *START_TAG* - Parser has stopped parsing at a start tag.
- *END_TAG* - Parser has stopped parsing at an end tag
- *TEXT* - Parser has stopped parsing to report text between tags.
- *END_DOKUMENT* - Parser has finished parsing without error.
- *PARSE_ERROR* - Parser stopped parsing at XML error in input.

input *data*
This method is only valid in state *READY*. It prepares the parser to use *data* as XML input to parse and switches the parser into state START_DOCUMENT.

inputchannel *channel*
This method is only valid in state *READY*. It prepares the parser to read the XML input to parse out of *channel* and switches the parser into state START_DOCUMENT.

inputfile *filename*
This method is only valid in state *READY*. It open *filename* and prepares the parser to read the XML input to parse out of that file. The method returns TCL_ERROR, if the file could not be open in read mode. Otherwise it switches the parser into state START_DOCUMENT.

next

This method is valid in state *START_DOCUMENT*, *START_TAG*, *END_TAG* and *TEXT*. It continues parsing of the XML input until the next event, which it will return.

tag
This method is only valid in states *START_TAG* and *END_TAG*. It returns the tag name of the current start or end tag.

attributes
This method is only valid in state *START_TAG*. It returns all attributes of the element in a name value list.

text
This method is only valid in state *TEXT*. It returns the character data of the event. There will be always at most one TEXT event between START_TAG and the next START_TAG or END_TAG event.

skip
This method is only valid in state *START_TAG*. It skips to the corresponding end tag and ignores all events (but not XML parsing errors) on the way and returns the new state END_TAG.

find-element ? tagname | -names tagnames ?
This method is only valid in states *START_DOCUMENT*, *START_TAG* and *END_TAG*. It skips forward until the next element start tag with tag name *tagname* and returns the new state START_TAG. If a list of tagnames is provided with the *-names* option, any of the *tagnames* match. If there isn't such an element the parser stops at the end of the input and returns END_DOCUMENT.

reset
This method is valid in all parser states. It resets the parser into READY state and returns that.

delete
This method is valid in all parser states. It deletes the parser command. Miscellaneous methods:

line
This method is valid in all parser states except READY and TEXT. It returns the line number of the parsing position. Line counting starts with 1.

column
This method is valid in all parser states except READY and TEXT. It returns the offset, from the beginning of the current line, of the parsing position. Column counting starts with 0.

Keywords: pullparser, XML, pull, parsing

schema

Name

tdom::schema - Creates a schema validation command

Synopsis

package require tdom

tdom::schema ?*create*? *cmdName*

Description

Every call of this command creates a new validation command. A validation command has methods to define a schema and is able to validate XML data or to post-validate a tDOM DOM tree (and to some degree other kind of hierarchical data) against this schema.

Also, a validation command may be used as argument to the *-validateCmd* option of the *dom parse* and the *expat* commands to enable validation additionally to what they do otherwise.

The methods of created commands are:

prefixns*?prefixUriList?*
: This method controls prefix (or abbreviation) to namespace URI mapping. Wherever a namespace argument is expected in the schema command methods the "prefix" could be used instead of the namespace URI. If the list maps the same prefix to different namespace URIs, the first one wins. If there is no such prefix, the namespace argument is used literally as namespace URI. If the method is called without argument, it returns the current prefixUriList. If the method is called with the empty string, any namespace URI arguments are used literally. This is the default.

defelement *name?namespace? <definition script>*
: This method defines the element *name* (optional in the namespace *namespace*) in the schema. The *definition script* is evaluated and defines the content model of the element. If the *namespace* argument is given, any *element* or *ref* references in the definition script not wrapped inside a *namespace* command are resolved in that namespace. If there is already a element definition for the name/namespace combination, the command raises error.

defelementtype *typename?namespace? <definition script>*
: This method defines the element type *typename* (optional in the namespace *namespace*) in the schema. If the element type is used in a definition script with the schema command element, the validation engine expects an element content according to content model *definition script*. Defining (and using) element types seems only sensible if you really have elements with the same name and namespace but different content models. The *definition script* is evaluated and defines the content model of the element it is assgned to. If the *namespace* argument is given, any *element* or *ref* references in the definition script not wrapped inside a *namespace* command are resolved in that namespace. If there is already an elementtype definition for the typename/namespace combination, the command raises error. The document element of any XML to validate cannot be a *defelementtype* defined element.

defpattern *name?namespace? <definition script>*
: This method defines a (maybe complex) content particle with the *name* (optional in the namespace *namespace*) in the schema, to be used in other definition scripts with the definition command *ref*. The *definition script* is evaluated and defines the content model of the content particle. If the *namespace* argument is given, any *element* or *ref* references in the definition script not wrapped inside a *namespace* command are resolved in that namespace. If there is already a pattern definition for the name/namespace combination, the command raises error.

deftexttype *name <constraint script>*
: This method defines a bundle of text constraints that can be referred to by *name* while defining constraints on text element or attribute values. If there is already a text type definition with this name, the command raises error. A text type may be referred before it is defined in the schema. If a referred text type isn't defined anywhere in the schema then any text will match this type during validation.

start *documentElement?namespace?*
This method defines the name and namespace of the root element of a tree to validate. If this method is used, the root element must match for validity. If *start* is not used, any element defined by *defelement* may be the root of a valid document. The *start* method may be used several times with varying arguments during the lifetime of a validation command. If the command is called with just the empty string (and no namespace argument), the validation constraint for the root element is removed and any defined element will be valid as root of a tree to validate.

define *<definition script>*
This method defines several elements or patterns or a whole schema with one call, by evaluating the *definition script*. All schema command methods so far (*prefixns*, *defelement*, *defelementtype*, *defpattern*, *deftexttype* and *start*) are allowed top level in the *definition script*. The *define* method itself isn't allowed recursively.

event *(start|end|text)?event specific data?*
This method enables validation of hierarchical data against the content constraints of the validation command.

start *name ?attributes? ?namespace?*
Checks if the current validation state allows the element *name* in the *namespace* to start here. It raises error if not.

end
Checks if the current innermost open element may end there in the current state without violation of validation constraints. It raises error if not.

text *text*
Checks if the current validation state allows the given text content. It raises error if not.

validate ?*options*? *<XML string>?objVar?*
Returns true if the *<XML string>* is valid, or false, otherwise. If validation has failed and the optional *objVar* argument is given, the variable with that name is set to a validation error message. If the XML string is valid and the optional *objVar* argument is given, the variable will be untouched.

The valid options are:

-baseurl *<baseURI>*
If *-baseurl <baseURI>* is specified, the baseURI is used as the base URI of the document. External entities references in the document are resolved relative to this base URI. This base URI is also stored within the DOM tree.

-externalentitycommand *<script>*
If *-externalentitycommand <script>* is specified, the specified Tcl script is called to resolve any external entities of the document. The default is "::tdom::extRefHandler", which is a simple file URL resolver defined by the script part of the package. Setting the option value to the empty string disables resolving of external entities. The actual evaluated command consists of this option followed by three arguments: the base uri, the system identifier of the entity and the public identifier of the entity. The base uri and the public identifier may be the empty list. The script has to return a Tcl list consisting of three elements. The first element of this list signals how the external entity is returned to the processor. Currently the two allowed types are "string" and "channel". The second element of the list has to be the (absolute) base

URI of the external entity to be parsed. The third element of the list are data, either the already read data out of the external entity as string in the case of type "string", or the name of a Tcl channel, in the case of type "channel". Note that if the script returns a Tcl channel, it will not be closed by the processor. It must be closed separately if it is no longer needed.

-paramentityparsing <*always*|*never*|*notstandalone*>

The *-paramentityparsing* option controls, if the parser tries to resolve the external entities (including the external DTD subset) of the document while building the DOM tree. *-paramentityparsing* requires an argument, which must be either "always", "never", or "notstandalone". The value "always" means that the parser tries to resolves (recursively) all external entities of the XML source. This is the default in case *-paramentityparsing* is omitted. The value "never" means that only the given XML source is parsed and no external entity (including the external subset) will be resolved and parsed. The value "notstandalone" means, that all external entities will be resolved and parsed, with the exception of documents, which explicitly states standalone="yes" in their XML declaration.

-useForeignDTD <*boolean*>

If <boolean> is true and the document does not have an external subset, the parser will call the -externalentitycommand script with empty values for the systemId and publicID arguments. Please note that if the document also doesn't have an internal subset, the -startdoctypedeclcommand and -enddoctypedeclcommand scripts, if set, are not called.

validatefile ?*options*? *filename?objVar?*

Returns true if the content of *filename* is valid, or false, otherwise. The given file is fed as binary stream to expat, therefore, only US-ASCII, ISO-8859-1, UTF-8 or UTF-16 encoded data will work with this method. If validation has failed and the optional *objVar* argument is given, the variable with that name is set to a validation error message. If the XML data is valid and the optional *objVar* argument is given, the variable will be untouched. The allowed options and their meaning are the same as for the *validate* method; see there for a description.

validatechannel ?*options*? *channel?objVar?*

Returns true if the content read from the Tcl channel *channel* is valid, or false, otherwise. Since data read out of a Tcl channel is UTF-8 encoded, any misleading encoding declaration at the beginning of the data will lead to errors. If the validation fails and the optional *objVar* argument is given, the variable with that name is set to a validation error message. If the XML data is valid and the optional *objVar* argument is given, the variable will be untouched. The allowed options and their meaning are the same as for the *validate* method; see there for a description.

domvalidate *domNode?objVar?*

Returns true if the first argument is a valid tree, or false, otherwise. If validation has failed and the optional *objVar* argument is given, the variable with that name is set to a validation error message. If the dom tree is valid and the optional *objVar* argument is given, the variable with that name is set to the empty string.

reportcmd*?cmd?*

This method expects the name of a Tcl command to be called in case of validation error. The command will be called with two arguments appended: the schema command which raises the validation error, and a validation error code.

The possible error codes are:

MISSING_ELEMENT
...see next...
MISSING_TEXT
...see next...
UNEXPECTED_ELEMENT
...see next...
UNEXPECTED_ROOT_ELEMENT
...see next...
UNEXPECTED_TEXT
...see next...
UNKNOWN_ROOT_ELEMENT
...see next...
UNKNOWN_ATTRIBUTE
...see next...
MISSING_ATTRIBUTE
...see next...
INVALID_ATTRIBUTE_VALUE
...see next...
DOM_KEYCONSTRAINT
...see next...
DOM_XPATH_BOOLEAN
...see next...
INVALID_KEYREF
...see next...
INVALID_VALUE
...see next...
UNKNOWN_GLOBAL_ID
...see next...
UNKNOWN_ID
For more detailed information see section Recovering.

delete
This method deletes the validation command.

info*?args?*
This method bundles methods to query the state of and details about the schema command.

validationstate
This method returns the state of the validation command with respect to validation state. The possible return values and their meanings are:

READY
The validation command is ready to start validation
VALIDATING
The validation command is in the process of validating input.
FINISHED
The validation has finished, no further events are expected.

vstate
This method is a shorter alias for validationstate; see there.

line
If the schema command is currently validating, this method returns the line part of the parsing position information, and the empty string in all other cases. If the schema command is currently post-validating a DOM tree, there may be no position information stored at some or all nodes. The empty string is returned in these cases.

column
If the schema command is currently validating this method returns the column part of the parsing position information, and the empty string in all other cases. If the schema command is currently post-validating a DOM tree, there may be no position information stored at some or all nodes. The empty string is returned in these cases.

byteIndex
If the schema command is currently validating this method returns the byte position of the parsing position information, and the empty string in all other cases. If the schema command is currently post-validating a DOM tree, there may be no position information stored at some or all nodes. The empty string is returned in these cases.

domNode
If the schema command isn't currently post-validating a DOM tree this method returns the empty string. Otherwise, if the schema command waits for the reportcmd script to finish while recovering from a validation error it returns the node on which the validation engine is currently looking at in case the node is an ELEMENT_NODE or, if not, its parent node. It is recommended that you do not use this method. Or at least leave the DOM tree alone, use it read-only.

nrForwardDefinitions
Returns how many elements, element types and ref patterns are referenced that aren't defined so far (summed together).

definedElements
Returns in no particular order the defined elements in the grammar as list. If an element is namespaced, its list entry will be itself a list with two elements, with the name as first and the namespace as second element.

definedElementtypes
Returns in no particular order the defined element types in the grammar as list. If an element type is namespaced, its list entry will be itself a list with two elements, with the name as first and the namespace as second element.

definedPatterns
Returns in no particular order the defined named pattern in the grammar as list. If a named pattern is namespaced, its list entry will be itself a list with two elements, with the name as first and the namespace as second element.

expected
Returns in no particular order all possible next events (since the last successful event match, if there was one) as a list. If an element is namespaced its list entry will be itself a list with two elements, with the name as first and the namespace as second element. If text is a possible next event, the list entry will be a two elements list, with #text as first element and the empty string as second. If an any element constraint is possible the list entry will be a two

elements list, with <any> as first element and the empty string as second. If an any element in a certain namespace constraint is possible, the list entry will be a two elements list, with <any> as first element and the namespace as second. If element end is a possible event, the list entry will be a two elements list with <elementend> as first element and the empty string as second element.

definition name ?namespace?
Returns the code that defines the given element. The command raises error if there is no definition of that element.

typedefinition name ?namespace?
Returns the code that defines the given element type definition. The command raises error if there is no definition of that element.

patterndefinition name ?namespace?
Returns the code that defines the given pattern definition. The command raises error if there is no definition of a pattern with that name and, if given, namespace.

vaction ?name|namespace|text?
This method returns useful information only if the schema command waits for the reportcmd script to finish while recovering from a validation error. Otherwise it returns NONE.

If the command is called without the optional argument the possible return values and their meanings are:

NONE
The schema command currently does not recover from a validation event.

MATCH_ELEMENT_START
Element start event, which includes looking for missing or unknown attributes.

MATCH_ELEMENT_END
Element end event.

MATCH_TEXT
Validating text between tags.

MATCH_ATTRIBUTE_TEXT
Attribute text value constraint check

MATCH_GLOBAL
Checking global IDs

MATCH_DOM_KEYCONSTRAINT
Checking domunique constraint

MATCH_DOM_XPATH_BOOLEAN
Checking domxpathboolean constant If called with one of the possible optional arguments, the command returns detail information depending on current action.

name
Returns the name of the element that has to match in case of MATCH_ELEMENT_START. Returns the name of the closed element in case of MATCH_ELEMENT_END. Returns the name of the attribute in case of MATCH_ATTRIBUTE_TEXT. Returns the name of the parent element in case of MATCH_TEXT.

namespace
Returns the namespace of the element that has to match in case of MATCH_ELEMENT_START. Returns the namespace of the closed element in case of MATCH_ELEMENT_END. Returns the

namespace of the attribute in case of MATCH_ATTRIBUTE_TEXT. Returns the namespace of the parent element in case of MATCH_TEXT.

text
Returns the text to match in case of MATCH_TEXT. Returns the value of the attribute in case of MATCH_ATTRIBUTE_TEXT.

stack top|inside|associated
In Tcl scripts evaluated by validation this method provides information about the current validation stack. Called outside this context the method returns the empty string.

top
Returns the element whose content is currently checked (the open element tag at this moment).

inside
Returns all currently open elements as a list.

associated
Returns the data associated with the current top most stack content particle or the empty string if there isn't any.

reset
This method resets the validation command into state READY (while preserving the defined grammar).

Schema definition scripts

Schema definition scripts are ordinary Tcl scripts evaluated in the namespace tdom::schema. The schema definition commands listed below in this Tcl namespace allow the definition of a wide variety of document structures. Every schema definition command establishes a validation constraint on the content which has to match or must be optional to qualify the content as valid. It is a validation error if there is additional (not matched) content. White-space-only text (in the XML sense of white space) between any different tags is ignored, with the exception of text only elements (for which even white-space-only text will be considered as significant content).

The schema definition commands are:

element *name?quant? (?<definition script>|*(lqtype*(lq typename)?*
If neither the optional argument *definition script* nor the string "type" and a *typename* is given this command refers to the element defined with *defelement* with the name *name* in the current context namespace.

If the string "type" and a *typename* is given then the content of the element is described by the content model defined with *defelementtype* with the name *typename* in the current context namespace.

If the *defelement script* argument is given, the validation constraint expects an element with the name *name* in the current namespace with content "locally" defined by the *definition script*. Forward references to so far not defined elements or patterns or other local definitions of the same name inside the *definition script* are allowed. If a forward referenced element is not defined until validation, only an empty element with name *name* and namespace *namespace* and no attributes matches.

ref *name?quant?*

This command refers to the content particle defined with *defpattern* with the name *name* in the current context namespace. Forward references to a so far not defined pattern and recursive references are allowed. If a forward referenced pattern is not defined until validation no content whatsoever is expected ("empty match").

group*?quant? <definition script>*
This method group a sequence of content particles defined by the *definition script*, which have to match in this sequence order.

choice*?quant? <definition script>*
This schema constraint matches if one of the top level content particles defined by the *definition script* matches. If one of this top level content particle is optional this constraint matches the "empty match".

interleave*?quant? <definition script>*
This schema constraint matches after every of the required top level content particles defined by the *definition script* have matched (and, optional, some or all other) in any arbitrary order.

mixed*?quant? <definition script>*
This schema constraint matches for any text (including the empty one) and every top level content particle defined by the *definition script* with default quantifier *.

text*?<constraint script>|*(lqtype*(lq typename?*
Without the optional constraint script this validation constraint matches every string (including the empty one). With *constraint script* or with a given text type argument a text matching this script or the text type is expected.

any ?*options*??<namespace list>??quant?*
Without arguments the any command matches every element. If the *<namespace list>* argument is given, this matches any element in a namespace out of that list. The empty string means elements with no namespace. If additionally the option *-not* is given then this maches every element with a namespace not in the list. The only other recognized option is -- which signals the end of any options. Please note that in case of no *namespace* argument is given that means that the quantifier * and + will eat up any elements until the enclosing element ends. If you really have a namespace that looks like a valid tDOM schema quantifier you will have to spell out always both arguments.

attribute *name?quant? (?<constraint script>|*(lqtype*(lq typename?)*
The attribute command defines an attribute (in no namespace) to the enclosing element. The first definition of *name* inside an element definition wins; later definitions of the same name are silently ignored. After the *name* argument there may be one of the quantifiers ? or!. If there is, it will be used. Otherwise the attribute will be required (must be present in the XML source). If there is one argument more this argument is evaluated as constraint script, defining the value constraints of the attribute. Otherwise, if there are two more arguments and the first of them is the bare-word "type" the following argument is used as a text type name. This command is only allowed at top level in the definition script of a defelement/element script.

nsattribute *name namespace?quant? (?<constraint script>|*(lqtype*(lq typename?)*
This command does the same as the command *attribute*, for the attribute *name* in the namespace *namespace*.

namespace *URI <definition script>*
Evaluates the *definition script* with context namespace *URI*. Every element, element type or

ref command name will be looked up in the namespace *URI*, and local defined elements will be in that namespace. An empty string as *URI* means no namespace.

tcl *tclcmd?arg arg ...?*
Evaluates the Tcl script *tclcmd arg arg ...*. This validation command is only allowed in strict sequential context (not in choice, mixed and interleave). If the return code is something else than TCL_OK, this is an error (which is not caught and reported by reportcmd).

self
Returns the schema command.

associate *data*
This command is only allowed top-level inside definition scripts of the element, elementtype, pattern or interleave content particles. Associates the *data* given as argument with the currently defined content particle and may be requested in scripts evaluated while validating the content of that particle with the schema command method call *info stack associated*.

domunique *selector fieldlist?name??*(lqIGNORE_EMPTY_FIELD_SET*(lq|(\ *(lqEMPTY_FIELD_SET_VALUE*(lq emptyFieldSetValue)?*
If not postvalidating a DOM tree with *domvalidate* this constraint always matches. If postvalidating this constraint resembles the xsd key/keyref mechanism. The *selector* argument may be any valid XPath expression (without the xsd limits). Several *domunique* commands within one element definition are allowed. They are checked in definition order. The argument name is available in the recovering script per *info vaction name*. If the *fieldlist* does not select something for a node of the result set of the *selector* the key value will be the empty string by default. If the arguments *EMPTY_FIELD_SET_VALUE <value>* are given an empty node set will have the key value *value*. If instead the flag *IGNORE_EMPTY_FIELD_SET* flag is given an empty node set result will not have any key value.

domxpathboolean *XPath_expr?name?*
If not postvalidating a DOM tree with *domvalidate* this constraint always matches. If postvalidating the *XPath_expr* argument is evaluated (with the node matching the schema parent of the *domxpathboolean* command as context node). The constraint maches if the result of this XPath expression, converted to boolean by XPath rules, is true. Several *domxpathboolean* commands within one element definition are allowed. They are checked in definition order.

This enables checks depending on more than one element. Consider

```
tdom::schema s
s define {
    defelement doc {
        element a ! text
        element b ! text
        element c ! text
        domxpathboolean "a * b * c >= 20000" volume
        domxpathboolean "a > b and b > c" sequence
    }
}
```

jsontype *JSON structure type*
If not postvalidating a DOM tree with *domvalidate* this constraint always matches. If

postvalidating the constraint matches if the enclosing element has the JSON type given as argument to the structure constraint. The possible JSON structure types are *NONE*, *OBJECT* and *ARRAY*. This constraint is only allowed as direct child of a defelement, defelementtype or local element definition.

prefixns*?prefixUriList?*
This defines a prefix to namespace URI mapping exactly as a *schemacmd prefixns* would. It is meant as top-level command of a *schemacmd define* script. This command is not allowed nested in another definition script command and will raise error, if you call it there.

defelement *name?namespace? <definition script>*
This defines an element exactly as a *schemacmd defelement* call would. It is meant as top-level command of a *schemacmd define* script. This command is not allowed nested in another definition script command and will raise error, if you call it there.

defelementtype *typename?namespace? <definition script>*
This defines an elementtype exactly as a *schemacmd defelementtype* call would. It is meant as top-level command of a *schemacmd define* script. This command is not allowed nested in another definition script command and will raise error, if you call it there.

defpattern *name?namespace? <definition script>*
This defines a named pattern exactly as a *schemacmd defpattern* call would. It is meant as top-level command of a *schemacmd define* script. This command is not allowed nested in another definition script command and will raise error, if you call it there.

deftexttype *name <constraint script>*
This defines a named bundle of text constraints exactly as a *schemacmd deftexttype* call would. It is meant as top-level command of a *schemacmd define* script. This command is not allowed nested in another definition script command and will raise error, if you call it there.

start *name?namespace?*
This command works exactly as a *schemacmd start* call would. It is meant as top-level command of a *schemacmd define* script. This command is not allowed nested in another definition script command and will raise error, if you call it there.

Quantity specifier

Several schema definition commands expect a quantifier as one of their arguments which determines how often the content particle specified by the command is expected. The valid values for a *quant* argument are:

!
The content particle has to occur exactly once in valid documents.

?
The content particle may not occur more than once in valid documents - the particle is optional.

*****
The content particle may occur zero or more times in a row in valid documents.

+
The content particle may occur one or more times in a row in valid documents.

n
The content particle must occur n times in a row in valid documents. The quantifier must be an integer greater zero.

{n m}
The content particle must occur at least n and at most m times in a row in valid documents. The quantifier must be a Tcl list with two elements. The first element of this list must be an integer with n >= 0. If the second list element is the character *, then there is no upper limit. Otherwise the second list element must be an integer with n < m. If an optional quantifier is not given, it defaults to * in case of the *mixed* command and to! for all other commands.

Text constraint scripts

Text (parsed character data, as XML calls it) sometimes has to be of a certain kind or comply with certain rules to be valid. The text constraint script arguments to text, attribute, nsattribute and deftexttype commands are evaluated in the Tcl namespace *tdom::schema::text* namespace and allow the ensuing text constraint commands to check text for certain properties. The commands are defined in the Tcl namespace *tdom::schema::text*. They raise error in case they are called outside of a text constraint script.

A few of the ensuing text type commands are exposed as general Tcl commands. They are defined in the namespace tdom::type and are called as documented below with the text to check appended to the argument list. They return a logical value. Please note that the commands may not accept starting or ending white space. If a command is available in the tdom::type namespace is recorded in its documentation.

The tcl text constraint command

The *tcl* text constraint command dispatches the check to an arbitrary Tcl command, thus enable any programmable decision rules.

tcl *tclcmd?arg arg ...?*
Evaluates the Tcl script *tclcmd arg arg ...* and the text to validate appended to the argument list. The return value of the Tcl command is interpreted as a boolean.

Basic xml types

name
<URL: https://www.w3.org/TR/xml/#NT-Name> "" This text constraint matches if the text value matches the XML name production. This means that the text value must start with a letter, underscore (_), or colon (:), and may contain only letters, digits, underscores (_), colons (:), hyphens (-), and periods (.).

ncname
<URL: https://www.w3.org/TR/xml-names/#NT-NCName> "" This text constraint matches if the text value matches the XML ncname production. This means that the text value must start with a letter or underscore (_), and may contain only letters, digits, underscores (_), hyphens (-), and periods (.) (The only difference to the name constraint is that colons are not permitted.)

qname
<URL: https://www.w3.org/TR/xml-names/#NT-QName> "" This text constraint matches if the text value matches the XML qname production. This means that the text value is either a ncname or two ncnames joined by a colon (:).

nmtoken
<URL: https://www.w3.org/TR/xml/#NT-Nmtoken> "" This text constraint matches if the text value matches the XML nmtoken production

nmtokens
<URL: https://www.w3.org/TR/xml/#NT-Nmtokens> "" This text constraint matches if the text value matches the XML nmtokens production

Basic type tests

integer*?(xsd|tcl)?*
This text constraint matches if the text value could be parsed as an integer. If the optional argument to the command is *tcl*, everything that returns TCL_OK if fed into Tcl_GetInt() matches. If the optional argument to the command is *xsd*, the constraint matches if the value is a valid xsd:integer. Without argument *xsd* is the default.

negativeInteger*?(xsd|tcl)?*
This text constraint matches the same text values as the *integer* text constraint (see there), with the additional constraint, that the value must be < zero.

nonNegativeInteger*?(xsd|tcl)?*
This text constraint matches the same text values as the *integer* text constraint (see there), with the additional constraint, that the value must be >= zero.

nonPositiveInteger*?(xsd|tcl)?*
This text constraint matches the same text values as the *integer* text constraint (see there), with the additional constraint, that the value must be <= zero.

positiveInteger*?(xsd|tcl)?*
This text constraint matches the same text values as the *integer* text constraint (see there), with the additional constraint, that the value must be > zero.

number*?(xsd|tcl)?*
This text constraint matches if the text value could be parsed as a number. If the optional argument to the command is *tcl*, everything that returns TCL_OK if fed into Tcl_GetDouble() matches. If the optional argument to the command is *xsd*, the constraint matches if the value is a valid xsd:decimal. Without argument *xsd* is the default.

boolean*?(xsd|tcl)?*
This text constraint matches if the text value could be parsed as a boolean. If the optional argument to the command is *tcl*, everything that returns TCL_OK if fed into Tcl_GetBoolean() matches. If the optional argument to the command is *xsd*, the constraint matches if the value is a valid xsd:boolean. Without argument *xsd* is the default.

date
This text constraint matches if the text value is a xsd:date, which is basically like an ISO 8601 date of the form YYYY-MM-DD, with optional time zone part (either the letter Z or plus (+) or minus (-) followed by hh:mm and with maximum allowed positive or negative time zone 14:00). It follows the date rules of the Gregorian calendar for all dates. A preceding minus sign for bce dates is allowed. There is no year 0. The year may have more than 4 digits, but only if needed (no extra leading zeros). This is available as common Tcl command tdom::type::date.

time
This text constraint matches if the text value is a xsd:time, which is basically like an ISO 8601 time of the form hh:mm:ss with optional time zone part. The time zone part follow the rules of the *date* command; see there. All three parts of the time value (hours, minutes, seconds) must be spelled out with 2 digits. Additional fractional seconds (with a point ('.') as separator)

are allowed, but not just a dangling point. The time value 24:00:00 (without fractional part) is allowed. This is available as common Tcl command tdom::type::time.

dateTime
This text constraint matches if the text value is a xsd:dateTime, which is basically like an ISO 8601 date time of the form YYYY-MM-DDThh:mm:ss with optional time zone part. The date and time zone parts follows the rules of the *date* and *time* command; see there. The time part (including the signaling 'T' character) is mandatory. This is available as common Tcl command tdom::type::dateTime.

duration
This text constraint matches if the text value is a xsd:duration, which is basically like an ISO 8601 duration of the form PnYnMnDTnHnMnS. All parts other than the starting P and - if one of H, M or S is given - T are optional. In case the following sign letter is S, n may be a decimal (with at least one digit before and after the dot), otherwise it must be a (positive) integer. This is available as common Tcl command tdom::type::duration.

base64
This text constraint matches if text is valid according to RFC 4648. This is available as common Tcl command tdom::type::base64.

hexBinary
This text constraint matches if text is a sequence of binary octets in hexadecimal encoding, where each binary octet is a two-character hexadecimal number. Lowercase and uppercase letters A through F are permitted.

unsignedByte
This text constraint matches if the text value is a xsd:unsignedByte. This is an integer between 0 and 255, both included, optionally preceded by a + sign and leading zeros.

unsignedShort
This text constraint matches if the text value is a xsd:unsignedShort. This is an integer between 0 and 65535, both included, optionally preceded by a + sign and leading zeros.

unsignedInt
This text constraint matches if the text value is a xsd:unsignedInt. This is an integer between 0 and 4294967295, both included, optionally preceded by a + sign and leading zeros.

unsignedLong
This text constraint matches if the text value is a xsd:unsignedLong. This is an integer between 0 and 18446744073709551615, both included, optionally preceded by a + sign and leading zeros.

byte
This text constraint matches if the text value is a xsd:byte. This is an integer between -128 and 127, both included, optionally preceded by a + or a - sign and leading zeros.

short
This text constraint matches if the text value is a xsd:short. This is an integer between -32768 and 32767, both included, optionally preceded by a + or a - sign and leading zeros.

int
This text constraint matches if the text value is a xsd:int. This is an integer between -2147483648 and 2147483647, both included, optionally preceded by a + or a - sign and leading zeros.

long

This text constraint matches if the text value is a xsd:long. This is an integer between -9223372036854775808 and 9223372036854775807, both included, optionally preceded by a + or a - sign and leading zeros.

Logical constructs

oneOf *<constraint script>*

This text constraint matches if one of the text constraints defined in the argument *constraint script* matches the text. It stops after the first matches and probes the text constraints in the order of definition.

allOf *<constraint script>*

This text constraint matches if all of the text constraints defined in the argument *constraint script* matches the text. It stops after the first match failure and probes the text constraints in the order of definition. Since the schema definition command *text* also expects all text constraints to match the text constraint, *allOf* is useful mostly in connection with the *oneOf* text constraint command.

not *<constraint script>*

This text constraint matches if none of the text constraints defined in the argument *constraint script* matches the text. It stops after the first matching constraint in the *constraint script* and reports validation error. The text constraints in the *constraint script* are probed in the order of definition.

type *text type name*

This text constraint matches if the text type given as argument matches.

Constraints on processed text value

whitespace *(preserve|replace|collapse) <constraint script>*

This text constraint command does white-space (#x20 (space, ' '), #x9 (tab, \t), #xA (linefeed, \n), and #xD (carriage return, \r) normalization to the text value and checks the resulting text with the text constraints of the constraint script argument. The normalization method *preserve* keeps everything as it is; this is another way to say *allOf*. The *replace* normalization method replaces any single white-space character (as above) to a space. The *collapse* normalization method removes all leading and trailing white-space, and all the other sequences of contiguous white-space are replaced by a single space.

split*?type ?args??<constraint script>*

This text constraint command splits the text to test into a list of values and tests all elements of that list for the text constraints in the evaluated *constraint script*.

The available types are:

whitespace

The text to split is stripped of all white space at start and end and split into a list at any successive white space.

tcl tclcmd ?arg ...?

The text to split is handed to the *tclcmd*, which is evaluated on global level, appended with every given arg and the text to split as last argument. This call must return a valid Tcl list whose elements are tested. The default in case no split type argument is given is *whitespace*.

strip *<constraint script>*

This text constraint command tests all text constraints in the evaluated *constraint script* with

the text to test stripped of all white space at start and end.

Various other string properties

fixed *value*

The text constraint only matches if the text value is string equal to the given value.

enumeration *list*

This text constraint matches if the text value is equal to one element (respecting case and any white-space) of the argument *list*, which has to be a valid Tcl list.

match*?-nocase? glob_style_match_pattern>*

<URL: https://www.tcl.tk/man/tcl8.6/TclCmd/string.htm#M35> "" This text constraint matches if the text value matches the glob style pattern given as argument. It follows the rules of the Tcl [string match] command, see.

regexp *expression*

<URL: https://www.tcl.tk/man/tcl8.6/TclCmd/re_syntax.htm> "" This text constraint matches if the text value matches the regular expression given as argument. describes the regular expression syntax

length *length*

This text constraint matches if the length of the text value (in characters, not bytes) is *length*. The length argument must be a positive integer or zero.

maxLength *length*

This text constraint matches if the length of the text value (in characters, not bytes) is at most *length*. The length argument must be an integer greater zero.

minLength *length*

This text constraint matches if the length of the text value (in characters, not bytes) is at least *length*. The length argument must be an integer greater zero.

id*?keySpace?*

This text constraint command marks the text as a document wide ID (to be referenced by an idref). Every ID value within a document must be unique. It isn't an error if the ID isn't actually referenced within the document. The optional argument *keySpace* does all this for a named key space. The key space "" (the empty sting) is another key space then the *id* command without keySpace argument.

idref*?keySpace?*

This text constraint command expects the text to be a reference to an ID within the document. The referenced ID may appear later in the document than the reference. Several references within the document to one ID are possible. For the optional *keySpace* argument see *id*

jsontype *<JSON text type>*

If not postvalidating a DOM tree with *domvalidate* this constraint always matches. If postvalidating the current TEXT_NODE to check must have the JSON text type given as argument to the text constraint command. The possible types are *NULL*, *TRUE*, *FALSE*, *STRING* and *NUMBER*.

Local key constraints

Document wide uniqueness and foreign key constraints are available with the text constraint commands id and idref. Keyspaces allow for sub-tree local uniqueness and foreign key

constraints.

keyspace *<names list> <constraint script>*
Any number of keyspaces are possible. A keyspace is either active or not. An inside a *constraint script* called keyspace with the same name does nothing. This text constraint commands work with keyspaces:

key *<name>*
If the keyspace with the name <*name*> is not active the constraint always matches. If the keyspace is active, reports error if there is already a key with the value. Otherwise it stores the value as key in this keyspace and matches.

keyref *<name>*
If the keyspace with the name <*name*> is not active always matches. If the keyspace is active then reports error if there is still no key as the value at the end of the keyspace <*name*>. Otherwise, it matches.

Recovering

By default the validation engine stops at the first detected validation violation and reports that finding. It does so by return false (and sets, if given, the result variable with an error message) in case the schema command itself is used to validate input. If the schema command is used by a SAX parser or the DOM parser, it does so by throwing error.

If a *reportcmd* is set this command is called on global level appended with the schema command and an error type as arguments in case a validation violation is detected. Then the validation recovers from the error and continues. For some validation errors the recover strategy can be determined with the script result of the reportcmd.

With a *reportcmd* (as long as the *reportcmd* does not throw error while called) the validation engine will never report validation failure to its caller. The validation engine recovers, continues, and reports the next error (if occurring) and so on until the end of the input. The schema command will return true and the SAX parser and DOM builder will process normally until the end of the input, as if there had not been a validation error.

Please note that this happens only for validation errors. It is not possible to recover from well-formedness errors. If the input is not well-formed, the schema command returns false and sets (if given) the result variable with an error message about the well-formedness error.

If the *reportcmd* throws error while called by the validation engine then validation stops and the schema command throws error with the error message of the script.

While validating basically three events can happen: an element start tag has to match, a piece of text has to match or an element end tag has to match. The method *info vaction* called in the recovering script or any script code called from there returns, which event has triggered the error report (MATCH_ELEMENT_START, MATCH_TEXT, MATCH_ELEMENT_END, respectively). While the command walks throu the schema looking whether the event matches other, data driven events (as, for example checking, if any keyref within a keyspace exists) may happen.

Several of the validation error codes, appended as second argument to the *reportcmd* calls, may happen at more than one kind of validation event. The *info vaction* method and its subcommands provide information about the current validation event, if called from the report command.

If a structural validation error happens, the default recovering strategy is to ignore any following

(or missing) content within the current subtree and to continue with the element end event of the subtree.

Returning "ignore" from the recovering script in case of error type MISSING_ELEMENT recovers by ignoring the failed constraint and continues to match the event further against the schema.

Returning "vanish" from the recover script in case of the error types MISSING_ELEMENT and UNEXPECTED_ELEMENT recovers by ignoring the event.

Examples

<URL: > The XML Schema Part 0: Primer Second Edition () starts with this example schema:

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema">

  <xsd:annotation>
    <xsd:documentation xml:lang="en">
     Purchase order schema for Example.com.
     Copyright 2000 Example.com. All rights reserved.
    </xsd:documentation>
  </xsd:annotation>

  <xsd:element name="purchaseOrder" type="PurchaseOrderType"/>

  <xsd:element name="comment" type="xsd:string"/>

  <xsd:complexType name="PurchaseOrderType">
    <xsd:sequence>
      <xsd:element name="shipTo" type="USAddress"/>
      <xsd:element name="billTo" type="USAddress"/>
      <xsd:element ref="comment" minOccurs="0"/>
      <xsd:element name="items"  type="Items"/>
    </xsd:sequence>
    <xsd:attribute name="orderDate" type="xsd:date"/>
  </xsd:complexType>

  <xsd:complexType name="USAddress">
    <xsd:sequence>
      <xsd:element name="name"   type="xsd:string"/>
      <xsd:element name="street" type="xsd:string"/>
      <xsd:element name="city"   type="xsd:string"/>
      <xsd:element name="state"  type="xsd:string"/>
      <xsd:element name="zip"    type="xsd:decimal"/>
    </xsd:sequence>
    <xsd:attribute name="country" type="xsd:NMTOKEN"
                   fixed="US"/>
  </xsd:complexType>

  <xsd:complexType name="Items">
```

```
    <xsd:sequence>
      <xsd:element name="item" minOccurs="0" maxOccurs="unbounded">
        <xsd:complexType>
          <xsd:sequence>
            <xsd:element name="productName" type="xsd:string"/>
            <xsd:element name="quantity">
              <xsd:simpleType>
                <xsd:restriction base="xsd:positiveInteger">
                  <xsd:maxExclusive value="100"/>
                </xsd:restriction>
              </xsd:simpleType>
            </xsd:element>
            <xsd:element name="USPrice"  type="xsd:decimal"/>
            <xsd:element ref="comment"   minOccurs="0"/>
            <xsd:element name="shipDate" type="xsd:date"
    minOccurs="0"/>
          </xsd:sequence>
          <xsd:attribute name="partNum" type="SKU" use="required"/>
        </xsd:complexType>
      </xsd:element>
    </xsd:sequence>
  </xsd:complexType>

  <!-- Stock Keeping Unit, a code for identifying products -->
  <xsd:simpleType name="SKU">
    <xsd:restriction base="xsd:string">
      <xsd:pattern value="\d{3}-[A-Z]{2}"/>
    </xsd:restriction>
  </xsd:simpleType>

</xsd:schema>
```

A simple one-to-one translation of that into a tDOM schema definition script would be:

```
tdom::schema schema
schema define {

    # Purchase order schema for Example.com.
    # Copyright 2000 Example.com. All rights reserved.

    defelement purchaseOrder {ref PurchaseOrderType}

    foreach elm {comment name street city state product} {
        defelement $elm text
    }

    defpattern PurchaseOrderType {
```

```
        element shipTo ! {ref USAddress}
        element billTo ! {ref USAddress}
        element comment ?
        element items
        attribute orderDate date
    }

    defpattern USAddress {
        element name
        element street
        element city
        element state
        element zip ! {text number}
        attribute country {fixed "US"}
    }

    defelement items {
        element item * {
            element product
            element quantity ! {text positiveInteger}
            element USPrice ! {text number}
            element comment
            element shipDate ? {text date}
            attribute partNum {regexp "^\d{3}-[A-Z]{2}$"}
        }
    }
}
```

<URL: > The RELAX NG Tutorial () starts with this example:

```
Consider a simple XML representation of an email address book:

<addressBook>
  <card>
    <name>John Smith</name>
    <email>js@example.com</email>
  </card>
  <card>
    <name>Fred Bloggs</name>
    <email>fb@example.net</email>
  </card>
</addressBook>

The DTD would be as follows:

<!DOCTYPE addressBook [
<!ELEMENT addressBook (card*)>
```

```
<!ELEMENT card (name, email)>
<!ELEMENT name (#PCDATA)>
<!ELEMENT email (#PCDATA)>
]>

A RELAX NG pattern for this could be written as follows:

<element name="addressBook"
    xmlns="http://relaxng.org/ns/structure/1.0">
  <zeroOrMore>
    <element name="card">
      <element name="name">
        <text/>
      </element>
      <element name="email">
        <text/>
      </element>
    </element>
  </zeroOrMore>
</element>
```

This schema definition script will do the same:

```
tdom::schema schema
schema define {
    defelement addressBook {
        element card *
    }
    defelement card {
        element name
        element email
    }
    foreach e {name email} {
        defelement $e text
    }
}
```

Keywords: schema, Validation, Postvalidation, DOM, SAX

tdomcmd

Name

tdom - tdom is an expat parser object extension to create an in-memory DOM tree from the input while parsing.

Synopsis

package require tdom

set parser [expat]

tdom $parser enable

Description

tdom adds the C handler set "tdom" to an tcl expat parser obj. This handler set builds an in-memory DOM tree out of the input, parsed by the parser. A DOM tree created this way behave exactly like a DOM tree created by the "dom" command (see there). In fact, tdom is only another interface to the same functionality; it uses the code behind the **dom** code for building the DOM tree.

tdom *parserObj* enable
Adds the tdom C handler set to a Tcl expat parser object. Next time, the parser parses input, the tdom C handler functions create an in-memory DOM tree.

tdom *parserObj* getdoc
Returns the DOM tree as domDoc (see there) object.

tdom *parserObj* setStoreLineColumn ?*boolean*?
See the method **setStoreLineColumn** of the **dom** command.

tdom *parserObj* remove
Removes the tdom C handler set from the parser object.

tdom *parserObj* keepEmpties
See the option **-keepEmpties** of the **dom** command.

tdom *parserObj* setExternalEntityResolver *script*
Keywords: tdomcmd, DOM, SAX, C handler set

tnc

Name

tnc - tnc is an expat parser object extension, that validates the XML stream against the document DTD while parsing.

Synopsis

package require tdom
package require tnc

set parser [expat]

tnc $parser enable

Description

tnc adds the C handler set "tnc" to a tcl expat parser obj. This handler set is a simple DTD validator. If the validator detects a validation error, it sets the interp result, signals error and stops parsing. There isn't any validation error recovering. As a consequence, only valid documents are completely parsed.

This handler set has only three methods:

tnc *parserObj* enable
Adds the tnc C handler set to a Tcl expat parser object.

tnc *parserObj* remove
Removes the tnc validatore from the parser *parserObj* and frees all information, stored by it.

tnc *parserObj* getValidateCmd?validateCmdName?

Returns a new created validation command, if one is available from the parser command, otherwise it signals error. The name of the validation command is the *validateCmdName*, if this optional argument was given, or a random chosen name. A validation command is available in a parser command, if the parser with tnc enabled was previously used, to parse an XML document with a valid doctype declaration, a valid external subset, if one was given by the doctype declaration, and a valid internal subset. The further document doesn't need to be valid, to make the validation command available. The validation command can only get received one time from the parser command. The created validation command has this syntax:

```
validationCmd method ?args?
```

The valid methods are:

validateDocument *domDocument?varName?*
Checks, if the given domDocument is valid against the DTD information represented by the validation command. Returns 1, if the document ist valid, 0 otherwise. If the *varName* argument is given, then the variable it names is set to the detected reason for the validation error or to the empty string in case of a valid document.

validateTree *elementNode?varName?*
Checks, if the given subtree with *domNode* as root element is a possible valid subtree of a document conforming to the DTD information represented by the validation command. IDREF could not checked, while validating only a subtree, but it is checked, that every known ID attribute in the subtree is unique. Returns 1, if the subtree is OK, 0 otherwise. If the *varName* argument is given, then the variable it names is set to the detected reason for the validation error or to the empty string in case of a valid subtree.

validateAttributes *elementNode?varName?*
Checks, if there is an element declaration for the name of the *elementNode* in the DTD represented by the validation command and, if yes, if the attributes of the *elementNode* are conform to the ATTLIST declarations for that element in the DTD. Returns 1, if the attributes and there value types are OK, 0 otherwise. If the *varName* argument is given, then the variable it names is set to the detected reason for the validation error or to the empty string in case the element has all its required attributes, only declared attributes and the values of the attributes matches there type.

delete
Deletes the validation command and frees the memory used by it. Returns the empty string.

Bugs

The validation error reports could be much more informative and user-friendly.

The validator doesn't detect ambiguous content models (see XML recomendation Section 3.2.1 and Appendix E). Most Java validators also doesn't, but handle such content models right anyhow. Tnc does not; if your DTD has such ambiguous content models, tnc can not used to validate documents against such (not completely XML spec compliant) DTDs.

It isn't possible to validate XML documents with standalone="yes" in the XML Declaration

Violations of the validity constraints Proper Group/PE Nesting and Proper Conditional Section/PE Nesting are not detected. They could only happen inside a invalid DTD, not in the content of a document.

Keywords: tnc, Validation, DTD

xpathFunc

Name

::dom::xpathFunc - Scripted XPath functions

Synopsis

```
::dom::xpathFunc::funcName ctxNode pos nodeListType nodeList ?type arg
    type arg...?
::dom::xpathFunc::namespaceURL::funcName ctxNode pos nodeListType
    nodeList ?type arg type arg...?
```

Description

tDOM's XPath engine supports custom XPath functions implemented by Tcl commands. When it encounters a function call to an unknown function name without a namespace qualifier, the XPath engine looks for a Tcl command of the same name in the **::dom::xpathFunc** namespace. If it encounters an unknown namespace-qualified function call, it looks for a Tcl namespace with a name equal to the function's full namespace URI inside **::dom::xpathFunc**, and a Tcl command named for the local part of the function's name inside that namespace. If it finds such a Tcl command, it is executed with at least the following arguments, to describe the current XPath context:

ctxNode
: The **domNode** object command of the XPath context node.

pos
: The XPath context position, in zero-based form - that is, the return value of the standard XPath **position()** function in the context would be **$pos - 1**.

nodeListType
: The type name for *nodeList*, as for the **selectNodes** method's *typeVar*.

nodeList
: The list of the current worked on XPath step result (as **selectNodes** would return them) in document order. *ctxNode* will be equal to **[lindex $nodeList $pos]**. If the function call includes any arguments, two arguments will be appended to the command's argument list for each one:

type
: The argument's type name, as for **selectNodes**'s *typeVar*.

val
: The argument's value, as **selectNodes** would return it in an XPath expression's result set.
The command is required to return a 1- or 2-element Tcl list to provide the result of the XPath function call. If the result list has two elements, the first is the result's XPath type name, and the second is an appropriately encoded value (note that the *attrnodes* type name is not supported):

bool
: Tcl boolean value acceptable to **Tcl_GetBooleanFromObj**.

number

Tcl numeric value acceptable to **Tcl_GetSizeIntFromObj** or **Tcl_GetDoubleFromObj**.

string
Simple string.

nodes
Tcl list of **domNode** object commands.

attrvalues
Alias for *string*. If the result list has only one element, it is treated as a simple string. It is an error for the result to have zero elements, more than two elements, or to be invalid as a Tcl list, and it is an error if the *val* of a two-part result does not match the requirements described above for its *type*.

Examples

A simple workalike for the XPath 2/3 **fn:matches()** function, not otherwise available in an XPath 1.0 engine such as tDOM's:

```
proc ::dom::xpathFunc::regmatch {
    ctxNode pos nodeListType nodeList
    inputType inputVal patternType patternVal
} {
    set input [::dom::xpathFuncHelper::coerce2string $inputType
    $inputVal]
    set pattern [::dom::xpathFuncHelper::coerce2string $patternType
    $patternVal]
    return [list bool [regexp -- $pattern $input]]
}
```

Helper procs

The **::dom::xpathFuncHelper** namespace contains helper procs for the convenience of scripted XPath functions:

coerce2number *type val*
Given a *type* and *val* as provided to scripted XPath functions in their argument lists, convert the *val* to a number in a manner appropriate for its *type*:

empty
Always zero.

number
Unchanged.

string
Unchanged. (Note that no error is raised if *val* is not numeric.)

attrvalues
value's first element.

nodes
The result of the **number()** XPath function called on the first node in the list.

attrnodes
The value of the single attribute assumed to be in *value*. Note that no error is raised if this value is non-numeric.

coerce2string ***type val***
As for **coerce2number**, but convert *val* to a pure string:

empty
The empty string.

number
Unchanged.

string
Unchanged.

attrvalues
value's first element.

nodes
The result of the **string()** XPath function called on the first node in the list.

attrnodes
The value of the single attribute assumed to be in *value*.

Limits

Custom XPath function names are limited to 200 characters, including any namespace URI and the **::** Tcl namespace separator between it and the local part. Function calls may have a maximum of 22 arguments (the argument values itself may be large nodesets). If you really need more this limits may be adjusted by build time defines. Tcl commands created with the deprecated **Tcl_CreateCommand** interface cannot be used as scripted XPath functions.

You **must not** alter any of the DOM trees from which nodes are provided to a scripted XPath function as argument (this is true for the ctxNode argument as well as for the nodes in the nodeList argument). Use them strictly read-only. Ignoring this advice may have any unpredictable results including segmentation faults or security problems.

Keywords: xpathFunc, XPath

thread

thread

Name

thread - Extension for script access to Tcl threading

Synopsis

package require Tcl 9.0
package require thread ?**3.0**?
thread::create ?*-joinable*? ?*-preserved*? ?*script*?
thread::preserve ?*id*?
thread::release ?*-wait*? ?*id*?
thread::id
thread::errorproc ?*procname*?
thread::cancel ?*-unwind*? *id* ?*result*?
thread::unwind
thread::exit ?*status*?

thread::names
thread::exists *id*
thread::send ?*-async*? ?*-head*? *id script* ?*varname*?
thread::broadcast *script*
thread::wait
thread::eval ?*-lock mutex*? *arg* ?*arg ...*?
thread::join *id*
thread::configure *id* ?*option*? ?*value*? ?*... ...*?
thread::transfer *id channel*
thread::detach *channel*
thread::attach *channel*
thread::mutex
thread::mutex create ?*-recursive*?
thread::mutex destroy *mutex*
thread::mutex lock *mutex*
thread::mutex unlock *mutex*
thread::rwmutex
thread::rwmutex create
thread::rwmutex destroy *mutex*
thread::rwmutex rlock *mutex*
thread::rwmutex wlock *mutex*
thread::rwmutex unlock *mutex*
thread::cond
thread::cond create
thread::cond destroy *cond*
thread::cond notify *cond*
thread::cond wait *cond mutex* ?*ms*?

Description

The **thread** extension creates threads that contain Tcl interpreters, and it lets you send scripts to those threads for evaluation. Additionally, it provides script-level access to basic thread synchronization primitives, like mutexes and condition variables.

Commands

This section describes commands for creating and destroying threads and sending scripts to threads for evaluation.

thread::create ?-joinable? ?-preserved? ?script?

This command creates a thread that contains a Tcl interpreter. The Tcl interpreter either evaluates the optional **script**, if specified, or it waits in the event loop for scripts that arrive via the **thread::send** command. The result, if any, of the optional **script** is never returned to the caller. The result of **thread::create** is the ID of the thread. This is the opaque handle which identifies the newly created thread for all other package commands. The handle of the thread goes out of scope automatically when thread is marked for exit (see the **thread::release** command below). If the optional **script** argument contains the **thread::wait** command the thread will enter into the event loop. If such command is not found in the **script** the thread will run the **script** to the end and exit. In that case, the handle

may be safely ignored since it refers to a thread which does not exists any more at the time when the command returns. Using flag **-joinable** it is possible to create a joinable thread, i.e. one upon whose exit can be waited upon by using **thread::join** command. Note that failure to join a thread created with **-joinable** flag results in resource and memory leaks. Threads created by the **thread::create** cannot be destroyed forcefully. Consequently, there is no corresponding thread destroy command. A thread may only be released using the **thread::release** and if its internal reference count drops to zero, the thread is marked for exit. This kicks the thread out of the event loop servicing and the thread continues to execute commands passed in the **script** argument, following the **thread::wait** command. If this was the last command in the script, as usually the case, the thread will exit. It is possible to create a situation in which it may be impossible to terminate the thread, for example by putting some endless loop after the **thread::wait** or entering the event loop again by doing an vwait-type of command. In such cases, the thread may never exit. This is considered to be a bad practice and should be avoided if possible. This is best illustrated by the example below:

```
# You should never do ...
set tid [thread::create {
    package require Http
    thread::wait
    vwait forever ; # <-- this!
}]
```

The thread created in the above example will never be able to exit. After it has been released with the last matching **thread::release** call, the thread will jump out of the **thread::wait** and continue to execute commands following. It will enter **vwait** command and wait endlessly for events. There is no way one can terminate such thread, so you wouldn't want to do this! Each newly created has its internal reference counter set to 0 (zero), i.e. it is unreserved. This counter gets incremented by a call to **thread::preserve** and decremented by a call to **thread::release** command. These two commands implement simple but effective thread reservation system and offer predictable and controllable thread termination capabilities. It is however possible to create initially preserved threads by using flag **-preserved** of the **thread::create** command. Threads created with this flag have the initial value of the reference counter of 1 (one), and are thus initially marked reserved.

thread::preserve ?id?

This command increments the thread reference counter. Each call to this command increments the reference counter by one (1). Command returns the value of the reference counter after the increment. If called with the optional thread **id**, the command preserves the given thread. Otherwise the current thread is preserved. With reference counting, one can implement controlled access to a shared Tcl thread. By incrementing the reference counter, the caller signalizes that he/she wishes to use the thread for a longer period of time. By decrementing the counter, caller signalizes that he/she has finished using the thread.

thread::release ?-wait? ?id?

This command decrements the thread reference counter. Each call to this command decrements the reference counter by one (1). If called with the optional thread **id**, the command releases the given thread. Otherwise, the current thread is released. Command returns the value of the reference counter after the decrement. When the reference counter

reaches zero (0), the target thread is marked for termination. You should not reference the thread after the **thread::release** command returns zero or negative integer. The handle of the thread goes out of scope and should not be used any more. Any following reference to the same thread handle will result in Tcl error. Optional flag **-wait** instructs the caller thread to wait for the target thread to exit, if the effect of the command would result in termination of the target thread, i.e. if the return result would be zero (0). Without the flag, the caller thread does not wait for the target thread to exit. Care must be taken when using the **-wait**, since this may block the caller thread indefinitely. This option has been implemented for some special uses of the extension and is deprecated for regular use. Regular users should create joinable threads by using the **-joinable** option of the **thread::create** command and the **thread::join** to wait for thread to exit.

thread::id

This command returns the ID of the current thread.

thread::errorproc ?procname?

This command sets a handler for errors that occur in scripts sent asynchronously, using the **-async** flag of the **thread::send** command, to other threads. If no handler is specified, the current handler is returned. The empty string resets the handler to default (unspecified) value. An uncaught error in a thread causes an error message to be sent to the standard error channel. This default reporting scheme can be changed by registering a procedure which is called to report the error. The *procname* is called in the interpreter that invoked the **thread::errorproc** command. The *procname* is called like this:

```
myerrorproc thread_id errorInfo
```

thread::cancel ?-unwind? *id* ?result?

This command requires Tcl version 8.6 or higher. Cancels the script being evaluated in the thread given by the *id* parameter. Without the **-unwind** switch the evaluation stack for the interpreter is unwound until an enclosing catch command is found or there are no further invocations of the interpreter left on the call stack. With the **-unwind** switch the evaluation stack for the interpreter is unwound without regard to any intervening catch command until there are no further invocations of the interpreter left on the call stack. If *result* is present, it will be used as the error message string; otherwise, a default error message string will be used.

thread::unwind

Use of this command is deprecated in favour of more advanced thread reservation system implemented with **thread::preserve** and **thread::release** commands. Support for **thread::unwind** command will disappear in some future major release of the extension. This command stops a prior **thread::wait** command. Execution of the script passed to newly created thread will continue from the **thread::wait** command. If **thread::wait** was the last command in the script, the thread will exit. The command returns empty result but may trigger Tcl error with the message "target thread died" in some situations.

thread::exit ?status?

Use of this command is deprecated in favour of more advanced thread reservation system implemented with **thread::preserve** and **thread::release** commands. Support for **thread::exit** command will disappear in some future major release of the extension. This command forces a thread stuck in the **thread::wait** command to unconditionally exit. The

thread's exit status defaults to 666 and can be specified using the optional *status* argument. The execution of **thread::exit** command is guaranteed to leave the program memory in the inconsistent state, produce memory leaks and otherwise affect other subsystem(s) of the Tcl application in an unpredictable manner. The command returns empty result but may trigger Tcl error with the message "target thread died" in some situations.

thread::names

This command returns a list of thread IDs. These are only for threads that have been created via **thread::create** command. If your application creates other threads at the C level, they are not reported by this command.

thread::exists *id*

Returns true (1) if thread given by the *id* parameter exists, false (0) otherwise. This applies only for threads that have been created via **thread::create** command.

thread::send ?-async? ?-head? *id script* ?varname?

This command passes a *script* to another thread and, optionally, waits for the result. If the **-async** flag is specified, the command does not wait for the result and it returns empty string. The target thread must enter it's event loop in order to receive scripts sent via this command. This is done by default for threads created without a startup script. Threads can enter the event loop explicitly by calling **thread::wait** or any other relevant Tcl/Tk command, like **update**, **vwait**, etc. Optional **varname** specifies name of the variable to store the result of the *script*. Without the **-async** flag, the command returns the evaluation code, similarly to the standard Tcl **catch** command. If, however, the **-async** flag is specified, the command returns immediately and caller can later **vwait** on ?varname? to get the result of the passed *script*

```
set t1 [thread::create]
set t2 [thread::create]
thread::send -async $t1 "set a 1" result
thread::send -async $t2 "set b 2" result
for {set i 0} {$i < 2} {incr i} {
    vwait result
}
```

In the above example, two threads were fed work and both of them were instructed to signalize the same variable "result" in the calling thread. The caller entered the event loop twice to get both results. Note, however, that the order of the received results may vary, depending on the current system load, type of work done, etc, etc. Many threads can simultaneously send scripts to the target thread for execution. All of them are entered into the event queue of the target thread and executed on the FIFO basis, intermingled with optional other events pending in the event queue of the target thread. Using the optional ?-head? switch, scripts posted to the thread's event queue can be placed on the head, instead on the tail of the queue, thus being executed in the LIFO fashion.

thread::broadcast *script*

This command passes a *script* to all threads created by the package for execution. It does not wait for response from any of the threads. The sending thread (the one executing the thread::broadcast command) does not send the script to itself. Each receiving thread is directed to place evaluation of the send script at the head of its event queue, stepping in front of any other events awaiting processing.

thread::wait

This enters the event loop so a thread can receive messages from the **thread::send** command. This command should only be used within the script passed to the **thread::create** . It should be the very last command in the script. If this is not the case, the exiting thread will continue executing the script lines past the **thread::wait** which is usually not what you want and/or expect.

```
set t1 [thread::create {
    #
    # Do some initialization work here
    #
    thread::wait ; # Enter the event loop
}]
```

thread::eval ?-lock mutex? *arg* ?arg ...?

This command concatenates passed arguments and evaluates the resulting script under the mutex protection. If no mutex is specified by using the ?-lock mutex? optional argument, the internal static mutex is used.

thread::join *id*

This command waits for the thread with ID *id* to exit and then returns it's exit code. Errors will be returned for threads which are not joinable or already waited upon by another thread. Upon the join the handle of the thread has gone out of scope and should not be used any more.

thread::configure *id* ?option? ?value? ?...?

This command configures various low-level aspects of the thread with ID *id* in the similar way as the standard Tcl command **fconfigure** configures some Tcl channel options. Options currently supported are: **-eventmark** and **-unwindonerror**. When **-eventmark** is provided with a value greater than 0 (zero), that value is the maximum number of asynchronously posted scripts that may be pending for the thread. **thread::send -async** blocks until the number of pending scripts in the event loop drops below the **-eventmark** value. When **-unwindonerror** is provided with a value of true, an error result in a script causes the thread to unwind, making it unavailable to evaluate additional scripts.

thread::transfer *id channel*

This moves the specified *channel* from the current thread and interpreter to the main interpreter of the thread with the given *id*. After the move the current interpreter has no access to the channel any more, but the main interpreter of the target thread will be able to use it from now on. The command waits until the other thread has incorporated the channel. Because of this it is possible to deadlock the participating threads by commanding the other through a synchronous **thread::send** to transfer a channel to us. This easily extends into longer loops of threads waiting for each other. Other restrictions: the channel in question must not be shared among multiple interpreters running in the sending thread. This automatically excludes the special channels for standard input, output and error. Due to the internal Tcl core implementation and the restriction on transferring shared channels, one has to take extra measures when transferring socket channels created by accepting the connection out of the **socket** commands callback procedures:

```
    socket -server _Accept 2200
    proc _Accept {s ipaddr port} {
        after idle [list Accept $s $ipaddr $port]
    }
    proc Accept {s ipaddr port} {
        set tid [thread::create]
        thread::transfer $tid $s
    }
```

thread::detach *channel*

This detaches the specified *channel* from the current thread and interpreter. After that, the current interpreter has no access to the channel any more. The channel is in the parked state until some other (or the same) thread attaches the channel again with **thread::attach**. Restrictions: same as for transferring shared channels with the **thread::transfer** command.

thread::attach *channel*

This attaches the previously detached *channel* in the current thread/interpreter. For already existing channels, the command does nothing, i.e. it is not an error to attach the same channel more than once. The first operation will actually perform the operation, while all subsequent operation will just do nothing. Command throws error if the *channel* cannot be found in the list of detached channels and/or in the current interpreter.

thread::mutex

Mutexes are most common thread synchronization primitives. They are used to synchronize access from two or more threads to one or more shared resources. This command provides script-level access to exclusive and/or recursive mutexes. Exclusive mutexes can be locked only once by one thread, while recursive mutexes can be locked many times by the same thread. For recursive mutexes, number of lock and unlock operations must match, otherwise, the mutex will never be released, which would lead to various deadlock situations. Care has to be taken when using mutexes in an multithreading program. Improper use of mutexes may lead to various deadlock situations, especially when using exclusive mutexes. The **thread::mutex** command supports following subcommands and options:

thread::mutex create ?-recursive?

Creates the mutex and returns it's opaque handle. This handle should be used for any future reference to the newly created mutex. If no optional ?-recursive? argument was specified, the command creates the exclusive mutex. With the ?-recursive? argument, the command creates a recursive mutex.

thread::mutex destroy *mutex*

Destroys the *mutex*. Mutex should be in unlocked state before the destroy attempt. If the mutex is locked, the command will throw Tcl error.

thread::mutex lock *mutex*

Locks the *mutex*. Locking the exclusive mutex may throw Tcl error if on attempt to lock the same mutex twice from the same thread. If your program logic forces you to lock the same mutex twice or more from the same thread (this may happen in recursive procedure invocations) you should consider using the recursive mutexes.

thread::mutex unlock *mutex*

Unlocks the *mutex* so some other thread may lock it again. Attempt to unlock the already

unlocked mutex will throw Tcl error.

thread::rwmutex
This command creates many-readers/single-writer mutexes. Reader/writer mutexes allow you to serialize access to a shared resource more optimally. In situations where a shared resource gets mostly read and seldom modified, you might gain some performance by using reader/writer mutexes instead of exclusive or recursive mutexes. For reading the resource, thread should obtain a read lock on the resource. Read lock is non-exclusive, meaning that more than one thread can obtain a read lock to the same resource, without waiting on other readers. For changing the resource, however, a thread must obtain a exclusive write lock. This lock effectively blocks all threads from gaining the read-lock while the resource is been modified by the writer thread. Only after the write lock has been released, the resource may be read-locked again. The **thread::rwmutex** command supports following subcommands and options:

thread::rwmutex create
Creates the reader/writer mutex and returns it's opaque handle. This handle should be used for any future reference to the newly created mutex.

thread::rwmutex destroy *mutex*
Destroys the reader/writer *mutex*. If the mutex is already locked, attempt to destroy it will throw Tcl error.

thread::rwmutex rlock *mutex*
Locks the *mutex* for reading. More than one thread may read-lock the same *mutex* at the same time.

thread::rwmutex wlock *mutex*
Locks the *mutex* for writing. Only one thread may write-lock the same *mutex* at the same time. Attempt to write-lock same *mutex* twice from the same thread will throw Tcl error.

thread::rwmutex unlock *mutex*
Unlocks the *mutex* so some other thread may lock it again. Attempt to unlock already unlocked *mutex* will throw Tcl error.

thread::cond
This command provides script-level access to condition variables. A condition variable creates a safe environment for the program to test some condition, sleep on it when false and be awakened when it might have become true. A condition variable is always used in the conjunction with an exclusive mutex. If you attempt to use other type of mutex in conjunction with the condition variable, a Tcl error will be thrown. The command supports following subcommands and options:

thread::cond create
Creates the condition variable and returns it's opaque handle. This handle should be used for any future reference to newly created condition variable.

thread::cond destroy *cond*
Destroys condition variable *cond*. Extreme care has to be taken that nobody is using (i.e. waiting on) the condition variable, otherwise unexpected errors may happen.

thread::cond notify *cond*
Wakes up all threads waiting on the condition variable *cond*.

thread::cond wait *cond mutex* ?ms?

This command is used to suspend program execution until the condition variable *cond* has been signalled or the optional timer has expired. The exclusive *mutex* must be locked by the calling thread on entrance to this command. If the mutex is not locked, Tcl error is thrown. While waiting on the *cond*, the command releases *mutex*. Before returning to the calling thread, the command re-acquires the *mutex* again. Unlocking the *mutex* and waiting on the condition variable *cond* is done atomically. The **ms** command option, if given, must be an integer specifying time interval in milliseconds the command waits to be signalled. Otherwise the command waits on condition notify forever. In multithreading programs, there are many situations where a thread has to wait for some event to happen until it is allowed to proceed. This is usually accomplished by repeatedly testing a condition under the mutex protection and waiting on the condition variable until the condition evaluates to true:

```
set mutex [thread::mutex create]
set cond  [thread::cond  create]

thread::mutex lock $mutex
while {<some_condition_is_true>} {
    thread::cond wait $cond $mutex
}
# Do some work under mutex protection
thread::mutex unlock $mutex
```

Repeated testing of the condition is needed since the condition variable may get signalled without the condition being actually changed (spurious thread wake-ups, for example).

Discussion

The fundamental threading model in Tcl is that there can be one or more Tcl interpreters per thread, but each Tcl interpreter should only be used by a single thread which created it. A "shared memory" abstraction is awkward to provide in Tcl because Tcl makes assumptions about variable and data ownership. Therefore this extension supports a simple form of threading where the main thread can manage several background, or "worker" threads. For example, an event-driven server can pass requests to worker threads, and then await responses from worker threads or new client requests. Everything goes through the common Tcl event loop, so message passing between threads works naturally with event-driven I/O, **vwait** on variables, and so forth. For the transfer of bulk information it is possible to move channels between the threads.

For advanced multithreading scripts, script-level access to two basic synchronization primitives, mutex and condition variables, is also supported.

Keywords: thread, events, message passing, mutex, synchronization

tpool

Name

tpool - Part of the Tcl threading extension implementing pools of worker threads.

Synopsis

package require Tcl 9.0
package require thread ?**3.0**?

tpool::create ?*options*?
tpool::names
tpool::post ?*-detached*? ?*-nowait*? *tpool script*
tpool::wait *tpool joblist* ?*varname*?
tpool::cancel *tpool joblist* ?*varname*?
tpool::get *tpool job*
tpool::preserve *tpool*
tpool::release *tpool*
tpool::suspend *tpool*
tpool::resume *tpool*

Description

This package creates and manages pools of worker threads. It allows you to post jobs to worker threads and wait for their completion. The threadpool implementation is Tcl event-loop aware. That means that any time a caller is forced to wait for an event (job being completed or a worker thread becoming idle or initialized), the implementation will enter the event loop and allow for servicing of other pending file or timer (or any other supported) events.

Commands

tpool::create ?options?

This command creates new threadpool. It accepts several options as key-value pairs. Options are used to tune some threadpool parameters. The command returns the ID of the newly created threadpool. Following options are supported:

-minworkers *number*

Minimum number of worker threads needed for this threadpool instance. During threadpool creation, the implementation will create somany worker threads upfront and will keep at least number of them alive during the lifetime of the threadpool instance. Default value of this parameter is 0 (zero). which means that a newly threadpool will have no worker threads initially. All worker threads will be started on demand by callers running **tpool::post** command and posting jobs to the job queue.

-maxworkers *number*

Maximum number of worker threads allowed for this threadpool instance. If a new job is pending and there are no idle worker threads available, the implementation will try to create new worker thread. If the number of available worker threads is lower than the given number, new worker thread will start. The caller will automatically enter the event loop and wait until the worker thread has initialized. If. however, the number of available worker threads is equal to the given number, the caller will enter the event loop and wait for the first worker thread to get idle, thus ready to run the job. Default value of this parameter is 4 (four), which means that the threadpool instance will allow maximum of 4 worker threads running jobs or being idle waiting for new jobs to get posted to the job queue.

-idletime *seconds*

Time in seconds an idle worker thread waits for the job to get posted to the job queue. If no job arrives during this interval and the time expires, the worker thread will check the number of currently available worker threads and if the number is higher than the number set by the **minthreads** option, it will exit. If an **exitscript** has been defined, the exiting worker thread will first run the script and then exit. Errors from the exit script, if any, are ignored. The idle

worker thread is not servicing the event loop. If you, however, put the worker thread into the event loop, by evaluating the **vwait** or other related Tcl commands, the worker thread will not be in the idle state, hence the idle timer will not be taken into account. Default value for this option is unspecified.

-initcmd *script*
Sets a Tcl script used to initialize new worker thread. This is usually used to load packages and commands in the worker, set default variables, create namespaces, and such. If the passed script runs into a Tcl error, the worker will not be created and the initiating command (either the **tpool::create** or **tpool::post**) will throw error. Default value for this option is unspecified, hence, the Tcl interpreter of the worker thread will contain just the initial set of Tcl commands.

-exitcmd *script*
Sets a Tcl script run when the idle worker thread exits. This is normally used to cleanup the state of the worker thread, release reserved resources, cleanup memory and such. Default value for this option is unspecified, thus no Tcl script will run on the worker thread exit.

tpool::names
This command returns a list of IDs of threadpools created with the **tpool::create** command. If no threadpools were found, the command will return empty list.

tpool::post ?-detached? ?-nowait? *tpool script*
This command sends a *script* to the target *tpool* threadpool for execution. The script will be executed in the first available idle worker thread. If there are no idle worker threads available, the command will create new one, enter the event loop and service events until the newly created thread is initialized. If the current number of worker threads is equal to the maximum number of worker threads, as defined during the threadpool creation, the command will enter the event loop and service events while waiting for one of the worker threads to become idle. If the optional ?-nowait? argument is given, the command just places the job in the pool's job queue without waiting for an idle worker or creating a new one, even if the pool would still have room for additional threads. The command returns the ID of the posted job. This ID is used for subsequent **tpool::wait**, **tpool::get** and **tpool::cancel** commands to wait for and retrieve result of the posted script, or cancel the posted job respectively. If the optional ?-detached? argument is specified, the command will post a detached job. A detached job can not be cancelled or waited upon and is not identified by the job ID. If the threadpool *tpool* is not found in the list of active thread pools, the command will throw error. The error will also be triggered if the newly created worker thread fails to initialize.

tpool::wait *tpool joblist* ?varname?
This command waits for one or many jobs, whose job IDs are given in the *joblist* to get processed by the worker thread(s). If none of the specified jobs are ready, the command will enter the event loop, service events and wait for the first job to get ready. The command returns the list of completed job IDs. If the optional variable ?varname? is given, it will be set to the list of jobs in the *joblist* which are still pending. If the threadpool *tpool* is not found in the list of active thread pools, the command will throw error.

tpool::cancel *tpool joblist* ?varname?
This command cancels the previously posted jobs given by the *joblist* to the pool *tpool*. Job cancellation succeeds only for job still waiting to be processed. If the job is already being executed by one of the worker threads, the job will not be cancelled. The command returns

the list of cancelled job IDs. If the optional variable ?varname? is given, it will be set to the list of jobs in the *joblist* which were not cancelled. If the threadpool *tpool* is not found in the list of active thread pools, the command will throw error.

tpool::get *tpool job*
: This command retrieves the result of the previously posted *job*. Only results of jobs waited upon with the **tpool::wait** command can be retrieved. If the execution of the script resulted in error, the command will throw the error and update the **errorInfo** and **errorCode** variables correspondingly. If the pool *tpool* is not found in the list of threadpools, the command will throw error. If the job *job* is not ready for retrieval, because it is currently being executed by the worker thread, the command will throw error.

tpool::preserve *tpool*
: Each call to this command increments the reference counter of the threadpool *tpool* by one (1). Command returns the value of the reference counter after the increment. By incrementing the reference counter, the caller signalizes that he/she wishes to use the resource for a longer period of time.

tpool::release *tpool*
: Each call to this command decrements the reference counter of the threadpool *tpool* by one (1).Command returns the value of the reference counter after the decrement. When the reference counter reaches zero (0), the threadpool *tpool* is marked for termination. You should not reference the threadpool after the **tpool::release** command returns zero. The *tpool* handle goes out of scope and should not be used any more. Any following reference to the same threadpool handle will result in Tcl error.

tpool::suspend *tpool*
: Suspends processing work on this queue. All pool workers are paused but additional work can be added to the pool. Note that adding the additional work will not increase the number of workers dynamically as the pool processing is suspended. Number of workers is maintained to the count that was found prior suspending worker activity. If you need to assure certain number of worker threads, use the **minworkers** option of the **tpool::create** command.

tpool::resume *tpool*
: Resume processing work on this queue. All paused (suspended) workers are free to get work from the pool. Note that resuming pool operation will just let already created workers to proceed. It will not create additional worker threads to handle the work posted to the pool's work queue.

Discussion

Threadpool is one of the most common threading paradigm when it comes to server applications handling a large number of relatively small tasks. A very simplistic model for building a server application would be to create a new thread each time a request arrives and service the request in the new thread. One of the disadvantages of this approach is that the overhead of creating a new thread for each request is significant; a server that created a new thread for each request would spend more time and consume more system resources in creating and destroying threads than in processing actual user requests. In addition to the overhead of creating and destroying threads, active threads consume system resources. Creating too many threads can cause the system to run out of memory or trash due to excessive memory consumption.

A thread pool offers a solution to both the problem of thread life-cycle overhead and the problem

of resource trashing. By reusing threads for multiple tasks, the thread-creation overhead is spread over many tasks. As a bonus, because the thread already exists when a request arrives, the delay introduced by thread creation is eliminated. Thus, the request can be serviced immediately. Furthermore, by properly tuning the number of threads in the thread pool, resource thrashing may also be eliminated by forcing any request to wait until a thread is available to process it.

Keywords: tpool, thread, threadpool

tsv

Name

tsv - Part of the Tcl threading extension allowing script level manipulation of data shared between threads.

Synopsis

package require Tcl 9.0
package require thread ?**3.0**?
tsv::names ?*pattern*?
tsv::object *varname element*
tsv::set *varname element* ?*value*?
tsv::get *varname element* ?*namedvar*?
tsv::unset *varname* ?*element*?
tsv::exists *varname element*
tsv::pop *varname element*
tsv::move *varname oldname newname*
tsv::incr *varname element* ?*count*?
tsv::append *varname element value* ?*value* ...?
tsv::lock *varname arg* ?*arg* ...?
tsv::handlers
tsv::lappend *varname element value* ?*value* ...?
tsv::linsert *varname element index value* ?*value* ...?
tsv::lreplace *varname element first last* ?*value* ...?
tsv::llength *varname element*
tsv::lindex *varname element* ?*index*?
tsv::lrange *varname element from to*
tsv::lsearch *varname element* ?*options*? *pattern*
tsv::lset *varname element index* ?*index* ...? *value*
tsv::lpop *varname element* ?*index*?
tsv::lpush *varname element* ?*index*?
tsv::array set *varname list*
tsv::array get *varname* ?*pattern*?
tsv::array names *varname* ?*pattern*?
tsv::array size *varname*
tsv::array reset *varname list*
tsv::array bind *varname handle*
tsv::array unbind *varname*

tsv::array isbound *varname*
tsv::keyldel *varname keylist key*
tsv::keylget *varname keylist key* ?*retvar*?
tsv::keylkeys *varname keylist* ?*key*?
tsv::keylset *varname keylist key value* ?*key value*..?

Description

This section describes commands implementing thread shared variables. A thread shared variable is very similar to a Tcl array but in contrast to a Tcl array it is created in shared memory and can be accessed from many threads at the same time. Important feature of thread shared variable is that each access to the variable is internally protected by a mutex so script programmer does not have to take care about locking the variable himself.

Thread shared variables are not bound to any thread explicitly. That means that when a thread which created any of thread shared variables exits, the variable and associated memory is not unset/reclaimed. User has to explicitly unset the variable to reclaim the memory consumed by the variable.

Element commands

tsv::names ?pattern?
Returns names of shared variables matching optional ?pattern? or all known variables if pattern is omitted.

tsv::object *varname element*
Creates object accessor command for the *element* in the shared variable *varname*. Using this command, one can apply most of the other shared variable commands as method functions of the element object command. The object command is automatically deleted when the element which this command is pointing to is unset.

```
% tsv::set foo bar "A shared string"
% set string [tsv::object foo bar]
% $string append " appended"
=> A shared string appended
```

tsv::set *varname element* ?value?
Sets the value of the *element* in the shared variable *varname* to *value* and returns the value to caller. The *value* may be omitted, in which case the command will return the current value of the element. If the element cannot be found, error is triggered.

tsv::get *varname element* ?namedvar?
Retrieves the value of the *element* from the shared variable *varname*. If the optional argument *namedvar* is given, the value is stored in the named variable. Return value of the command depends of the existence of the optional argument *namedvar*. If the argument is omitted and the requested element cannot be found in the shared array, the command triggers error. If, however, the optional argument is given on the command line, the command returns true (1) if the element is found or false (0) if the element is not found.

tsv::unset *varname* ?element?
Unsets the *element* from the shared variable *varname*. If the optional element is not given, it deletes the variable.

tsv::exists *varname element*
Checks whether the *element* exists in the shared variable *varname* and returns true (1) if it does or false (0) if it doesn't.

tsv::pop *varname element*
Returns value of the *element* in the shared variable *varname* and unsets the element, all in one atomic operation.

tsv::move *varname oldname newname*
Renames the element *oldname* to the *newname* in the shared variable *varname*. This effectively performs an get/unset/set sequence of operations but all in one atomic step.

tsv::incr *varname element* ?count?
Similar to standard Tcl **incr** command but increments the value of the *element* in shared variable *varname* instead of the Tcl variable.

tsv::append *varname element value* ?value ...?
Similar to standard Tcl **append** command but appends one or more values to the *element* in shared variable *varname* instead of the Tcl variable.

tsv::lock *varname arg* ?arg ...?
This command concatenates passed arguments and evaluates the resulting script under the internal mutex protection. During the script evaluation, the entire shared variable is locked. For shared variable commands within the script, internal locking is disabled so no deadlock can occur. It is also allowed to unset the shared variable from within the script. The shared variable is automatically created if it did not exists at the time of the first lock operation.

```
% tsv::lock foo {
    tsv::lappend foo bar 1
    tsv::lappend foo bar 2
    puts stderr [tsv::set foo bar]
    tsv::unset foo
}
```

tsv::handlers
Returns the names of all persistent storage handlers enabled at compile time. See **ARRAY COMMANDS** for details.

List commands

Those command are similar to the equivalently named Tcl command. The difference is that they operate on elements of shared arrays.

tsv::lappend *varname element value* ?value ...?
Similar to standard Tcl **lappend** command but appends one or more values to the *element* in shared variable *varname* instead of the Tcl variable.

tsv::linsert *varname element index value* ?value ...?
Similar to standard Tcl **linsert** command but inserts one or more values at the *index* list position in the *element* in the shared variable *varname* instead of the Tcl variable.

tsv::lreplace *varname element first last* ?value ...?
Similar to standard Tcl **lreplace** command but replaces one or more values between the *first* and *last* position in the *element* of the shared variable *varname* instead of the Tcl variable.

tsv::llength *varname element*

Similar to standard Tcl **llength** command but returns length of the *element* in the shared variable *varname* instead of the Tcl variable.

tsv::lindex *varname element* ?index?
Similar to standard Tcl **lindex** command but returns the value at the *index* list position of the *element* from the shared variable *varname* instead of the Tcl variable.

tsv::lrange *varname element from to*
Similar to standard Tcl **lrange** command but returns values between *from* and *to* list positions from the *element* in the shared variable *varname* instead of the Tcl variable.

tsv::lsearch *varname element* ?options? *pattern*
Similar to standard Tcl **lsearch** command but searches the *element* in the shared variable *varname* instead of the Tcl variable.

tsv::lset *varname element index* ?index ...? *value*
Similar to standard Tcl **lset** command but sets the *element* in the shared variable *varname* instead of the Tcl variable.

tsv::lpop *varname element* ?index?
Similar to the standard Tcl **lindex** command but in addition to returning, it also splices the value out of the *element* from the shared variable *varname* in one atomic operation. In contrast to the Tcl **lindex** command, this command returns no value to the caller.

tsv::lpush *varname element* ?index?
This command performs the opposite of the **tsv::lpop** command. As its counterpart, it returns no value to the caller.

Array commands

This command supports most of the options of the standard Tcl **array** command. In addition to those, it allows binding a shared variable to some persistent storage databases. Currently the persistent options supported are the famous GNU Gdbm and LMDB. These options have to be selected during the package compilation time. The implementation provides hooks for defining other persistency layers, if needed.

tsv::array set *varname list*
Does the same as standard Tcl **array set**.

tsv::array get *varname* ?pattern?
Does the same as standard Tcl **array get**.

tsv::array names *varname* ?pattern?
Does the same as standard Tcl **array names**.

tsv::array size *varname*
Does the same as standard Tcl **array size**.

tsv::array reset *varname list*
Does the same as standard Tcl **array set** but it clears the *varname* and sets new values from the list atomically.

tsv::array bind *varname handle*
Binds the *varname* to the persistent storage *handle*. The format of the *handle* is <handler>:<address>, where <handler> is "gdbm" for GNU Gdbm and "lmdb" for LMDB and <address> is the path to the database file.

tsv::array unbind *varname*

Unbinds the shared *array* from its bound persistent storage.

tsv::array isbound *varname*

Returns true (1) if the shared *varname* is bound to some persistent storage or zero (0) if not.

Keyed list commands

Keyed list commands are borrowed from the TclX package. Keyed lists provide a structured data type built upon standard Tcl lists. This is a functionality similar to structs in the C programming language.

A keyed list is a list in which each element contains a key and value pair. These element pairs are stored as lists themselves, where the key is the first element of the list, and the value is the second. The key-value pairs are referred to as fields. This is an example of a keyed list:

```
    {{NAME  {Frank  Zappa}} {JOB {musician and composer}}}
```

Fields may contain subfields; `.' is the separator character. Subfields are actually fields where the value is another keyed list. Thus the following list has the top level fields ID and NAME, and subfields NAME.FIRST and NAME.LAST:

```
    {ID 106} {NAME {{FIRST Frank} {LAST Zappa}}}
```

There is no limit to the recursive depth of subfields, allowing one to build complex data structures. Keyed lists are constructed and accessed via a number of commands. All keyed list management commands take the name of the variable containing the keyed list as an argument (i.e. passed by reference), rather than passing the list directly.

tsv::keyldel *varname keylist key*

Delete the field specified by *key* from the keyed list *keylist* in the shared variable *varname*. This removes both the key and the value from the keyed list.

tsv::keylget *varname keylist key* ?retvar?

Return the value associated with *key* from the keyed list *keylist* in the shared variable *varname*. If the optional *retvar* is not specified, then the value will be returned as the result of the command. In this case, if key is not found in the list, an error will result. If *retvar* is specified and *key* is in the list, then the value is returned in the variable *retvar* and the command returns 1 if the key was present within the list. If *key* isn't in the list, the command will return 0, and *retvar* will be left unchanged. If {} is specified for *retvar*, the value is not returned, allowing the Tcl programmer to determine if a *key* is present in a keyed list without setting a variable as a side-effect.

tsv::keylkeys *varname keylist* ?key?

Return the a list of the keys in the keyed list *keylist* in the shared variable *varname*. If *key* is specified, then it is the name of a key field whose subfield keys are to be retrieved.

tsv::keylset *varname keylist key value* ?key value..?

Set the value associated with *key*, in the keyed list *keylist* to *value*. If the *keylist* does not exists, it is created. If *key* is not currently in the list, it will be added. If it already exists, *value* replaces the existing value. Multiple keywords and values may be specified, if desired.

Discussion

The current implementation of thread shared variables allows for easy and convenient access to data shared between different threads. Internally, the data is stored in Tcl objects and all

package commands operate on internal data representation, thus minimizing shimmering and improving performance. Special care has been taken to assure that all object data is properly locked and deep-copied when moving objects between threads.

Due to the internal design of the Tcl core, there is no provision of full integration of shared variables within the Tcl syntax, unfortunately. All access to shared data must be performed with the supplied package commands. Also, variable traces are not supported. But even so, benefits of easy, simple and safe shared data manipulation outweighs imposed limitations.

Credits

Thread shared variables are inspired by the nsv interface found in AOLserver, a highly scalable Web server from America Online.

Keywords: tsv, locking, synchronization, thread shared data, threads

ttrace

Name

ttrace - Trace-based interpreter initialization

Synopsis

package require Tcl 9.0
package require thread ?**3.0**?
ttrace::eval *arg* ?*arg* ...?
ttrace::enable
ttrace::disable
ttrace::cleanup
ttrace::update ?*epoch*?
ttrace::getscript
ttrace::atenable *cmd arglist body*
ttrace::atdisable *cmd arglist body*
ttrace::addtrace *cmd arglist body*
ttrace::addscript *name body*
ttrace::addresolver *cmd arglist body*
ttrace::addcleanup *body*
ttrace::addentry *cmd var val*
ttrace::getentry *cmd var*
ttrace::getentries *cmd* ?*pattern*?
ttrace::delentry *cmd*
ttrace::preload *cmd*

Description

This package creates a framework for on-demand replication of the interpreter state across threads in an multithreading application. It relies on the mechanics of Tcl command tracing and the Tcl **unknown** command and mechanism.

The package requires Tcl threading extension but can be alternatively used stand-alone within the AOLserver, a scalable webserver from America Online.

In a nutshell, a short sample illustrating the usage of the ttrace with the Tcl threading extension:

```
% package require ttrace
3.0.1

% set t1 [thread::create {package require ttrace; thread::wait}]
tid0x1802800

% ttrace::eval {proc test args {return test-[thread::id]}}
% thread::send $t1 test
test-tid0x1802800

% set t2 [thread::create {package require ttrace; thread::wait}]
tid0x1804000

% thread::send $t2 test
test-tid0x1804000
```

As seen from above, the **ttrace::eval** and **ttrace::update** commands are used to create a thread-wide definition of a simple Tcl procedure and replicate that definition to all, already existing or later created, threads.

User commands

This section describes user-level commands. Those commands can be used by script writers to control the execution of the tracing framework.

ttrace::eval *arg* ?arg ...?
: This command concatenates given arguments and evaluates the resulting Tcl command with trace framework enabled. If the command execution was ok, it takes necessary steps to automatically propagate the trace epoch change to all threads in the application. For AOLserver, only newly created threads actually receive the epoch change. For the Tcl threading extension, all threads created by the extension are automatically updated. If the command execution resulted in Tcl error, no state propagation takes place. This is the most important user-level command of the package as it wraps most of the commands described below. This greatly simplifies things, because user need to learn just this (one) command in order to effectively use the package. Other commands, as described below, are included mostly for the sake of completeness.

ttrace::enable
: Activates all registered callbacks in the framework and starts a new trace epoch. The trace epoch encapsulates all changes done to the interpreter during the time traces are activated.

ttrace::disable
: Deactivates all registered callbacks in the framework and closes the current trace epoch.

ttrace::cleanup
: Used to clean-up all on-demand loaded resources in the interpreter. It effectively brings Tcl interpreter to its pristine state.

ttrace::update ?epoch?
: Used to refresh the state of the interpreter to match the optional trace ?epoch?. If the optional ?epoch? is not given, it takes the most recent trace epoch.

ttrace::getscript

Returns a synthesized Tcl script which may be sourced in any interpreter. This script sets the stage for the Tcl **unknown** command so it can load traced resources from the in-memory database. Normally, this command is automatically invoked by other higher-level commands like **ttrace::eval** and **ttrace::update**.

Callback commands

A word upfront: the package already includes callbacks for tracing following Tcl commands: **proc** , **namespace**, **variable**, **load**, and **rename**. Additionally, a set of callbacks for tracing resources (object, classes) for the XOTcl v1.3.8+, an OO-extension to Tcl, is also provided. This gives a solid base for solving most of the real-life needs and serves as an example for people wanting to customize the package to cover their specific needs.

Below, you can find commands for registering callbacks in the framework and for writing callback scripts. These callbacks are invoked by the framework in order to gather interpreter state changes, build in-memory database, perform custom-cleanups and various other tasks.

ttrace::atenable *cmd arglist body*
Registers Tcl callback to be activated at **ttrace::enable**. Registered callbacks are activated on FIFO basis. The callback definition includes the name of the callback, *cmd*, a list of callback arguments, *arglist* and the *body* of the callback. Effectively, this actually resembles the call interface of the standard Tcl **proc** command.

ttrace::atdisable *cmd arglist body*
Registers Tcl callback to be activated at **ttrace::disable**. Registered callbacks are activated on FIFO basis. The callback definition includes the name of the callback, *cmd*, a list of callback arguments, *arglist* and the *body* of the callback. Effectively, this actually resembles the call interface of the standard Tcl **proc** command.

ttrace::addtrace *cmd arglist body*
Registers Tcl callback to be activated for tracing the Tcl **cmd** command. The callback definition includes the name of the Tcl command to trace, *cmd*, a list of callback arguments, *arglist* and the *body* of the callback. Effectively, this actually resembles the call interface of the standard Tcl **proc** command.

ttrace::addscript *name body*
Registers Tcl callback to be activated for building a Tcl script to be passed to other interpreters. This script is used to set the stage for the Tcl **unknown** command. Registered callbacks are activated on FIFO basis. The callback definition includes the name of the callback, *name* and the *body* of the callback.

ttrace::addresolver *cmd arglist body*
Registers Tcl callback to be activated by the overloaded Tcl **unknown** command. Registered callbacks are activated on FIFO basis. This callback is used to resolve the resource and load the resource in the current interpreter.

ttrace::addcleanup *body*
Registers Tcl callback to be activated by the **trace::cleanup**. Registered callbacks are activated on FIFO basis.

ttrace::addentry *cmd var val*
Adds one entry to the named in-memory database.

ttrace::getentry *cmd var*
Returns the value of the entry from the named in-memory database.

ttrace::getentries *cmd* ?pattern?
Returns names of all entries from the named in-memory database.

ttrace::delentry *cmd*
Deletes an entry from the named in-memory database.

ttrace::preload *cmd*
Registers the Tcl command to be loaded in the interpreter. Commands registered this way will always be the part of the interpreter and not be on-demand loaded by the Tcl **unknown** command.

Discussion

Common introspective state-replication approaches use a custom Tcl script to introspect the running interpreter and synthesize another Tcl script to replicate this state in some other interpreter. This package, on the contrary, uses Tcl command traces. Command traces are registered on selected Tcl commands, like **proc**, **namespace**, **load** and other standard (and/or user-defined) Tcl commands. When activated, those traces build an in-memory database of created resources. This database is used as a resource repository for the (overloaded) Tcl **unknown** command which creates the requested resource in the interpreter on demand. This way, users can update just one interpreter (master) in one thread and replicate that interpreter state (or part of it) to other threads/interpreters in the process.

Immediate benefit of such approach is the much smaller memory footprint of the application and much faster thread creation. By not actually loading all necessary procedures (and other resources) in every thread at the thread initialization time, but by deferring this to the time the resource is actually referenced, significant improvements in both memory consumption and thread initialization time can be achieved. Some tests have shown that memory footprint of an multithreading Tcl application went down more than three times and thread startup time was reduced for about 50 times. Note that your mileage may vary. Other benefits include much finer control about what (and when) gets replicated from the master to other Tcl thread/interpreters.

Keywords: ttrace, command tracing, introspection

License

bwidget license terms

BWidget ToolKit

Copyright (c) 1998-1999 UNIFIX.

Copyright (c) 2001-2002 ActiveState Corp.

The following terms apply to all files associated with the software

unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute,

and license this software and its documentation for any purpose, provided

that existing copyright notices are retained in all copies and that this

notice is included verbatim in any distributions. No written agreement,

license, or royalty fee is required for any of the authorized uses.

Modifications to this software may be copyrighted by their authors
and need not follow the licensing terms described here, provided that
the new terms are clearly indicated on the first page of each file where
they apply.
IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY
FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES
ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY
DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE
POSSIBILITY OF SUCH DAMAGE.
THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES,
INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE
IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE
NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR
MODIFICATIONS.
GOVERNMENT USE: If you are acquiring this software on behalf of the
U.S. government, the Government shall have only "Restricted Rights"
in the software and related documentation as defined in the Federal
Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you
are acquiring the software on behalf of the Department of Defense, the
software shall be classified as "Commercial Computer Software" and the
Government shall have only "Restricted Rights" as defined in Clause
252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the
authors grant the U.S. Government and others acting in its behalf
permission to use and distribute the software in accordance with the
terms specified in this license.

expect license terms

Expect

Written by: Don Libes, libes at nist.gov, NIST

Design and implementation of this program was paid for by U.S. tax dollars. Therefore it is public domain. However, the author and NIST would appreciate credit if this program or parts of it are used.

itcl, itcl-c license terms

This is a mostly rewritten version of [incr Tcl], which is copyrighted by Arnulf P. Wiedemann (c) Copyright 2008. It is derived from a version written by Lucent Technologies, Inc., and other parties see that copyright below.

The rewritten version is copyrighted with BSD license or Public Domain at your choice.

The original version of this software is copyrighted by Lucent Technologies, Inc., and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

itk license terms

This software is copyrighted by Lucent Technologies, Inc., and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT,

UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

mclistbox license terms

Copyright (c) 1999, Bryan Douglas Oakley All Rights Reserved.

This software is provided AS-IS with no waranty expressed or implied. This software may be used free of charge, though I would appreciate it if you give credit where credit is due and mention my name when you use it.

mpexpr license terms

This software is copyrighted by Tom Poindexter and Talus Technologies, Inc., 1998. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

tclhttpd license terms

TCLHTTPD License

This software is copyrighted by Brent Welch, Sun Microsystems, Inc, Scriptics, and Panasas. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

tcltls license terms

This software is copyrighted by Matt Newman <matt@novadigm.com> and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT

NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

tcludp license terms

Copyright (c) 1999-2003 by Columbia University; all rights reserved Copyright (c) 2003-2006 Pat Thoyts <patthoyts@users.sourceforge.net>

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

tclvfs license terms

This software is copyrighted by the Vince Darley, and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT

NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

tclws license terms

This software is copyrighted by the Gerald W. Leser and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7014 (b) (3) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

tdbc, tdbc-c, tdbcmysql, tdbcodbc license terms

This software is copyrighted by Kevin B. Kenny, and by other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software

and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

tdbcpostgres license terms

This software is copyrighted by Slawomir Cygan, and by other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are

acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

tdbcsqlite3 license terms

This software is copyrighted by the Scriptics Corporation, and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

tdom license terms

The tDOM specific source files come with the Mozilla Public License Version 1.1. If you want to distribute/use tDOM under a different license please feel free to contact the author (loewerj@hotmail.com).

For Expat XML parser package license look into the directory expat.

Mozilla Public License Version 1.1

1. Definitions.

1.0.1. "Commercial Use" means distribution or otherwise making the Covered Code available to a third party.

1.1. "Contributor" means each entity that creates or contributes to the creation of Modifications.

1.2. "Contributor Version" means the combination of the Original Code, prior Modifications used by a Contributor, and the Modifications made by that particular Contributor.

1.3. "Covered Code" means the Original Code or Modifications or the combination of the Original Code and Modifications, in each case including portions thereof.

1.4. "Electronic Distribution Mechanism" means a mechanism generally accepted in the software development community for the electronic transfer of data.

1.5. "Executable" means Covered Code in any form other than Source Code.

1.6. "Initial Developer" means the individual or entity identified as the Initial Developer in the Source Code notice required by Exhibit A.

1.7. "Larger Work" means a work which combines Covered Code or portions thereof with code not governed by the terms of this License.

1.8. "License" means this document.

1.8.1. "Licensable" means having the right to grant, to the maximum extent possible, whether at the time of the initial grant or subsequently acquired, any and all of the rights conveyed herein.

1.9. "Modifications" means any addition to or deletion from the substance or structure of either the Original Code or any previous Modifications. When Covered Code is released as a series of files, a Modification is: Any addition to or deletion from the contents of a file containing Original Code or previous Modifications. Any new file that contains any part of the Original Code or previous Modifications.

1.10. "Original Code" means Source Code of computer software code which is described in the Source Code notice required by Exhibit A as Original Code, and which, at the time of its release under this License is not already Covered Code governed by this License.

1.10.1. "Patent Claims" means any patent claim(s), now owned or hereafter acquired, including without limitation, method, process, and apparatus claims, in any patent Licensable by grantor.

1.11. "Source Code" means the preferred form of the Covered Code for making modifications to it, including all modules it contains, plus any associated interface definition files, scripts used to control compilation and installation of an Executable, or source code differential comparisons against either the Original Code or another well known, available Covered Code of the Contributor's choice. The Source Code can be in a compressed or archival form, provided the appropriate decompression or de-archiving software is widely available for no charge.

1.12. "You" (or "Your") means an individual or a legal entity exercising rights under, and complying with all of the terms of, this License or a future version of this License issued under Section 6.1. For legal entities, "You" includes any entity which controls, is controlled by, or is under common control with You. For purposes of this definition, "control" means (a) the power, direct or indirect, to cause the direction or management of such entity, whether by contract or otherwise, or (b) ownership of more than fifty percent (50%) of the outstanding shares or beneficial ownership of such entity.

1. Source Code License.

2.1. The Initial Developer Grant. The Initial Developer hereby grants You a world-wide, royalty-free, non-exclusive license, subject to third party intellectual property claims:

a. under intellectual property rights (other than patent or trademark) Licensable by Initial

Developer to use, reproduce, modify, display, perform, sublicense and distribute the Original Code (or portions thereof) with or without Modifications, and/or as part of a Larger Work; and b. under Patents Claims infringed by the making, using or selling of Original Code, to make, have made, use, practice, sell, and offer for sale, and/or otherwise dispose of the Original Code (or portions thereof). c. the licenses granted in this Section 2.1 (a) and (b) are effective on the date Initial Developer first distributes Original Code under the terms of this License. d. Notwithstanding Section 2.1 (b) above, no patent license is granted: 1) for code that You delete from the Original Code; 2) separate from the Original Code; or 3) for infringements caused by: i) the modification of the Original Code or ii) the combination of the Original Code with other software or devices.

2.2. Contributor Grant. Subject to third party intellectual property claims, each Contributor hereby grants You a world-wide, royalty-free, non-exclusive license

a. under intellectual property rights (other than patent or trademark) Licensable by Contributor, to use, reproduce, modify, display, perform, sublicense and distribute the Modifications created by such Contributor (or portions thereof) either on an unmodified basis, with other Modifications, as Covered Code and/or as part of a Larger Work; and b. under Patent Claims infringed by the making, using, or selling of Modifications made by that Contributor either alone and/or in combination with its Contributor Version (or portions of such combination), to make, use, sell, offer for sale, have made, and/or otherwise dispose of: 1) Modifications made by that Contributor (or portions thereof); and 2) the combination of Modifications made by that Contributor with its Contributor Version (or portions of such combination). c. the licenses granted in Sections 2.2 (a) and 2.2 (b) are effective on the date Contributor first makes Commercial Use of the Covered Code. d. Notwithstanding Section 2.2 (b) above, no patent license is granted: 1) for any code that Contributor has deleted from the Contributor Version; 2) separate from the Contributor Version; 3) for infringements caused by: i) third party modifications of Contributor Version or ii) the combination of Modifications made by that Contributor with other software (except as part of the Contributor Version) or other devices; or 4) under Patent Claims infringed by Covered Code in the absence of Modifications made by that Contributor.

1. Distribution Obligations.

3.1. Application of License. The Modifications which You create or to which You contribute are governed by the terms of this License, including without limitation Section 2.2. The Source Code version of Covered Code may be distributed only under the terms of this License or a future version of this License released under Section 6.1, and You must include a copy of this License with every copy of the Source Code You distribute. You may not offer or impose any terms on any Source Code version that alters or restricts the applicable version of this License or the recipients' rights hereunder. However, You may include an additional document offering the additional rights described in Section 3.5.

3.2. Availability of Source Code. Any Modification which You create or to which You contribute must be made available in Source Code form under the terms of this License either on the same media as an Executable version or via an accepted Electronic Distribution Mechanism to anyone to whom you made an Executable version available; and if made available via Electronic Distribution Mechanism, must remain available for at least twelve (12) months after the date it initially became available, or at least six (6) months after a subsequent version of that particular Modification has been made available to such recipients. You are responsible for ensuring that the Source Code version remains available even if the Electronic Distribution Mechanism is

maintained by a third party.

3.3. Description of Modifications. You must cause all Covered Code to which You contribute to contain a file documenting the changes You made to create that Covered Code and the date of any change. You must include a prominent statement that the Modification is derived, directly or indirectly, from Original Code provided by the Initial Developer and including the name of the Initial Developer in (a) the Source Code, and (b) in any notice in an Executable version or related documentation in which You describe the origin or ownership of the Covered Code.

3.4. Intellectual Property Matters

(a) Third Party Claims If Contributor has knowledge that a license under a third party's intellectual property rights is required to exercise the rights granted by such Contributor under Sections 2.1 or 2.2, Contributor must include a text file with the Source Code distribution titled "LEGAL" which describes the claim and the party making the claim in sufficient detail that a recipient will know whom to contact. If Contributor obtains such knowledge after the Modification is made available as described in Section 3.2, Contributor shall promptly modify the LEGAL file in all copies Contributor makes available thereafter and shall take other steps (such as notifying appropriate mailing lists or newsgroups) reasonably calculated to inform those who received the Covered Code that new knowledge has been obtained.

(b) Contributor APIs If Contributor's Modifications include an application programming interface and Contributor has knowledge of patent licenses which are reasonably necessary to implement that API, Contributor must also include this information in the LEGAL file.

(c) Representations. Contributor represents that, except as disclosed pursuant to Section 3.4 (a) above, Contributor believes that Contributor's Modifications are Contributor's original creation(s) and/or Contributor has sufficient rights to grant the rights conveyed by this License.

3.5. Required Notices. You must duplicate the notice in Exhibit A in each file of the Source Code. If it is not possible to put such notice in a particular Source Code file due to its structure, then You must include such notice in a location (such as a relevant directory) where a user would be likely to look for such a notice. If You created one or more Modification(s) You may add your name as a Contributor to the notice described in Exhibit A. You must also duplicate this License in any documentation for the Source Code where You describe recipients' rights or ownership rights relating to Covered Code. You may choose to offer, and to charge a fee for, warranty, support, indemnity or liability obligations to one or more recipients of Covered Code. However, You may do so only on Your own behalf, and not on behalf of the Initial Developer or any Contributor. You must make it absolutely clear than any such warranty, support, indemnity or liability obligation is offered by You alone, and You hereby agree to indemnify the Initial Developer and every Contributor for any liability incurred by the Initial Developer or such Contributor as a result of warranty, support, indemnity or liability terms You offer.

3.6. Distribution of Executable Versions. You may distribute Covered Code in Executable form only if the requirements of Sections 3.1, 3.2, 3.3, 3.4 and 3.5 have been met for that Covered Code, and if You include a notice stating that the Source Code version of the Covered Code is available under the terms of this License, including a description of how and where You have fulfilled the obligations of Section 3.2. The notice must be conspicuously included in any notice in an Executable version, related documentation or collateral in which You describe recipients' rights relating to the Covered Code. You may distribute the Executable version of Covered Code or ownership rights under a license of Your choice, which may contain terms different from this

License, provided that You are in compliance with the terms of this License and that the license for the Executable version does not attempt to limit or alter the recipient's rights in the Source Code version from the rights set forth in this License. If You distribute the Executable version under a different license You must make it absolutely clear that any terms which differ from this License are offered by You alone, not by the Initial Developer or any Contributor. You hereby agree to indemnify the Initial Developer and every Contributor for any liability incurred by the Initial Developer or such Contributor as a result of any such terms You offer.

3.7. Larger Works. You may create a Larger Work by combining Covered Code with other code not governed by the terms of this License and distribute the Larger Work as a single product. In such a case, You must make sure the requirements of this License are fulfilled for the Covered Code.

1. Inability to Comply Due to Statute or Regulation.

If it is impossible for You to comply with any of the terms of this License with respect to some or all of the Covered Code due to statute, judicial order, or regulation then You must: (a) comply with the terms of this License to the maximum extent possible; and (b) describe the limitations and the code they affect. Such description must be included in the LEGAL file described in Section 3.4 and must be included with all distributions of the Source Code. Except to the extent prohibited by statute or regulation, such description must be sufficiently detailed for a recipient of ordinary skill to be able to understand it.

1. Application of this License. This License applies to code to which the Initial Developer has attached the notice in Exhibit A and to related Covered Code.
2. Versions of the License.

6.1. New Versions Netscape Communications Corporation ("Netscape") may publish revised and/or new versions of the License from time to time. Each version will be given a distinguishing version number.

6.2. Effect of New Versions Once Covered Code has been published under a particular version of the License, You may always continue to use it under the terms of that version. You may also choose to use such Covered Code under the terms of any subsequent version of the License published by Netscape. No one other than Netscape has the right to modify the terms applicable to Covered Code created under this License.

6.3. Derivative Works If You create or use a modified version of this License (which you may only do in order to apply it to code which is not already Covered Code governed by this License), You must (a) rename Your license so that the phrases "Mozilla", "MOZILLAPL", "MOZPL", "Netscape", "MPL", "NPL" or any confusingly similar phrase do not appear in your license (except to note that your license differs from this License) and (b) otherwise make it clear that Your version of the license contains terms which differ from the Mozilla Public License and Netscape Public License. (Filling in the name of the Initial Developer, Original Code or Contributor in the notice described in Exhibit A shall not of themselves be deemed to be modifications of this License.)

1. DISCLAIMER OF WARRANTY COVERED CODE IS PROVIDED UNDER THIS LICENSE ON AN "AS IS" BASIS, WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, WITHOUT LIMITATION, WARRANTIES THAT THE COVERED CODE IS FREE OF DEFECTS, MERCHANTABLE, FIT FOR A PARTICULAR PURPOSE OR NON-INFRINGING. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE COVERED CODE IS WITH YOU. SHOULD ANY COVERED CODE PROVE DEFECTIVE IN ANY RESPECT, YOU (NOT THE INITIAL DEVELOPER OR

ANY OTHER CONTRIBUTOR) ASSUME THE COST OF ANY NECESSARY SERVICING, REPAIR OR CORRECTION. THIS DISCLAIMER OF WARRANTY CONSTITUTES AN ESSENTIAL PART OF THIS LICENSE. NO USE OF ANY COVERED CODE IS AUTHORIZED HEREUNDER EXCEPT UNDER THIS DISCLAIMER.

2. Termination

8.1. This License and the rights granted hereunder will terminate automatically if You fail to comply with terms herein and fail to cure such breach within 30 days of becoming aware of the breach. All sublicenses to the Covered Code which are properly granted shall survive any termination of this License. Provisions which, by their nature, must remain in effect beyond the termination of this License shall survive.

8.2. If You initiate litigation by asserting a patent infringement claim (excluding declatory judgment actions) against Initial Developer or a Contributor (the Initial Developer or Contributor against whom You file such action is referred to as "Participant") alleging that:

a. such Participant's Contributor Version directly or indirectly infringes any patent, then any and all rights granted by such Participant to You under Sections 2.1 and/or 2.2 of this License shall, upon 60 days notice from Participant terminate prospectively, unless if within 60 days after receipt of notice You either: (i) agree in writing to pay Participant a mutually agreeable reasonable royalty for Your past and future use of Modifications made by such Participant, or (ii) withdraw Your litigation claim with respect to the Contributor Version against such Participant. If within 60 days of notice, a reasonable royalty and payment arrangement are not mutually agreed upon in writing by the parties or the litigation claim is not withdrawn, the rights granted by Participant to You under Sections 2.1 and/or 2.2 automatically terminate at the expiration of the 60 day notice period specified above. b. any software, hardware, or device, other than such Participant's Contributor Version, directly or indirectly infringes any patent, then any rights granted to You by such Participant under Sections 2.1(b) and 2.2(b) are revoked effective as of the date You first made, used, sold, distributed, or had made, Modifications made by that Participant.

8.3. If You assert a patent infringement claim against Participant alleging that such Participant's Contributor Version directly or indirectly infringes any patent where such claim is resolved (such as by license or settlement) prior to the initiation of patent infringement litigation, then the reasonable value of the licenses granted by such Participant under Sections 2.1 or 2.2 shall be taken into account in determining the amount or value of any payment or license.

8.4. In the event of termination under Sections 8.1 or 8.2 above, all end user license agreements (excluding distributors and resellers) which have been validly granted by You or any distributor hereunder prior to termination shall survive termination.

1. LIMITATION OF LIABILITY UNDER NO CIRCUMSTANCES AND UNDER NO LEGAL THEORY, WHETHER TORT (INCLUDING NEGLIGENCE), CONTRACT, OR OTHERWISE, SHALL YOU, THE INITIAL DEVELOPER, ANY OTHER CONTRIBUTOR, OR ANY DISTRIBUTOR OF COVERED CODE, OR ANY SUPPLIER OF ANY OF SUCH PARTIES, BE LIABLE TO ANY PERSON FOR ANY INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES OF ANY CHARACTER INCLUDING, WITHOUT LIMITATION, DAMAGES FOR LOSS OF GOODWILL, WORK STOPPAGE, COMPUTER FAILURE OR MALFUNCTION, OR ANY AND ALL OTHER COMMERCIAL DAMAGES OR LOSSES, EVEN IF SUCH PARTY SHALL HAVE BEEN INFORMED OF THE POSSIBILITY OF SUCH DAMAGES. THIS LIMITATION OF LIABILITY SHALL NOT APPLY TO LIABILITY FOR DEATH OR PERSONAL INJURY RESULTING FROM SUCH PARTY'S NEGLIGENCE TO THE EXTENT APPLICABLE LAW

PROHIBITS SUCH LIMITATION. SOME JURISDICTIONS DO NOT ALLOW THE EXCLUSION OR LIMITATION OF INCIDENTAL OR CONSEQUENTIAL DAMAGES, SO THIS EXCLUSION AND LIMITATION MAY NOT APPLY TO YOU.

2. U.S. government end users The Covered Code is a "commercial item," as that term is defined in 48 C.F.R. 2.101 (Oct. 1995), consisting of "commercial computer software" and "commercial computer software documentation," as such terms are used in 48 C.F.R. 12.212 (Sept. 1995). Consistent with 48 C.F.R. 12.212 and 48 C.F.R. 227.7202-1 through 227.7202-4 (June 1995), all U.S. Government End Users acquire Covered Code with only those rights set forth herein.
3. Miscellaneous This License represents the complete agreement concerning subject matter hereof. If any provision of this License is held to be unenforceable, such provision shall be reformed only to the extent necessary to make it enforceable. This License shall be governed by California law provisions (except to the extent applicable law, if any, provides otherwise), excluding its conflict-of-law provisions. With respect to disputes in which at least one party is a citizen of, or an entity chartered or registered to do business in the United States of America, any litigation relating to this License shall be subject to the jurisdiction of the Federal Courts of the Northern District of California, with venue lying in Santa Clara County, California, with the losing party responsible for costs, including without limitation, court costs and reasonable attorneys' fees and expenses. The application of the United Nations Convention on Contracts for the International Sale of Goods is expressly excluded. Any law or regulation which provides that the language of a contract shall be construed against the drafter shall not apply to this License.
4. Responsibility for claims As between Initial Developer and the Contributors, each party is responsible for claims and damages arising, directly or indirectly, out of its utilization of rights under this License and You agree to work with Initial Developer and Contributors to distribute such responsibility on an equitable basis. Nothing herein is intended or shall be deemed to constitute any admission of liability.
5. Multiple-licensed code Initial Developer may designate portions of the Covered Code as "Multiple-Licensed". "Multiple-Licensed" means that the Initial Developer permits you to utilize portions of the Covered Code under Your choice of the MPL or the alternative licenses, if any, specified by the Initial Developer in the file described in Exhibit A.

Exhibit A - Mozilla Public License.

"The contents of this file are subject to the Mozilla Public License Version 1.1 (the "License"); you may not use this file except in compliance with the License. You may obtain a copy of the License at http://www.mozilla.org/MPL/

Software distributed under the License is distributed on an "AS IS" basis, WITHOUT WARRANTY OF ANY KIND, either express or implied. See the License for the specific language governing rights and limitations under the License.

The Original Code is _____.

The Initial Developer of the Original Code is _____. Portions created by _____ are Copyright (C) _____ _____. All Rights Reserved.

Contributor(s): _____.

Alternatively, the contents of this file may be used under the terms of the _____ license (the "[___] License"), in which case the provisions of [_____] License are applicable instead of those above. If you wish to allow use of your version of this file only under the terms of the [____]

License and not to allow others to use your version of this file under the MPL, indicate your decision by deleting the provisions above and replace them with the notice and other provisions required by the [___] License. If you do not delete the provisions above, a recipient may use your version of this file under either the MPL or the [___] License."

NOTE: The text of this Exhibit A may differ slightly from the text of the notices in the Source Code files of the Original Code. You should use the text of this Exhibit A rather than the text found in the Original Code Source Code for Your Modifications.

thread license terms

This software is copyrighted by the Regents of the University of California, Sun Microsystems, Inc., Scriptics Corporation, and other parties. The following terms apply to all files associated with the software unless explicitly disclaimed in individual files.

The authors hereby grant permission to use, copy, modify, distribute, and license this software and its documentation for any purpose, provided that existing copyright notices are retained in all copies and that this notice is included verbatim in any distributions. No written agreement, license, or royalty fee is required for any of the authorized uses. Modifications to this software may be copyrighted by their authors and need not follow the licensing terms described here, provided that the new terms are clearly indicated on the first page of each file where they apply.

IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR MODIFICATIONS.

GOVERNMENT USE: If you are acquiring this software on behalf of the U.S. government, the Government shall have only "Restricted Rights" in the software and related documentation as defined in the Federal Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you are acquiring the software on behalf of the Department of Defense, the software shall be classified as "Commercial Computer Software" and the Government shall have only "Restricted Rights" as defined in Clause 252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the authors grant the U.S. Government and others acting in its behalf permission to use and distribute the software in accordance with the terms specified in this license.

Index

A

Archetype 203, 206, 208, 211
ArrowButton 9
asynchronous I/O 241, 245

B

bind 241, 245
body 148
bound variable 304, 307
Button 14
ButtonBox 17
BWidget 11

C

C handler set 357, 380
call 304, 307
callback 157
Calling_a_Web_Service 270
certificate 241
channel 241, 245
Class 193
class 148, 156, 159, 164, 165, 167, 191, 197, 209
code 157, 192
ComboBox 21
command tracing 404
component 167, 211
configbody 159
configure 159
connection 241, 245, 297, 302, 304, 307, 311, 314, 316, 317
connectivity 297, 302, 304, 307, 311, 314, 316, 317
contents 102
Creating_a_Tcl_Web_Service 276
Creating_a_Web_Service_Type 279

D

database 297, 302, 304, 307, 308, 309, 311, 314, 316, 317
Defining_Types 282
delegation 170
delete 160
Dialog 24
Dictionary_Representation_of_XML_Arrays 283
document 327, 343
document element 335
DOM 343, 379, 380
dom 327

DOM node creation 335
domain 245
domain name 241
domDoc 335
domNode 343
DragSite 26
DropSite 29
DTD 382
DynamicHelp 31

E

Embedded_Web_Service 285
ensemble 160, 163
Entry 35
events 392
expat 353
expatapi 357
expect 137
extendedclass 167, 178, 180

F

file 247, 252, 256, 257
filesystem 247, 252, 256, 257
find 164, 193
free 194, 195
ftp 247, 257

H

host 241, 245
HTTP 227
http 247, 257
httpd 227
https 241

I

I/O 241, 245
import 164
index 102, 296
info 163
introspection 404
IP Address 241, 245
is 165
itcl 167, 182, 210
itclcomponent 167
itcldelegate 170
itclextendedclass 178
itcloption 180
itclvars 182
itclwidget 189

itk 203, 206, 208, 209, 210, 211
itkvars 210

L

Label 39
LabelEntry 41
LabelFrame 43
libexpect 146
List 193
ListBox 49
local 191
locking 401

M

MainFrame 53
mclistbox 217
mega-widget 167, 203, 206, 208, 209, 211
memory 194, 195
message passing 392
MessageDlg 56
metakit 247
method 170
mpexpr 224
mutex 392
MySQL 311

N

name 245
namespace 157, 160, 167, 192, 247, 257
navtree 103
network 241, 245
network address 241, 245
networking 245
node 327, 343
NoteBook 59

O

Object 194
object 148, 156, 159, 164, 165, 167, 178, 189, 191, 197, 209
object-oriented 156, 167, 178, 189, 209
ODBC 314
OpenSSL 241
option 170, 180
options 111

P

PagesManager 61
PanedWindow 63
PanelFrame 65

parsing 327, 343, 359
part 163
PasswdDlg 67
Postgres 316
Postvalidation 379
Preserve 195
private 157
proc 160, 170
procedure 148, 191
ProgressBar 69
ProgressDlg 71
protected 157
public 157
pull 359
pullparser 359
push 353
pushparser 353

R

RegisterC 197
Rest_flavor_service_response 286
resultset 297, 302, 304, 307

S

SAX 353, 379, 380
schema 379
scope 157, 192
ScrollableFrame 74
ScrolledWindow 76
ScrollView 72
search 164
SelectColor 78
SelectFont 79
Separator 80
socket 241, 245
SpinBox 83
SQL 297, 302, 304, 307, 308, 309, 311, 314, 316, 317
SQLite3 317
SSL 241
Stack 198
state 302
statement 297, 302, 304, 307
StatusBar 85
stored procedure 304, 307
synchronization 392, 401

T

tar 247, 257

Tcl_Web_Service_Example 289
Tcl_Web_Service_Math_Example 292
TclHttpd 227
TclTLS 241
TclUDP 245
TCP 241
TDBC 302, 304, 307, 308, 309, 311, 314, 316, 317
tdbc 297
tdbc_connection 302
Tdbc_Init 309
tdbc_mapSqlState 302
tdbc_mysql 311
tdbc_odbc 314
tdbc_postgres 316
tdbc_resultset 304
tdbc_sqlite3 317
tdbc_statement 307
tdbc_tokenize 308
tdomcmd 380
thread 392, 396
thread shared data 401
threadpool 396
threads 401
TitleFrame 86
tls 241
tnc 382
tokenize 308, 309
Toplevel 203, 206
tpool 396
Tree 95
tsv 401
ttrace 404

U

udp 245
Using_Options 295
usual 211

V

Validation 379, 382
variable 159, 160, 192
variables 182, 210
vfs 247, 252, 256, 257
vfs-filesystems 247
vfs-fsapi 252
vfslib 247, 257

W

Web Server 227
webdav 247, 257
Widget 100, 203, 206, 208
widget 167, 180, 189
widgetadaptor 167, 180

X

XML 327, 343, 359
XPath 384
xpathFunc 384

Z

zip 247, 257